AGENT ORCHESTRATOR · JAVA 21 · USER GUIDE & TECHNICAL REFERENCE

the complete **spectroscope** user guide

One headless core, many faces. Every feature of the full build, from the terminal, the web UI, the desktop shell, the Lab, subagents, skills, MCP, voice and vision to the fleet on the bus, explained with real screenshots, and every protocol, event and endpoint documented down to the wire.

7	18	22+	3
MODULES ON ONE STREAM	RUNEVENT TYPES	TOOLS (+ MCP)	LLM BACKENDS (5 PRESETS)
678	383	3	7
JUNIT TESTS	VITEST TESTS	DESIGNS (DARK · BRIGHT · WHITE)	BUILT-IN SCENARIOS

ABOUT THIS GUIDE

How to read this book

A NOTE ON THE SCREENSHOTS

Every screenshot is a real capture of the running app, taken by the repository's own capture pipeline (`capture_screens.mjs`) and reshot per release. The two editions of this guide carry matching sets: the dark pages show **spectro dark**, the light pages **spectro bright**.

spectroscope is the reference harness of the workshop "build an agent harness from scratch" — and the full build in the `spectroscope/` folder is its final form: everything the ten core stages and six bonus stages construct, plus the conveniences that never fit a single stage. This guide documents that full build. Not the journey — the machine.

The book has five parts, ordered from "I just want to use it" to "I want to know exactly what crosses which wire":

- **Part I — Meet spectroscope**: what it is, how to start it, what it can do.
- **Part II — The faces**: the terminal, the web UI with all its tabs and panels, the Lab, session management, the design system, the desktop app.
- **Part III — The agent's powers**: tools, permissions, hooks, subagents, skills, providers, vision, voice, image generation, MCP and scheduling.
- **Part IV — The file system**: where everything lives on disk and what each file looks like inside.
- **Part V — Under the hood**: the complete technical reference — the architecture dossier, every event, every socket frame, every REST endpoint, every configuration key.

HONESTY RULES OF THIS DOCUMENT

Every screenshot in this guide is a real capture of the running application (English chrome, taken with the reproducible scripts in `docs/guide-assets/`; the main suite, the tutorial, chat and state graph plates were re-taken on 2026-09-29 against the release before this one). Every terminal block is real command output. Every name, constant, wire string and default was read from the migration phase-6 source tree (2026-07-20) — not from memory. Where behaviour is a deliberate limitation, the guide says so.

Conventions

- `monospace` marks anything that exists literally in code, config or on the wire: tool names, event types, JSON keys, commands.
- Chips classify tools: `free` needs no permission, `gated` asks first, `gated + prefix rule` asks first and remembers approvals scoped to a prefix.
- German UI labels appear in parentheses where the chrome ships bilingual — the screenshots use the English chrome.
- File paths are relative to the `spectroscope/` root unless they start with `~`.

CONTENTS

The map of this book

0 **How to read this book**
- Conventions

PART I **Meet spectroscope**

1 **What is spectroscope?**
- One core, five faces
- Additive forever
- Tool inputs are model output
- The stack, in one breath

2 **Quick start**
- The launcher
- The thirty-second start
- What doctor tells you
- The web face is a command group
- The provider matrix

3 **The capability map**

PART II **The faces**

4 **The command line**
- Commands and global flags
- The REPL
- Slash commands
- The twelve doctor checks
- The tour

5 **The web face: Chat**
- The anatomy of the screen
- The conversation
- Pictures out of an imported transcript
- Messages between agents
- The composer
- Commands
- Goal and loop
- The permission gate
- Live indicators

6 **The web face: tabs & panels**
- The Graph tab
- The Trace tab
- The Text tab
- The right panel
- Images

6b **the spectrum and the new lenses**
- the spectrum tab
- the reasoning lens
- the causal chain and the replay scrubber
- the gate surface
- the explain panel
- spectro doctor, the web face

7 **The Lab**
- The dam
- The transport
- compact | expanded
- The Flow map (React Flow)
- The keymap
- Under the hood: the Petri marking
- Scenarios: deterministic demo runs

6c **the tutorial**
- learn, light or developer
- the tutorial question
- the seven states
- progression with receipts
- reaching a step
- turning it off

8 **Sessions: continue, delete, import**
- The list
- Continue, with the re-upload made visible
- Several at once
- Delete
- Import
- What an imported transcript says
- The agents beside a session
- The address of what you are looking at
- New chat semantics

9 **The design system**
- The token base
- Three designs
- The effects layer

10 **The desktop face**
- The supervision sequence
- A self-contained run kit
- Tray, notifications, quitting

11 **Two languages**

PART III **The agent's powers**

12 **The tool belt**
- The shared rules
- Files & search
- Mutation & shell
- Metadata & delegation
- Who gets which tools
- Tool groups

13 **Permissions, allowlist & hooks**
- The guard pipeline
- Permission modes
- The allowlist
- The net fence
- Hooks
- The audit trail

14 **Subagents & the A2A protocol**
- Two roles
- The A2A-lite lifecycle
- The development tools

15 **Skills**
- The package format
- Progressive disclosure
- What a fresh home gets
- The bundled catalogue
- Installing one

15b **the fleet: many agents, one spectrum**
- what rides the bus
- spectro node: the process fleet
- code fleets on the hub
- entering a fleet
- the fleet bar
- the rail and its cards
- one agent's tab
- talking to a node
- the control plane

15c **The state graph: drawn before the run**
- Why this one is not a reconstruction
- Reaching it, and the address it does not have
- Five bundled runs, under two sets of labels
- The pair of files, and why the shape alone is enough
- Four tiers, and why the default is off
- The transport, and the speed that is not one
- Reading the drawing
- The panel: what one node wrote at this visit
- A node that failed
- Export: two files, no options
- Why the layout is not dagre
- Two things called graph

15d **Playbooks: a way of working, drawn**
- What a playbook is
- The folder
- The developer mode
- The spectro playbook
- Copying it into a folder
- Spectrolyzr
- Editing a playbook
- The bundle tabs
- Playbook contents
- Running a playbook
- Running a workflow
- What a playbook file may not carry
- What version 1 does not do

16 **Providers & models**
- The backends
- First run — choosing a backend
- Putting a key in the .env
- Switching mid-session
- The provider overview
- Retry — a loop that doesn't tip over
- Prompt caching
- The wrapper chain

16b **The built-in models**
- Choosing a model
- Where a model comes from
- The runtime underneath
- Honest capability flags, per model

17 **Seeing, hearing, speaking, painting**
- Vision — image input
- Voice input — two routes to the same text
- Live transcription — text while you are still speaking
- Voice output — spoken answers
- Image generation

18 **MCP — external tool servers**
- Configuration
- Call semantics: at-most-once
- The example server: spectro-mcp-notes

19 **Scheduling**

PART IV **The file system**

20 **Where everything lives**
- Inside a session file
- Blobs and generated images
- The workspace: where the agent actually works
- Lifecycle: create, continue, delete
- The jq cookbook

PART V **Under the hood**

21 **The architecture dossier**
- The big picture
- The protocol layer
- The model side
- The loop and the belt
- Delegation
- The web face
- Runtime and integration
- The tutorial
- The codebase itself

22 **The RunEvent protocol**
- The wire rules
- The catalog
- The cross-edition proof
- Resume: lines become a conversation

23 **Streams & cancellation**
- EventStream
- MergedEventStream
- CancelSignal

24 **The provider wire**
- The port
- Backend mappings

25 **The WebSocket protocol**
- Client → server
- Server → client
- Threading model

26 **The REST API**

26b **The llm-wire**
- The sidecar file
- The three line types
- Fidelity and transport
- Always on, with one exception
- What is redacted
- Reading a recorded exchange
- The three reads
- What it does not record

27 **The agent loop, hop by hop**
- The turn, precisely
- The guard pipeline (per tool call)
- The headless variant

28 **Context: compaction, caching, introspection**
- Compaction
- Prompt caching
- Introspection

29 **MCP internals**
- JSON-RPC framing
- Client semantics
- The reference server

30 **Configuration reference**
- The layers
- Every key
- Observability: streaming to OTLP
- Environment-only variables
- SPECTRO.md

31 **Build & test inventory**
- Modules
- Version pins
- The test suite

32 **Releases & working with the source**
- What a release ships
- The libraries on Maven Central
- The desktop run kit
- Working with the source
- The release ritual
- The brand mark

33 **Saving a key, safely**
- Where the key goes
- Why it works without a restart
- The two fences on the write
- What an attacker can and cannot do
- You never have to use it

A **Troubleshooting**

B **Reproducing this guide**
- Credits and colophon

PART I

Meet spectroscope

What the harness is, the one idea everything hangs on, how to start it in thirty seconds, and a map of everything it can do.

1 What is spectroscope?

2 Quick start

3 The capability map

CHAPTER 1

What is spectroscope?

spectroscope is an agent harness: the machinery around a large language model that turns “a model that can talk” into “an agent that can work” — read files, run commands, ask for permission, delegate to subagents, remember sessions, and show you every step it takes while doing so.

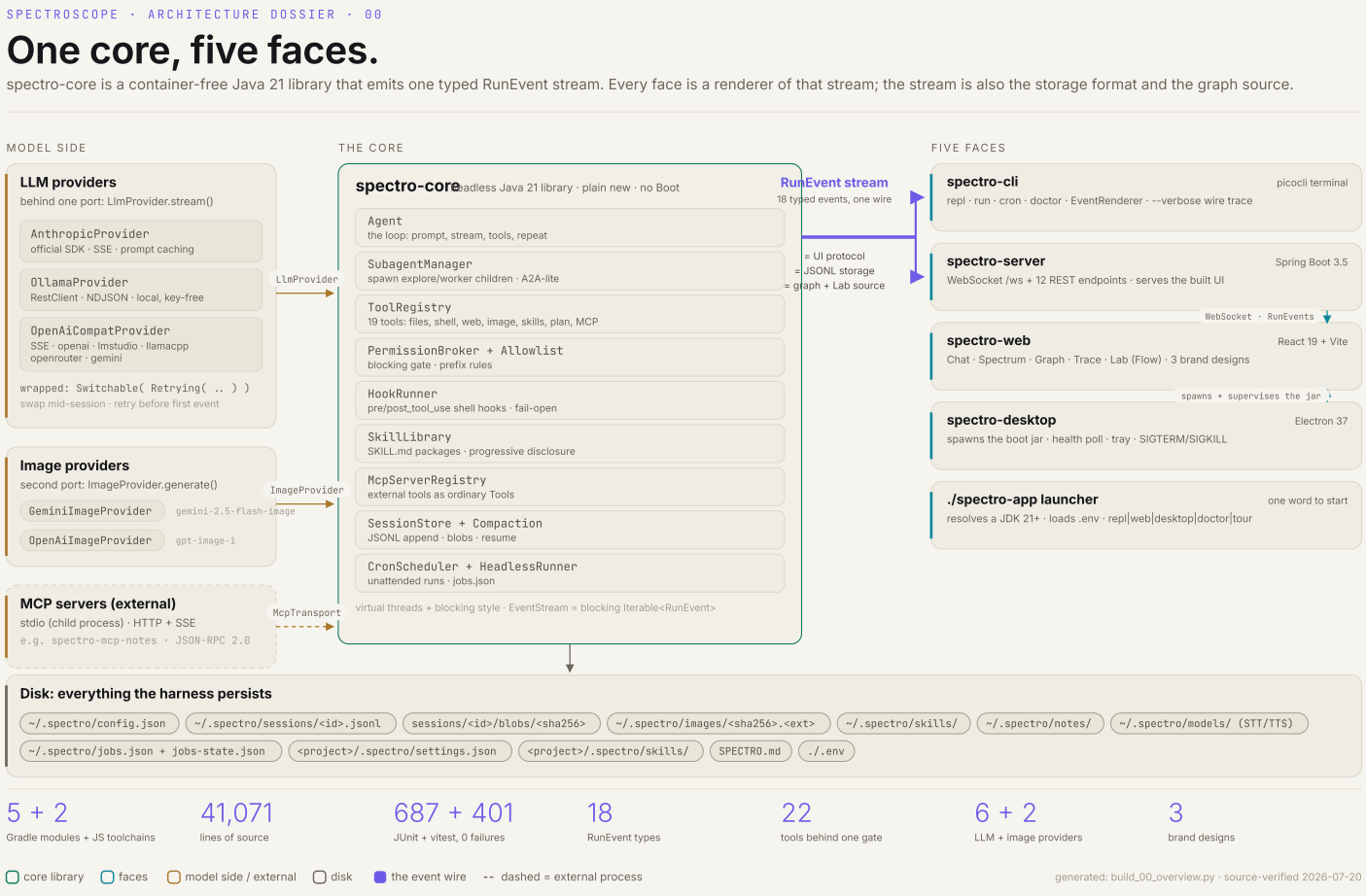
It was built from scratch across a sixteen-stage workshop (Java 21, Gradle, Spring only where it earns its place), and the `spectroscope/` full build is the sum of all of it. Three properties define the design:

One core, five faces

The heart of spectroscope is `spectro-core` — a headless Java library with no container, no UI and no I/O of its own. Everything in it is constructible with plain `new`. When the agent runs, the core produces exactly one thing: a typed stream of **RunEvents**. Everything else in the project is a *renderer* of that stream:

- the **CLI** (`spectro-cli`) renders it as ANSI text with tool cards and a spinner,
- the **web backend** (`spectro-server`) pushes it over a WebSocket,
- the **browser UI** (`spectro-web`) folds it into chat, graph, trace, Lab and panels,
- the **desktop shell** (`spectro-desktop`) wraps that same web UI in a supervised window,
- the **session store** writes it line by line into a JSONL file.

That last renderer is the trick: the same event stream is simultaneously the **UI protocol**, the **storage format** and the **data source for every visualisation**. Replaying an old session is not a separate code path — it is the same array of events, read from a file instead of a socket.



The big picture: the container-free core, the RunEvent stream as the one wire, the five faces, the provider ports and the disk.

Additive forever

The JSONL wire format is shared byte-for-byte with the sibling TypeScript edition of the workshop. That makes the format a contract: new event types and new optional fields may be added, but nothing is ever renamed, removed or reinterpreted, and every consumer silently skips what it does not know. Sessions recorded in 2026 replay forever; sessions recorded by the other edition replay here (proven by a byte-for-byte round-trip test).

Tool inputs are model output

The course maxim that shapes every security decision in this book: whatever a tool receives as input was written by the model, and the model writes what it wants. So every file path passes a sandbox, every mutation passes a permission gate, network egress asks first, hooks can veto calls before the gate, and even the “always allow” button remembers its approval scoped to a prefix instead of a blanket. You will meet this maxim in almost every chapter.

The stack, in one breath

LANGUAGE	Java 21 — virtual threads + blocking style; no reactive frameworks. The stream type is a blocking `Iterable<RunEvent>`, cancellation is a cooperative `CancelSignal`.
BUILD	Gradle 9.6.1, Kotlin DSL, version catalog, five JVM modules + two npm toolchains (Vite/React web UI, Electron shell).
LLM ACCESS	One narrow `LlmProvider` port; behind it Anthropic (official SDK, SSE), Ollama (NDJSON) and any OpenAI-compatible server (SSE) — switchable mid-session.
FRAMEWORKS	Spring *Framework* as a library in the core (RestClient + declarative HTTP interfaces); Spring *Boot* 3.5.3 only in `spectro-server`; picocli in the CLI; React 19 + React Flow in the browser.
TESTS	678 JUnit + 383 vitest, all key-free: fake providers, scripted HTTP servers, process seams. `./gradlew build` needs no API key and no network model.

CHAPTER 2

Quick start

One executable starts everything. The `./spectro-app` launcher resolves a JDK, loads your `.env`, and dispatches to the face you ask for — you never have to touch the Gradle task zoo.

`./spectro-app` is the developer wrapper in a clone of the repository; `spectro` is the shipped CLI. The command you install from a release is called `spectro` and this guide calls it that everywhere else. The wrapper in the repository root carries a different name so that it reads as one of a set with its two siblings — `./spectro-serve` for the server's lifecycle and `./spectro-env` for the local docker stacks. Same product, two entry points, and the names say which one you are holding.

The launcher

```
● $ ./spectro-app
```

```
◆ spectroscope

  repl          interactive agent REPL — /voice push-to-talk, --speak reads aloud
                ./gradlew -q --console=plain :spectro-cli:run --args="..."
  run           headless run: ./spectro-app run -p "task" [--json] [--image path] [--speak]
                ./gradlew -q --console=plain :spectro-cli:run --args="run -p ..."
  node          fleet node: ./spectro-app node -p "task" --hub host:port --context <fleet>
                ./gradlew -q --console=plain :spectro-cli:run --args="node -p ..."
  cron          scheduler: list / status / run <id> (~/.spectro/jobs.json)
                ./gradlew -q --console=plain :spectro-cli:run --args="cron list"
  sessions      list the stored JSONL sessions (~/.spectro/sessions)
                ./gradlew -q --console=plain :spectro-cli:run --args="sessions"
  resume        continue a session by id: ./spectro-app resume <id>
                ./gradlew -q --console=plain :spectro-cli:run --args="--resume <id>"
  doctor        check Java, config layers, provider reachability
                ./gradlew -q --console=plain :spectro-cli:run --args="doctor"
  level         the tutorial with its ticks and receipts (no server needed)
                ./gradlew -q --console=plain :spectro-cli:run --args="level"
  tour          guided feature tour — menu, tips, settings
                ./gradlew -q --console=plain tour
  web           web face: a command group — start, dev, stop, status, logs (./spectro-app web)
                ./spectro-app web  ·  ./spectro-app web start --port 8090
  desktop       desktop face — Electron shell + managed Spring Boot process
                ./gradlew :spectro-server:bootJar && (cd spectro-desktop && npm start)
  mcp-notes     build the example MCP server (stdio) into a runnable dist
                ./gradlew -q --console=plain :spectro-mcp-notes:installDist

Extra arguments pass through: ./spectro-app repl --verbose · ./spectro-app run -p 'Say OK.' --json
Vision needs a vision model (attach via --image or the web composer); voice: bash scripts/setup-stt.sh (in) and scripts/setup-tts.sh (out).
MCP: configure external servers in .spectro/settings.json ('mcpServers'); ./spectro-app mcp-notes builds the example; in the REPL, /mcp lists them.
```

Before dispatching, the launcher fixes the two classic host problems for you:

- **JDK auto-resolution.** The modules compile with `--release 21`, but many machines default to an older `java`. If `JAVA_HOME` is not already a JDK 21+, the launcher probes `/usr/libexec/java_home`, the Homebrew `openjdk@21` keg (preferred: a stable versioned keg survives brew upgrades), newer kegs up to 25, and the Cellar globs — and exports the first hit. Verified up to JDK 26; a miss prints a note and continues.
- **`.env` loading for every face.** The gitignored `spectroscope/.env` holds API keys and provider switches. The launcher parses it (comments skipped, one quote layer stripped, empty values ignored so `KEY=` can never blank a shell export) and exports the pairs — deliberately including the `desktop` face, which spawns the server through Electron and would otherwise bypass Gradle's own injection.

The thirty-second start

```
cd spectroscope

# 1 — pick a model. Local (no key at all):
#    .env:  SPECTRO_PROVIDER=ollama · SPECTRO_MODEL=qwen3 · SPECTRO_BASE_URL=http://localhost:11434
#    cloud: ANTHROPIC_API_KEY=sk-ant-…      (model defaults to claude-opus-4-8)

# 2 — check the environment
./spectro-app doctor

# 3 — go
./spectro-app web start  # browser UI on http://localhost:8080, in the background
./spectro-app web        # what the group can do, and whether it is running
./spectro-app repl       # terminal REPL
./spectro-app run -p "Say OK."   # headless one-shot
./spectro-app desktop    # the Electron app
```

What doctor tells you

`./spectro-app doctor` is the honest pre-flight: it probes what your configuration actually selects — it loads the config hierarchy with your flags, pings the provider you chose, connects to every configured MCP server, and write-tests the sessions directory. This is real output from the machine this guide was built on:

```
● $ ./spectro-app doctor

◆ spectro doctor
  ✓ Java 25.0.2
  ✓ config: provider=ollama model=qwen3.5:27b permissionMode=ask autoApprove=2 rule(s)
    layers: user config absent · project settings present
  ✓ ollama at http://localhost:11434 (version 0.24.0)
    images: gemini — GEMINI_API_KEY not set; generate_image will return a readable error
  ✓ skills: 4 installed (brainstorming, test-driven-development, verification, writing-plans)
  ✓ hooks: 0 configured
  ✓ mcp: notes reachable at /Users/you/spectroscope/spectro/spectro-mcp-notes/build/install/spectro-mcp-notes/bin/spectro-mcp-notes (2 tools)
    vision: local provider — attach images only with a vision model (e.g. ollama pull qwen3-vl); a text-only model fails fast
  ✓ voice input: whisper-cli + ggml-small.bin ready (/voice)
    voice output: piper missing · voice missing — run bash scripts/setup-tts.sh to enable --speak
  ✓ sessions dir writable: /Users/you/.spectro/sessions (7 session(s))
  ✓ jobs.json: 0 job(s)

Everything looks good.
```

Green checks fail the exit code when broken; the indented dim lines are information, not failures — a missing image-generation key, for example, is deliberately not unhealthy (the tool will return a readable error instead). Chapter 4 documents all twelve checks.

The web face is a command group

Typing `./spectro-app web` does not start anything. It shows what the group can do and whether a server is already up, because a face that fires a process the moment you name it is a face you cannot ask a question of.

```
● $ ./spectro-app web

◆ spectro-app web

  ○ stopped
  port  8199   health --

  start     background: the packaged jar, health-waited, browser opened
            --port N · --env-file F · --no-open
  dev       foreground: gradle bootRun from source, ctrl-c stops it
            --port N · this is what a bare ./spectro-app web used to do
  stop      stop it, and free the port if something still holds it
  restart   stop, then start again on the same port
            --port N
  status    the block above, on demand
  logs      the tail of logs/spectro-server.log
            -f follows
  open      open the url in the browser
  doctor    what the SERVER needs to boot: java, the jar, a backend
            ./spectro-app doctor checks the agent instead

A bare './spectro-app web' no longer starts anything: 'start' backgrounds it, 'dev' holds the terminal.
Every verb but 'dev' runs ./spectro-serve, which owns the pid file, the port file and the log.
```

Two of those verbs start a server and they are not the same thing. `start` runs the packaged jar in the background, waits for health and opens the browser, which is what you want when you are using spectroscope. `dev` runs Gradle's `bootRun` in the foreground from source, which is what you want when you are changing it: the terminal stays attached and ctrl-c stops it. Everything else, `stop`, `restart`, `status`, `logs`, `open` and `doctor`, reaches `./spectro-serve`. That script owns the pid file, the port file and the log, and it stays the only implementation of the lifecycle so the two can never disagree about what is running.

> WHAT CHANGED, AND WHAT IT COSTS YOU
>
> Before this, a bare `./spectro-app web` started the server in the foreground. That behaviour is now `./spectro-app web dev`. If you have the old form in a script, it fails with one line naming the fix rather than printing help and exiting cleanly, because a script that believes it started a server and did not is worse than a script that stops.

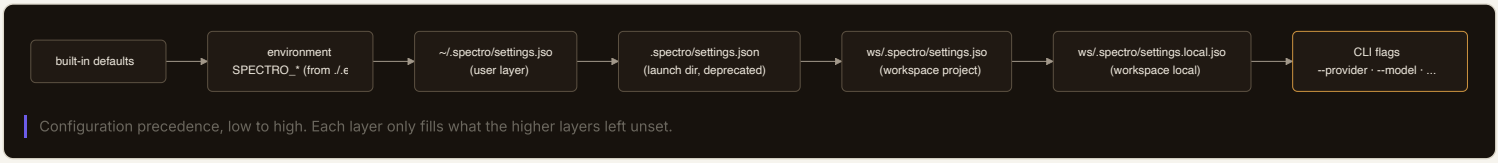
Note that `./spectro-app doctor` and `./spectro-app web doctor` answer different questions. The first checks the agent: Java, the config layers, whether your provider replies, whether the sessions directory is writable. The second checks only what the server needs in order to boot at all.

The provider matrix

PROVIDER	TRANSPORT	NEEDS	DEFAULT MODEL
`anthropic`	official SDK, SSE streaming	`ANTHROPIC_API_KEY`	`claude-opus-4-8`
`ollama`	Spring RestClient, NDJSON	local Ollama on :11434	`qwen3`
`openai`	Spring RestClient, SSE	OpenAI itself, or any compatible server you point `baseUrl` at; key optional	`local-model`
`lmstudio`	Spring RestClient, SSE	LM Studio's server on :1234, or another compatible one	`local-model`
`llamacpp`	Spring RestClient, SSE	your own `llama-server` on :8080	`local-model`

More ids exist than these five. Chapter 12 compares every backend and chapter 18 lists them exhaustively; this table is the quickstart set, and it is deliberately not a second copy of that list.

Selection rides the settings hierarchy (chapter 30 has every key): defaults < environment (`SPECTRO_PROVIDER` / `SPECTRO_MODEL` / `SPECTRO_BASE_URL`, usually from `./.env` — the BASE layer) < `~/.spectro/settings.json` (user) < launch-dir `.spectro/settings.json` (deprecated) < workspace `.spectro/settings.json` + `settings.local.json` < CLI flags — any settings file outranks the env. “Local Ollama for this checkout” is three uncommented lines in `.env` as long as no settings file overrides them — and in the web UI you can switch provider and model mid-session from the model chip under the message box.



Configuration precedence, low to high. Each layer only fills what the higher layers left unset.

CHAPTER 3

The capability map

Everything the full build can do, on two pages. Each row names the chapter where the details live.

CAPABILITY	IN ONE SENTENCE	CH.
Terminal REPL	Interactive agent with tool cards, spinner, slash commands, voice input and spoken answers.	4
Headless runs	`spectro run -p "…"` — one prompt, exit codes for CI, NDJSON event output with `--json`.	4
Cron scheduling	Jobs in `~/.spectro/jobs.json` run unattended with desktop notifications and audit sessions.	19
Web chat	Streaming chat with markdown answers, collapsible reasoning, tool cards with gate chips, image attachments, push-to-talk.	5
Graph view	Every run as a BPMN-style flow overview or a dagre DAG — live and for any archived session.	6
Trace view	The Wireshark of the harness: every wire frame both directions, filters, Δt, LLM-direction column, three detail modes.	6
Right panel	Agents roster, the agent's live plan, the exact system context per agent, a sandboxed file browser and the generated images, side by side or one at a time as tabs.	6
The Lab	A step-through debugger for agent runs: dam up the event stream, advance it click by click across three synchronized views.	7
Scenarios	Seven deterministic, compiled demo runs — no LLM, no key — for teaching every mechanism.	7
Sessions	Every run is a JSONL file with one row in the list: click it and keep typing (the context re-upload stays visible), run several at once, each on its own provider and model, delete after a confirmation.	8
Import	Replay foreign files: raw spectroscope JSONL verbatim, Claude Code transcripts through an adapter.	8
Design system	Three brand designs — spectro dark, spectro bright, spectro white — switchable in place, particle/scroll effects, one-step persistence.	9
Desktop app	Electron shell that spawns and supervises the server JVM: health checks, tray, native cron notifications.	10
DE/EN chrome	The whole UI chrome switches between German and English live; content keeps its language.	11
Tool belt	39 built-in tools in the app: files, shell, grep/glob, web fetch + search, page browse, image + document view, plan, skills, spawns, messages, code graph, plus every connected MCP tool.	12
Permission system	ask/auto/readonly/extended modes, a blocking broker, prefix-scoped allowlist, “always allow” with optional persistence.	13
Hooks	Config-driven shell guards around every tool call: `pre_tool_use` can block before the gate, `post_tool_use` observes.	13
Subagents	Explore/worker children with their own context, visible A2A task/status/result protocol, four development role tools.	14
Skills	`SKILL.md` packages with progressive disclosure — catalog in the prompt, body on demand.	15
Provider switch	Change LLM backend and model mid-session from the model chip under the message box; history intact; refused cleanly without a key.	16
Resilience	Transient-error retry with backoff (spectroscope owns it, SDK retry off) and Anthropic prompt caching with an honest compaction trigger.	16
Vision	Attach images in the composer or `--image`; blobs stay next to the session file, prompts stay small.	17
Voice in/out	Local whisper.cpp push-to-talk (editable transcript) and piper speech output that reads answers while they stream.	17
Image generation	`generate_image` behind its own provider port (Gemini / OpenAI), content-addressed store, an Images panel in the dock.	17
MCP client	External tool servers over stdio or HTTP/SSE, Claude-Desktop-shaped config, at-most-once calls, example notes server included.	18
Context management	Automatic compaction at 100k tokens, forced `/compact`, live context ring with per-part introspection.	28
Wire tracing	`--verbose` mirrors the full agent↔provider protocol to stderr; the Trace tab does it in the browser.	4, 6

PART II

The faces

Five renderers of one event stream: the terminal, the web UI with its tabs and panels, the step-through Lab, the session archive, the design system, and the desktop shell.

4 The command line
5 The web face: Chat
6 The web face: tabs & panels
7 The Lab
8 Sessions: continue, delete, import
9 The design system
10 The desktop face
11 Two languages

CHAPTER 4

The command line

The CLI is the first face the workshop builds and the most direct one: a picocli application whose root command is an interactive REPL, with subcommands for headless runs, scheduling and diagnostics. Everything it prints is a rendering of the same RunEvent stream that the web UI folds and the session store writes.

Commands and global flags

INVOCATION	WHAT IT DOES
`spectroscope`	the interactive REPL (root command, no subcommand)
`spectro run -p "…"`	headless single prompt (see below)
`spectroscope cron [list|status|--once id]`	the scheduler — chapter 19
`spectro sessions`	list stored sessions (a positional, not a subcommand)
`spectroscope --resume <id>`	continue a stored session in the REPL
`spectro doctor`	the environment check
`./spectro-app tour`	the interactive menu tour of all faces

Global flags on the root command apply to every entry point (subcommands reach them via picocli's `@ParentCommand` — so `flags > env` holds on `run` and `doctor` too):

FLAG	MEANING
`--provider` / `--model` / `--base-url`	override the configured backend for this invocation
`--compaction-threshold <n>`	compaction trigger in input tokens (default 100 000)
`--resume <id>`	rebuild the conversation from a session file and continue it
`--verbose`	mirror the agent↔provider wire protocol to stderr in cyan
`--speak`	read answers aloud while they stream (voice output, chapter 17)

The REPL

Startup prints a banner naming the provider and model (for Ollama it live-probes the server version), the session id and file path, and the essential key bindings. The prompt is a coral `❯`. Each turn drives three consumers in order — the JSONL store, the speech renderer, the ANSI renderer — the CLI-side proof of "one stream, many renderers".

```
◆ spectroscope  full build
  ollama · gpt-oss:20b · ollama 0.24.0 · images: gemini · skills: 4 · allowlist active
  session 20260716-091500-3fa4b2c1 · ~/.spectro/sessions/20260716-091500-3fa4b2c1.jsonl
  /help for commands · /mcp servers · /voice push-to-talk · /speak on|off reads answers aloud · Ctrl+C aborts a run
```

Rendering rules worth knowing (all in `EventRenderer`):

- **Thinking is dimmed.** A reasoning model's `thinking_delta` stream prints dim under a `· thinking` label, visually apart from the answer.
- **Tool cards.** Each call prints `⚒ name` (coral hammer) with a 100-char input preview; the result line shows `✓/✗ <ms> · output preview`.
- **Permissions inline.** A gated call prints a sand `run <tool>? [y/N]` prompt; only `y` approves. Allowlisted calls print `✓ auto-approved by allowlist` instead — the decision is still recorded in the stream.
- **Subagents in brackets.** Child events render as `[worker-1] …` lines with line-buffered text so parallel children stay readable; A2A task/status/result messages appear as one dim line each.
- **The plan pretty-print.** An `update_plan` call renders `◇ plan (N steps)` with `[x]` done, coral `[~]` running, `[ ]` open.
- **Degradation.** Colors and the braille spinner appear only on a real TTY (and never with `NO_COLOR` or `TERM=dumb`); piped output is plain text.

Slash commands

Handled entirely in the CLI — none of these ever reaches the model:

COMMAND	BEHAVIOUR
`/help`, `/commands`	lists every command of the terminal with a one-liner, then the web chat's own commands under "In the web chat (web only)". The list is the one the web chat's picker and `/help` read, so both show the same commands
`/cost`	cumulative session tokens: `Session usage: X in / Y out`
`/model`	current provider · model (+ base URL for local providers)
`/sessions`	the stored-session listing, same as `spectro sessions`

`/skills`	installed skills with descriptions (chapter 15)
`/mcp`	connected MCP servers, reachability, and every `mcp__…` tool (chapter 18)
`/think on|off`	toggles reasoning visibility; rebuilds the agent and reloads the session history so nothing is lost
`/voice`	push-to-talk: record → transcribe → editable confirmation (chapter 17)
`/speak on|off`	voice output at runtime; `off` also stops the current sentence
`/goal`	states this session's outcome and the command that checks it, shows it, or clears it (card 267)
`/loop [interval] <prompt>`	runs the prompt now and again: with an interval on a timer thread of the REPL, without one on the model's `schedule_wakeup`. While a loop runs, the prompt also wakes for a fired wakeup. `/loop stop` ends it; `/clear` leaves it running (card 504)
`/compact`	force a context compaction now (`Agent.compactNow()`); prints the result or "Nothing to compact"
`/clear`	a fresh context in the same session, as in the web chat: the session keeps its id, its file, its folder, its goal and its tools, and only the history goes. The file records a `context_cleared` line, so a later resume starts after it. A running loop and a goal stay
`/peer deliver|decline <id>`	answers a message from another session that waits for you (chapter 4)
`/exit`	leave (empty line or Ctrl-D work too)

Headless runs: `spectro run`

The automation face. One prompt, one process, honest exit codes:

```
# plain: prints only the final answer text
./spectro-app run -p "Count the Java files under spectro-core."

# NDJSON: every RunEvent on stdout, one per line — byte-identical to the JSONL format
./spectro-app run -p "Say OK." --json | jq -c '{type}'

# guard rails
./spectro-app run -p "…" --max-turns 5 --permissions readonly
```

FLAG	MEANING
`-p, --prompt`	required task text
`--json`	NDJSON events on stdout; ALL diagnostics go to stderr, so `| jq` pipelines stay clean
`--max-turns <n>`	cancel when a turn exceeds the limit (stop reason `max_turns`)
`--permissions readonly|auto|extended`	the headless policy. Default **readonly** — "capability before convenience": unattended runs deny mutations unless you opt in. `extended` approves like `auto` and lets the file tools read and write outside the working folder
`--image <path>`	attach an image (repeatable; jpg/jpeg/png/webp/gif) — vision, chapter 17
`--mcp` / `--no-mcp`	mount the configured MCP servers for this run, overriding the `headlessMcp` setting (chapter 30) — or decline them. A permission, not a convenience: with `--permissions auto` this approves every tool every configured server offers, unwatched. Absent, the setting decides; the run prints what it mounted on stderr
`--verbose` / `--speak`	as on the REPL

Exit code contract: 0 only when the run ended with a regular `end_turn`. Anything else (aborted, max turns, error) exits 1 and names the stop reason and the session id on stderr — the session file is always there for post-mortem inspection.

The twelve doctor checks

#	CHECK	WHAT IT VERIFIES
1	Java runtime	runtime major version ≥ 21
2	Config hierarchy	the layered config loads; shows provider/model/permissionMode/allowlist count and which layers exist. A broken config stops doctor immediately — nothing below would make sense
3	Provider reachability	anthropic: key present · ollama: live version probe · llamacpp: `GET /props`, which answers only once a model is loaded · every other OpenAI-compatible id: GET `/v1/models` (2 s/3 s timeouts)
4	Image provider	key for gemini/openai; a missing key is informational, not unhealthy
5	Skills	count + names of installed skills
6	Hooks	count + `event:matcher` pairs
7	MCP servers	connects each configured server: reachable + tool count, or a red UNREACHABLE line (never a crash)
8	Vision	informational hint matching your provider (vision model needed locally)
9	Voice input	`whisper-cli` on PATH + the ggml model file present
10	Voice output	piper binary + voice model present
11	Sessions dir	creates the dir, writes and deletes a probe file, counts sessions
12	jobs.json	the cron file parses (loudly invalid otherwise)

Wire tracing with `--verbose`

A decorator around the provider mirrors the whole protocol to **stderr** in cyan: the full outgoing request before each call (message count, tool names, clipped system prompt, every content block) and each incoming provider event as it streams (`← text_delta "…"`, `← tool_call name {…}`, `← usage 123 in / 45 out`, `← stop end_turn`). Because it writes to stderr, `spectro run --json | jq` keeps a clean stdout. In the REPL the traced instance is shared with subagents — you see the

children's requests too.

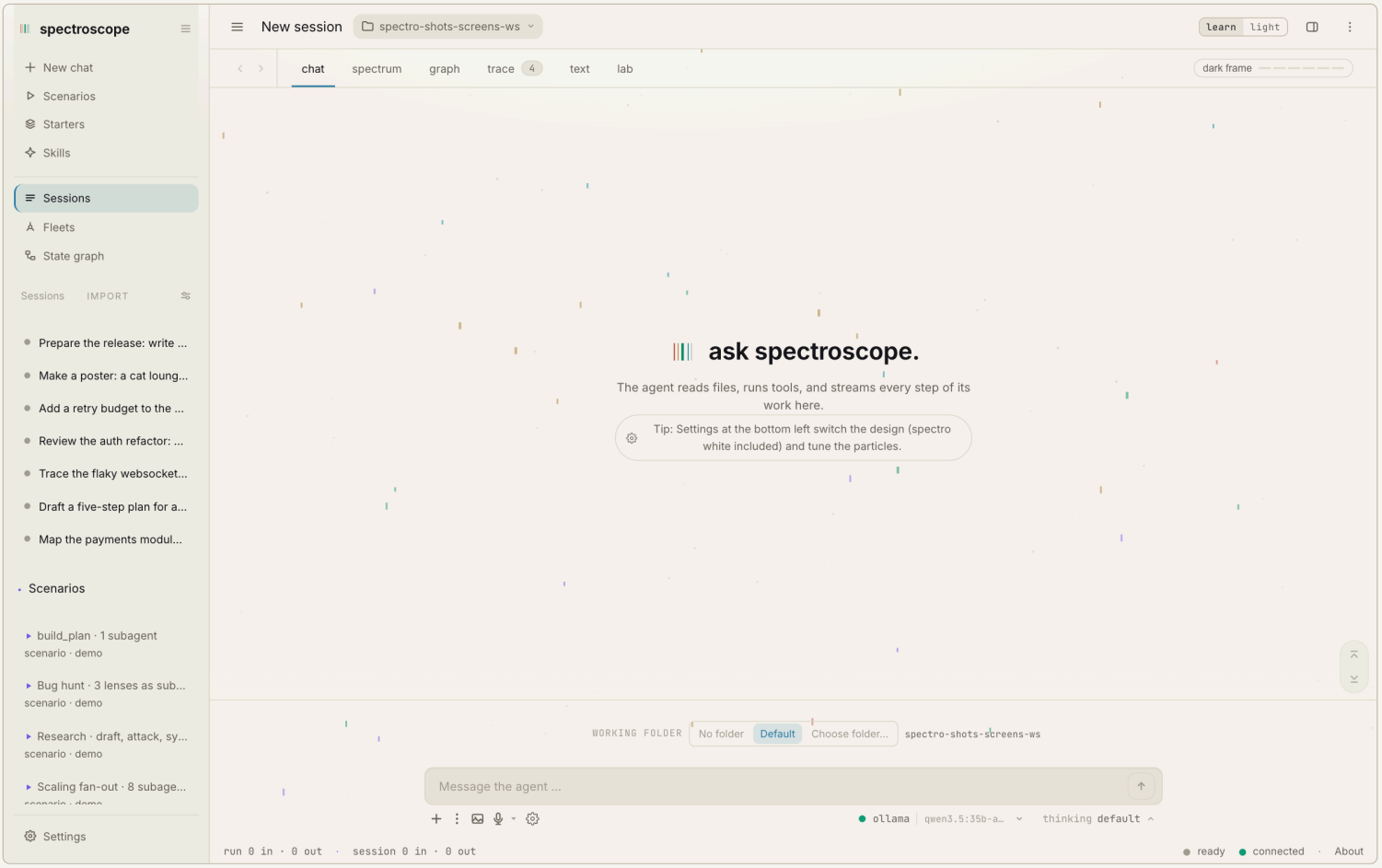
The tour

`./spectro-app tour` is a menu-driven guide around all faces: launch the REPL, list or resume sessions, do a headless run, drive cron, start the web jar, run doctor — plus a settings screen that captures a provider choice and API key for the session (hidden input, optional save to `./.env` with mode 600) and stage-appropriate “try this” tips. The same file ships in every workshop stage and detects at runtime which features exist; in the full build it shows everything.

CHAPTER 5

The web face: Chat

One Spring Boot jar serves everything: the built React UI, a handful of REST endpoints, and one WebSocket per open session that carries RunEvents in one direction and a small set of client frames in the other. Open `http://localhost:8080` after `./spectro-app web` and you are looking at the chat.



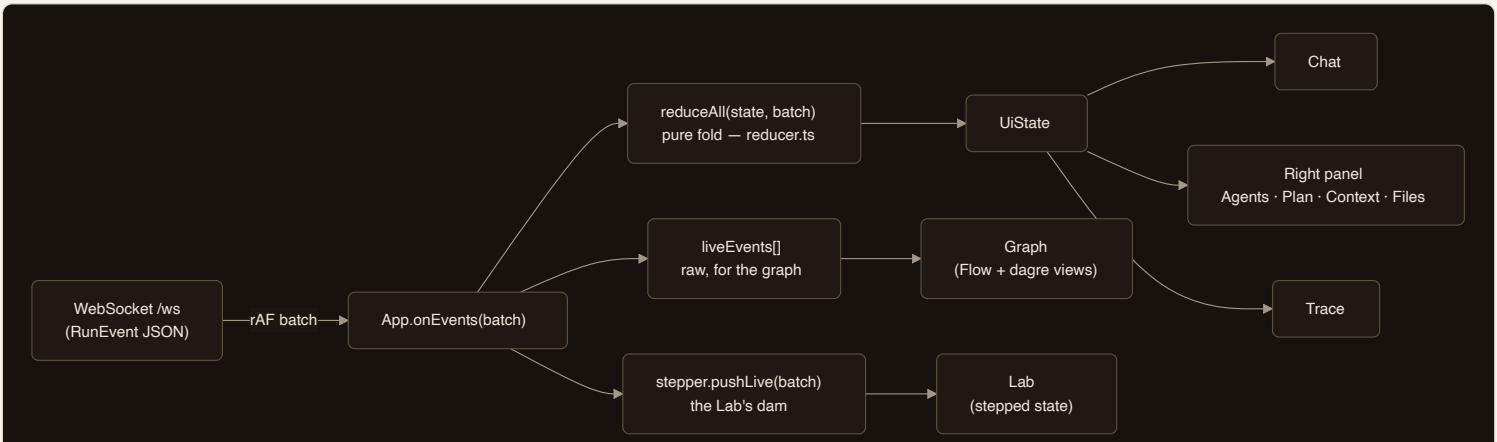
The empty app: the rail with its actions, the session list under its own head row and Settings at its foot, the header with the title, the working folder chip, the learn and light switch, the side panel toggle and the ⋮ menu, the six tabs, and the composer waiting with the model and the thinking level under it. “Ask spectroscope.”

The anatomy of the screen

- **Sidebar** (left, resizable, collapsible): “New chat”, the **Scenarios** row (chapter 7), and the session list. A head row above the list shows “Sessions”, the Import button (chapter 8) and the list options. The list has one row per session. The list options set its density: minimal shows each session's name, its state dot and the badge that says it waits for you; normal adds the folder and the state; extended adds the time, the turns and the tokens. The rows of the other sessions on this machine follow the same density. A new chat gets its own row once it has started, and a session that is running keeps pulsing while you look at another one. Settings sits at the foot of the sidebar, behind the gear.
- **Header**: the sidebar toggle, the session title (the first prompt), the **working folder** chip, the back and forward buttons, left of the mode switch, the mode switch (learn, light, developer), the side panel toggle and the ⋮ **menu**. In developer mode a playbook dropdown and a play button stand beside the folder chip (chapter 15d). The back and forward buttons are in the header in every mode and on every screen, the Playbook segment included, and ⌘← and ⌘→ do the same. The folder chip names the folder the session works in. Its menu shows the folder in Finder, copies its path, changes it (only before the first message) or opens Terminal there. Finder and Terminal work on macOS, once the session has started. The ⋮ menu lists every panel of the dock (Work in the v2 chat view, Agents, Plan, System context, Files, Terminal, Browser and Images), each with its icon, its name and a check mark while it is open. Below a separator it offers Keyboard shortcuts and spectro doctor. The code graph rows stand above them, and the folder chip menu has the same rows under its four actions. A folder without `graphify-out/graph.html` gets one row, Build code graph, which opens the build sheet on a full build. A folder with a graph gets two: Show code graph asks the Browser panel for the graph the same way the Graph ready chip does, and Update code graph opens the sheet on an update. A graph page too large for the server to serve counts as no graph. When graphify is not installed, Build and Update show the install command and the sheet cannot start; Show still opens the graph that is there. You set the language in Settings, on the General tab.
- **Under the message box**: on the left the plus menu, the view menu (⋮), the image button, the microphone with its device menu and the gear for the permission mode. On the right the **model chip** (provider and model), the **thinking** chip with its levels and the **context ring**, which appears once tokens flow. Every menu in this row opens upward. A stored session that you can continue shows the model chip with its menu before you send, set to the provider and model its last run used. A pick there makes the session live again without sending a message, its next run uses the picked pair, and the pick becomes the default for new chats, as a pick in a live session does. Opening that menu asks the recorded backend for its models, bounded by the connect limit that the stop button entry below names. A saved session that opens read-only (an import, a scenario, a session another window holds) shows its model and its ring in the bar at the bottom, without a menu.
- **Tab bar**: chat · spectrum · trace · graph · text · lab — six lowercase lenses over the same state (the labels are wire vocabulary; trace carries a live frame count). Entering a fleet swaps the whole row for the fleet's own bar — chapter 15b.

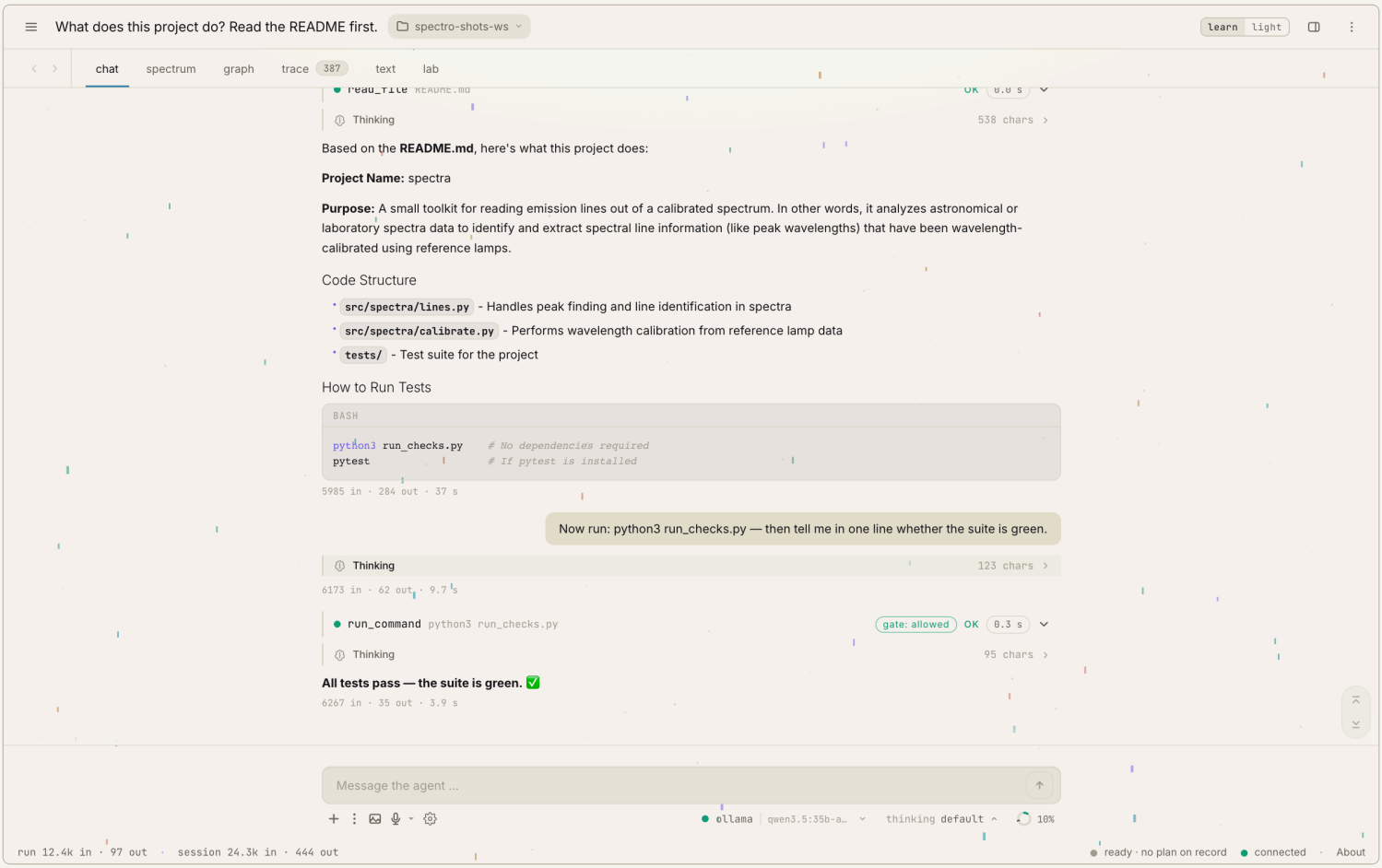
- **Footer**: run and session token totals in tabular numerals, a run status dot, and the connection dot.

Behind the scenes, every incoming socket frame is batched once per animation frame (a `text_delta` flood costs one React render per frame, not per token) and folded by one pure reducer into an immutable `UiState`. Live stream and archived replay produce the same state shape from the same code — which is why every view in this chapter works identically for both.



The state pipeline of the session in view: its socket, one rAF batch, one pure reducer. Every tab is a lens over the same UiState.

The conversation

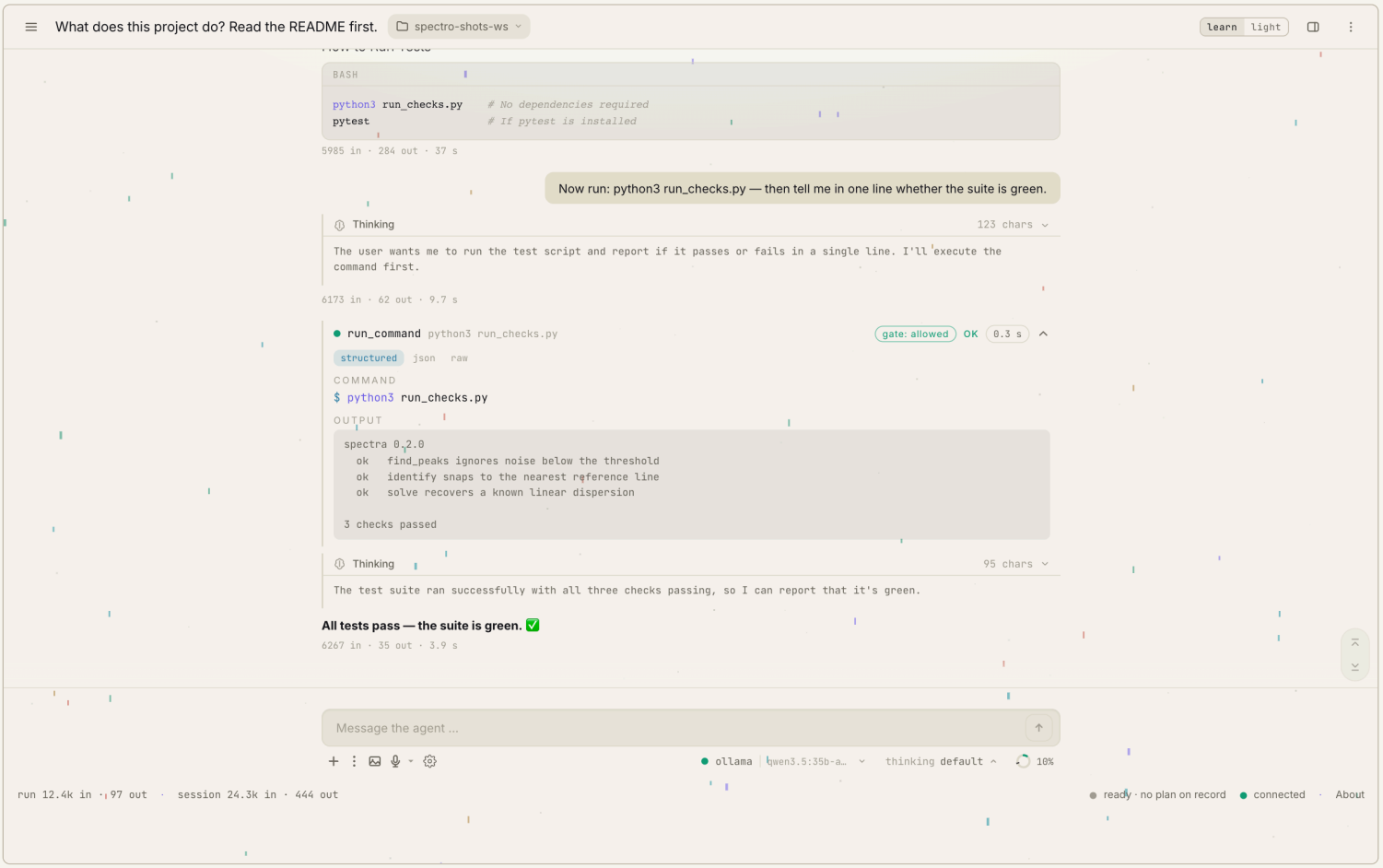


A real two-turn run against local Ollama: your prompt as a flat bubble on the right, the answer as bare text on the left, the finished reasoning collapsed to its character count, and each call as a hairline: read_file plain, run_command carrying "gate: allowed". The faint line under every turn is what it cost: tokens in and out and the duration.

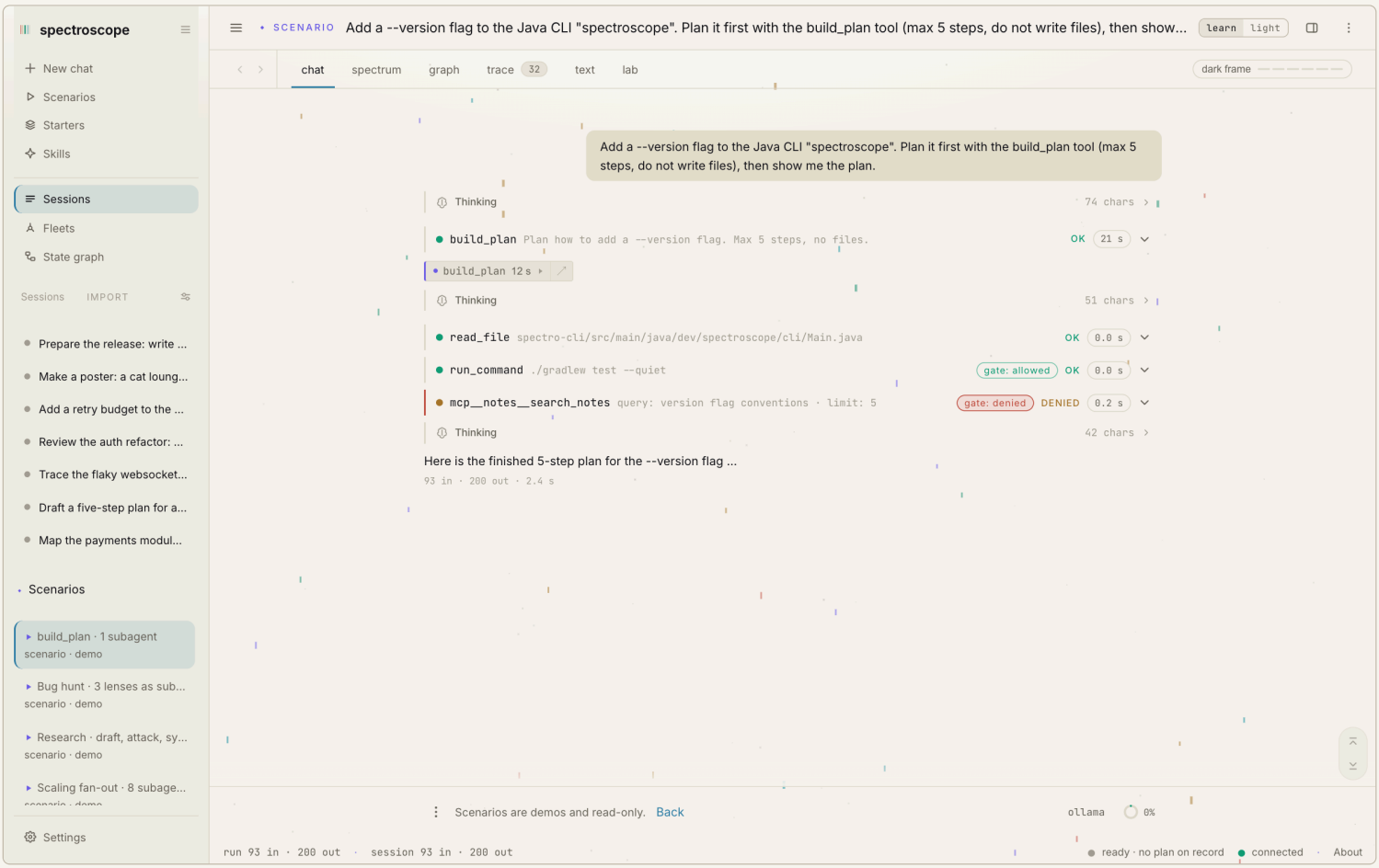
- **Markdown answers.** Assistant turns render through a dependency-free markdown parser (headings, streaming-safe fenced code with language chip and copy button, nested lists, GFM tables, quotes). Nothing ever becomes raw HTML, and link protocols are vetted — `javascript:` renders as plain text. Model output is untrusted end to end.
- **Thinking, apart from the answer.** While a reasoning model thinks, a pulsing "thinking …" dot is the live indicator; the finished reasoning collapses into a disclosure above the answer showing its size. The thinking chip under the message box sets the level for the next prompt, including off when the model has an off switch (`set_reasoning`).
- **Tool cards.** One collapsed card per call: status dot + name + input preview + **gate chip** + duration. The gate chip is the didactic point — *Gate: allowed / denied / waiting* appears only for permission-gated calls, so the three control paths (free tool, gated tool, hook-blocked call) read differently at a glance. Expanding gives the call **three faces**: `structured` draws it as the thing it is (a shell call as its command and its output, a read as the file and its content), `json` as collapsible input and

output trees, and `raw` as the untouched pair — with a copy button for the output beside them. The face is a reading mode rather than card state: switching it on one card switches every card, and the browser remembers the choice. Where an imported call handed back a picture, the picture sits under the body — the section after this one.

- **Subagent threads.** Consecutive turns of one child nest under a pastel-edged thread block (header: agent id, role label, task). Interleaving splits threads honestly — the Trace keeps the flat truth. More in chapter 14.
- **Info and error lines.** Spawns and compactions appear as quiet info lines (localized live); a failed run gets an error card with a “Send again” link.



One call opened: the face row reads structured · json · raw, and structured has drawn the shell call as COMMAND and OUTPUT instead of a JSON pair. The head keeps the gate chip, the OK and the duration, so a shut card still says what happened.



Subagent threads: the planner child the build_plan scenario spawns rides in its own edged block, here shut to its id and its 12 s. The parent's own calls read on around it, each carrying its gate chip — an allowed run_command, a denied MCP call.

Pictures out of an imported transcript

An imported Claude Code transcript usually carries pictures — a screenshot pasted into a prompt, a browser tool handing back what it saw — and the importer used to read the base64 block, count its bytes, write `[image/png · 31.0 KB]` and drop the bytes on the floor. The bytes are in the file and are already in memory by the time anything reads them, so they now ride on the frame itself (`attachment_image` , a type that exists for imports and never crosses the wire) and are drawn where they belong. That old sentence survives as the picture's label. Chapter 8 covers the import itself.

- **In the bubble.** A picture that arrived with a message sits above its words as a bounded thumbnail (at most 160 px wide, 120 px tall). Where the record said nothing else — no sentence, just the screenshot — the picture *is* the message and gets a bubble of its own instead of gluing itself to whatever was typed next, which is a different message. A picture pasted into a prompt that was queued behind a running turn arrives the same way.
- **On the tool card.** What a tool handed back is drawn under the card's body, bounded at 320 px tall and loaded lazily. This is the common case by a wide margin: of the 8,788 image blocks in the corpus this was measured against, roughly 7,300 sat in a tool result rather than in a person's message.

The same picture appears twice more — as its own section in the trace row that carried it (chapter 6), and on the Lab's agent card (chapter 7). In the bubble and on the card it also opens: the cursor becomes a magnifier over a picture that does, and an accent hairline draws itself on hover. Clicking one opens the **lightbox**, which is mounted at app level rather than inside the chat, because the walk has to cross surfaces. `←` and `→` step through every picture in the session in stream order — the ones a message brought and the ones a tool returned, interleaved as they happened — wrapping at both ends, with a counter saying which of how many; Esc closes. The bar above the picture names where it came from (“from a message”, or “from” and the tool's name) and carries the file's own note beside it.

A picture also has **three faces**, in the trace's own idiom: `picture` , `base64` and `file` . `base64` is the string itself, headed with the media type and its character count, copyable in one press. `file` is the `.jsonl` line the picture lives on, with three lines either side for orientation — the neighbours clipped, the line itself cut into segments of readable JSON with every base64 blob standing in as a measured absence you can click to land on the string. Every blob, not only this picture's: one record often carries several, and the head says how many. Nothing is fetched for any of this, because the import holds the file's lines byte for byte and knows which line produced which frame. The face survives a step to the next picture, deliberately — comparing where three screenshots sit in one file should not cost three presses.

Two buttons finish the bar. **save** writes bytes that are already in the page, so nothing leaves the machine and the picture really is a file afterwards. **folder** opens the *transcript's* folder, and its tooltip says why it is not the picture's: the picture is not a file on disk, it is base64 inside the `.jsonl` . That button appears only for a session opened from the store — a picked file and a paste have no address behind them, though both still get the `file` face, because the lines came in with them.

WHAT THE PICTURES DO NOT DO

Nothing reads what is *in* a picture: no OCR, no caption, no search by what a screenshot shows, and the viewer neither zooms nor pans. Generated images (chapter 17) keep their own Images panel in the dock and stay out of this walk. Handed over and painted here are different provenances, and the app refuses to blur them. And the frame is import-only by construction, so all three JSONL formats an export can write drop every picture; only the HTML export carries the bytes out.

Messages between agents

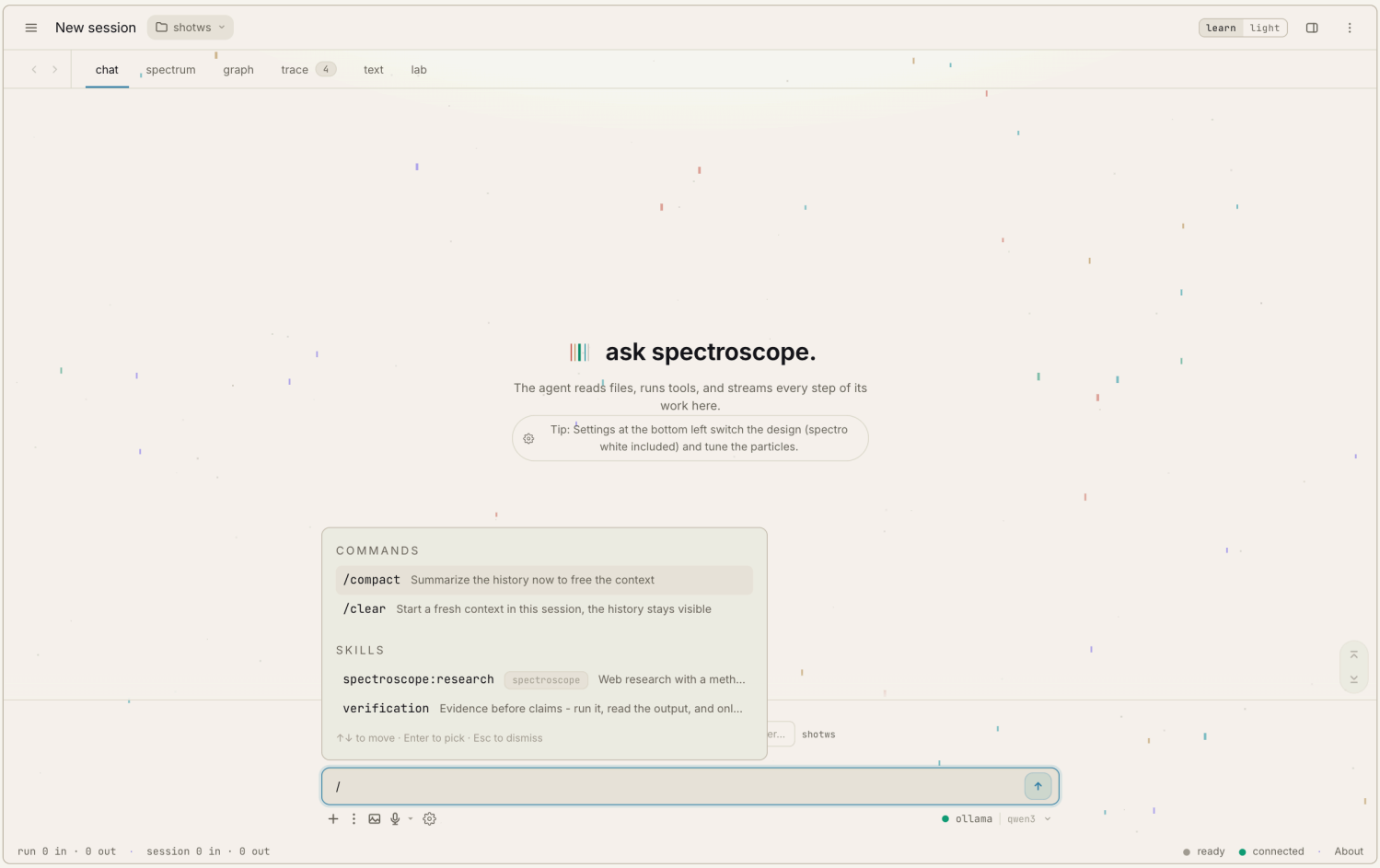
Sessions on this machine can message each other, and a run's children can message their parent (card 503, tools in chapter 9). Each message is a block in the chat: who sent it or who it went to, the time, and its status (queued, received, held, delivered, declined, expired, refused or dropped). A message from a session that runs with other permissions than this one waits for you: the block says so and offers Deliver and Decline, and the model here reads it only after Deliver. Every received message has Reply, which puts the sender above the message box; what you send then goes to that agent as your message, not as a prompt to the model. A message between the parent and a child sits in the child's thread and once more in the main thread as one line. The rail lists the sessions other processes host on this machine, with their folder and what they are doing. A stored session shows its messages again when you open it. The switch `agentMessaging` in the settings menu of the composer turns all of it off.

The composer

- **Enter sends, Shift+Enter breaks the line** — the only shortcut. The textarea autosizes up to 150 px; while a run streams and the box is empty, the Send button becomes the stop button (one run at a time per connection).
- **Image attachments.** The image button opens a file picker (jpeg/png/webp/gif, multiple); dragging files anywhere onto the chat works too. Every image is downscaled in the browser before sending (longest edge 1568 px, JPEG re-encode when useful; GIFs pass through to keep animation), previewed as removable chips, and never leaves the page until you hit Send. Attachments cross the socket as base64, land in the session's blob store, and the events carry only references.
- **Push-to-talk.** The mic button records (coral pulsing dot + `m:ss` timer), a second press stops; the audio is transcribed server-side by local whisper.cpp and the text lands *in the input field* — an STT error is reviewable text, never an agent instruction. If STT is not installed the button disables itself with a setup hint. Chapter 17 has the setup.

Commands

The composer's commands come from one list that the server sends, the same list the terminal's `/help` prints (chapter 3). The chat offers `/help` (also `/commands`), `/compact`, `/clear`, `/loop`, `/graph` and `/skills`, and in developer mode the playbook commands of chapter 15d. Type `/` at the start of an empty message and the picker lists the commands above the session's skills and commands, each with one line of help. A click runs a command. Enter and Tab write the highlighted command into the message box, and Enter on a message that is exactly the command runs it, so a slash and a stray Enter start nothing. A command that takes words, such as `/loop`, is only written into the message box. A command with subcommands lists them once its name is typed: `/graph` shows `/graph build`, `/graph ask` and the others, and the second word narrows the list, so `/graph b` leaves `/graph build`. A row that takes words shows its template under it, for example `/graph ask <question>`, and while you type those words the picker shows only the template and its help. Help and templates come from the same list, in English and German. The picker offers command rows only while the cursor is at the end of the message. With the cursor inside a message whose first word is a command, the picker offers nothing for that word, and Enter sends the message as it stands. Esc puts the list away, and a skill typed later in the same message opens its own list. Commands are read in any case, so `/GRAPH ask` is `/graph ask`. A message counts as a command only when it is exactly a command that takes no words, or starts with one that takes words. A sentence such as `please /clear the cache` is sent to the agent as written. `/help` answers with a list in the chat, and `/skills` opens the Skills and Commands page. No command reaches the model as text, except the question of `/graph ask`, which runs as a prompt, and while a run is active `/compact`, `/clear`, `/loop` and `/graph ask` answer "A run is already active, stop it first." Until the list has arrived the picker says “Loading the commands …”, and a message that starts with a slash waits instead of being sent. The commands work in learn and in light; the playbook commands need developer mode.



The picker on a slash: the two commands above the skills

/COMPACT

Summarizes the conversation now instead of waiting for the automatic compaction. The agent keeps the last four messages and a summary of everything before them. The summary is a model call, so while it runs the chat shows "Compacting the history", the footer reads "compacting" and the stop button takes the send button's seat. A sentence you type meanwhile waits and goes out once the compaction is over. Stop cancels the summary and leaves the history as it was; before the backend has sent its first byte the stop waits for it, as it does for a run. A stop that arrives after the summary is done comes too late: the history is compacted, and the chat says so. When it is done the chat shows the same line an automatic compaction leaves, for example "History compacted: 4 turns summarized into 50 characters", and the session file records a `compaction` event. With a short history there is nothing to fold, and the chat says so in one line. A compaction that fails changes nothing and leaves no line in the session file.

/CLEAR

Gives the agent a fresh context and keeps the session. The session id, its file and its llm-wire stay the same, so a session bound to one machine stays bound to it. The agent keeps its system prompt, its goal and its tools and forgets the conversation and its plan. The chat keeps every turn on screen and draws a divider "context cleared" with the time; the next prompt is sent with an empty history. The context ring stays on screen: until the next answer reports its usage, it shows an estimate of the system prompt and the tools (their characters divided by four). The session file records a `context_cleared` line, and a reload or a later resume starts the history after it. The terminal's `/clear` does the same (chapter 3).

/LOOP

Runs a prompt again and again (card 504). `/loop 5m check the build` runs the prompt now and again every five minutes; an interval is written as seconds, minutes or hours (`90s` , `5m` , `1h`) and is held between one minute and one hour. `/loop watch the build` , without an interval, lets the model pace itself: after each tick it calls `schedule_wakeup` with a delay and a reason, or ends the loop. `/loop stop` ends the loop, as Stop on the chip does. The section below says how a loop runs and ends.

/GRAPH

Works with the code graph of the session's folder (card 587). `/graph build` builds it and `/graph update` brings it up to date, as the Build code graph row of the folder chip does, without names for the communities. The chat says when the build is done, with the number of nodes, edges and communities; says the graph is already current when the structure of the code is the same as at the last build; says only that an update finished when graphify's output names neither; or names the exit code and the last line of a build that failed. These lines stay in this window, and after a reload the chip in the header shows how the build ended. `/graph show` opens the graph in the browser panel. `/graph ask <question>` sends the question to the model with a line that asks it to answer through the `code_graph` tool (chapter 9); where the folder has no graph or the program is not installed, the chat says so and how to go on. `/graph help` and `/graph` alone list the subcommands, and `/codegraph` and `/graphify` work the same as `/graph` . While the folder has a graph, the session lists the skill `code-graph` , built into the app, beside its own skills, and the list follows a build or a deletion within a few seconds.

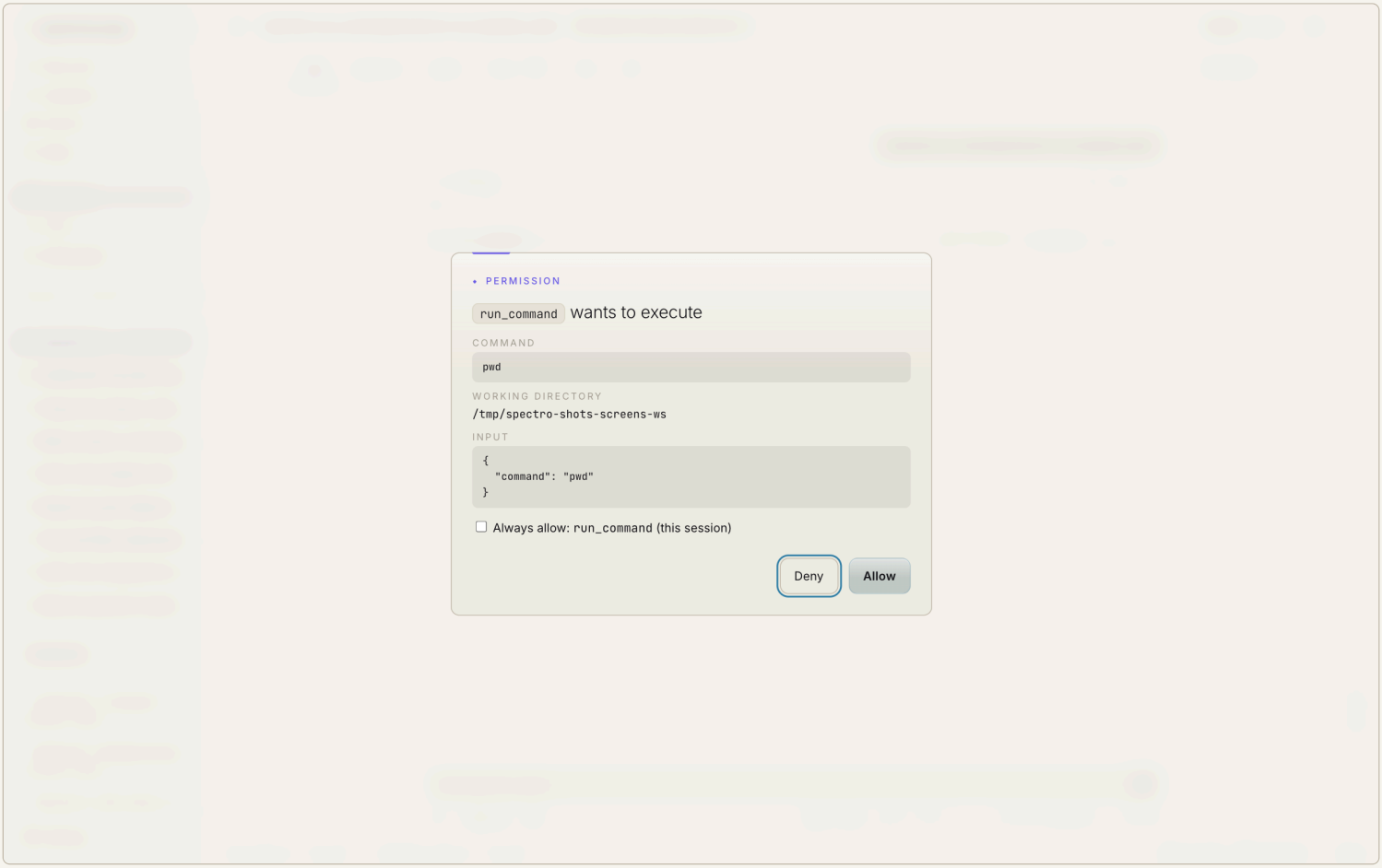
Goal and loop

The settings menu of the composer has a Goal section. Write what should be true and a check command, for example "The tests are green." and `npm test` , and press Set goal. The goal holds for every run of the session until you clear it, and it is kept in a file under `~/.spectro/goals` , which the section names. The harness runs the check when the model ends a turn without a tool call and nothing else waits for it: no sentence you typed during the turn, no message from another agent, no result of a background child. It also runs when the run reaches the turn cap or the reply stops at the output limit; the verdict is shown, and the run ends anyway. It does not run when you press Stop, when the run ends with an error, when the progress guard ends the run, or after a call to `schedule_wakeup` , which ends the run at once. If the check fails at a turn end below the cap, the run goes on with the check's output, and the chat says so. This does not use the leash's continuation budget: the turn cap stays the ceiling, 1,000 turns unless the session sets another. The run also ends when the check fails with exactly the same output twice in a row and no tool call succeeded in between. A check that prints its own duration, as most test runners do, never repeats that way, so such a goal runs until the check passes or the run reaches the turn cap. A line under the composer shows the goal and the verdict of its last check. While children of a background fan-out still run, the check waits; after 30 minutes, then 60, then 120 and every 120 after that, the model gets a check in that lists each child with its last status line. When the backend refuses the key, the account cannot pay, the context is full or the model is gone, the goal clears itself, its file goes, and the chat says "Goal cleared after an unrecoverable error". A failure that may pass, a message you type into a running turn and the turn cap leave the goal in place.

A loop lives in the server process that holds the session. Each wakeup lands on the whole minute at or after its delay. A wakeup fires only when the session is idle and a window has it open. One that falls due while no window is open waits, and fires when you open the session again. The chip under the composer shows the turn of the run in progress with the turn cap, when the loop wakes next and why, and Stop. The loop ends when you press Stop (in the chat or on the chip) or type `/loop stop` , when the model stops it with `schedule_wakeup` or ends two ticks in a row without a wakeup (after the first, the harness wakes it once more after 20 minutes), or seven days after it started. The chat names the reason, also for a loop that ended while no window had the session open: the line appears when you continue the session. A wakeup prompt the model wrote itself shows in the chat behind a line that says so. In a loop the goal is checked only on a tick whose run ends without `schedule_wakeup` . In a self paced loop that is a tick where the model sets no next wakeup. A loop with an interval needs no such call, so its check runs at the end of each tick unless the model stops the loop with `schedule_wakeup` . The record of a loop is `~/.spectro/loops/<session-id>.loop.jsonl` , one line per start, wakeup armed, wakeup held, wakeup fired and end, and one per goal check in. The switch `agentMessaging` does not touch a loop.

The permission gate

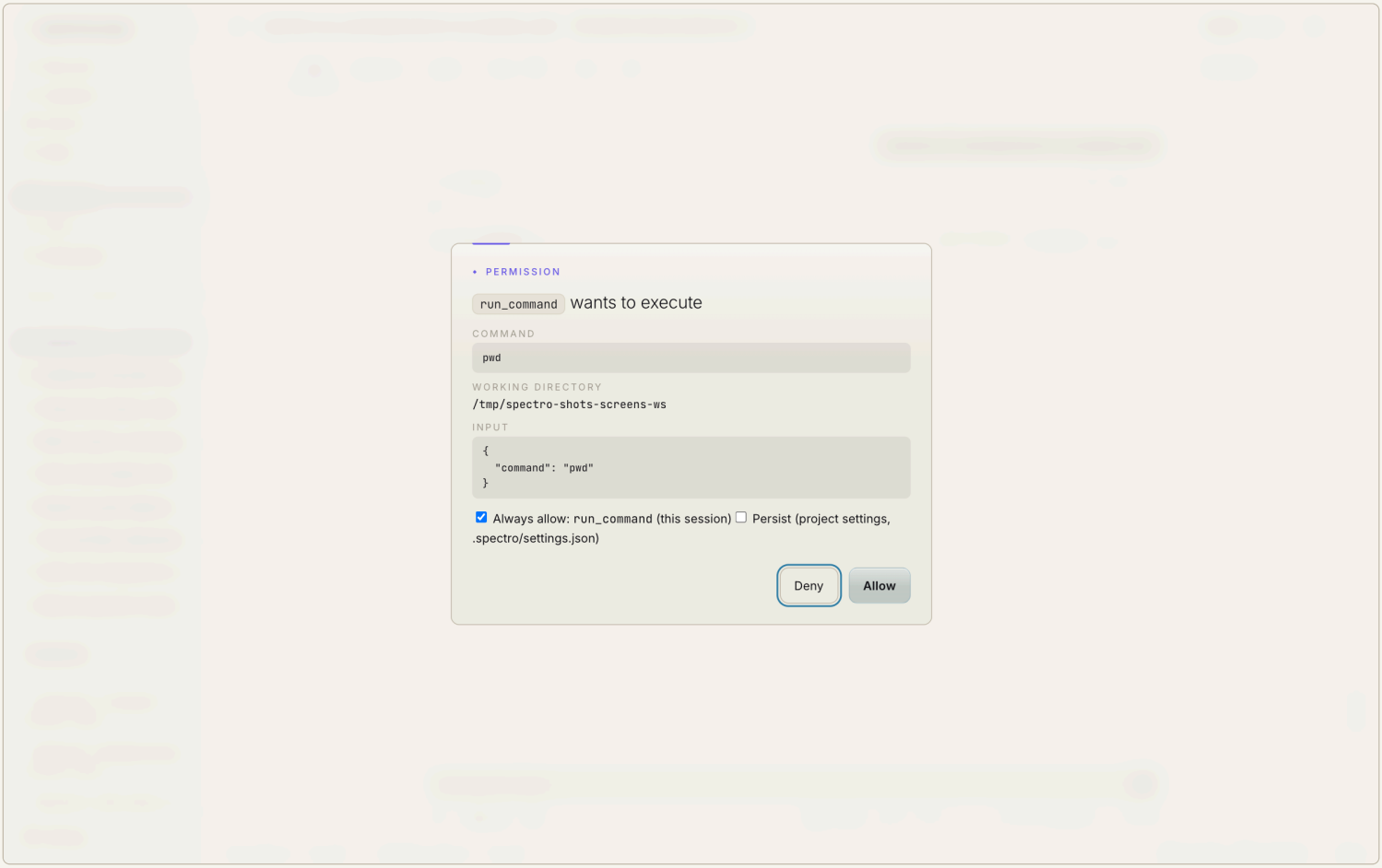
When the agent wants to run a gated tool and no allowlist rule covers it, the run genuinely pauses (the agent's virtual thread parks on a future server-side) and the **permission dialog** opens over the window. The scrim never closes it, so the decision has to be deliberate.



The serious moment: run_command wants to execute pwd. The dialog names the command, the working directory it would run in and the input in full. Deny sits left of Allow and holds the focus.

- The dialog leads with what the call reaches for: the command, a path, a URL, a server's tool or a skill. Below that stands the working directory and the input in full, as pretty JSON with every multi-line field as its own block. The box scrolls and the text is never clipped. You approve what you see, never a summary.
- Deny is the safe default: it holds the initial focus, and Esc denies too. Tab stays inside the dialog while the run waits.
- A subagent's request says who asked: “requested by” its id, the agent that spawned it and the task it was handed, so it is never read as the main agent's.
- Further requests queue behind the one on screen and say so: `1 of 3`.
- The dialog also keeps the **record**: the last six decisions of this session, each as allowed or denied with its tool and its agent.

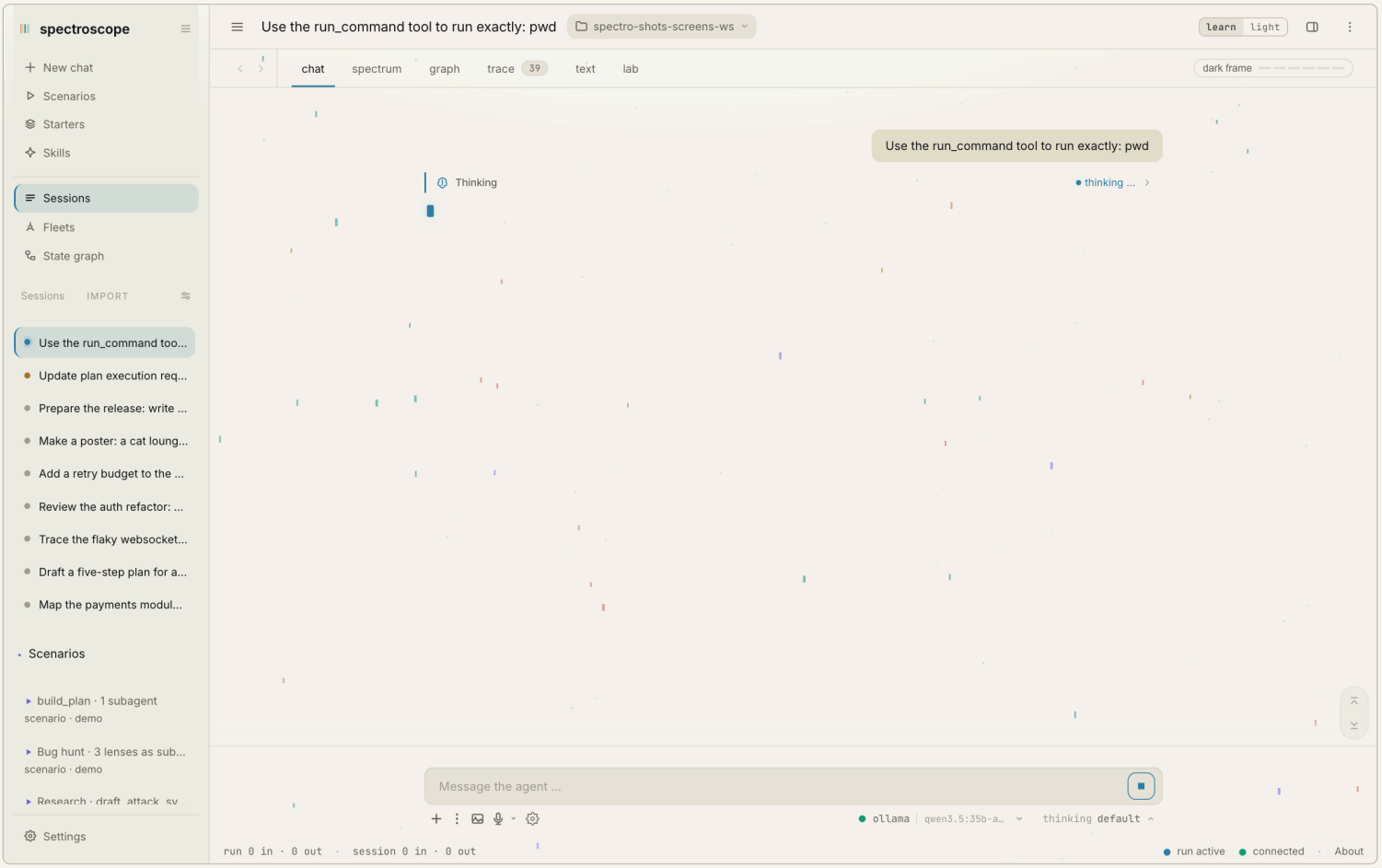
The two checkboxes appear one behind the other:



With “Always allow: run_command (this session)” ticked, the second box appears beside it: persist the rule in the project settings. Only Allow carries the two flags.

Always allow (this session) remembers the approval for this session only. **Persist**, which appears once the first is ticked and only for a session that has a real workspace, appends the rule to that workspace's `.spectro/settings.json`. That is the same `autoApprove` list the CLI reads, so a web decision is honored by the terminal on its next run. Remembered rules are **prefix-scoped** for risky tools (a `git status` approval becomes `run_command:git*`, never “all commands”); chapter 13 has the exact rules. A denied call returns `ERROR: the user denied the execution.` to the model, which reads it and adapts.

Live indicators



A live run against a local model through ollama: the reasoning streams into the Thinking disclosure, which reads “thinking …” while it lasts. The stop button sits in the message box while the run lasts, and the model and the thinking level sit in the row under it.

- The **context ring** at the right end of the row under the message box fills with the last reported input tokens against the compaction threshold (green < 70%, amber ≤ 90%, red above); clicking it opens the introspection popover above the ring (chapter 28).
- The **usage footer** counts run and session tokens live in tabular numerals; the status dot switches “run active → ready” and names any non-regular stop reason.
- The **connection banner** appears under the header when the socket drops, with a reconnect countdown (exponential backoff 1→15 s) and a “Reconnect now” action. Sessions survive: the JSONL file is the state, the socket is just transport.
- The **stop button** in the message box sends `abort`, which fires the run's CancelSignal — the run ends with `stopReason: "aborted"`, tools and children included. It also ends a run that waits for a backend that does not answer: while the connection is set up, while a connected server sends nothing, and in the pause between two attempts, for the OpenAI-compatible providers, Ollama, Anthropic and Copilot. No new attempt starts after Stop, and the session takes the next prompt at once. Without Stop, the OpenAI-compatible, Ollama and Anthropic providers give up connecting after 10 seconds per attempt, and a run makes up to three attempts.

CHAPTER 6

The web face: tabs & panels

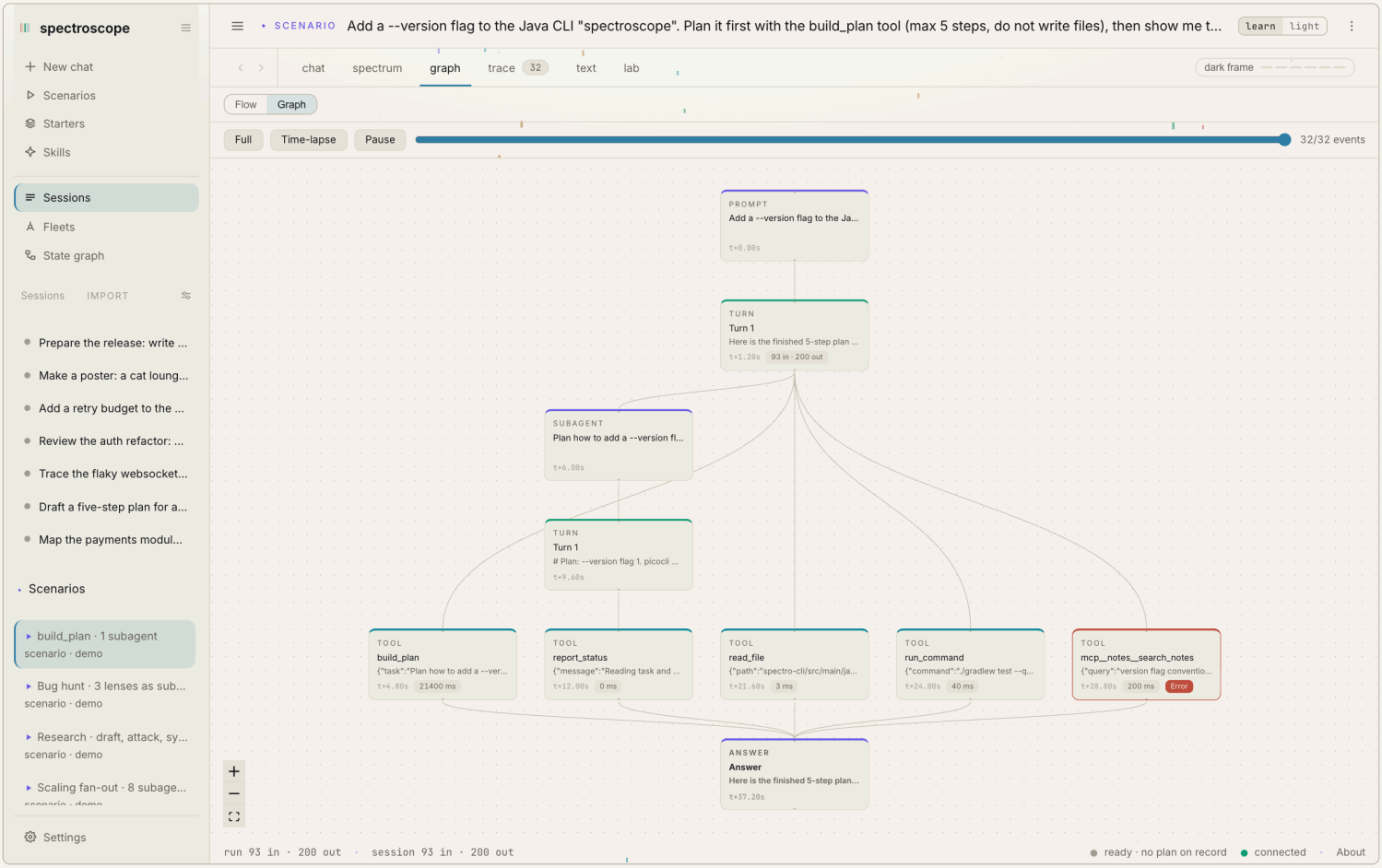
Chat is one lens. The Graph tab shows the same run as a process diagram, the Trace tab as wire frames, and the right-docked panel adds the agents roster, the live plan, the exact system context and a sandboxed file browser.

The Graph tab

Two views behind a *Flow* | *Graph* toggle (sticky per browser). For archived sessions a replay bar adds *Full*, *Time-lapse* (advances along the real timestamp gaps, accelerated), *Pause* and a scrubber.

Flow view: the whole session as a BPMN-style overview — activities on the main spine, one parallel split/join block per subagent wave, gate outcomes as dots.

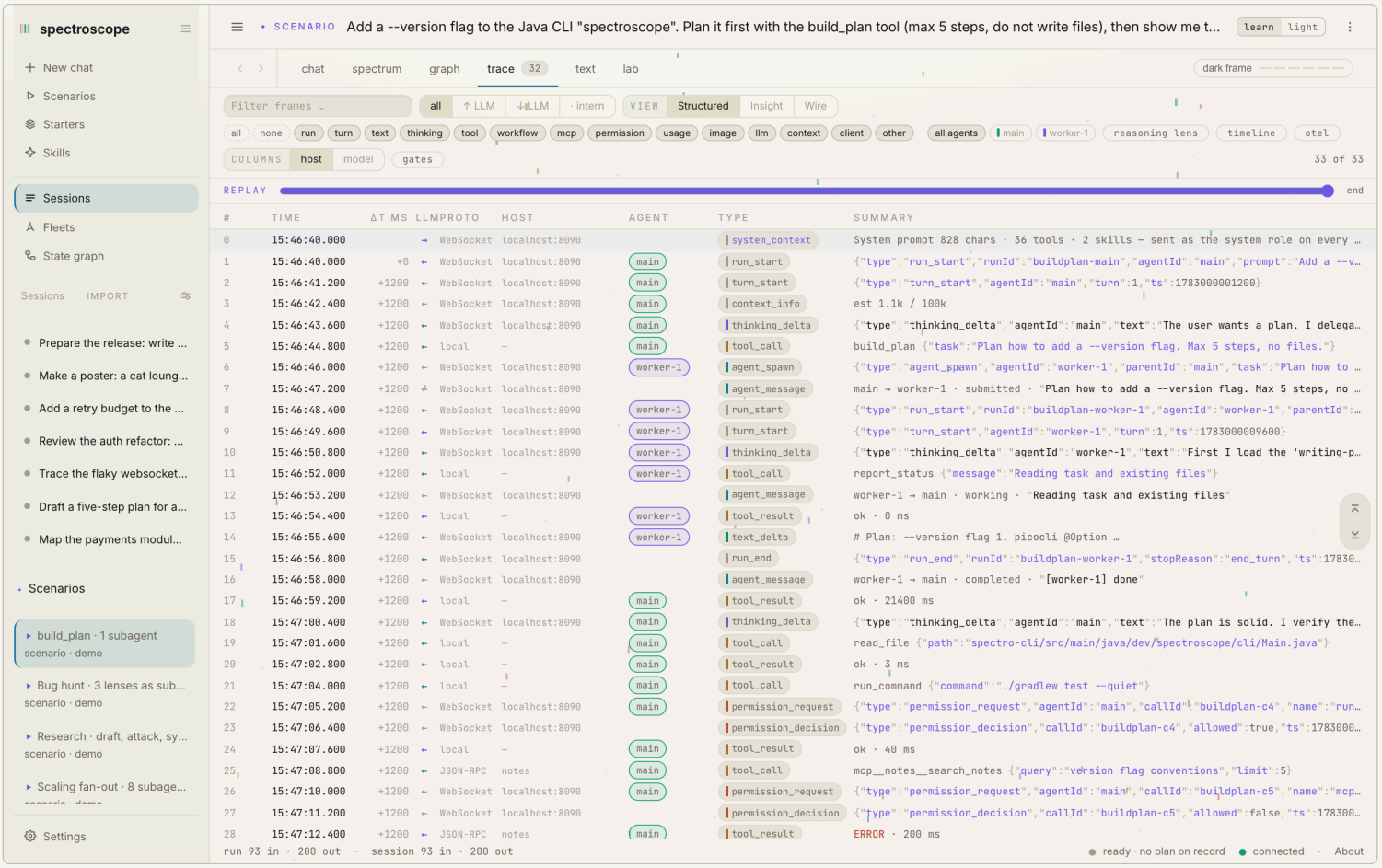
Flow is the minimap for humans: rounded activities (*user / think / tool / say / compact / end*) stack on the main spine; each subagent wave becomes a split block with one column per child and a join bar. Think and say activities accumulate their streamed text, so real runs get sentence labels instead of “token one”. Tool activities label themselves honestly — `read <file>`, `$ <command>`, `fetch <url>`, `server · tool` for MCP. Gate outcomes render as small dots (pending / allowed / denied). In a replay, clicking an activity seeks the timeline to its first event.



Graph view: the dagre DAG — prompt, turns, tools, subagent lanes and the answer; every node carries timing, token badges and status.

Graph is the classic DAG (React Flow + dagre): one node per prompt, turn, tool call, subagent and answer, with `t+` relative times, duration and token badges, error badges, and a pulsing outline for running nodes. Sequentially spawned subagent subtrees get their own side-by-side lanes. Clicking a node opens a detail panel with its timing line and its *raw evidence* — exactly the events that formed the node, each as a collapsible JSON tree. The right mouse button pans; the left button only clicks — on both React Flow canvases in the app.

The Trace tab



> The wire view of one run, 33 of 33 frames: clock, Δt, LLM direction, the proto and host each payload really rode, agent, type and a token-highlighted summary, with the synthetic system_context row first. The face row above sets every row at once, the chips filter by category — image among them — and in the columns control model is switched off, so that column is not in the rows at all. The slim rail on the right edge jumps to the first frame or the newest.

"What Wireshark is to packets, this is to the harness protocol." Every frame lands here — RunEvents inbound, client messages outbound — before the reducer even looks at it, so unknown event types are visible too.

- **Columns:** sequence · wall-clock time with milliseconds · Δt since the previous visible row · **LLM direction** (↑ going to the model with the next request, ↓ produced by the model, · internal) · **proto** · **host** · agent · type · summary. Host and **model** — the model serving each run — are the two optional ones, switched from the toolbar's columns control; a column that is off is left out of the row entirely.
- **The proto column** names the wire each payload actually rides, and it answers for the *frame* rather than for the work behind it. Most rows therefore read `WebSocket`: a `text_delta` is our own frame about the model's output, not the model's output arriving, and it never left this machine. Only a recorded exchange says what went out to the provider — `HTTPS/SSE` for the cloud streams (Claude and OpenAI-compatible), `HTTPS/NDJSON` for Ollama, plain `HTTPS` for the one-shot calls (transcription, an image) and `process` when the provider runs as a local child process. Tool rows name their *execution* transport instead: `JSON-RPC` for `mcp__*` (a result resolves its tool through the callId), `HTTP` for `web_fetch` and image generation, `local` for the file tools.
- **The host column** names the network counterpart. A frame that only rode the socket names this server's own origin; an import, which cannot know the machine the other app ran on, leaves it at a dash. LLM rows show where the request really goes (`api.anthropic.com`, `localhost:11434` …) — live from the socket-only `provider_info` frame the server sends on connect and after every provider switch, so a switch is a visible trace row, never a silent client-side swap. MCP rows name their server, `web_fetch` the fetched URL's host, image events their backend endpoint; replays only know the provider, so anthropic maps to its fixed endpoint and local backends show an honest `—`.
- **Filters:** free-text (matches type, agent and payload), an LLM-direction segment, and fourteen toggleable category chips — run, turn, text, thinking, tool, workflow, mcp, permission, usage, image, llm, context, client, other. Tool traffic has three of them rather than one, so the MCP calls and the workflow steps can be taken out without taking the local tools with them.
- **Summaries with token highlighting:** the model's own words in sand, punctuation faint, numbers tabular, the literal `ERROR` red — lossless, the full payload is one click away (collapsible JSON tree + copy).
- **The synthetic `system_context` row:** the system prompt rides with every request but is not a wire event — the view prepends one synthetic ↑ row (from `GET /api/context`) naming prompt size, tools, skills and MCP servers. Display-only: never in the reducer, never in the JSONL.
- **Reading flow:** the tab **opens at the first frame** — the story reads from the beginning. While a frame is open, **Space steps to the next visible one** (it opens, takes focus and centres itself; Enter still toggles a focused row), so the trace taps through like the Lab stepper. A slim **jump rail** on the right edge scrolls to the first or the newest frame.
- **Auto-follow** engages once you scroll down to the live end; while you are reading further up, arrivals accumulate into a "{n} new ↓" pill instead of yanking the view.
- **Four faces** per row: *structured* opens a frame as the thing it is (a call as its tool card, an answer as its text), *insight* as an expanded JSON tree, *wire* as plain text — exactly the lines that crossed the wire, one row each — and *source* as the line of the imported file the frame was read from. The view control above the list is a master switch: flipping it brings every row back to that face, and a single row can still be switched afterwards. A face with nothing behind it is left out rather than shown empty, so a recorded LLM exchange offers no *source*; the two retired names, `raw` and `compact`, both land on *wire*. The `session_resume` marker's detail carries the entire re-uploaded history as its JSONL lines — chapter 8.

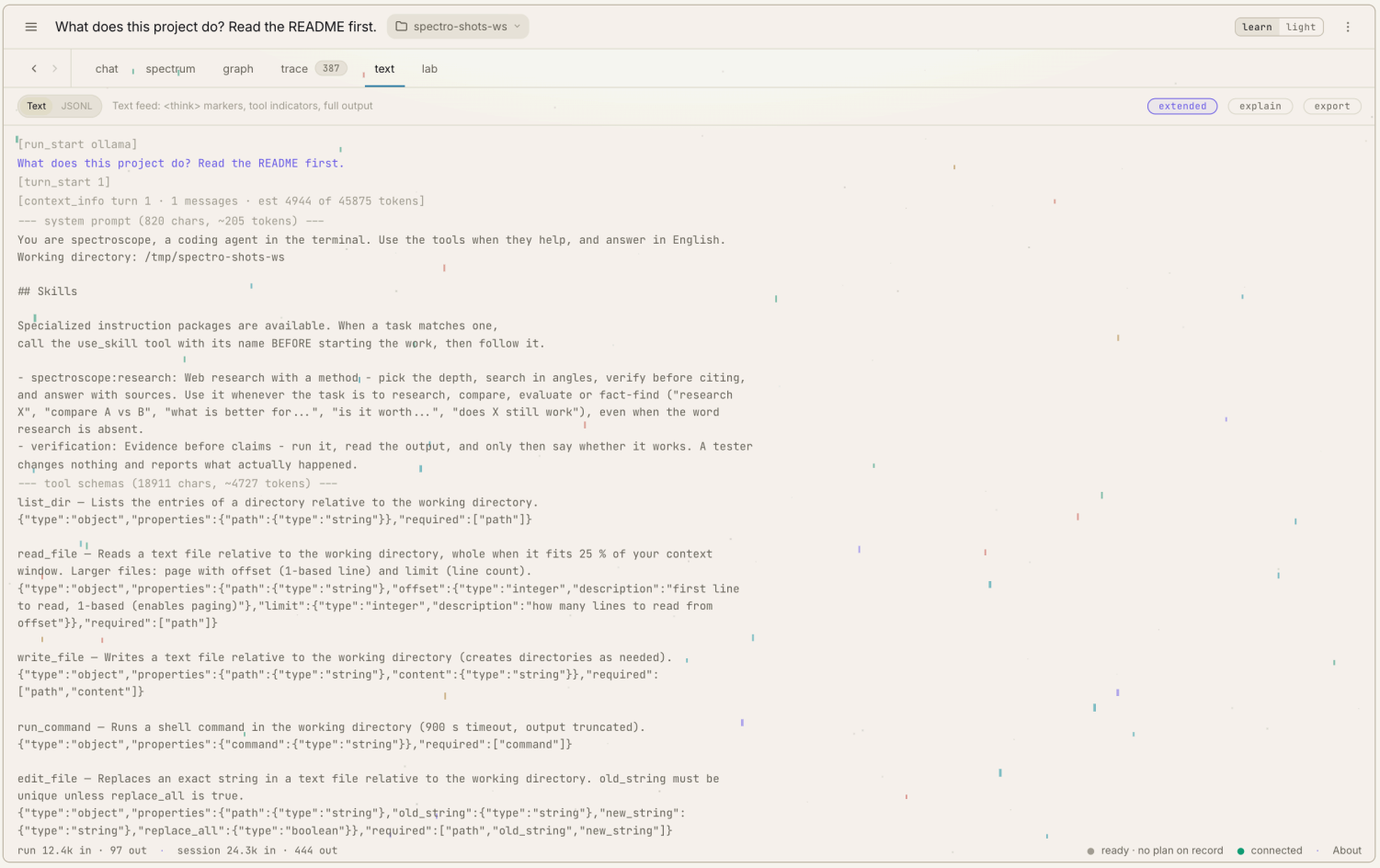
- **Pictures.** A picture an imported transcript carried is a frame like any other (`attachment_image`), and it answers to the `image` chip beside the generated ones — that chip is the one gesture a reader makes to find the pictures, and until it knew this type it hid every one of them. The collapsed row prints the file's own note, `[image/png · 31.0 KB]`, and never the bytes: a row's summary is also the text a find in the page matches, and one 140-picture file put 19.87 MB of base64 there before this had a case of its own. Open the row and the picture is drawn in a section of its own, labelled with the payload field it came out of, that note underneath. An imported picture carries its bytes on the frame, so clicking it opens the lightbox and the arrow-key walk (chapter 5); a generated one is a store URL rather than bytes in the page, so it renders and does not open.

The Text tab

Another lens on the same stream — where the trace shows frames, this shows the *text*. Two views behind a *Text* / *JSONL* toggle (sticky per browser), one copy button for the whole thing:

- **Text** is the whole session as one readable feed, with the protocol made visible as literal markers: every reasoning run sits between `<think>` and `</think>` — exactly the tags Ollama really streams inline; Anthropic's thinking blocks render to the same boundaries — followed by the answer text in plain type. Tool use appears as honest indicators with nothing hidden: `[tool_call read_file {"path":"…"}]`, then `[tool_result read_file · 9ms]` with the *full* output underneath. Run boundaries (`[run_start ollama]`, `[run_end end_turn]`), the permission gate, spawns, A2A messages, compaction and errors all keep their place in reading order; subagent lines carry their agent id as a prefix. Prompts render in sand, thinking dimmed italic, markers faint — but it is all ONE selectable text, and the copy button hands it out as plain text.
- **JSONL** is the session exactly as the file on disk stores it: one compact JSON line per wire event, newline per event — the NDJSON view of the truth. Socket-only UI frames (`provider_info`, `workspace_info`) are deliberately absent: they never enter the file, and this view IS the file.

An **extended** toggle sits above the feed, and it is the honesty control: the reading leaves things out, extended puts them back — the assembled request the model was handed, its parts each headed by a character count and a token estimate, plus the usage lines, the turn boundaries and the published plan. The setting travels into the exported document, so the file says what the screen said.



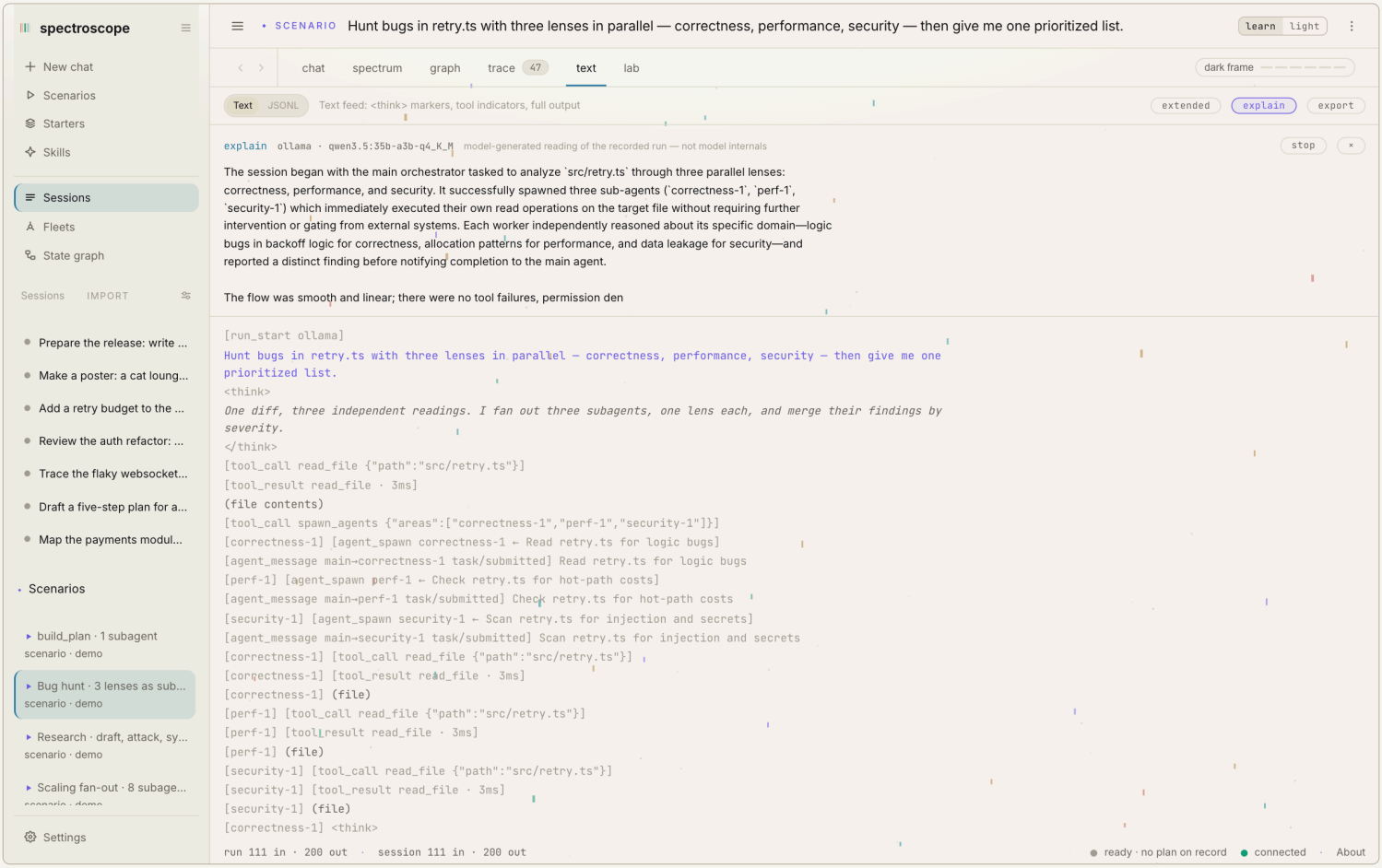
> The text feed with extended on. Above the conversation stands the request as it was actually assembled: the system prompt with its size in chars and estimated tokens, then the tool schemas with theirs, each tool with its real JSON schema. Normal reading starts at the run and never shows any of this.

Explain: have a model read the run

Above the feed sits an **explain** button. It does one thing: it sends a bounded digest of exactly what the Text view shows — the same prompts, reasoning runs, tool calls and gates, nothing hidden and nothing added — to your configured provider, and streams back a plain-prose reading of *why* the session went the way it did. What the agent was trying to do, why each step followed from the last, where it hesitated or was gated, where it recovered. Because the digest is built from the same feed you can read yourself, imported sessions explain just as well as live ones.

The panel is labelled for what it is — *"model-generated reading of the recorded run — not model internals"*. That boundary is not just a caption: it is written into the instructions the explain call runs under. The model is told it is reading a recording, that it cannot see hidden state, and that it must ground every claim in the transcript and say so where the transcript is ambiguous. If you want the deterministic view — which gate asked and why, folded straight from the stream with no model in the loop — that is the gates panel in the Trace tab, and it stays exactly as it was.

One honest caveat: the explain call spends your provider key, like any other model call. It carries the same local-origin fences as the key-writing endpoint (it is a loopback-only, same-origin request), and it needs a provider with a key set — otherwise the button reads “set a key in Settings” instead of running.

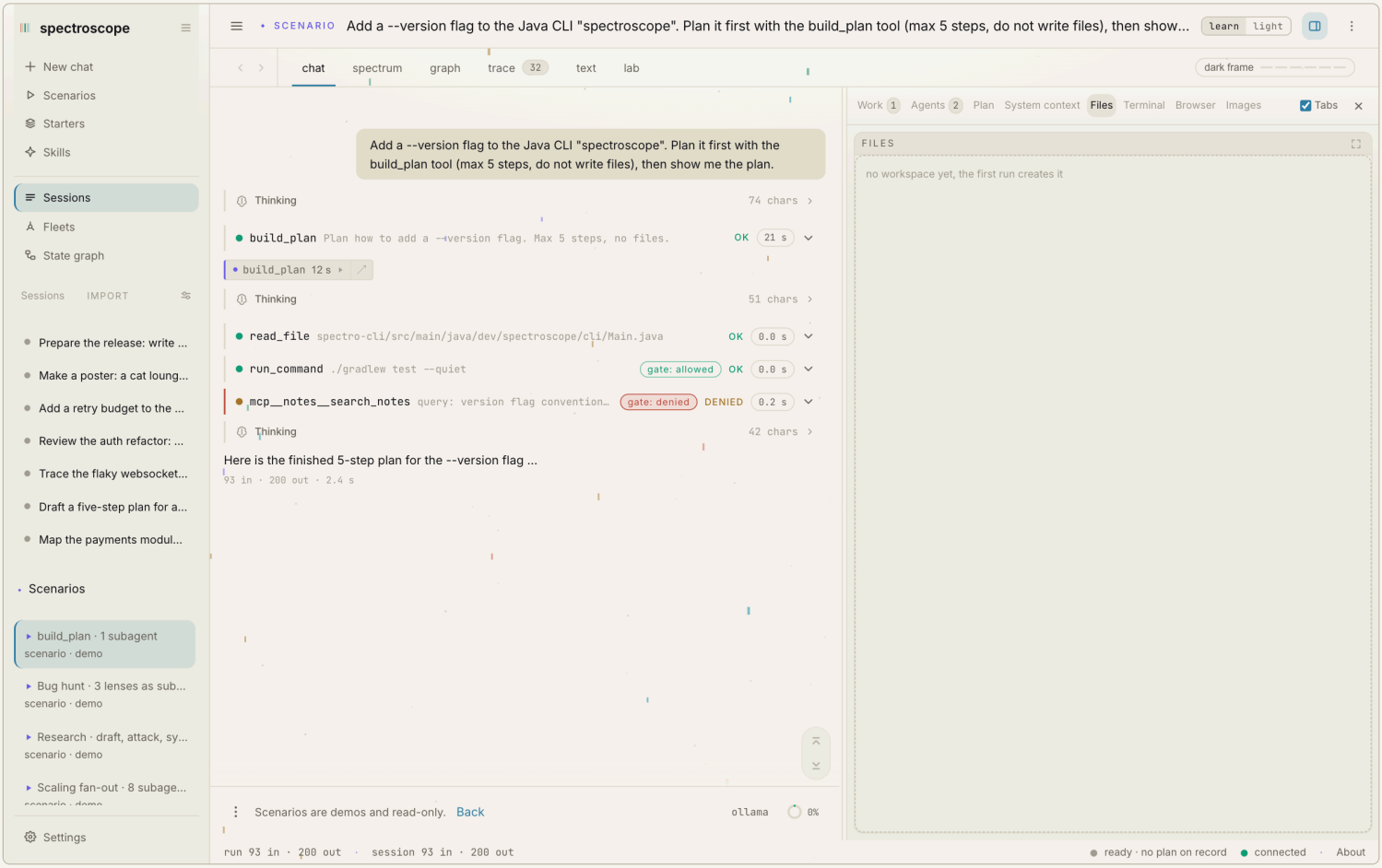


> The explain panel over the text feed: a model's reading of a real run, honestly labelled as an interpretation of the recording — not a window into the model's internals. The deterministic gates view lives in the Trace tab, untouched.

The right panel

The header's panel button opens the dock, a resizable panel beside the chat. Its strip lists the panels: Work, Agents, Plan, System context, Files, Terminal, Browser and Images. A fresh installation shows one panel at a time; an installation that already had a layout keeps the panels side by side, where each click on the strip opens or closes one of them.

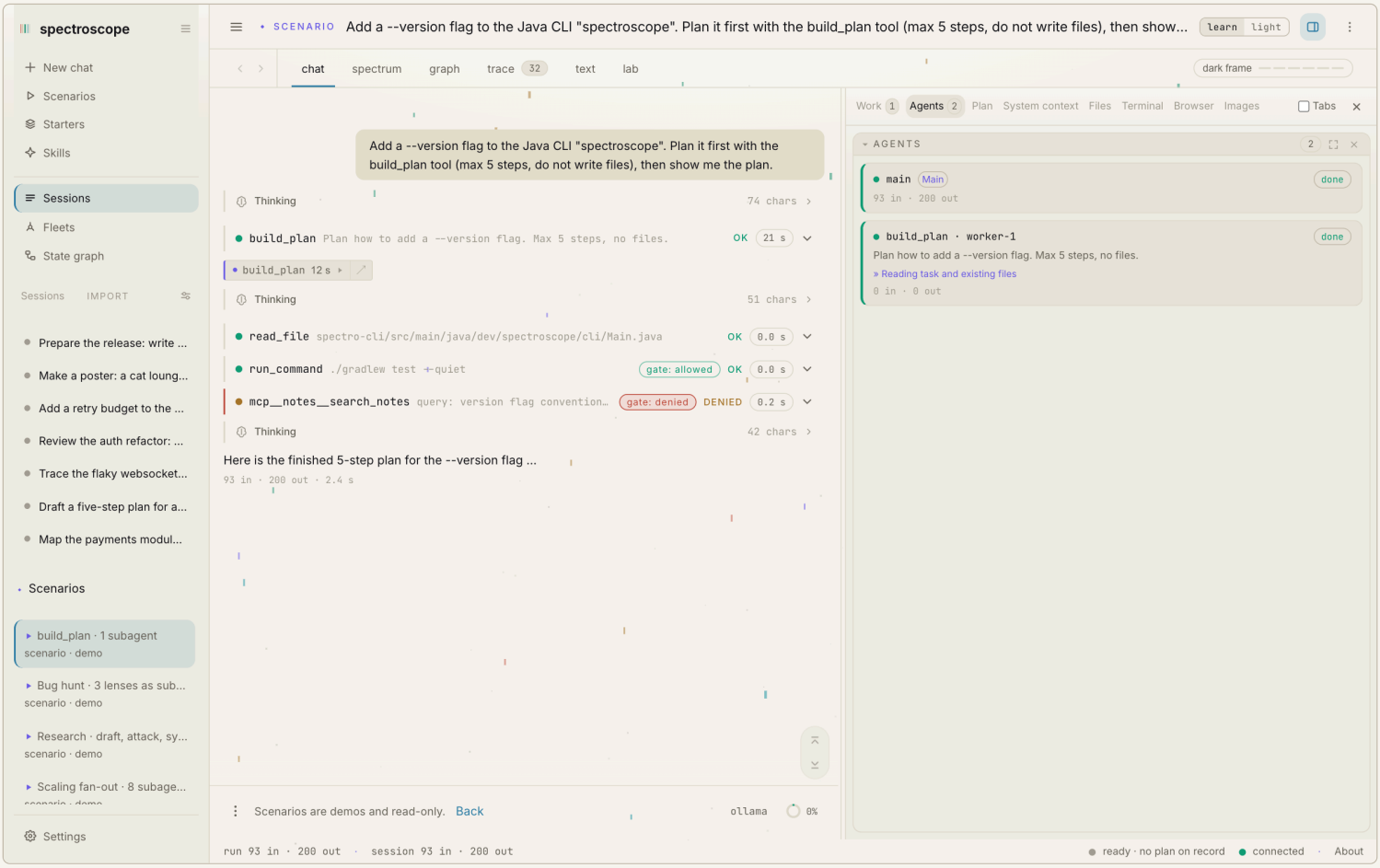
The **Tabs** checkbox at the top of the dock shows one panel at a time instead. A click on Files then shows only Files, a click on Plan shows only Plan, and the dock keeps its width. A terminal keeps its shell and a browser keeps its page when you switch to another tab and back. The choice is kept per browser. In a narrow window the dock opens on top of the chat instead of squeezing both.



The dock with Tabs checked: the strip reads like a row of tabs, only Files is open, and the dock has the same width it had side by side.

The sections below take the panels one at a time.

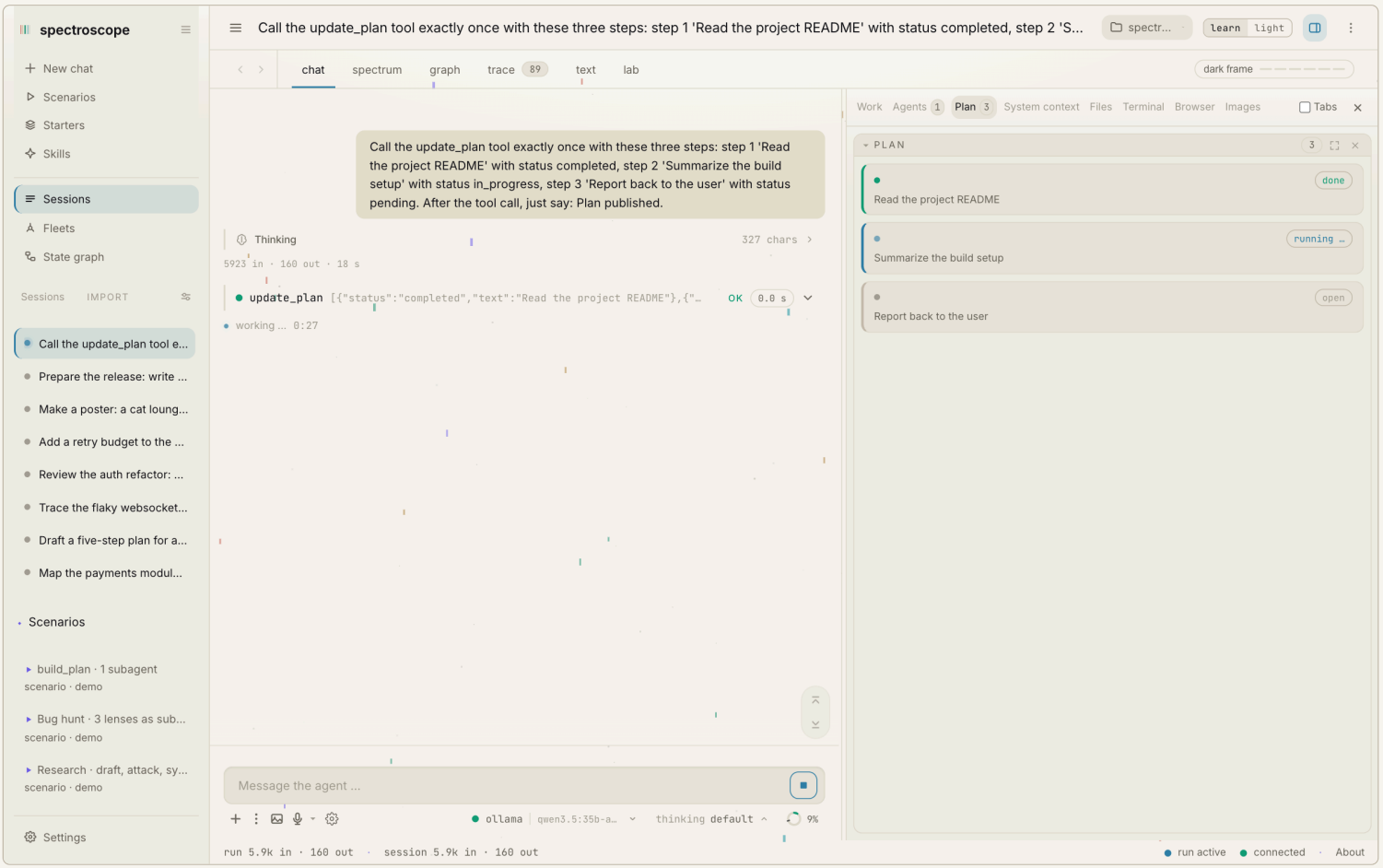
Agents



The session-wide roster: main plus every subagent that ever ran, with lifecycle badge, task, latest status line and token cost. It survives run ends — only a new chat clears it.

One card per agent — main first, then every child in spawn order — with its state dot and lifecycle badge (*submitted / working / done / failed*), its task, the latest `report_status` line, and its token cost (`1.2k in · 0.3k out`). Selecting a card jumps to the System context tab for that agent.

Plan



The agent's published plan, live: three steps in the three states (done, running …, open). The capture is from a real run against a local model through ollama, with one update_plan call.

While the session has a playbook or workflow run, the Plan panel shows the run. It opens by itself when a run starts, once per run: after you close it, later lines of the same run leave it closed, and a new run opens it again. The head names the run and its task, how long it has been running or how it ended (*ended after 4:12: done*, *stopped* or *failed*, read from how the run stopped), and a picker when the session holds more than one run. A strip below it has one mark per node of the forward path, decisions as diamonds: done, running, waiting for you, failed, skipped and not started.

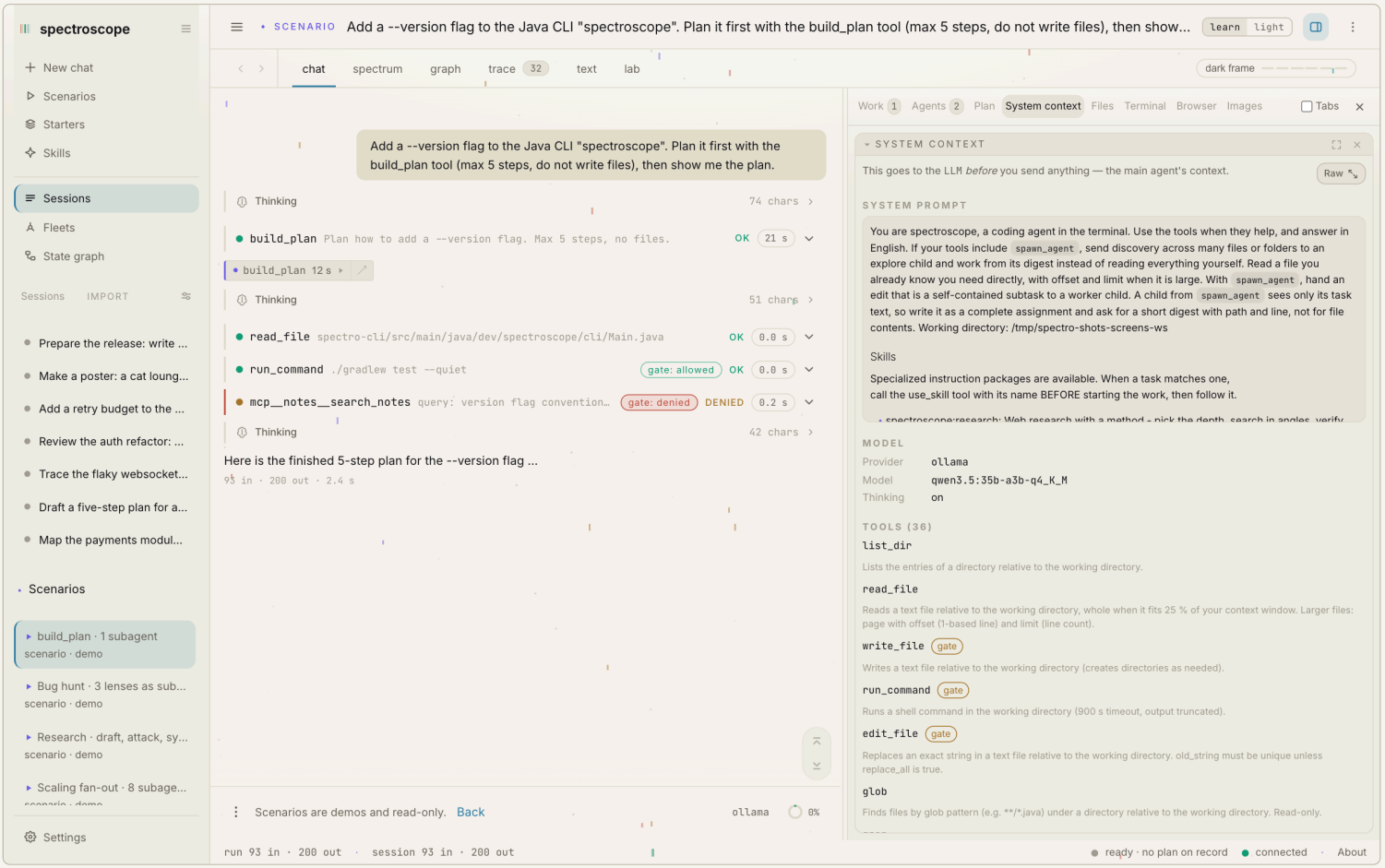
Then one card per entry, in the order the entries ran, and the nodes not started yet in grey. A loop gives a node a second card. A card says who works on the step (*Chat* or the helper with its role), the model, and *picked* when you chose it in the start sheet; under *in* the documents the step reads, under *out* the documents it wrote, each *new* or *changed*, then the files its agents changed with the write and edit tools and the number of shell commands, whose files are not captured. A running card shows the last line its agent wrote. A decision names its round and outcome, for example *round 2 of 5*, and a loop that ran out of rounds reads *limit of 2 rounds reached*. A press on a card shows the step in the chat and marks it for two seconds: the prompt a chat step sent, or the thread of a child step's helper. A press on a document or a file opens it in the Files panel. A path under a dot folder such as `.superpowers/` opens only when the run recorded it as a document it wrote (`GET /api/run-document`, chapter 26); everything else opens as the Files panel opens it.

wide puts the run in the chat's place. A playbook run draws the graph file its runner writes, a workflow run its steps as a chain. Under the graph one entry is shown in full: by default the one that runs, else the last. It has who and which model, the input text, the documents it reads, the output, the documents it wrote, the files and the helpers. A press on a node shows that node's latest entry. *In the chat* goes back to the chat at that entry, and **small** brings the chat back. The panel and the wide view show the same run: a run picked in one is picked in the other. The keys the State graph segment answers (space, the arrows, Home and End) do nothing here, and the view loads no files.

A plan the model writes with `update_plan` shows under the run as *Plan of the model*. A session without a run, among them every session recorded before runs wrote `run_step` lines, shows only that plan:

The read-only, latest-wins snapshot of the agent's plan, fed by the permission-free `update_plan` tool: each step with a status dot and badge — *done*, coral *running …*, *open*. The wire statuses stay canonical English (`pending / in_progress / completed`, enforced at the write boundary); only the labels localize. The tab badge counts steps; the CLI pretty-prints the same event.

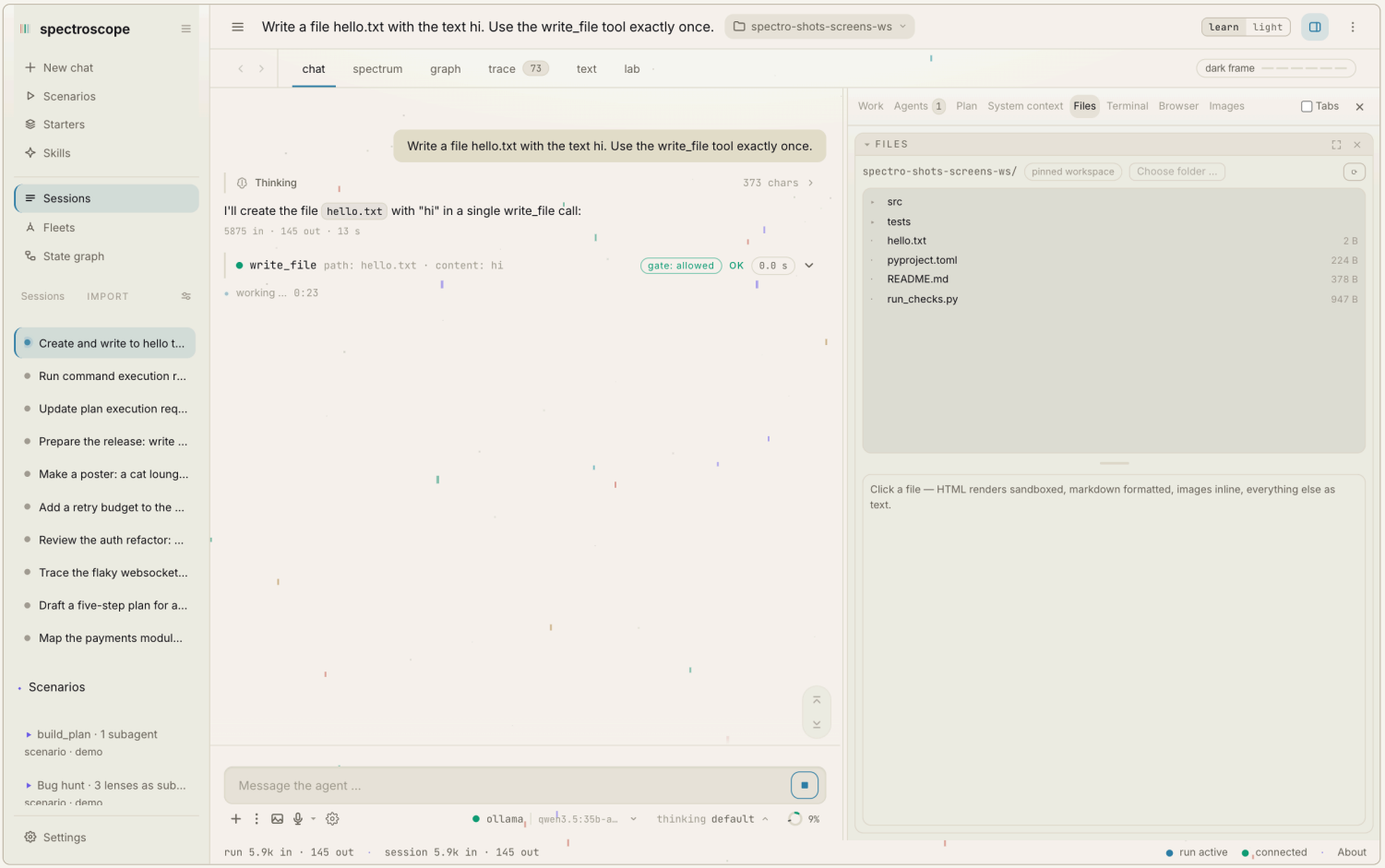
System context



"What goes to the LLM before any user message": the real system prompt (markdown-rendered), model + thinking state, all tools with gate markers, skills, MCP servers and the subagent role profiles.

The transparency panel: exactly what the LLM receives before any user message. For the **main agent**: the full system prompt (the paragraph on when to read a file directly and when to send an explore or worker child, working directory, project `SPECTRO.md` context, the skills catalog), provider/model/thinking, every tool in registration order (gate chip = needs your approval), skills, MCP servers, and the six subagent role profiles. For a **selected subagent**: its role prompt, its task, its reduced tool set — and the note that children never get the spawn tools (nesting ends at depth 1). The data comes from the stateless `GET /api/context`, assembled from the same constants the live tools use — what the panel shows is what the model gets. One row is a list rather than a state: the MCP servers are the ones your settings file declares, named whether or not they answered, and their tools are not in the tool list above them. Reachability is what `/mcp` and `spectro doctor` report (chapter 18). A fullscreen *Raw* modal shows the whole context untruncated with a character count and copy button.

Files

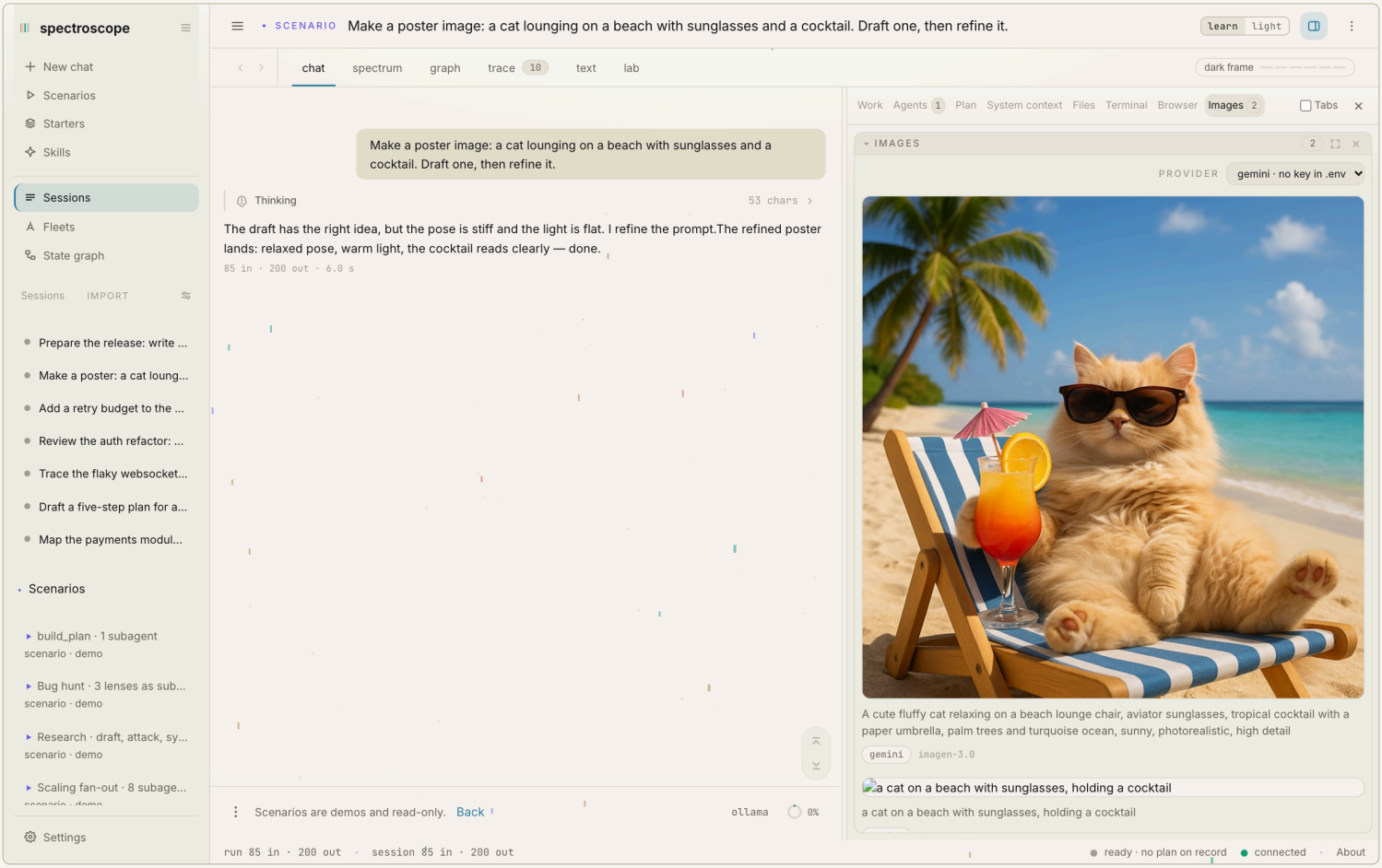


The Files tab is the agent's desk: the workspace this session runs in, refreshed live — hello.txt, which the agent wrote through an allowed gate seconds ago, is right there.

A read-only view of the agent's **workspace** — the same sandbox the file tools see. Every session gets its own folder (`<tmpdir>/spectroscope-ws/<session-id>` unless a `workspace` is configured — chapter 30), the server announces it over the socket (`workspace_info`, visible in the trace), and the tab follows it live: what the agent writes appears here, and the repo you started spectroscope in stays clean. The announcement leaves the panel as it was. The Files row of the header's ⋮ menu opens it on this tab, and the tab then shows the folder the run resolved. And you can **pick the desk per session**: the "Choose folder ..." button opens the *native* folder dialog on the spectroscope machine (`POST /api/pick-workspace`, macOS; a browser cannot hand out absolute paths, but spectroscope runs locally), and the picked path travels back as the additive `set_workspace` socket message. Change folder in the menu of the folder chip in the header opens the same dialog. Only before the first run: once the agent ran, the sandbox and every subagent are anchored, the button locks and a new chat is the way to a different folder. The pin is shared server state, so the tree you browse and the sandbox the tools see are always the same folder, and a continued session in the same server process finds the picked folder again. And the browser **remembers your pick**: every new chat lands on the same desk with zero clicks (a dashed "↺ folder" chip shows the memory; clicking it forgets, and the next chat gets its per-session temp folder again. A continued session keeps its own folder). Hidden entries and build folders (`.git`, `build`, `node_modules`, `target`, `dist`, `out`) are skipped, depth and entry count are capped honestly ("list truncated" note). Clicking a file previews it by type: HTML runs in a **CSP-sandboxed iframe** (scripts execute in an opaque origin — no cookies, no API, no parent access), markdown renders through the shared component, images inline, everything else as text. Binary answers 415, oversized 413 — and the defense in depth is real: the `.env` with your API key answers 404 *even by direct URL*, because every path segment is re-checked server-side against the hidden/ignored rule.

Images

When the agent generates images (chapter 17), they collect in the **Images** panel of the dock. It opens from the Images row of the header's ⋮ menu, which also shows how many pictures there are, from the dock's strip, or from View > Images in the desktop app, and it stays open after a reload. The panel shows cards newest-first with the prompt as caption, provider and model badges, full-size link, and a provider dropdown (`gemini` / `openai`) that switches the image backend for the next generation. Only generated images live here: a picture an imported transcript carried belongs to the message or the tool card it came with (chapter 5), and the two provenances are kept apart on purpose. Bytes never cross the socket — cards load from `GET /api/images/<sha256>.<ext>`, where the content address doubles as the traversal guard (64 hex chars or 400, before any filesystem access).



The Images panel in the dock, opened from the Images row of the header's ⋮ menu on the image generation scenario: the pictures as cards, newest first, with the image backend above them.

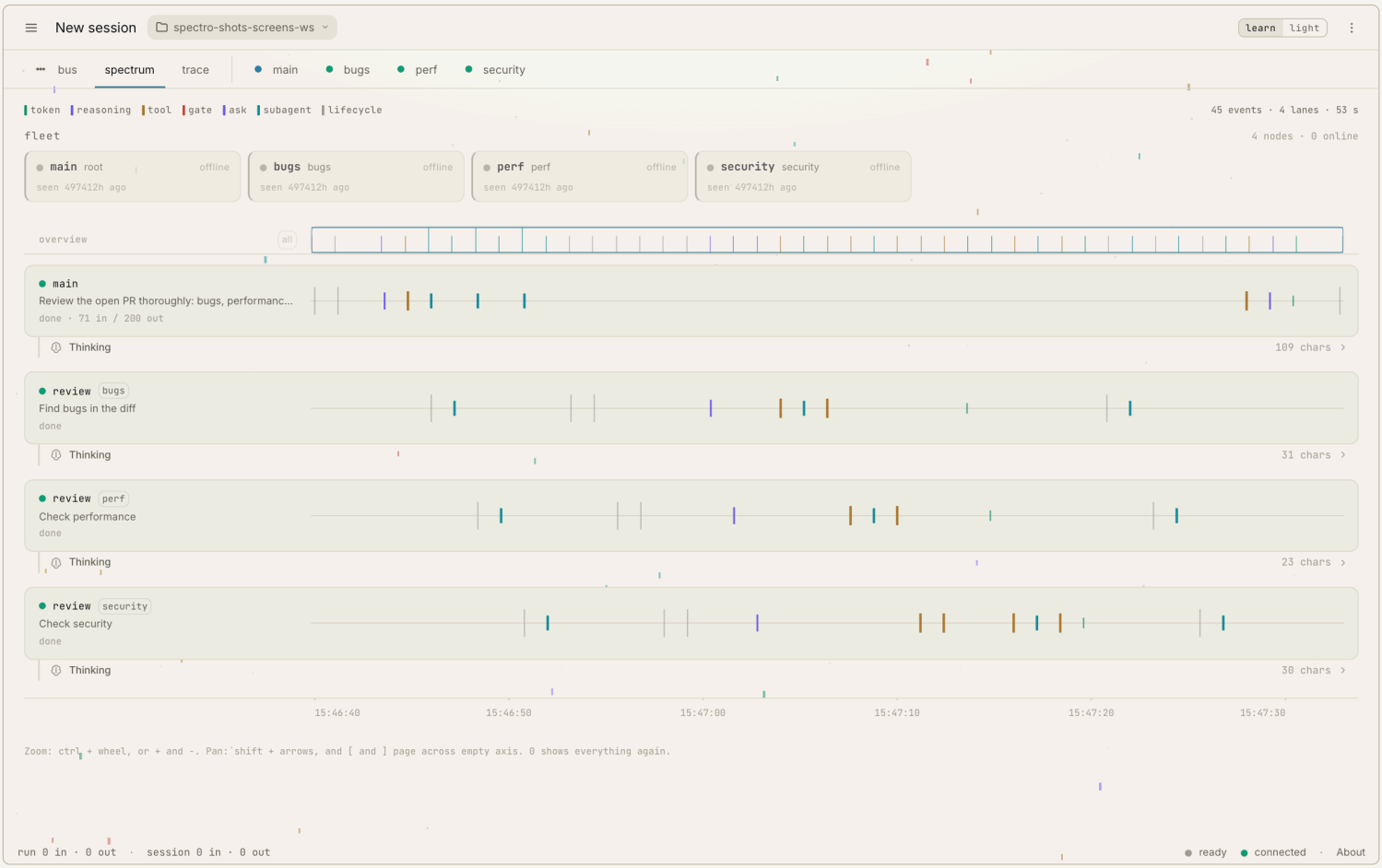
Each card also carries **"→ Workspace"**: it copies the image from the global store into *this session's workspace* — you pick the file name (keep the content-addressed original or type your own; a name without an extension inherits the original's), and the file appears in the Files tab, ready for the agent's tools. The endpoint keeps the house guards: only store names touch the disk, the target name is a single sanitized file name, and an existing file is never overwritten (409).

the spectrum and the new lenses

The design transplant (migration phase 4) brought the brand into the product and six new surfaces with it. All of them read the same event stream you already know; none of them adds a second source of truth.

the spectrum tab

The spectrum is the fleet on one screen. Every agent draws one horizontal lane; every event is a discrete mark on it, colored by type: token teal, reasoning violet, tool amber, gate red, subagent ocean, lifecycle faint. The lane rail carries the agent id, its task, its lifecycle state and its token totals. A pending permission turns the lane's gate mark violet and pulses until the decision lands.

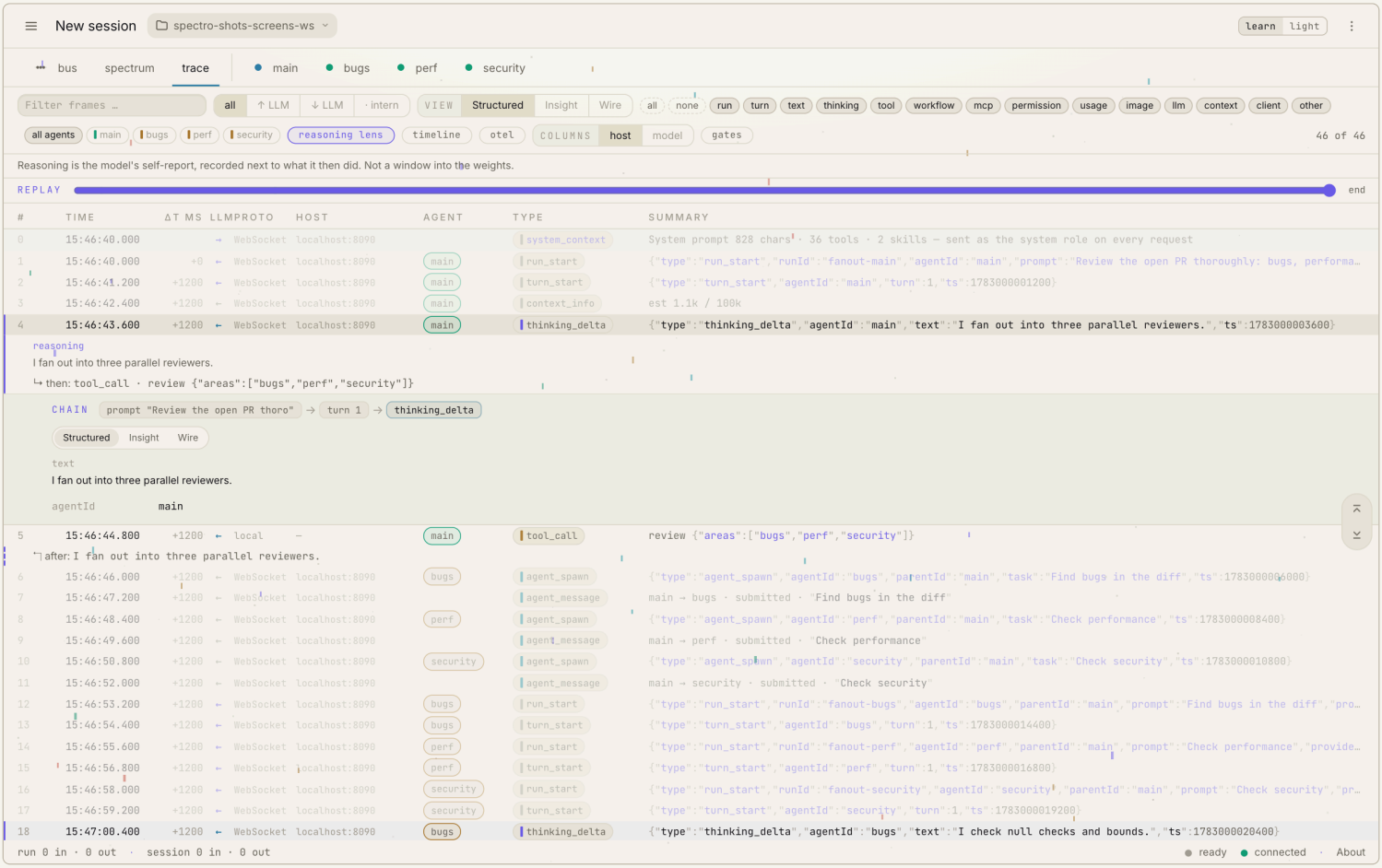


The spectrum tab: one lane per agent, every event a spectral mark. A real capture of a review fan-out with three subagents.

Clicking a lane opens the trace pinned to that agent: the new agent chip row filters the wire view to one lane while session-level frames (decisions, run ends) stay visible. Dense token streams are thinned for display; the counter names the drop, and the JSONL file keeps everything.

the reasoning lens

The reasoning lens is a trace mode for one question: what did the model say it was thinking, and what did it then do? Switched on, thinking frames come to the foreground in violet, everything else steps back, and tool calls and gate frames stay readable as anchors. Every block of thinking ends in a clickable "then:" line that jumps to the action that followed it.



The reasoning lens: thinking foregrounded in violet, the said-vs-did pairing line under the block, the causal chain in the open frame.

HONEST LABELING

Reasoning text is the model's self-report, recorded next to what it actually did. It is not a window into the weights, and the interface says so. With reasoning capture off, the lens tells you that too instead of showing an empty highlight.

The lens state persists with your design preferences, applies to live runs and replays alike, and combines with every other filter.

the causal chain and the replay scrubber

Every open trace frame now shows its walk-back: result, decision, request, call, turn, run, spawn, parent run. Each link is a chip; click it and the trace jumps there. The chain is computed from the recorded stream alone, with no model involved.

The replay scrubber caps the visible stream at any frame. Drag it back and you read the session exactly as far as it had happened, filters included.

the gate surface

A pending permission opens the permission dialog over whatever tab you are on (chapter 5). It shows the full input and the recorded outcomes of earlier gates, and Deny holds the focus. The Lab keeps its own stepper-owned dialog; replays never ask.

the explain panel

The explain button in the trace toolbar docks the why layer next to the stream: a run summary (duration, turns, tool calls by name, gate tallies, tokens, stop reason) and one entry per gate that answers "why did the gate ask" from the permission mode at ask time. Stored sessions carry no mode frames; the panel says "not recorded" instead of guessing. Everything in the panel folds deterministically from the stream.

spectro doctor, the web face

The spectro doctor row at the bottom of the header's ⋮ menu opens the calibration page: one measured line per subsystem, from this browser's viewpoint. Server api, live socket, llm backend, session store, default workspace, log level, permission mode. The page re-measures on every open; the CLI's `spectro doctor` checks the machine side.

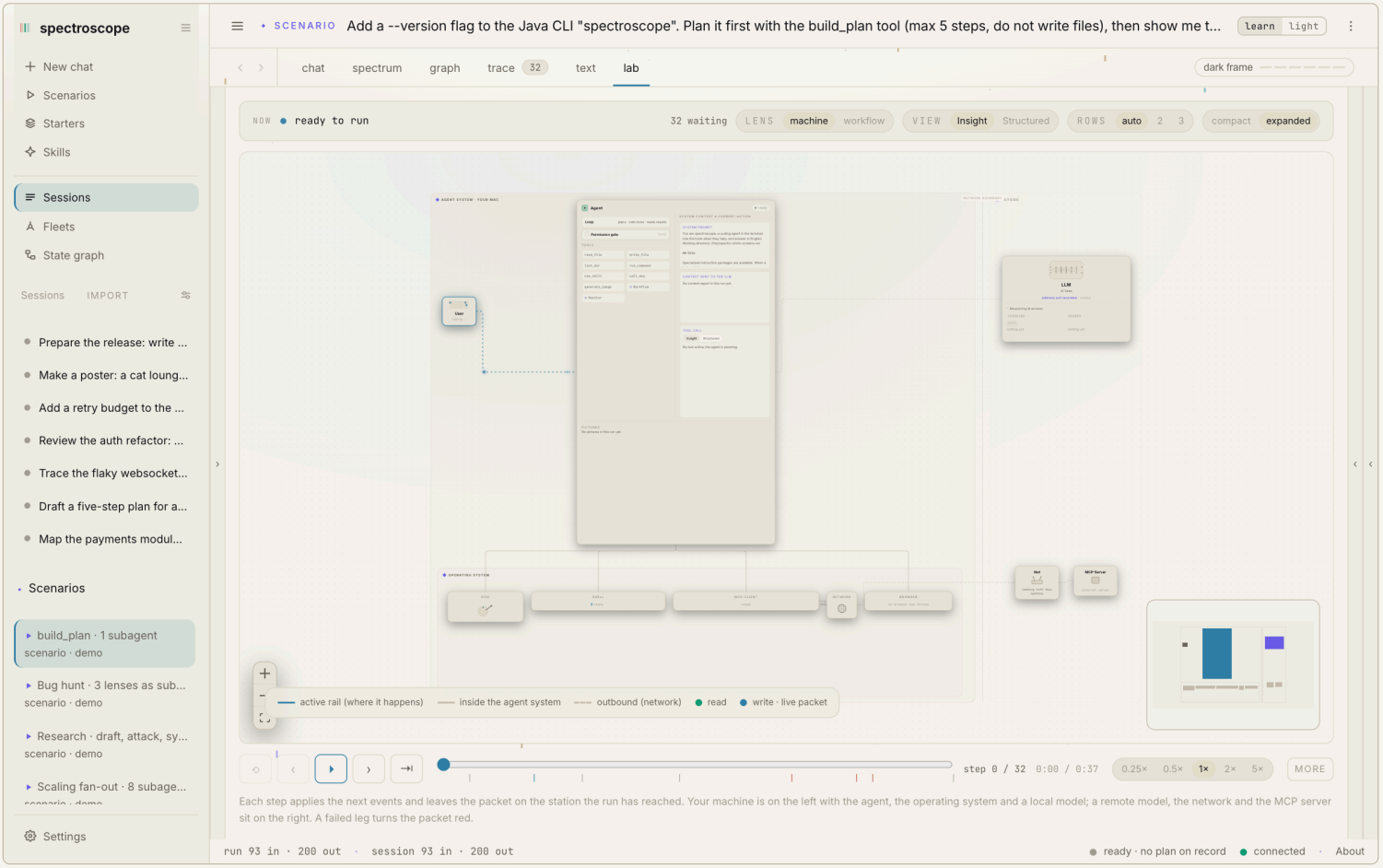
CHAPTER 7

The Lab

The Lab is spectroscope's step-through debugger for agent runs, and its best classroom. Events do not flow through automatically: they queue behind a client-side **dam**, and you advance them one boundary at a time while chat, map and JSONL strip move in perfect lockstep. It works on live runs and on any archived replay.

The dam

Live events keep queueing even while the Lab is closed. Each step applies the next event(s) through the *same pure reducer* the chat uses, plus two pure folds underneath: the scene model the Flow map paints, and a strict Petri marking. The invariant is literal: the stepped UI state always equals “fold of everything applied so far”. Because the server genuinely waits on permission futures, a token sitting at the gate is the real system state, not a simulation.



A freshly loaded scenario: the dam holds every event (the “now” band reads “ready to run” and “32 waiting”) and the map shows the idle system, waiting for the first step.

The transport

Two things frame the map. Above it floats the **“now” band**: a plain-language line of what the run is doing at the current step (“the model is thinking”, “writing hello.txt to disk”, “the permission gate is deciding”, “orchestrating 3 workers”, “done · control is with you”), and next to it the size of the dam (“`n waiting`”, or “`waiting for server …`” when a live run has drained the queue). Before the first step it reads “ready to run”. Because the band names the folded scene, a scrubbed replay always tells you where it is.

Below the map sits the **scrub bar**. Four buttons drive it: reset (`⟲`), step back (`‹`), play / pause (`▸ / ❚❚`) and step forward (`›`). A slider walks the run's **coarse-step boundaries**, so a drag always lands on a whole step, never mid-block; a “`step 4 / 27`” counter reads out where you are. Scrubbing pauses auto-play. Grain and speed are one rung down, behind a small **more** disclosure that a live run rarely needs:

CONTROL	BEHAVIOUR
› Step forward	applies the next block (or one JSONL line in Single grain); the “now” band, chat, map and strip all move together
‹ Step back	undoes the last step group, re-folding chat, map and strip backwards from scratch
▸ Play / ❚❚ Pause	opens the dam into a paced auto-play: a timer steps at the chosen speed instead of teleporting to the end
⟲ Reset	re-queues everything applied and returns to the start
Blocks | Single (more)	the grain: one click = one meaningful block (a whole thinking run, a whole answer run, or one event), or exactly one JSONL line
Speed (more)	the auto-play slider, roughly 0.5×–16×/s (default 0.8/s)

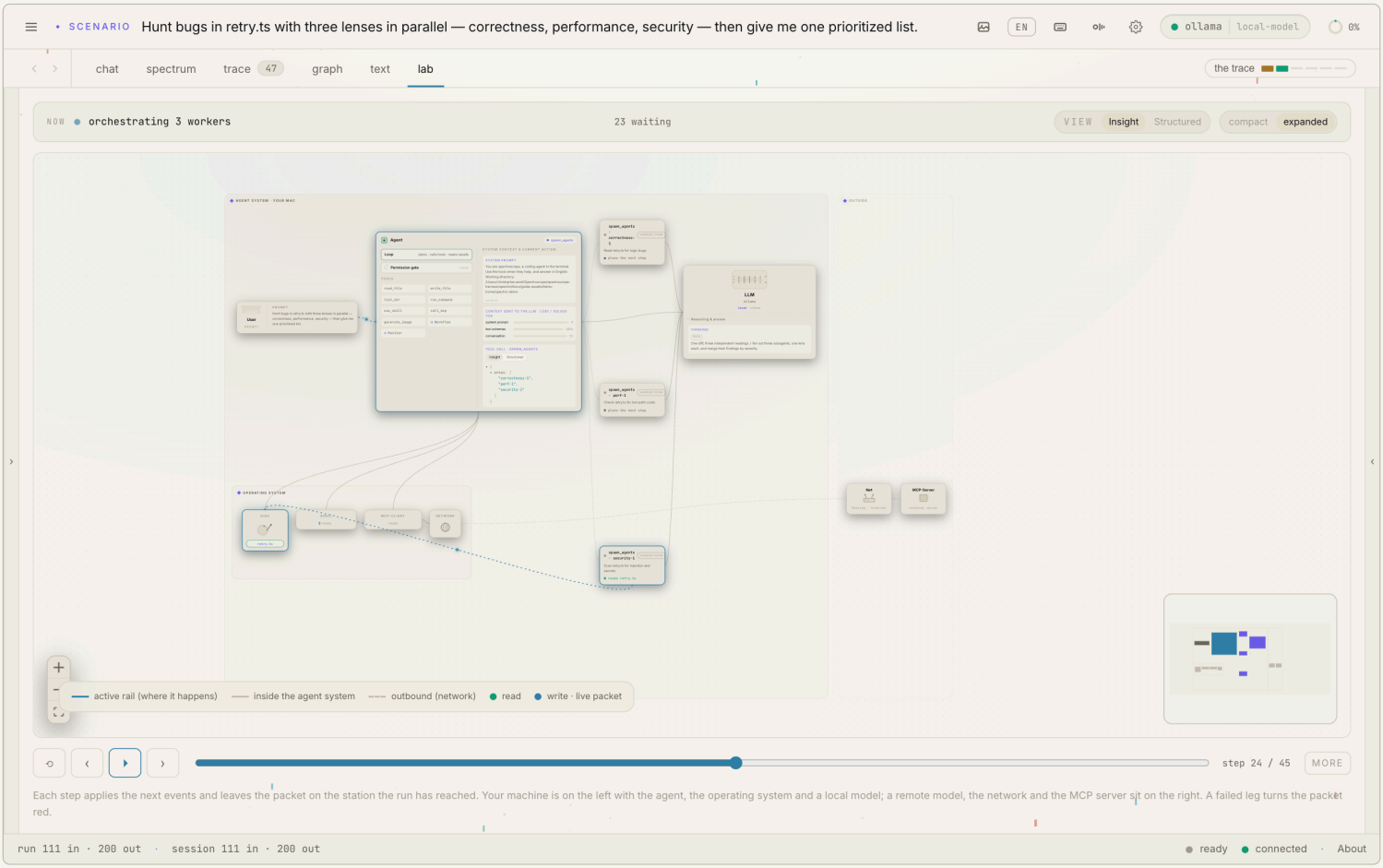
Every button has a keyboard twin, guarded so it never eats a keystroke while you type in a field: `space` or `→` steps forward, `←` steps back, `f` toggles flow (auto-play), and `r` resets. Press `?` at any time for the full sheet (see the keymap).

The panes left and right of the map, the fully functional chat and the JSONL strip, are drag-resizable and collapsed by default, so the map gets the room. The strip shows applied lines, the just-fired line highlighted, the dam divider ("`▮▮ dam · n waiting`"), then the dimmed queue; every row expands to the full event, and an "All" toggle drops the render window for very long sessions.

compact | expanded

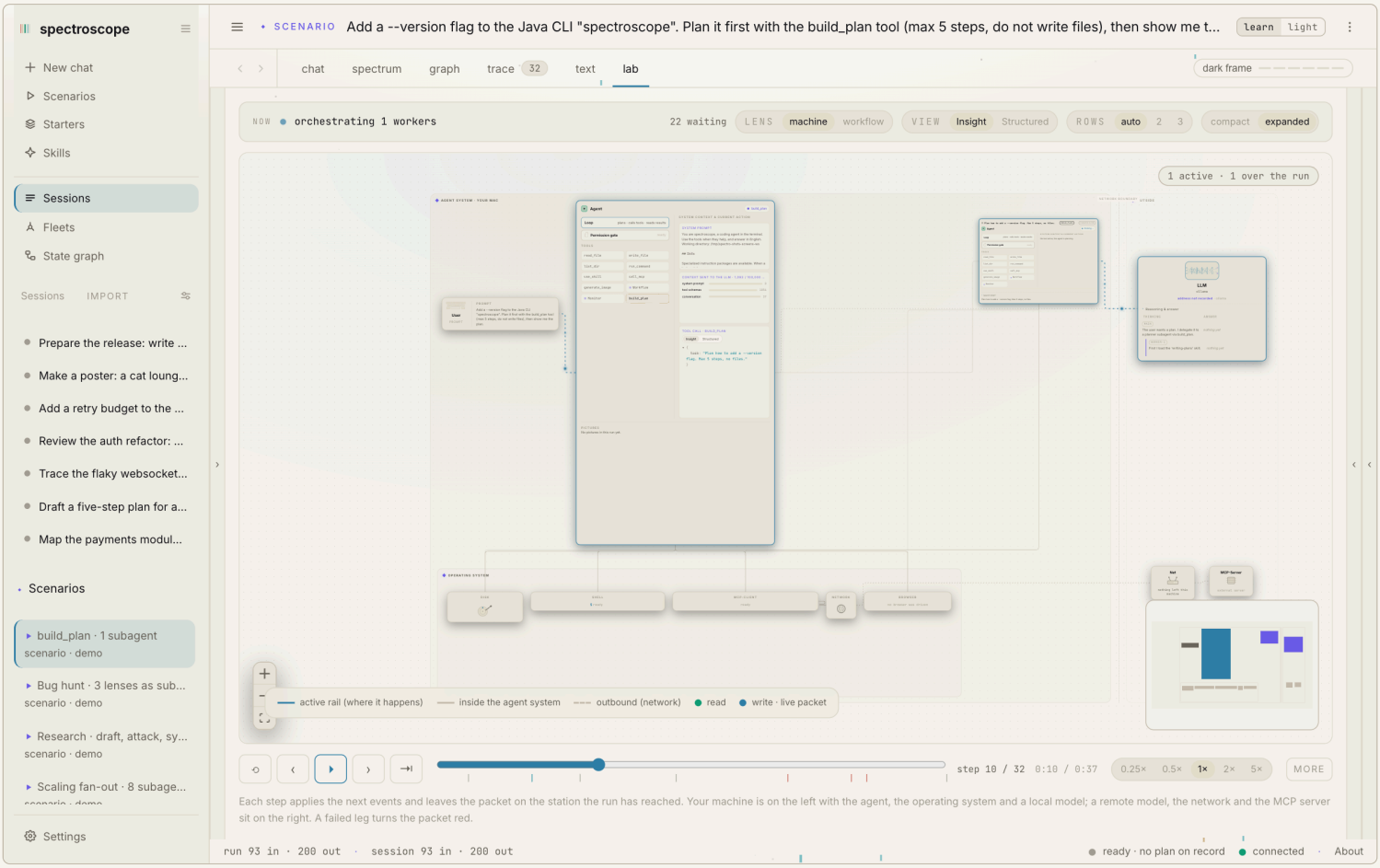
A small **compact | expanded** pill sits by the map (the same pill rides the fleet machine room, and the two share one preference, so the choice follows you between them). It changes how much of each node opens by default, not what the map means.

Compact is the working default: an agent card shows its identity, its state and its live token totals, and the surrounding stations (the context window, the system prompt, the user, the LLM) sit as tidy tiles. **Expanded** opens every card at once — the context window and the system prompt move to sit BESIDE the agent, the prompt beside the user, and each node card carries its task and history inline. The layout genuinely spreads to make the room, so nothing overlaps; it is the view for reading one run closely, where compact is the view for watching the shape of it. The same pill also carries the *Text* / *JSONL* toggle for the strip, so both grains are one click away.



> Expanded view: every node opened at once — the context window and system prompt beside the agent, the prompt beside the user, task and history inline on each card. The layout spreads so nothing overlaps; compact collapses it back to the tidy default.

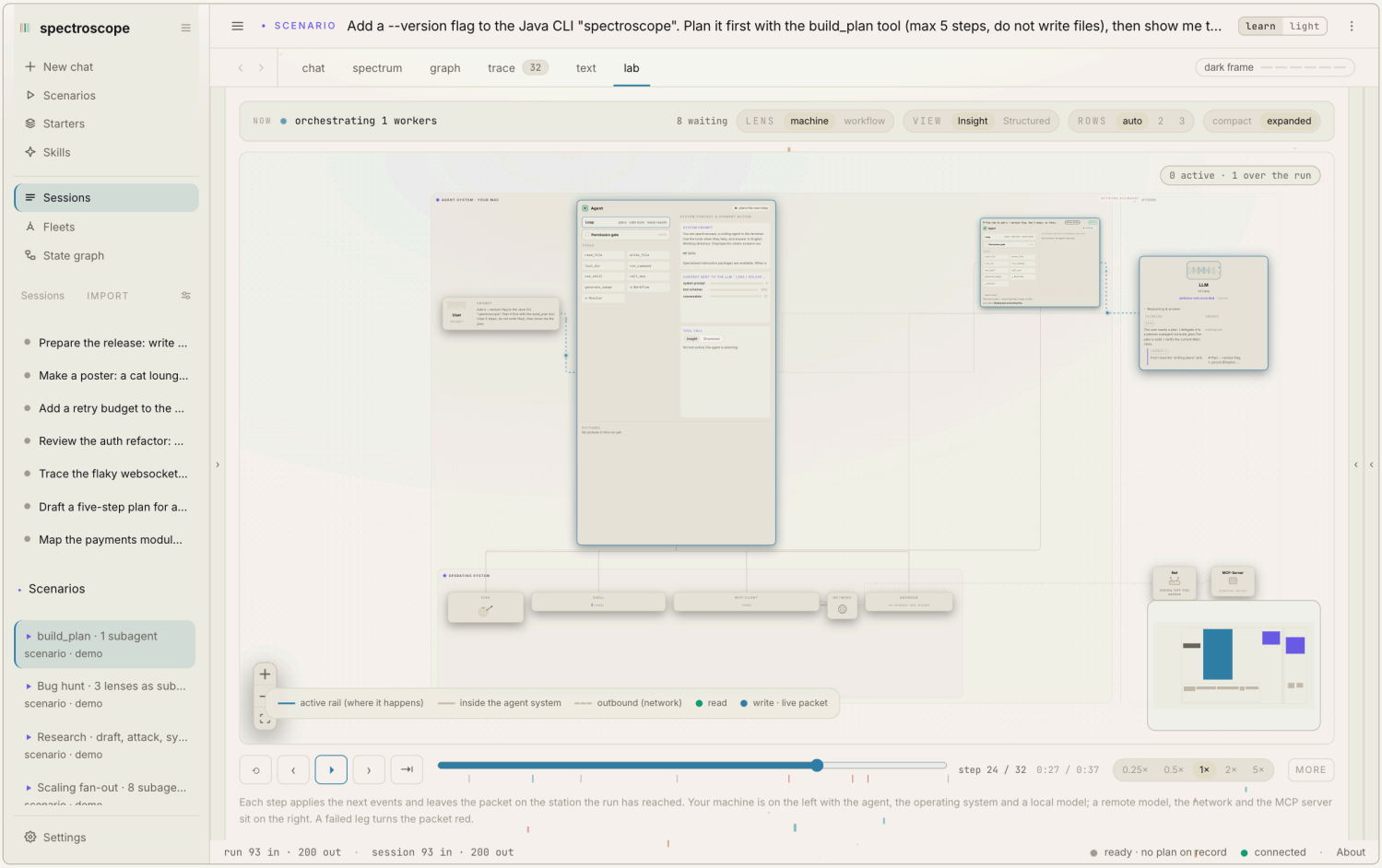
The Flow map (React Flow)



Step 10 of 32: the Agent hub with its Loop and Permission-gate rows and the tool belt below them, the current build_plan call beside them, the OS band underneath and the LLM on the right.

The centre is a literal systems map. On the left, the **AGENT SYSTEM · YOUR MAC** zone: the user, the **Agent hub** (live activity line, the Loop row, the **Permission gate** row that colors with the gate state, and the tool belt) and the **OPERATING SYSTEM** band, disk (animated platter, the touched file as a pill), shell (the running command), MCP client and network stack. On the right, **OUTSIDE**: the LLM (neural-net animation while thinking, model name, reasoning and answer stream slices per agent), the network and the MCP server.

One coral packet rides the rails to wherever the current event happens. The map is **honest about locality**: with a local provider (Ollama) the LLM node sits *inside* your machine and nothing crosses the boundary; switching to a cloud provider re-places it beyond a dashed **network boundary**, live, because the layout follows the selected provider.



A subagent at work, step 24 of 32: the planner child has its own loop card inside the machine zone, wired to the same LLM as its parent, whose card carries the reasoning of both.

Every node carries disclosures with the real data. The Agent hub now shows a **tool belt**: the chips `read_file`, `write_file`, `list_dir`, `run_command`, `use_skill`, `call_mcp` and `generate_image`, with the active one lit. Its *system context* disclosure holds the main agent's system prompt, the live context budget (estimated tokens against the compaction threshold, with a bar per segment) and the current tool call as a JSON tree, or, when the active tool is `generate_image`, a small generated-image thumbnail with its prompt caption. The LLM node reveals per-agent reasoning and answer streams; the disk shows its file, the shell its command, the MCP client its call. Subagents pop in as their own loop cards with lifecycle badges and disappear when the root run ends.

Two picture panels can sit on that same shelf, and the label says which is which. *generated image* is the last one the agent painted, captioned with the prompt that asked for it. *pictures handed over* is what an imported transcript handed the agent — a pasted screenshot, a browser tool's answer — captioned with the file's own note, in arrival order, and capped at six per card, because a session that pasted forty screenshots would otherwise draw forty on one node; the rest are still in the chat and the trace. The panel reads the stream's **root** agent rather than the literal `main`, and that distinction is the whole feature on an imported subagent transcript: such a file roots at the child's own id, and two thirds of the pictures in the measured corpus sit in exactly such a sidecar. In **expanded** both panels leave the context disclosure and span the whole card under the two columns. Neither thumbnail opens the lightbox — that walk belongs to the chat bubble, the tool card and the trace row (chapters 5 and 6).

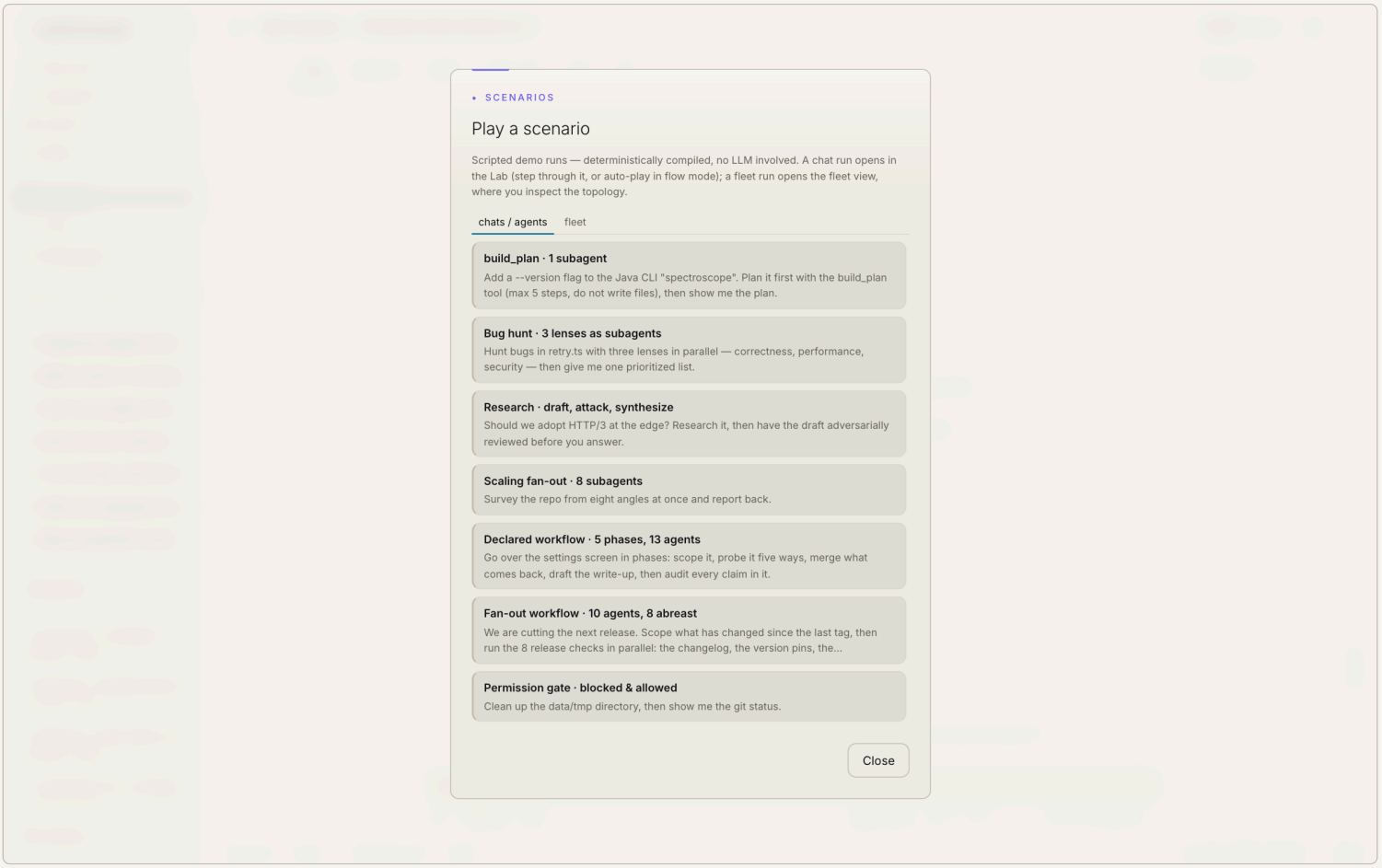
The keymap

Press `?` anywhere (or Keyboard shortcuts in the header's ⋮ menu) to open the **keymap**: a game-style shortcut sheet that pairs each keycap with a small icon of what it does. It documents the Lab's real keys, `space` / `→` step forward, `←` step back, `f` flow, and `r` reset, alongside `?` to reopen the sheet and `esc` to close it. Close it with `Esc`, the ×, or a backdrop click. The footer states the one rule worth knowing: shortcuts pause while you type in a field.

Under the hood: the Petri marking

The stepper also folds every event into a strict Petri net, for the formally minded: places are the stations a request passes through (*User, LLM, Tool, Gate*), transitions are the RunEvents, and one token moves. A **Log** place accumulates a token per fired transition, so its marking literally counts the JSONL lines, which is the invariant the strip's highlight rides on. Guards keep the net consistent: a move only fires when the source place holds the token, otherwise it degrades to a pulse (parallel tool calls, odd foreign replays). The marking has no view of its own; it is the Lab's bookkeeping, not its face.

Scenarios: deterministic demo runs



The scenario picker: seven scripted demo runs, compiled deterministically, no LLM, no API key. Each opens in the Lab with the dam closed.

The sidebar's **Scenarios** button opens seven built-in demo runs. Each is authored once in a bilingual DSL and *compiled* to a deterministic RunEvent stream: real wire format, fixed timestamps, no model involved. Picking one lands in the Lab with the stepper at event zero; it rides the same replay path as any stored session, so every tab renders it.

SCENARIO	WHAT IT DEMONSTRATES
`build_plan · 1 subagent`	dev-tool delegation: a planner child with status updates, verification by main, a gated test run, a *denied* MCP call and graceful continuation
`Review fan-out · 3 subagents`	parallel children (bugs / performance / security) running round-robin, then a prioritized consolidation
`Permission gate · blocked & allowed`	the red path: `rm -rf data/tmp` denied, the agent backs off to read-only listing, then an allowed `git status`
`Disk & shell · read / write`	the disk/shell stations: read → write → list → a gated `npm test`
`Coding · 4 phases, parallel workers`	a realistic dev run: explore → plan (subagent) → implement (two parallel workers writing files) → verify (gated test run)
`Research · consolidate + critical review`	parallel source sweep with MCP searches, a consolidation draft, an adversarial critic child that finds a contradiction, the corrected answer
`Context window · fill & compact`	no subagents: three big reads fill the context ring (ok → warn → high), then a compaction drops the meter and the run continues

WHY SCENARIOS EXIST

Live model behaviour is non-deterministic: a workshop demo that depends on the model calling exactly the right tool at the right moment will betray you. Scenarios make every mechanism of the harness (gates, subagents, MCP, compaction) steppable and repeatable, offline, in either language. (Most screenshots in Part II use the “coding” scenario for exactly this reason.)

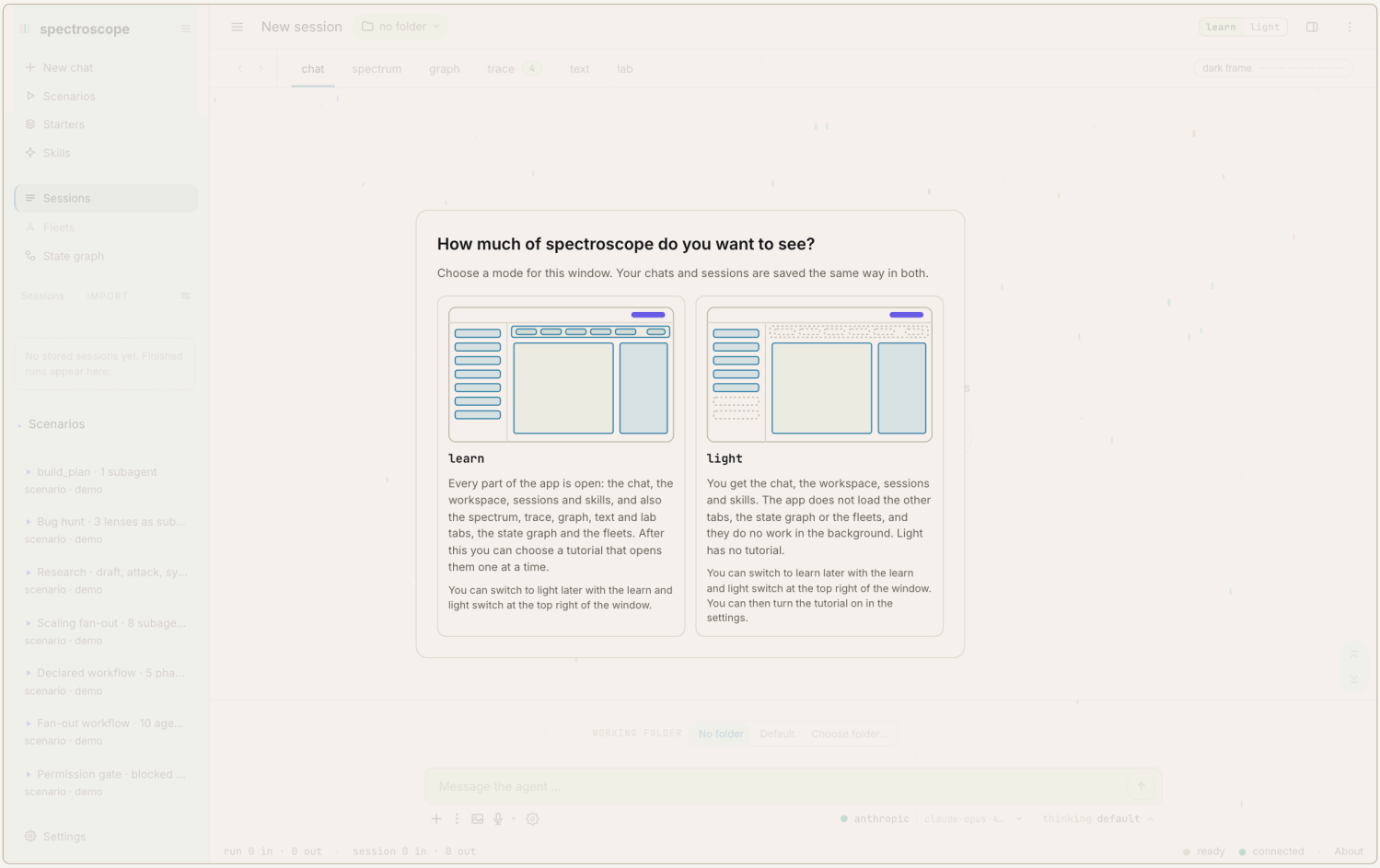
the tutorial

spectroscope has seven tabs, three lenses, a fleet canvas, a machine room and an observability page. Every one of them earned its place, and together they are a wall. A fresh install therefore starts small and opens up as you use it. That is the tutorial.

The twist is that there is nothing to sit through. The product already watches agents through an event stream; the tutorial watches you through the same instrument. You do not tick a checkbox to say you have read a trace. You read one, and the engine sees it in the events the interface was rendering anyway.

learn, light or developer

Before anything else, a new window asks “How much of spectroscope do you want to see?”. It shows the three modes side by side with the same weight, and none is suggested. **learn** opens every part of the app: the chat, the workspace, sessions and skills, and also the spectrum, trace, graph, text and lab tabs, the state graph and the fleets. **light** keeps the chat, the workspace, sessions and skills. The app does not load the rest, and those parts do no work in the background. **developer** opens everything learn opens, without the tutorial, and adds the playbook segment to the sidebar (chapter 15d). Each choice carries a small picture of the window in that mode, drawn from the same table the app uses to decide what a mode shows.

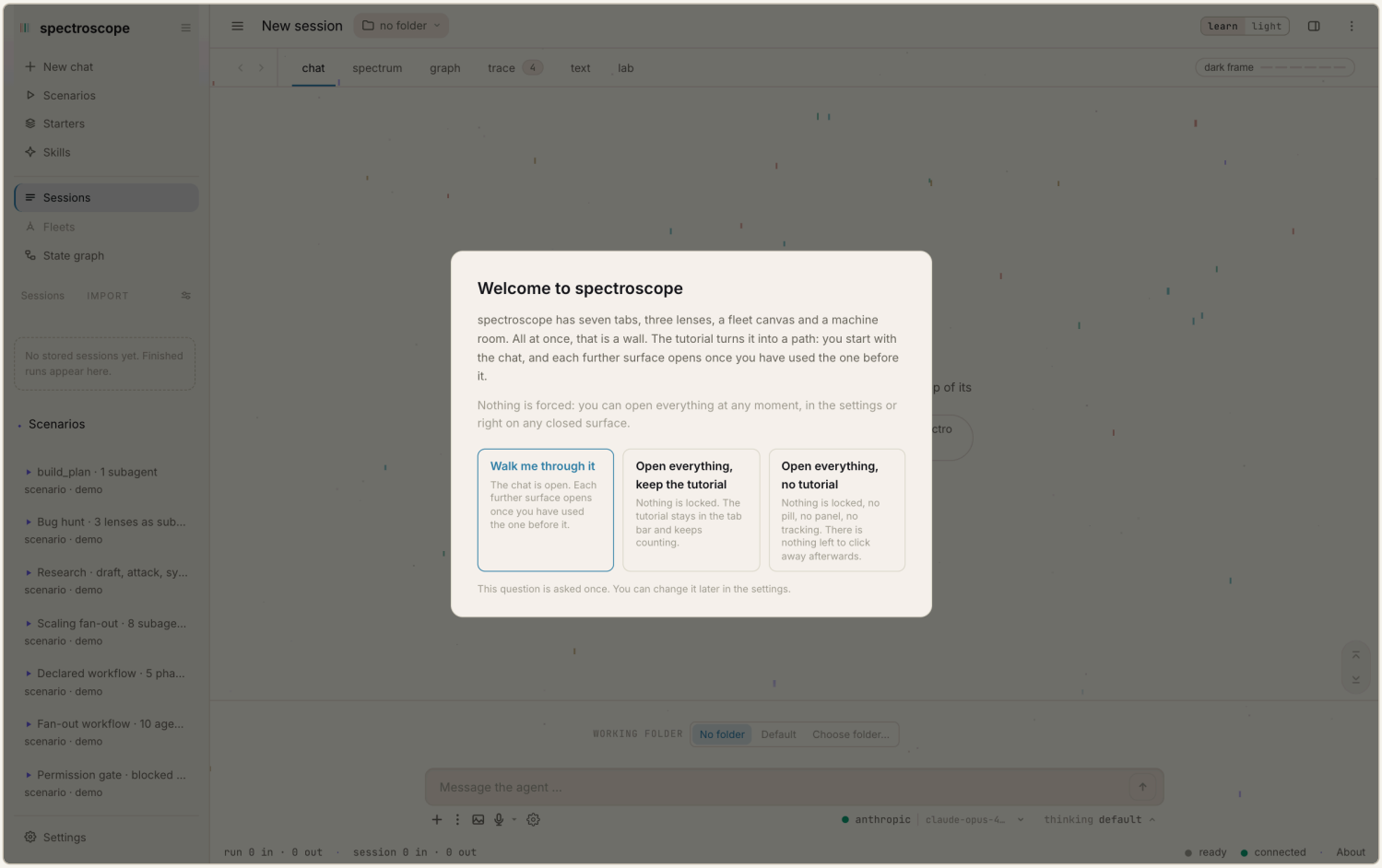


The first question of a new window: the modes with the same weight, each with a short explanation, where to switch later, and a picture of the window in that mode. This plate predates developer and shows learn and light.

Learn goes on to the tutorial question below. Light and developer skip it and turn the tutorial off, because light keeps closed the parts the tutorial would open, and developer opens all of them from the start. In developer and in light the row of tabs above the chat is drawn only while the tutorial is on, so a fresh developer window opens on the chat alone. A tab that was chosen earlier, by an address or a reload, shows the chat until the row is back. The Playbook segment keeps its own tabs either way. In learn the row is always there. The mode switch at the top right of the window changes the mode later. After a switch to learn, you turn the tutorial on in the settings. The answer is kept per window type, so the desktop app and a browser tab each ask once. An installation that was in use before this question existed sees it once as well: its mode stays as it was until you choose, and a tutorial question it already answered is not asked again.

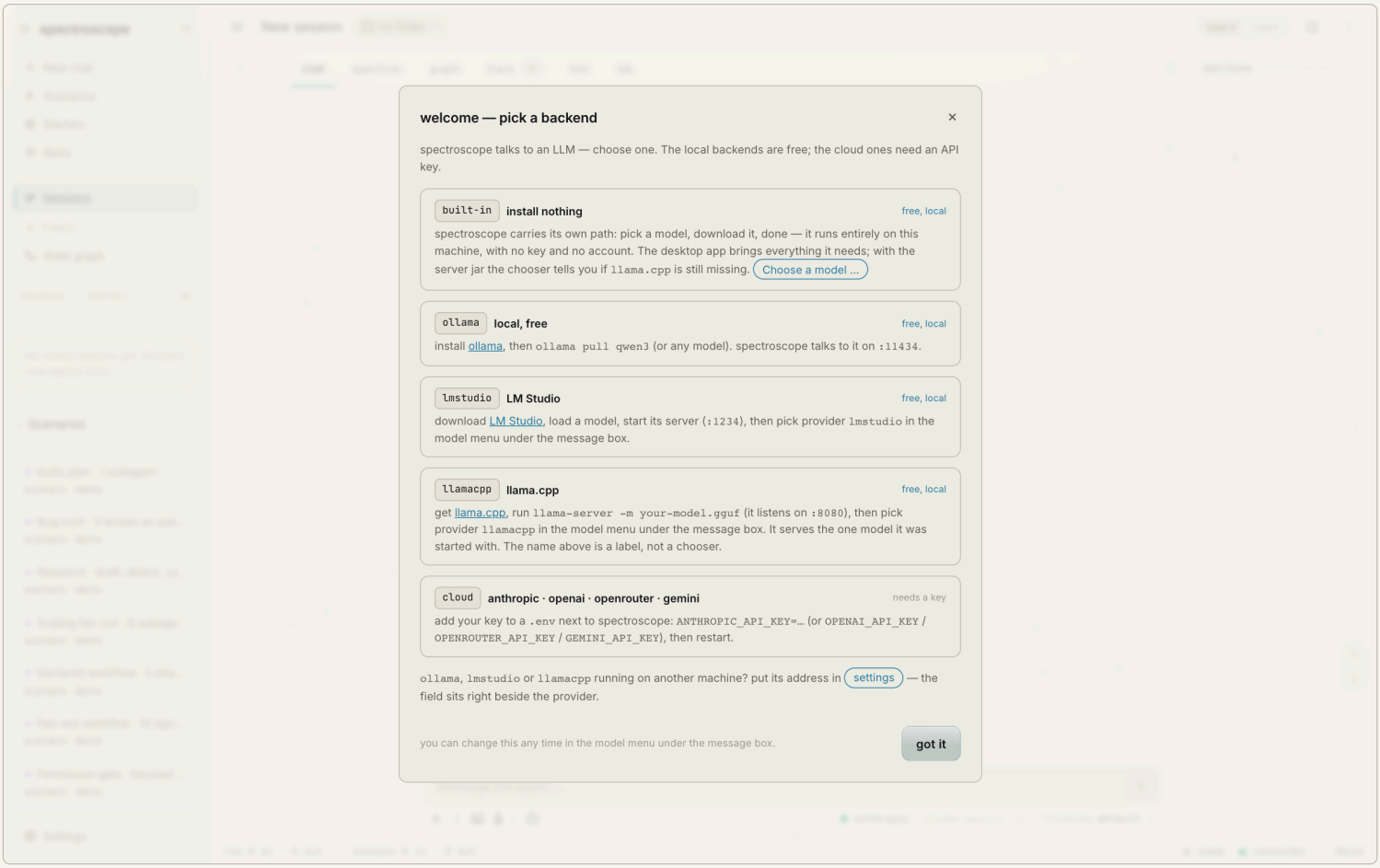
the tutorial question

After learn, a home that has never been used asks for one more decision: grow into it, or open everything now. There is no silent default in either direction, the question is asked exactly once, and the answer lives in the settings afterwards. A home that already has sessions or a settings file is never asked at all and never sees a lock. If you have been using spectroscope, nothing about this changes for you.

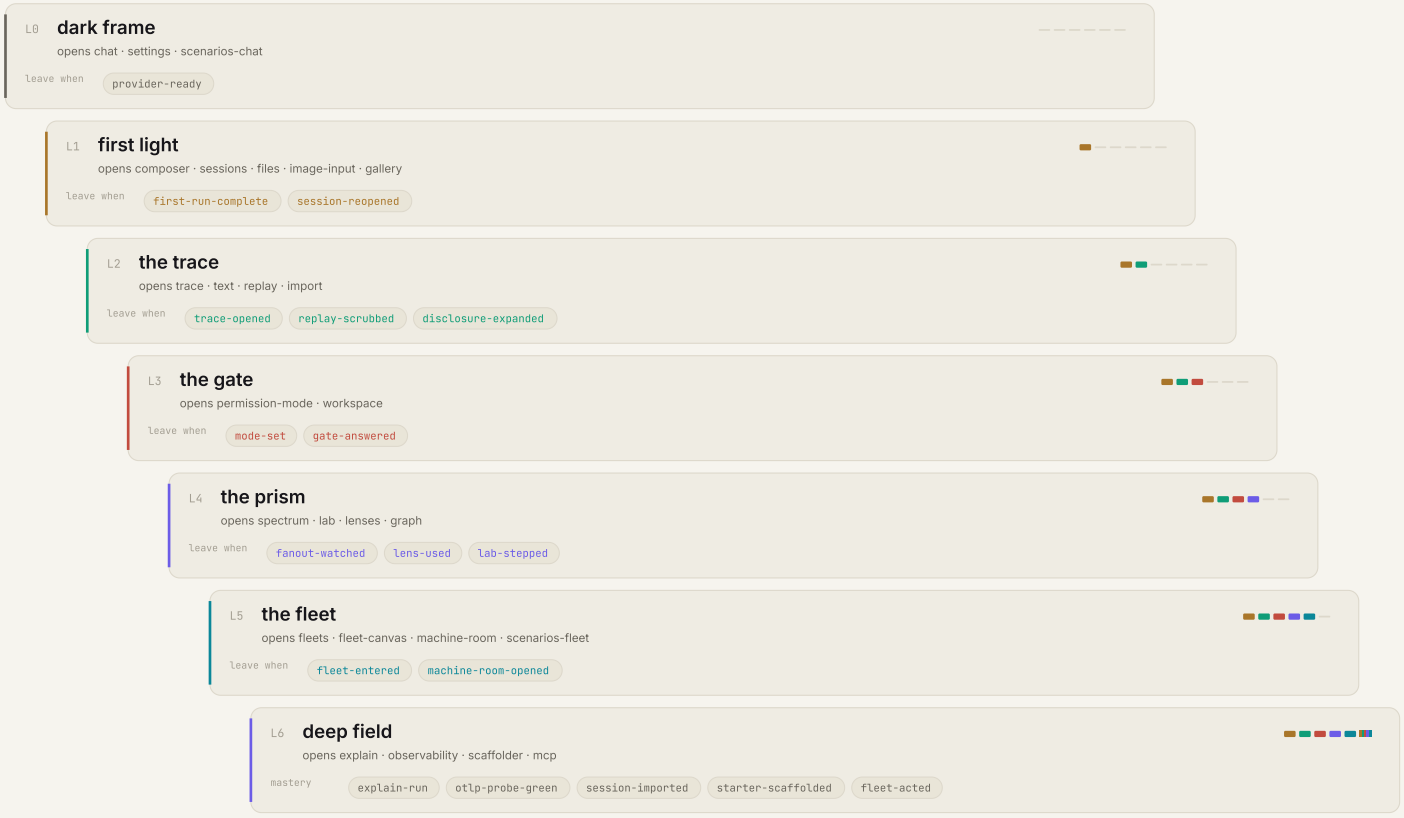


The question that follows learn on a fresh install: what the tutorial is, and a real choice with no default.

Each of the two questions is the *only* screen at its moment. A first run has a third thing to say: that spectroscope needs a backend and which ones are free. Two welcome dialogs on top of each other would be exactly the wall the tutorial tries to remove, so the backend sheet waits until the questions before it have been answered, and then takes the screen alone.



What follows once the questions are answered: the backend sheet, on its own. They never stack.



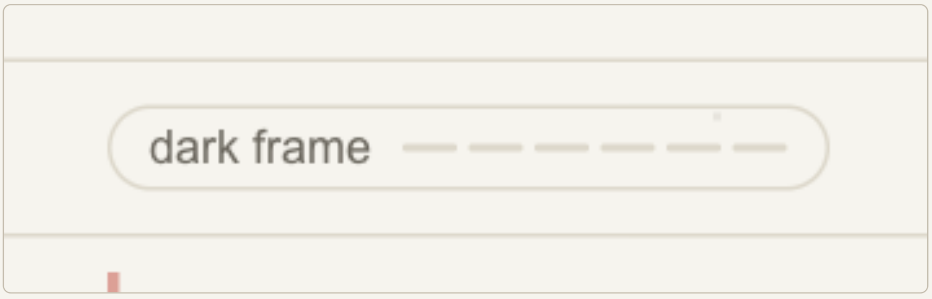
The tutorial as one picture, generated from the same levels.json the engine reads: each state, the spectral line it adds, and the criteria that leave it.

Four of the names are the product's own words. Three are borrowed from telescopes, because they describe the same progression.

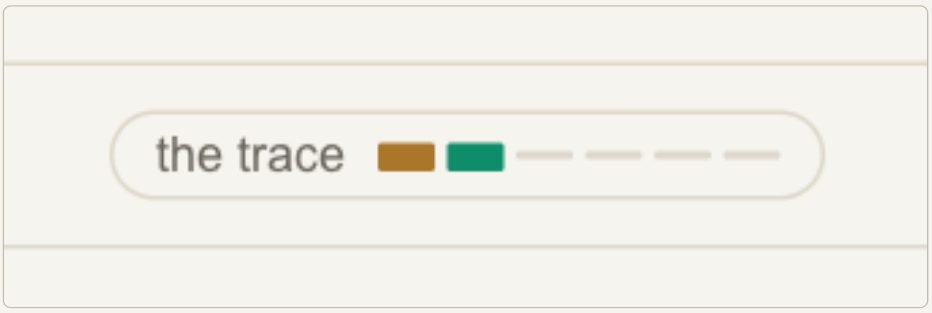
STATE	YOU CAN	IT OPENS
dark frame	set up a provider, watch scenarios	chat, settings, scenarios
first light	run an agent and read its answer	composer, sessions, files, gallery
the trace	read what actually happened	trace, text, replay, import
the gate	decide what the agent may do	permission mode, workspace
the prism	split one run into many lines	spectrum, lab, lenses, graph
the fleet	watch agents across processes	fleets, canvas, machine room
deep field	point the instrument outward	explain, observability, scaffolder, MCP

A **dark frame** is the calibration image a sensor takes with the shutter closed: the picture you make before you make a picture. **First light** is a telescope's first real image, the night it stops being an assembly of parts. The **deep field** is what appears when you point the thing at a patch of sky that looks empty and stare long enough.

The level pill at the right end of the tab row carries a small spectrum strip that gains one colored line per state reached. At the end of the tutorial you hold the full spectrum, which is the same five lines as the mark on the cover of this guide.



Six empty slots. A home that has not been used yet, and the pill saying so rather than pretending otherwise.



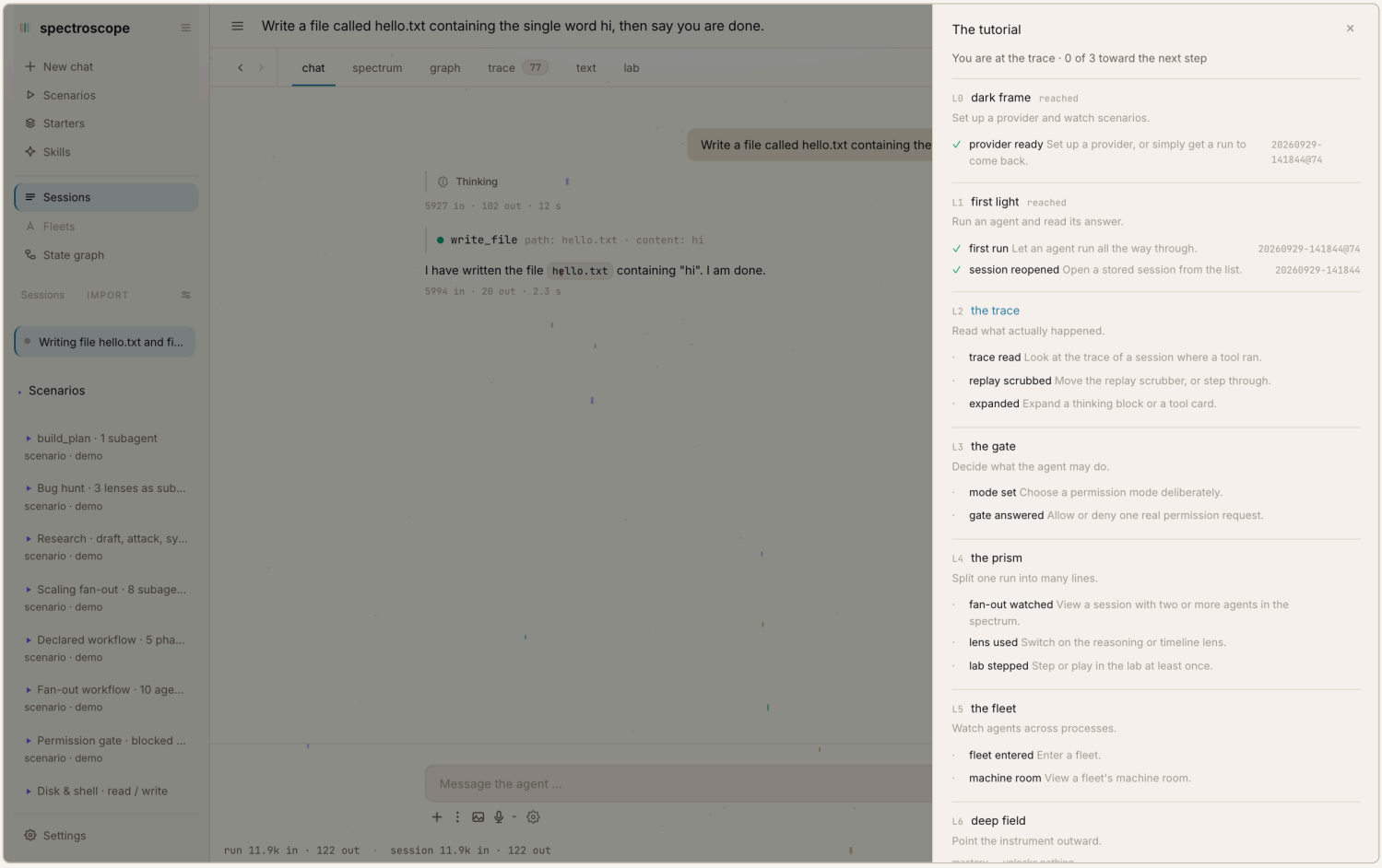
Two steps up: one line per state reached, in the order the mark uses.

The end of the tutorial — five lines plus the full band, which is the mark on this guide's cover.

No new colour was invented for any of this. The five lines are the app's existing event tokens, in the app's existing order, so the strip is correct in all three skins without a palette of its own.

progression with receipts

Click the pill and the panel shows every state, every criterion, and what each one counts. A met criterion carries its evidence: the session it happened in and the event inside it. The link opens that session on the trace, at that frame. The address is a plain hash (`#/session/<id>@<n>`), so it can be copied into a bug report like any other link.

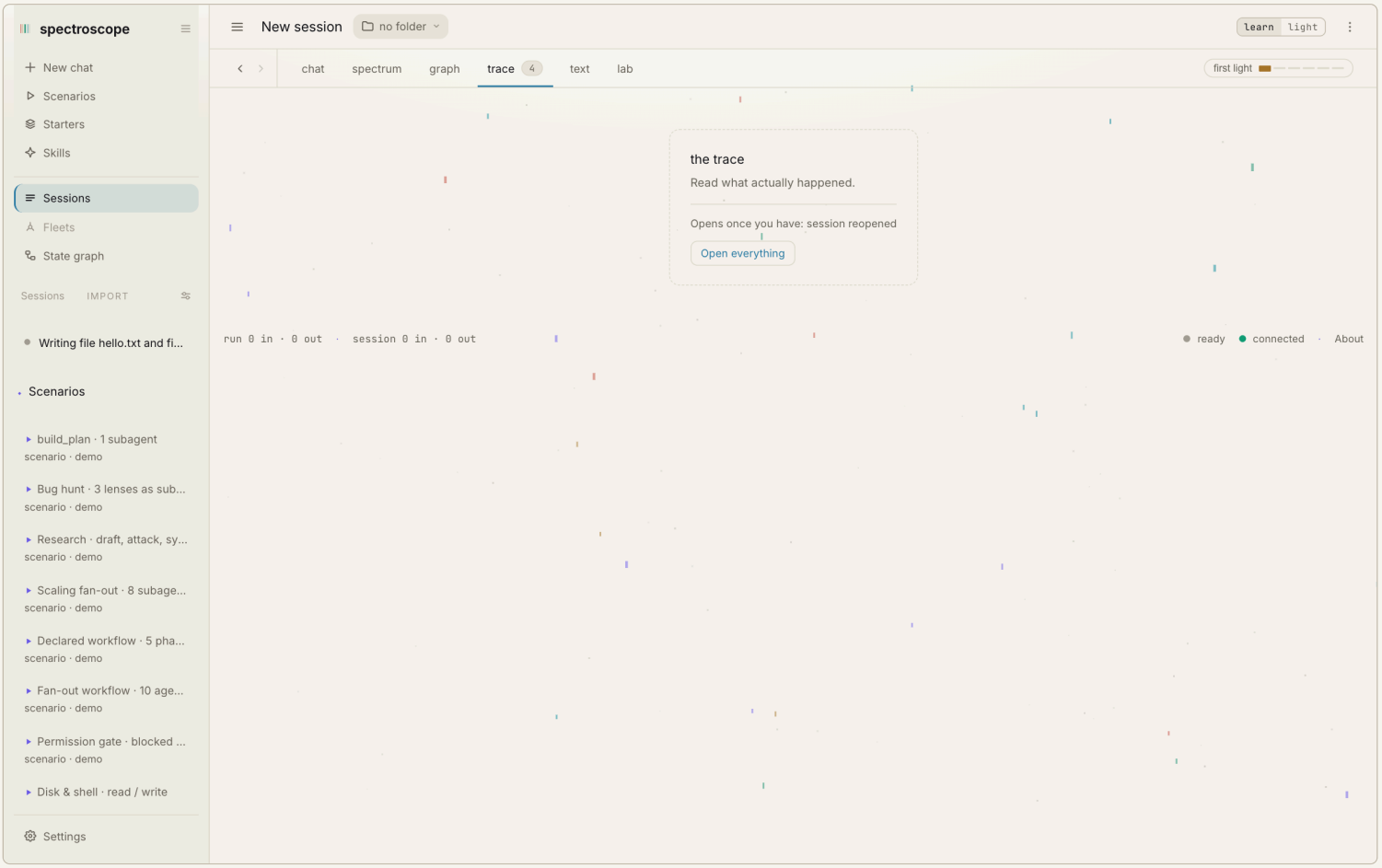


The progress panel: every step, every criterion with what it counts, and a receipt that opens the exact frame it was earned in.

A criterion you tick by hand says so, permanently. The tutorial distinguishes what it saw from what it was told, because a progress display that cannot tell the difference is decoration.

NOTHING IS EVER HIDDEN

A state you have not reached still shows its tab. Clicking it gives you a card naming what the surface is, what opens it, and a button that opens everything immediately and permanently. Two surfaces are never locked at any level: the settings, because that is where keys live, and the permission gate, because a run waiting on a decision must always be answerable.

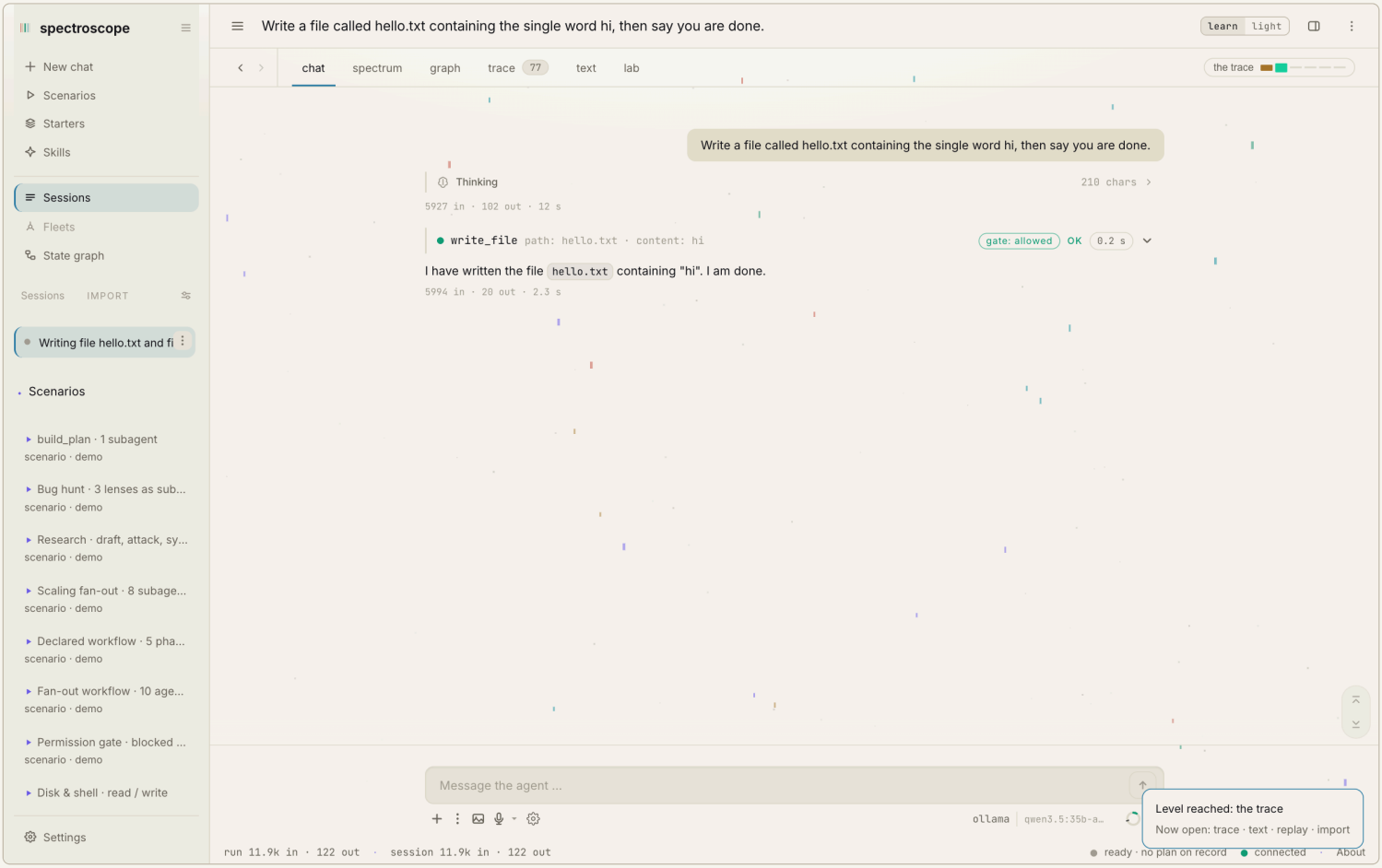


The trace at first light: the tab is there, it is selected, and it says what it is and what opens it. A locked surface argues its own case.

The card is the whole mechanism in one screen. The tab stays where it was, because a feature nobody can see is a feature nobody adopts. What it shows you is the one criterion still outstanding, not a percentage, and the button beside it ends the whole arrangement then and there.

reaching a step

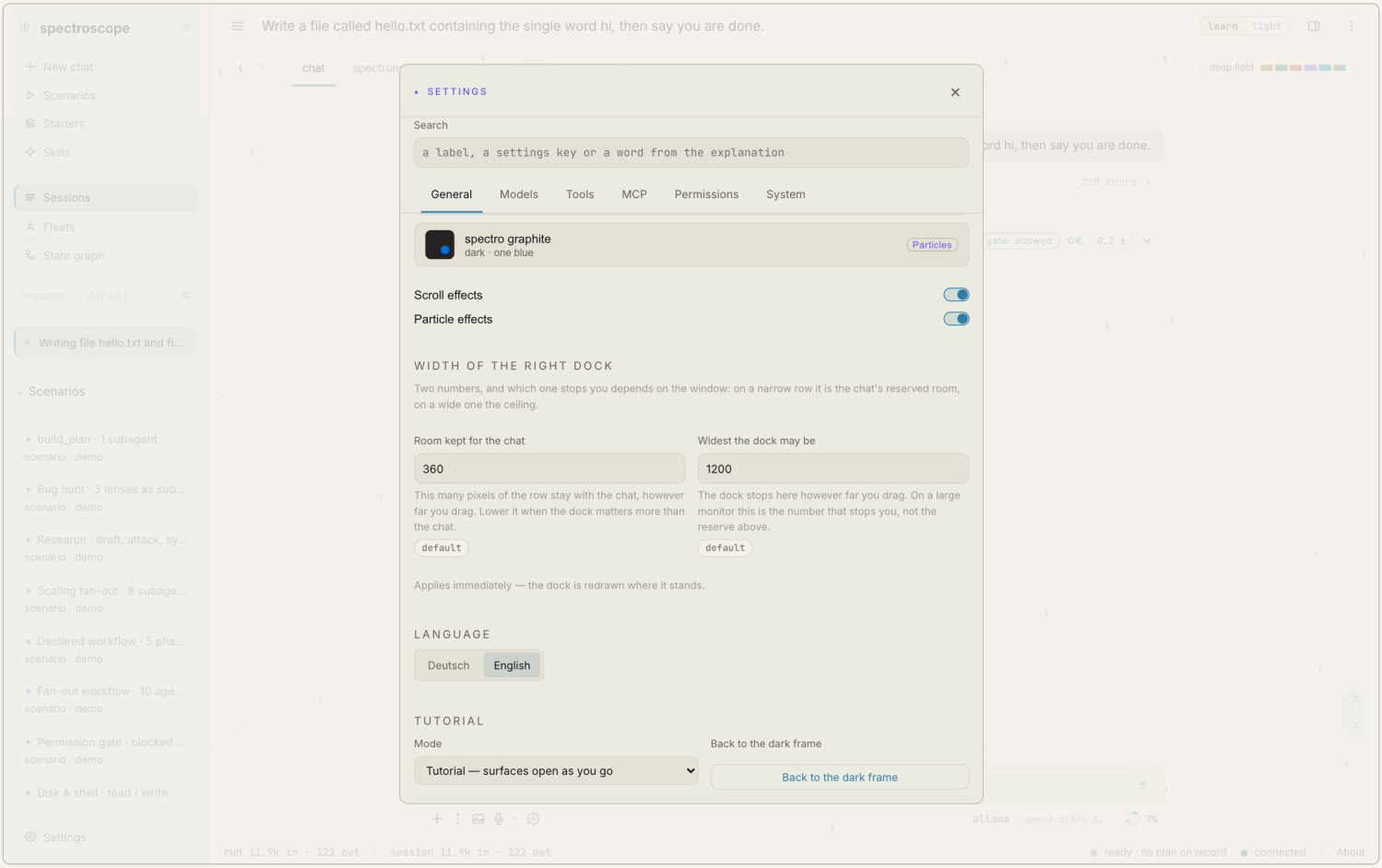
A climb is announced once, briefly, and names what it opened. The new line on the strip ignites as it lands. It blocks nothing and needs no dismissing, and after seven seconds it is gone again, because a tutorial that congratulates you at length is a tutorial standing in your way.



Opening a finished run settles the last criterion of first light. The toast names the step and what it opened; the second line on the pill is mid-ignite.

turning it off

Settings has three modes. **Tutorial** opens surfaces as you go. **Checklist** locks nothing but still tracks and celebrates, which is what every existing home gets. **Off** removes the pill and the panel and stops recording entirely. The same block restarts the tutorial from the dark frame; restarting does not ask the tutorial question again.



Two controls and no ceremony: which mode the tutorial runs in, and one button back to the dark frame.

The CLI reports the same state without a server: `spectro level` prints the tutorial with its ticks and receipts, and `spectro doctor` carries one line of it. All three faces read the same `levels.json` from the core, so they cannot drift apart.

NO KEY REQUIRED

Every criterion is reachable without a paid API key. A local model through any of the free backends counts as light, the bundled scenarios count as fan-outs and fleets, and the walkthrough from dark frame to deep field has been done that way.

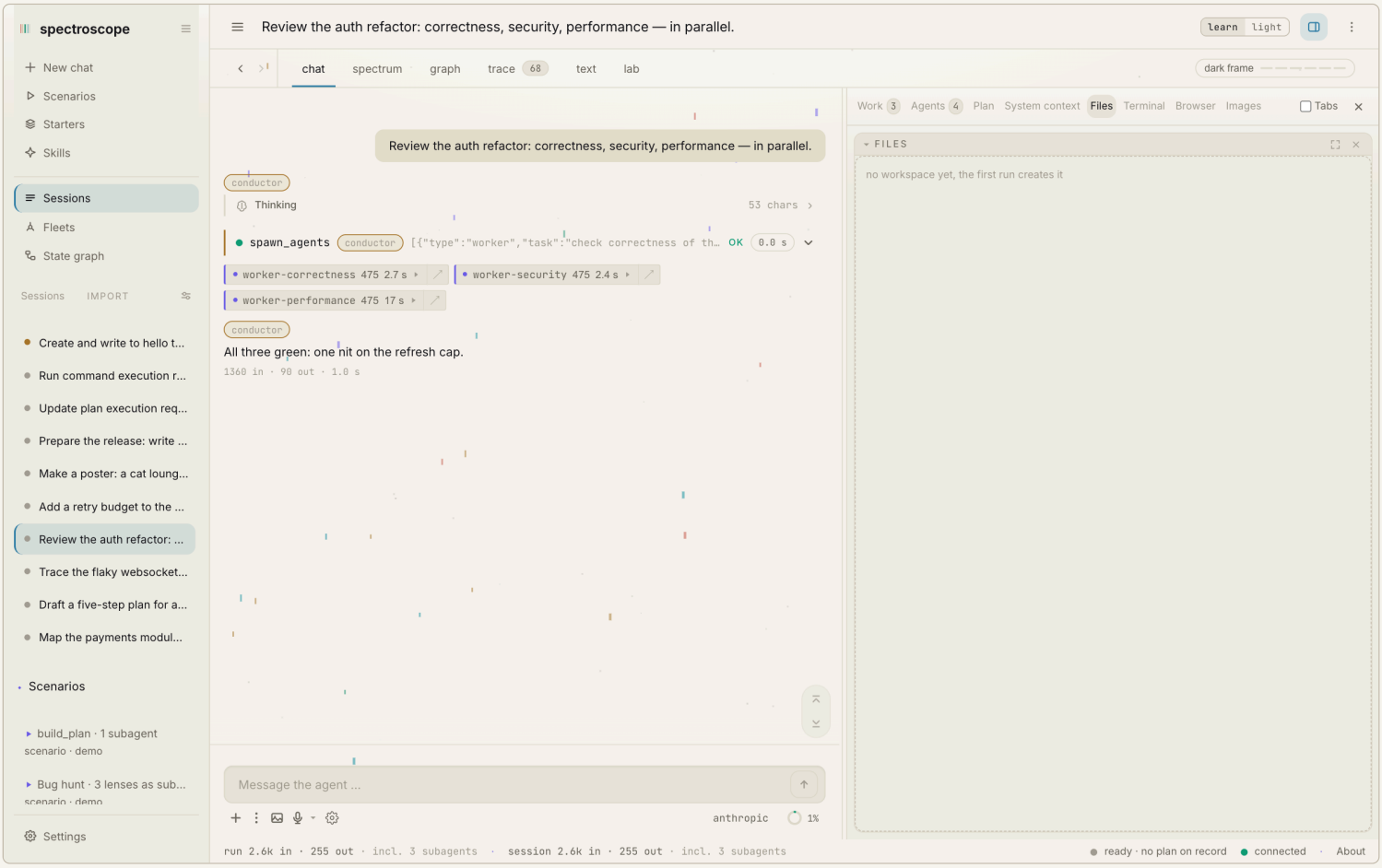
CHAPTER 8

Sessions: continue, delete, import

Every run writes a JSONL session file as it happens — there is no “save” step, the file *is* the state. This chapter is about what you can do with those files from the UI; Part IV shows them on disk, and chapter 22 documents every line.

The list

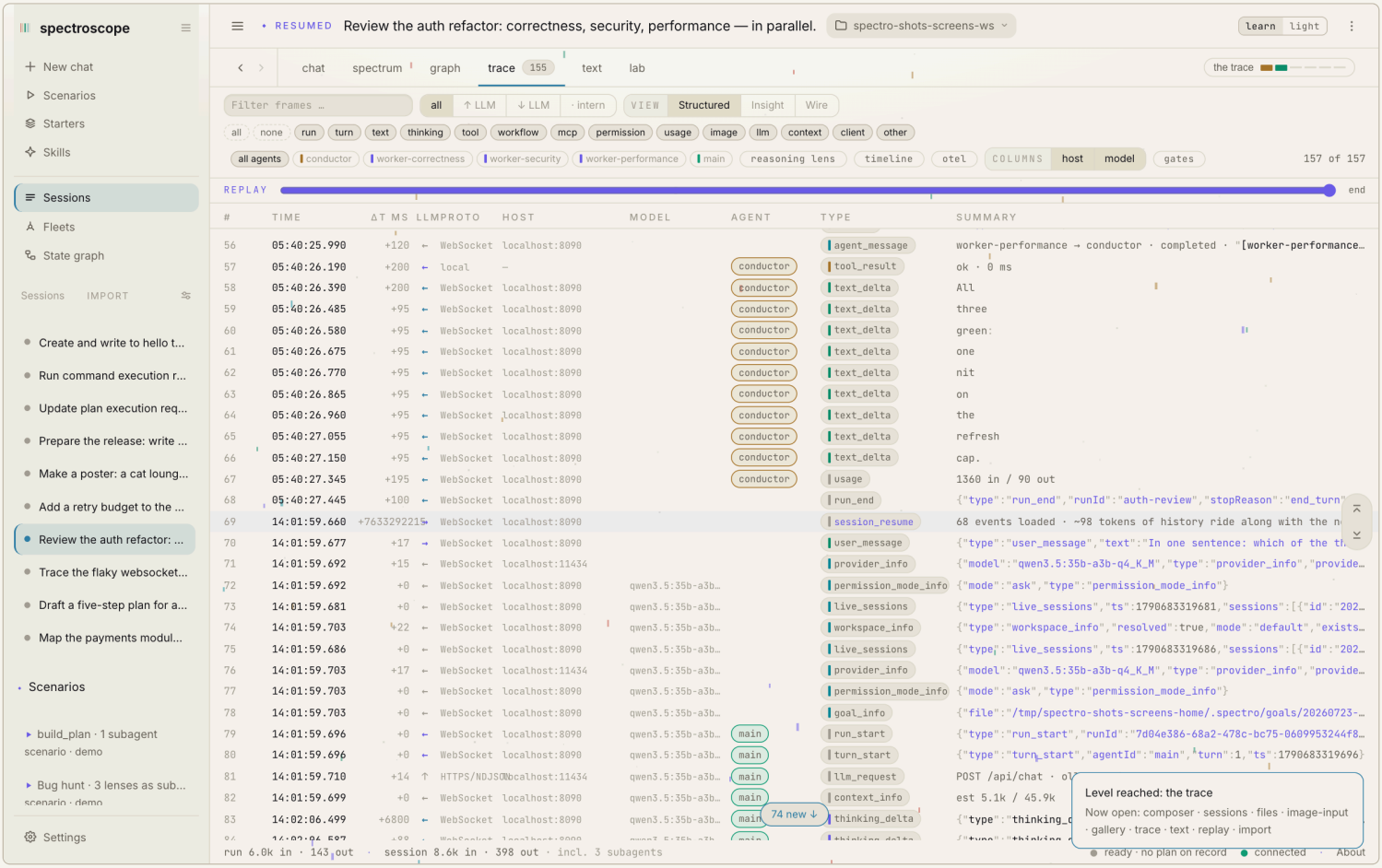
The sidebar lists every session newest-first, one row each: the title (the first prompt until the model suggests a better one), relative age, token total. A new chat gets its own row once it has started, and no session appears twice. Click a row and the session opens with its history and an empty composer. Chat, Graph, Trace and Lab all work on it.



A stored session, opened from its row: the history above, an empty composer below, ready for the next message. The list has one row per session and no separate row for this browser tab.

Continue, with the re-upload made visible

Type into a stored session and your message continues it. There is no Resume button. When you send it, the page opens a socket with `?resume=<id>`, the server reconstructs the provider history from the file (main agent only, orphaned tool calls dropped, see chapter 22), and new events *append to the same file*. The session runs on the provider and model its last run recorded, even when the window has used another pair since. The header eyebrow reads *Resumed*.



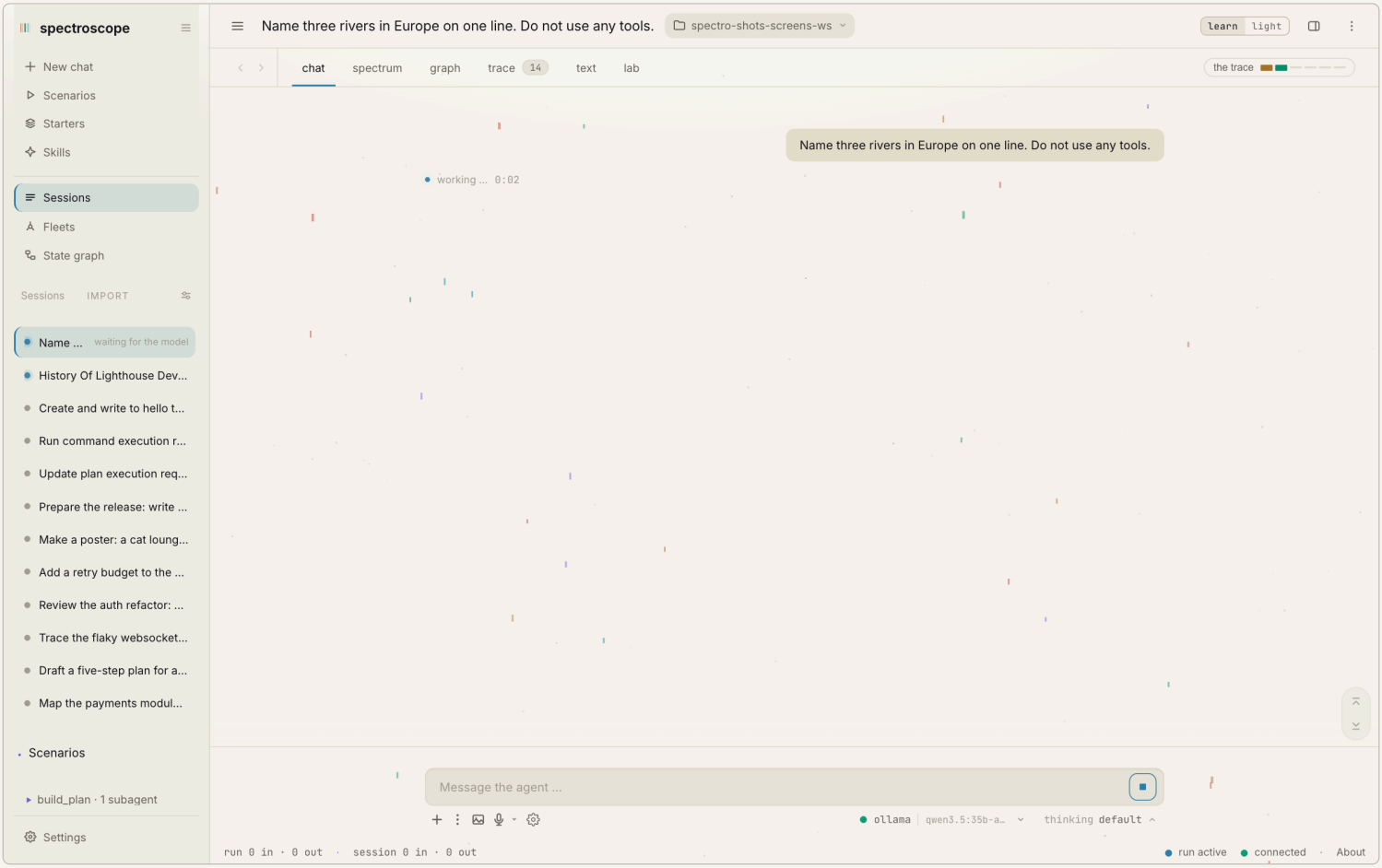
> After a stored session is continued, the trace shows the whole reconstructed history followed by the session_resume marker row: “n events loaded · ~t tokens of history ride along with the next request (plus system prompt & tool schemas on top).”

The didactic detail: continuing is not free. Your next prompt re-uploads the whole conversation to the model. The trace makes that visible with a UI-only `session_resume` **marker** between history and new traffic, summarizing how many events and roughly how many tokens now ride along. The estimate mirrors what the server actually reconstructs: main agent only (a subagent's inner steps never ride back up), thinking excluded (it never re-enters the provider history). The marker also says explicitly that the system prompt and tool schemas come on top, so the next `usage` event's real jump lands where you expect it. The marker's detail view carries the entire re-uploaded history as its JSONL lines.

Some sessions open read-only, and the line where the composer would be says why. Imported sessions and scenarios have no file on this machine to append to. A session that another window is working in stays read-only here until that window lets it go.

Several at once

Sessions run side by side. Start a long run in one session, then click another row or New chat and send a message there. The first run keeps going, and its row pulses until it is done. Each session keeps its own provider and model: one can run on a local model in LM Studio while another runs on an ollama cloud model, and the provider menu always applies to the session you are looking at.

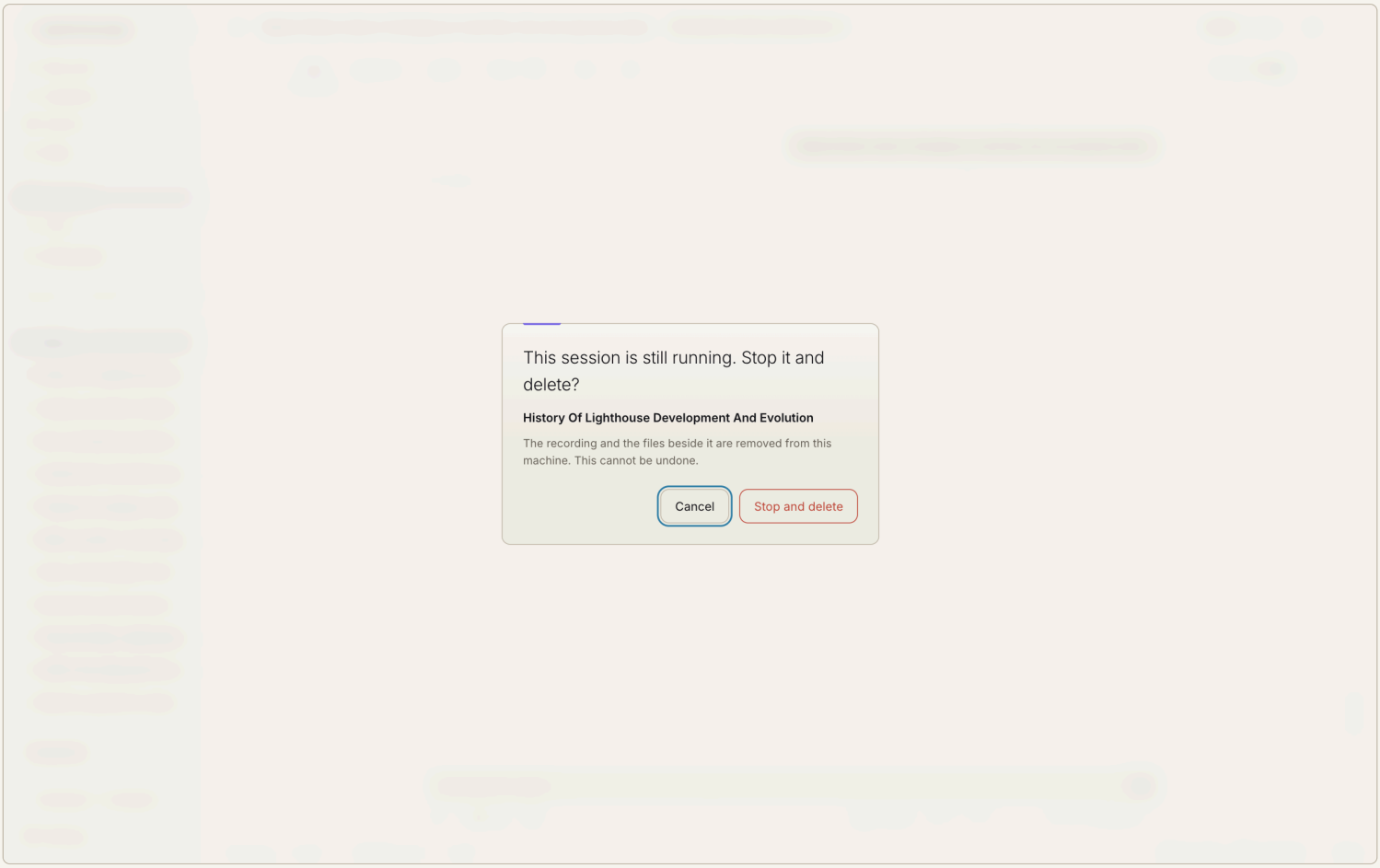


> Two runs at once on one local model: the first session (a long essay on lighthouses) keeps running while a second one starts in view, and both rows carry the running dot. While the model serves the first run, the second row can read "waiting for the model".

A row also tells you when a session needs something. *waiting for you* means it asked a question or waits for a permission; a banner with *open it* appears as well, whatever you are looking at. *waiting for the model* means the run has no output yet while another session streams on the same provider. The app infers that second label; the backend does not report it.

A few settings still apply to the whole window: the text you are typing (switching sessions keeps the draft), the image backend, the Thinking switch, and a base URL changed in your user settings. The desktop app asks before quitting while sessions are working ("2 sessions are still working. Quit anyway?"). Quitting stops those runs, and their history stays saved.

Delete



Delete from the row menu of a session that is still running: the dialog asks first and names what goes. Cancel keeps everything.

Each row has a menu at its right end: pin, rename, suggest a title, delete. Delete opens a dialog that names the session and says that the recording and the files beside it are removed from this machine for good. Nothing happens until you confirm. If the session is still running, the dialog asks “This session is still running. Stop it and delete?” and the button reads *Stop and delete*: the run is stopped first, then the files go.

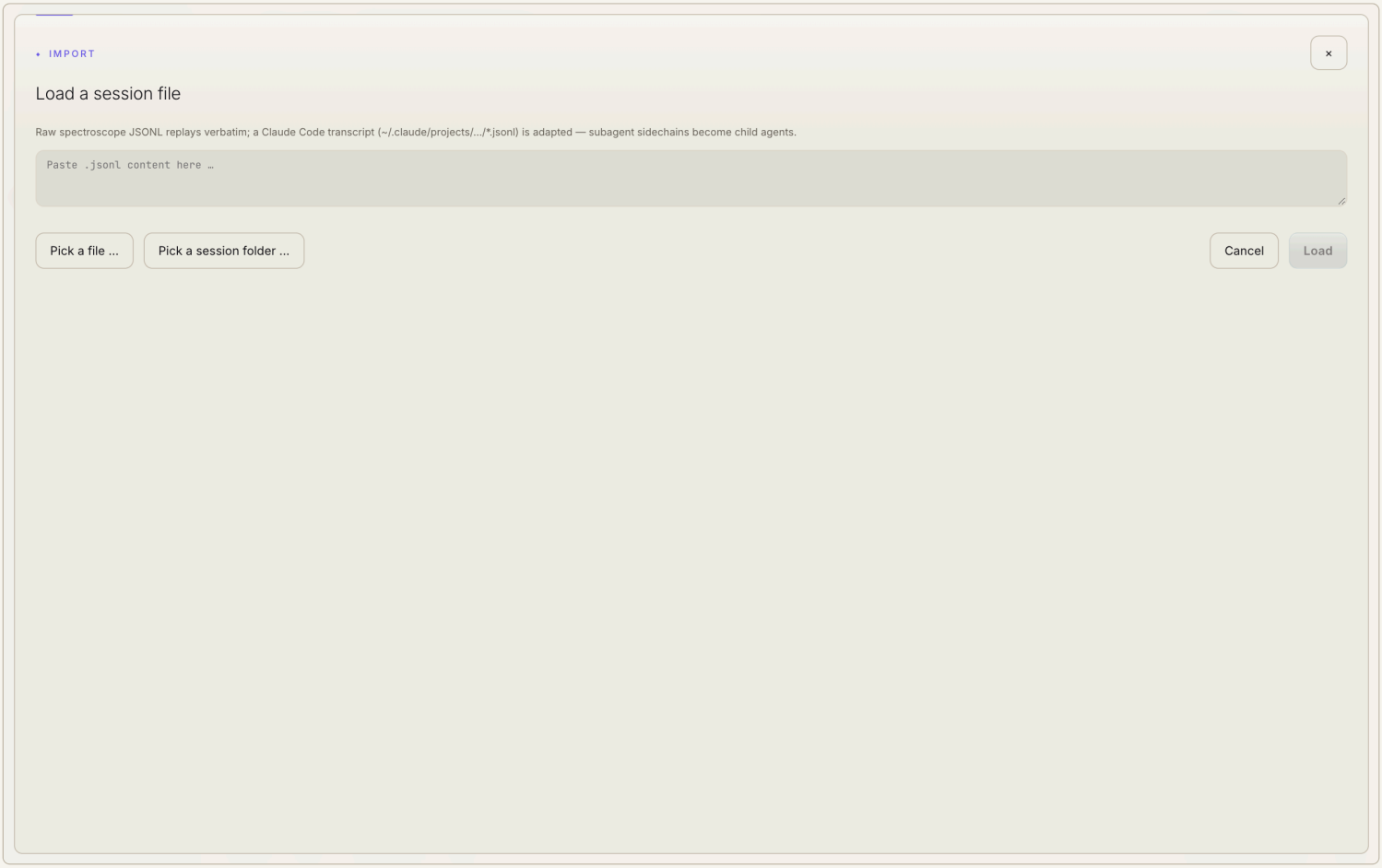
The server guards the delete as well: the REST endpoint (`DELETE /api/sessions/{id}`) guards the id shape before touching anything, and the store itself refuses any id that does not resolve to a direct child of the sessions directory, so a traversal id deletes nothing. Success removes the JSONL *and* the session's blob folder and refreshes the sidebar.

Import

The Import dialog replays *foreign* files through the same pipeline, and it is a room rather than a picker: full screen, with only the row list scrolling. Three ways in. The transcripts the server finds under `~/.claude/projects` are listed and load on one click — that folder is Finder-invisible, so a file picker cannot reach it — and beside the list sit an ordinary file picker and a paste box for everything else. Detection reads records rather than the first line, because a real transcript opens with metadata records before its first message, and the first record naming a format decides which of three it is.

- **Raw spectroscope JSONL** — recognized by its event types, eighteen of them, and replayed verbatim. Files from the TypeScript edition work identically (the format is the contract).
- **Claude Code transcripts** — recognized by their `message` records and translated by an adapter: tool uses become `tool_call` s, `Task` / `Agent` spawns become `agent_spawn` + A2A messages, thinking becomes `thinking_delta` , sidechains fold under their owning task. You can watch a real Claude Code session step through the Lab's map.
- **VS Code agent-mode exports** — dotted type names over a `data` object. That export records which tools ran and whether they succeeded, never what they returned, so the dialog states it in one line rather than leaving a screen of empty tool bodies to read as a broken import.

A file matching none of the three is refused by name: the message lists up to five of the type names it actually found and says where each format lives, because “unrecognized format” sends you back to the file with no idea what to look at.



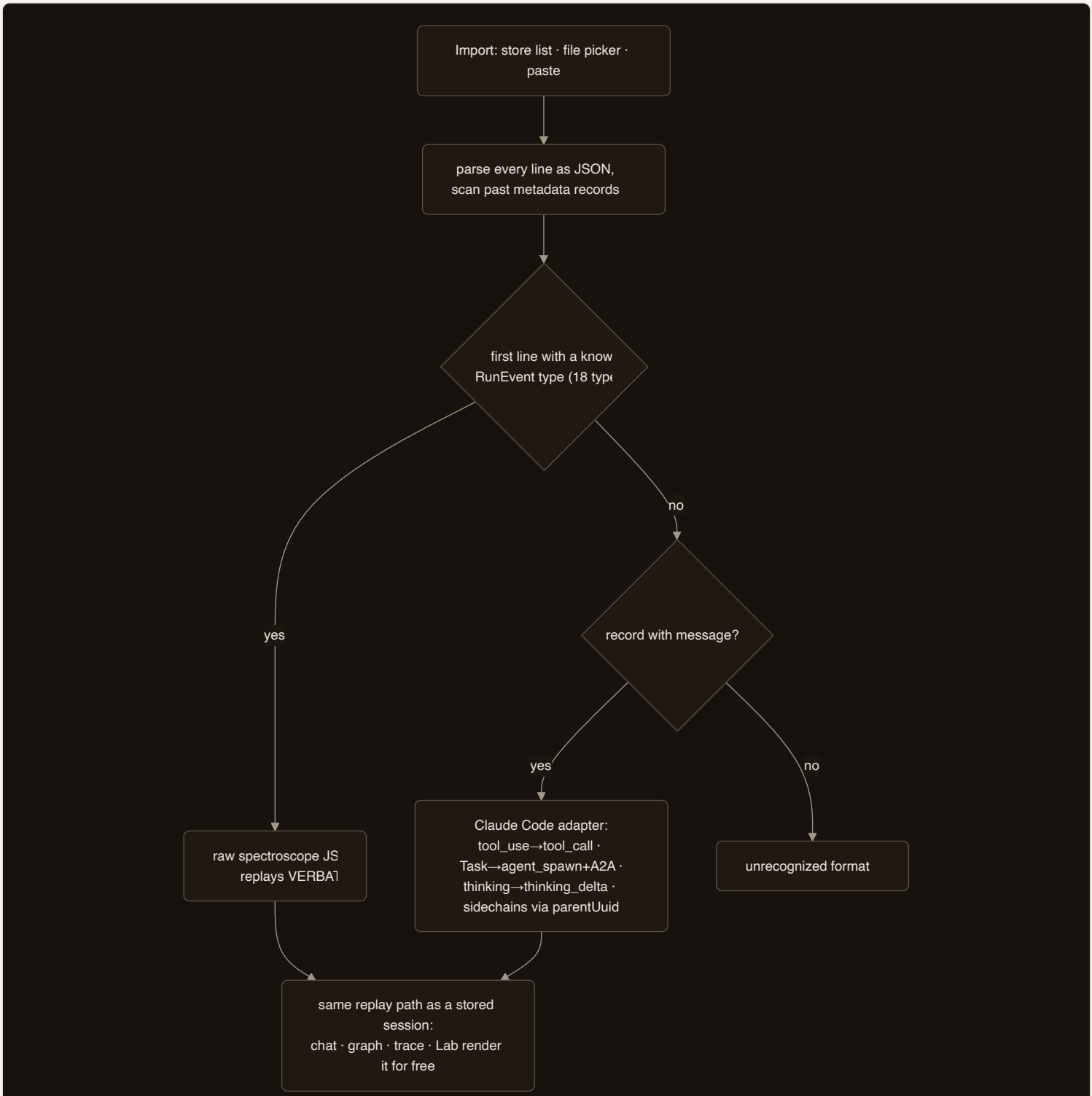
> Three ways in: the store list from ~/.claude/projects, an ordinary file picker, and a paste box. The filter bar, the statistics line and the filled-in rows described below stand where the list does, once the store has transcripts in it.

Two axes narrow the list, and they behave differently on purpose. The `model` chips are the families the facts have turned up so far, and picking a second one *widens* the result; the three `with` chips — `workflow`, `subagents`, `images` — all have to hold, so picking a second one cuts deeper. A search box matches file, project, model and opening prompt. One statistics line then adds up whatever is left standing: the transcripts, the span they cover, each model family with the number of sessions it spoke in, and the workflow, subagent and picture totals. Beside those stands "*n* not read yet", because a total that silently omits forty rows is not a total.

Facts arrive per visible row. The listing is one directory walk and answers in milliseconds; reading transcript bodies does not, so rows render from the listing and fill in behind it. They are batched as you scroll, twenty-four to a request, and the cap comes from a number the server publishes in its own answer so the client cannot drift past it. A row that has not answered is neither a match nor a miss — it is pending, and the dialog says how many are still out instead of quietly shrinking the list to whatever happened to be cached.

A row carries the file name, the opening prompt, the project, the age and the size, then chips for what is inside it: the model families (the tooltip keeps the verbatim ids), `workflow ×n`, `subagents ×n`, `workflow-agents ×n`, `images ×n` and the language. None of that says what the session was *about* — an opening prompt is often "read the card first" or four thousand characters of pasted log — so one button asks the configured model and writes one line per row. It is a press and never a side effect of opening the dialog, because it is the only thing on the row that costs a model call. One call per session, sixty per press at most, and the answer is capped at 120 tokens. The instruction is to write twelve words at most, and to answer *unclear from the opening prompt* rather than invent a subject. The lines live on disk keyed by each file's size and modification time, so a transcript that has grown is offered again and its old line is marked stale; the tooltip names the model that wrote it.

Two ceilings govern what you get to see. The listing is the three hundred newest transcripts, and it says so when that cap fired — a bigger store is never quietly reported as a store of three hundred. The content endpoint serves at most 128 MiB, so a transcript above that is disabled before the click, with the reason on the row rather than a status code after it. A refusal you still get means the file grew between the render and the press, and the server names both numbers. Nothing about an import is written to disk, and the bar over the replay says so. It names the file and counts the conversation in prompts and answers. The file's own counts sit behind the bar's Details control: how many lines arrived, how many frames the view is built from, and how many lines were not matched. On a Claude Code transcript the last number is almost never zero, and the bar does not warn about it. Those files also hold records that belong to no conversation, such as titles and reminders the client adds, and the importer skips them on purpose.



Detection scans past the metadata records for the first line that names a format: spectroscope JSONL replays verbatim, a Claude Code transcript runs through the adapter, and both feed the same replay path as a stored session. The picture predates the VS Code branch, which sits beside the Claude Code one.

What an imported transcript says

A Claude Code record carries the conversation and everything the client wrote down around it, and the second half used to reach nobody. It does now, and every reading below obeys the same two rules: a line that does not carry a field produces *nothing* — not an empty row, not a dash — and values travel verbatim, because each of these is a vocabulary somebody else extends without asking.

- **The tool's own return value.** A record with a tool result can carry a `toolUseResult` beside it, structured, while the block holds the flattened text the model was shown. A `Read` sheds the line-number gutter the block welds onto the body, and states the page that came back instead of counting the flattened text. `Bash` keeps `stdout` and `stderr` apart, which the block runs together with no marker. An edit's `structuredPatch` says where the change landed, where the block says only that the file was updated, and a task update carries the state it moved out of. Two shapes are refused rather than half-read — a string, which is always an error the block already states, and an array, which is MCP output the block duplicates byte for byte.
- **Compaction.** A boundary becomes the `compaction` frame the wire has carried all along, counting the turns that went and how long the summary ran. The machine-written summary is out of the chat bubble, where it had been rendering as your own words — and it is now readable nowhere in the app, because every face hangs off a row and a line that produces no frame has no row. The bytes stay in the transcript on disk. Giving them a carrier is an open call rather than an oversight.
- **Failures that read as failures.** An API error record, and the outage text the client writes into the assistant channel, both become `error` frames. A retry ladder used to be a silent gap in the clock, and an overload message used to read as the model's own answer.

- **What a subagent cost.** A modern transcript does not contain its own children. Everything about one therefore comes off the single record the parent wrote when the call came back: the child's token bill, which maps onto the wire's `usage` event under the child's own id, its wall time, its tool count, and whether it was launched in the background and never reported back inside this file. The model it ran on travels with it, verbatim down to the context-window suffix. It is read and never inherited: a child runs a different model from its parent often enough, and sometimes a whole tier down.
- **Who was driving.** Chips on the trace row name the skill, the MCP server and its tool, the effort the turn was told to spend, who wrote a user turn when it was not a person, an answer that stopped on a limit rather than on its own ending, and a model swapped under the run. Names travel with their plugin prefix; spectroscope looks nothing up.
- **Where the run stood.** The working directory, the git branch and the client version are announced once and again at every move, each with what it left behind. Nearly two thirds of the session transcripts this was measured over stand in more than one directory, and every relative path in every tool result after a move means something else than it did before.

None of this touches the wire. These are readings of somebody else's file, and the readings with no home on the wire ride frames that are import-only — no writer in the Java or Python core constructs them, and they can never land in a session file.

The agents beside a session

A session transcript holds only its own start. Every word its agents said lives in sibling files, under a folder derived from the transcript's own name — direct spawns in it, and the agents of a workflow run one level below, under the run id. Opening a session asks the server which of them are there, and that costs one directory walk and no transcript read. The work panel lists what came back: how many sit beside this file, each row an agent id and a size, each openable as a session in its own right. Where the task's own claim and the files disagree, both are shown and neither is corrected.

On the chat and on the trace, buttons open the folders a recorded session left behind — the transcript's folder, the workflow folder, the scratchpad. These are the ones the app reads constantly and could never reach, because the store hides under a dot-folder and the scratchpad under a temp path nobody would guess. The server decides which of the three exist, since only it can stat them, and only the *kind* travels back: an endpoint that opened a path off the wire would be a way to run your file manager on any file on the machine.

A standalone agent transcript imports as a session too, and the shape decides rather than the name. A file whose every record is a sidechain, with nothing in it that could own them, is not a session with orphans in it — it is one agent's transcript, and it names that agent on every line. The bar says which agent, what kind of agent it was and which session it ran under, for as much of that as the file states. A copied or renamed file is read for what it holds.

> KNOWN LIMITATION
>
> The agents beside a session are a *listing*, not a join. Opening one replaces the view, and the parent's trace, spectrum and graph gain no child rows, because a real join would mean reading several thousand files to open one session. The folder is derived from the transcript's location, so a session split across two project directories answers “none”, indistinguishable from one that spawned nothing. And a pasted or file-picked transcript has no address in the store at all: no facts row, no gist, and no agents beside it.

The address of what you are looking at

There is no copy-link button anywhere in spectroscope, and that is the design: the browser's address bar *is* the link, and the app keeps it current as you move. A stored session reads `#/session/<id>`, with `@<n>` for the frame you are standing on and a trailing segment for the tab — `chat`, `spectrum`, `graph`, `trace`, `text` or `lab`, the six literals being the six words in the tab row. A transcript opened from the store is an address as well, `#/import/<path>`, the path percent-encoded so its own slashes cannot be misread as the tab. A paste, a picked file and a scenario are views and not addresses: there is nothing on disk for them to point at, so the bar falls back to the plain address rather than invent an id. Two tabs add *how* they are being read — the trace its open row and the categories left on, the spectrum its zoom window as two fractions of the span. No other tab reports anything, so a lab or a graph link says only which session and which tab.

Following an address plans the smallest change that gets you there. One naming the session already on screen is a seek or a tab flip, never a refetch. Anything the app cannot believe — a malformed frame index, an unknown tab, a filter category this build does not know — is dropped in favour of what it does understand, because landing wrong is worse than landing home. Reading down a trace or dragging a zoom *replaces* the history entry instead of pushing a new one, so fifty readings of one place do not bury the place you came from. The header draws back and forward chevrons, dark when there is nothing behind them, and ⌘←/⌘→ do the same. The desktop shell is a bare window with no browser chrome, which is why a deep link is something you read in a browser at `localhost` and not in the packaged app.

New chat semantics

One socket connection is one server-side session: one agent, one JSONL file. The page keeps one socket for each session it has open. “New chat” therefore opens one more socket beside the others: fresh agent, fresh file, fresh state. The session you came from keeps its socket and keeps running if it was running. Two browser tabs are two independent windows with separate agents, files and remembered permission rules. A session that one window is working in opens read-only in the other, because the server refuses a second writer. Only persisted rules in `.spectro/settings.json` are shared, which is why that writer is serialized.

CHAPTER 9

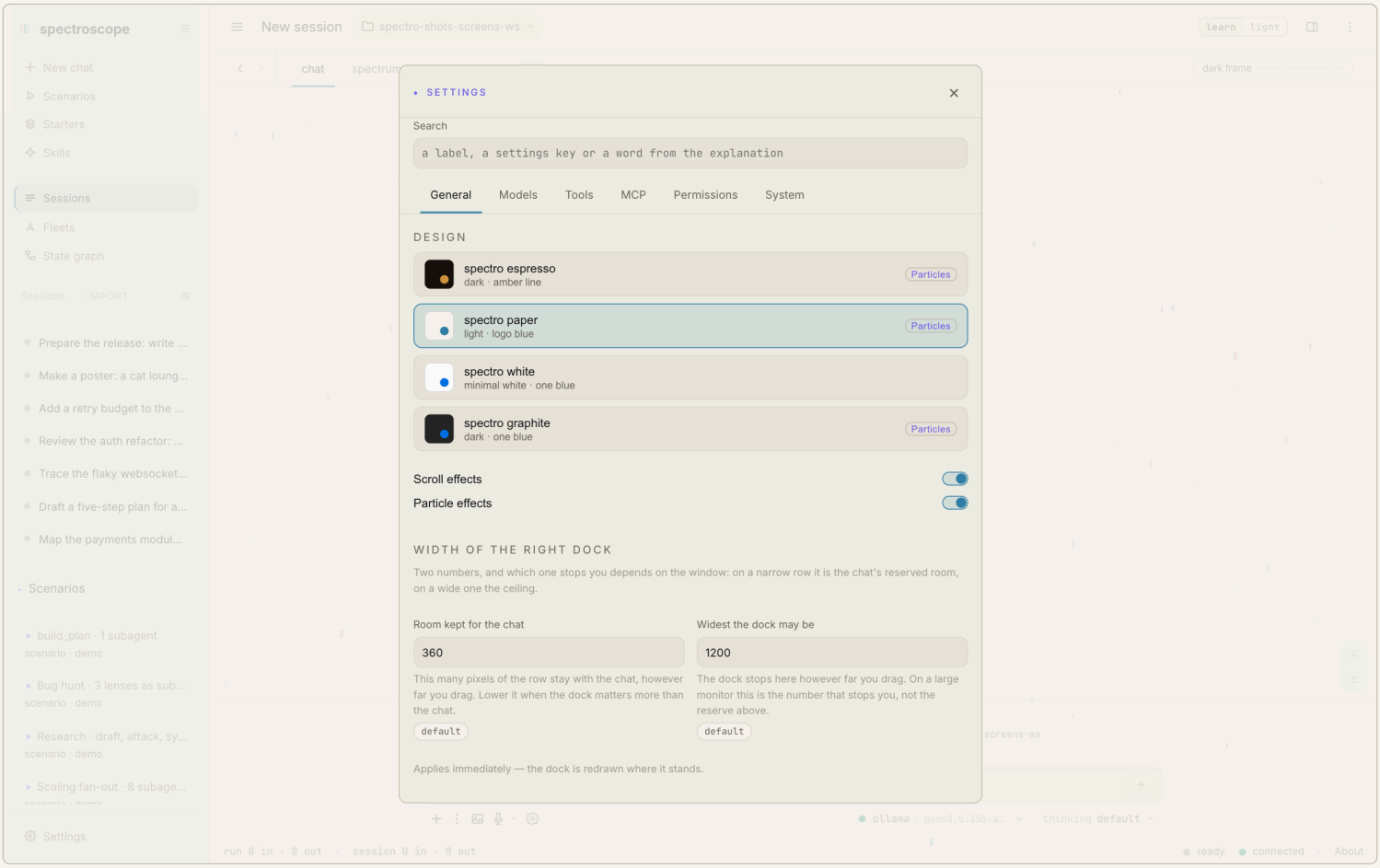
The design system

The whole UI reads its colours, fonts, radii and motion through CSS custom properties. That single indirection makes two things cheap: swapping the entire look (a *design*) and painting effects that always match.

The token base

The master vocabulary is brand-strict: espresso `#17120D` as the ground (never pure black), **the logo's amber line** `#CE9440` **exclusively for interaction and live elements** (links, buttons, focus, running indicators, the packet), **violet** `#8B7CF0` **as the editorial accent** (eyebrows, kickers — never clickable), desaturated status colours, and pastel agent accents that are decoration only. Focus is always visible and always the accent; reduced motion switches every animation off globally; numbers render tabular so nothing jitters mid-stream.

Three designs

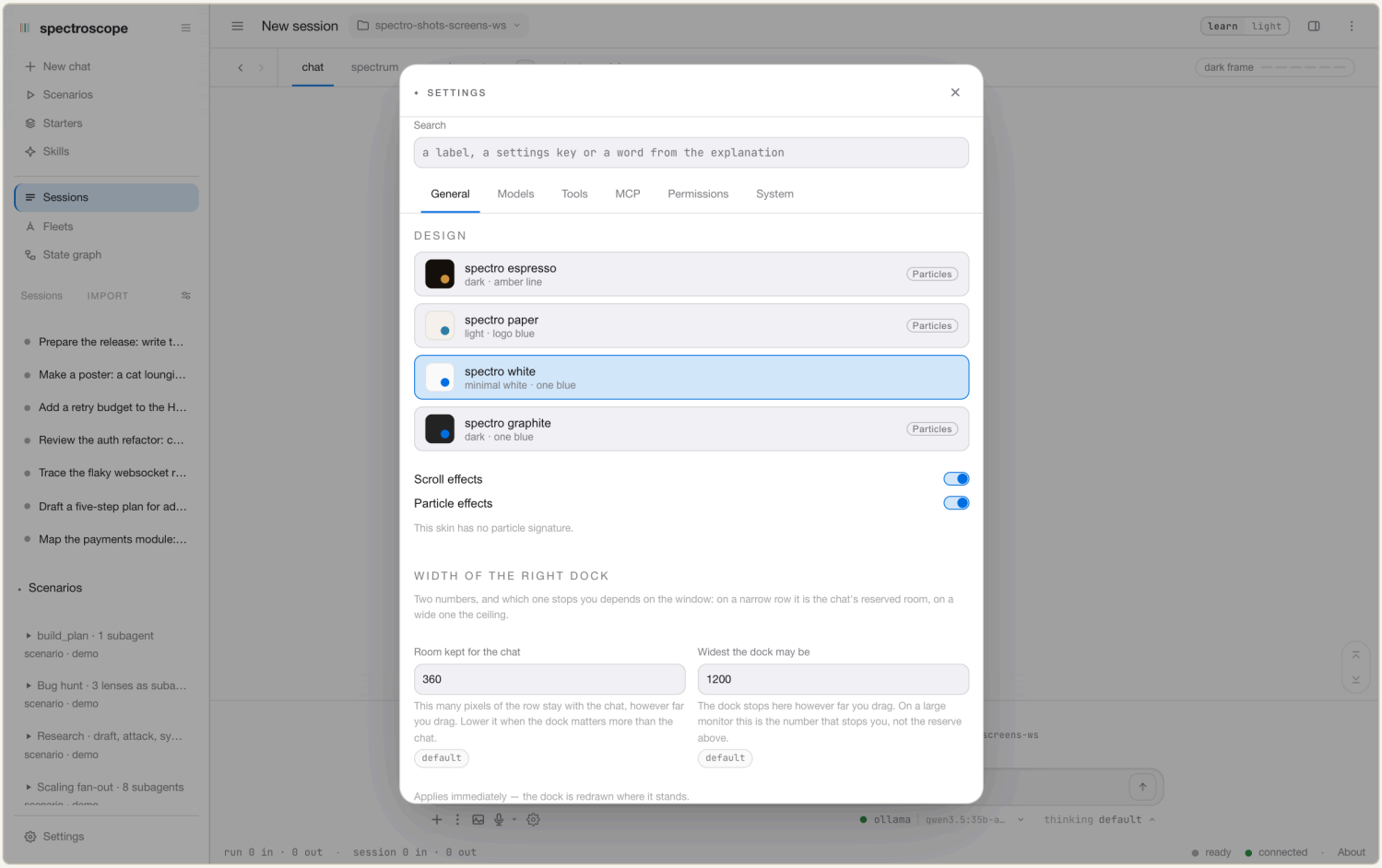


The design section of the settings page: four designs with swatches, the effect switches beneath them and the two widths of the right dock. A choice applies live and persists in one step.

A design is a token override set — nothing structural changes. Selecting one flips a `data-design` attribute on `<html>` and the UI re-expresses itself instantly, no reload; a FOUC guard applies the saved design before first paint. There used to be a shelf of extra skins; the picker was pruned to the three brand designs, and any retired id left in older browser state folds back to the default.

DESIGN	DATA-DESIGN	CHARACTER	GROUND / ACCENT	PARTICLES
spectro dark	`spectroscope`	espresso · amber line (the default)	`#17120D` / `#CE9440`	brand dust
spectro bright	`paper`	LIGHT — paper · the logo blue	`#F6F4EE` / `#2E7EA6`	ink dust
spectro white	`still`	LIGHT — minimal white, gray, one blue	`#fbfbfd` / `#0071e3`	— (deliberately still)

The two **light** designs flip `color-scheme` and re-derive the status colours, agent pastels and the whole shadow set for a light ground — proof that the token vocabulary carries a full polarity flip, not just a palette swap.



spectro white: the same UI in the quiet all-white design — every component reads var(--token), so the flip is total, live behind the settings page.

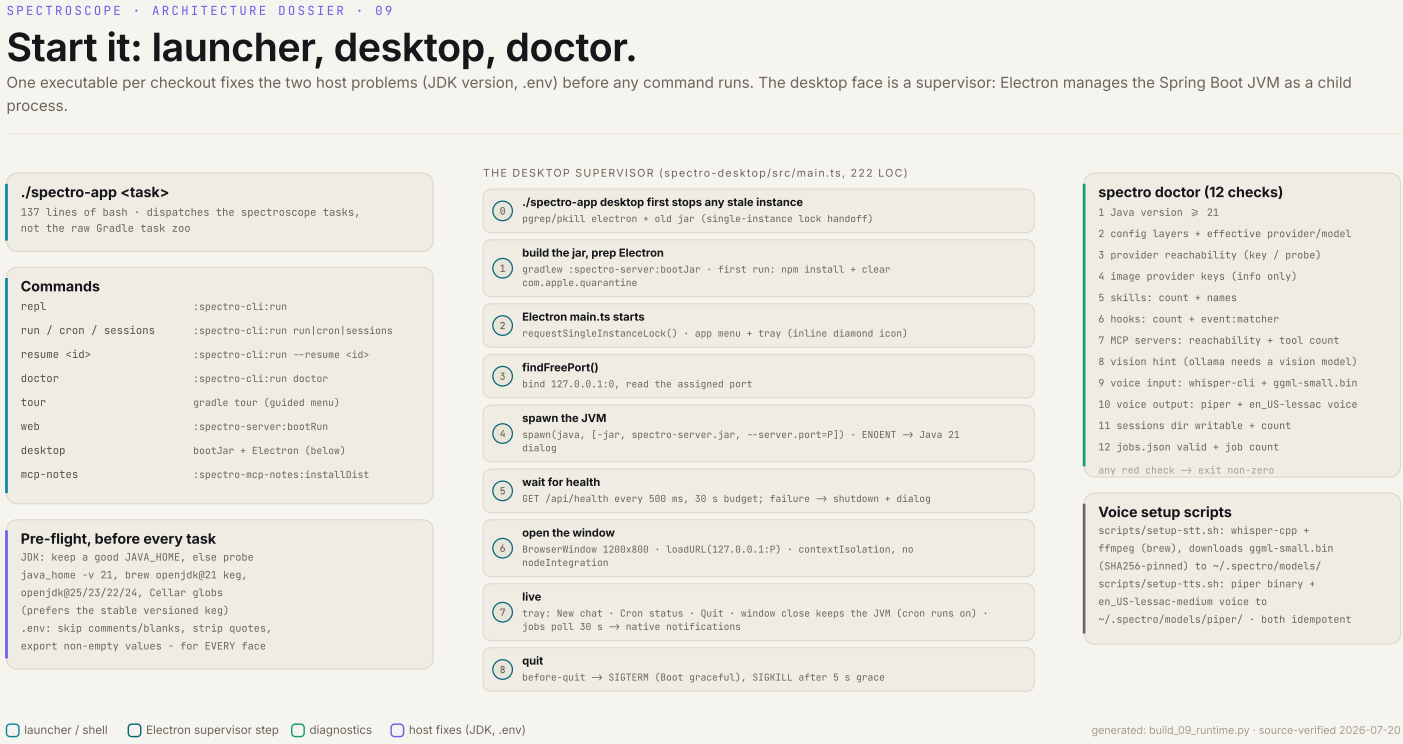
The effects layer

Two independently togglable effects, both reading the *active* design's colours live: the **particle field** (a canvas behind the UI with the brand dust drift — spectro white deliberately renders none) and the **scroll reveal** (panels fade/rise on entry). Both defer to reduced motion; particles pause in hidden tabs. A design choice from the settings page applies and persists in one step; the effect switches work the same way.

CHAPTER 10

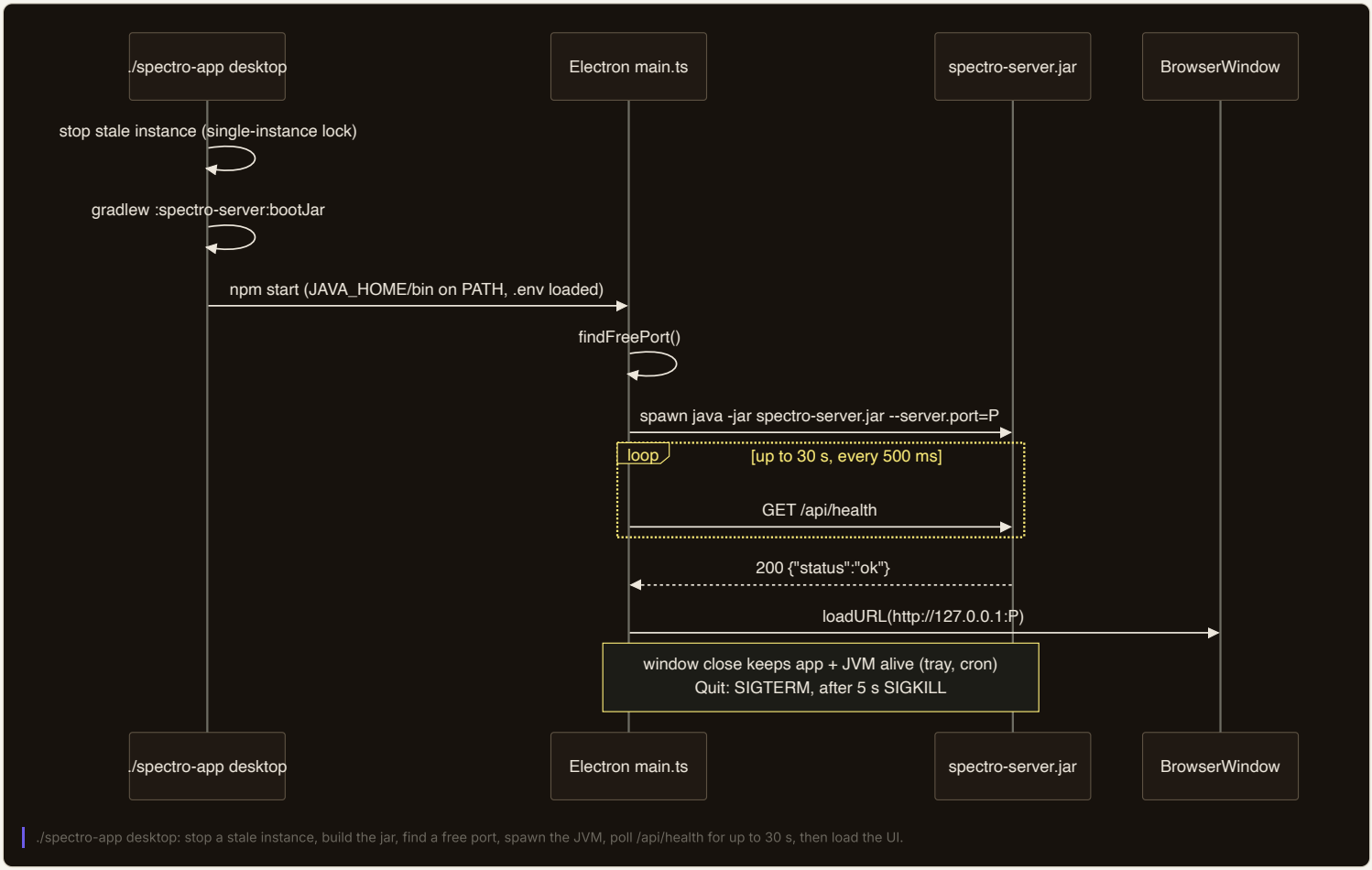
The desktop face

A JVM cannot live inside Electron's main process, so the desktop face is honest about it: a thin shell that *supervises* the real server as a child process and points a window at it. Packaged, it goes one step further and carries everything it supervises — the server jar and a whole Java runtime — so a fresh machine needs no system Java. Double-click, the server starts with it, the cockpit opens. The lesson of this face is process management, not IPC.



The launcher-and-desktop map. ./spectro-app desktop is one of the launcher's commands; the middle band traces the desktop supervisor step by step, from the stale-instance stop through the health poll to the live tray.

The supervision sequence



`./spectro-app desktop` prepares the ground (stops a stale instance — Electron's single-instance lock would otherwise hand a relaunch to the *old* build; builds the boot jar; clears the macOS quarantine flag from fresh Electron binaries on first run; puts the resolved JDK on the PATH). The Electron main process then finds a free port, spawns `java -jar spectro-server.jar --server.port=P`, polls `/api/health` every 500 ms for up to 30 s, and opens a 1200×800 window on `http://127.0.0.1:P` — the same React UI, talking over the same WebSocket. The renderer gets no Node API at all (`contextIsolation` on, `nodeIntegration` off): it is an ordinary web page.

A self-contained run kit

That dev flow expects a JDK on the machine. The packaged app does not. `scripts/build-desktop-runkit.sh` bundles two things into the `.dmg` as Electron `extraResources`: the **server jar** and a **jlink'd Java runtime** (a trimmed JRE built straight from your JDK). Run packaged, the shell resolves both from inside its own bundle — the jar from the app resources, `java` from the bundled `jre/bin/java` — so a fresh machine needs no system Java at all. Double-click the app, the server boots on the bundled runtime, the cockpit window opens: the same React UI over the same WebSocket. (In dev the shell falls back to the `java` on your PATH and the jar from the Gradle build.) The app wears the brand — `productName` spectroscope, a brand app icon that the build script regenerates from `icon.svg` when `rsvg-convert` is present.

- **First launch is a right-click.** The default build is *ad-hoc signed* (free, no Apple account). That clears the "is damaged" gate, but on first download macOS still calls it an unidentified developer: right-click the app → *Open* once (or `xattr -cr`), and it launches normally ever after.
- **Zero-warning distribution is a paid path.** A plain double-click with no prompt needs an Apple Developer ID plus notarization. The bundled JRE is the one twist — its dylibs and JIT need signing and the right entitlements. The full recipe, end to end, is in `docs/DESKTOP-SIGNING.md`.
- **Built for the host.** The bundled runtime and the Electron binary are OS- and CPU-specific, so the script builds for the machine you run it on; a Windows, Linux, or Intel kit means running the build there.

Tray, notifications, quitting

- **Closing the window does not quit.** The tray (an ebony diamond) keeps shell and server alive so cron jobs keep running. Tray menu: New chat, Cron status, Quit.
- **Native cron notifications.** Every 30 s the shell polls `GET /api/jobs/state` and fires a desktop notification for each job whose status *changed* — click to open the window.
- **Quit is graceful, then firm.** ⌘Q / tray-Quit sends the JVM a SIGTERM (Spring Boot closes sockets and finishes the current JSONL line); if the process still lives after five seconds, SIGKILL follows.
- **Startup failures clean up.** If health never turns green, the shell kills the JVM before showing the error — no headless server left behind. In dev a missing `java` shows "Java 21 is required"; the packaged run kit carries its own runtime, so it cannot miss.

CHAPTER 11

Two languages

The UI chrome ships fully bilingual — English (the default) and German — switched live in Settings, on the General tab under Language. This guide's screenshots use the English chrome.

- **Chrome only, content never.** Sidebar, header, dialogs, panels, tool cards, Lab toolbar, the in-map SVG texts, trace chrome, settings page, footer — all switch live (~460 dictionary entries). Chat content, tool payloads and the JSONL wire never run through translation: a session keeps its own language.
- **Wire values stay English.** Plan statuses (`pending / in_progress / completed`), A2A states and stop reasons are canonical English on the wire — only labels localize. Cross-edition compatibility depends on it.
- **Scenarios compile per language.** Picking a scenario compiles it in the current chrome language; the compiled session then keeps that language, like any other session.
- **Late-bound info lines.** Reducer-folded info lines (spawns, compactions) carry an additive key + variables, so lines already in the transcript re-render when you flip the language.

PART III

The agent's powers

Everything the model may do, and everything that keeps it honest: the tool belt, the permission system, hooks, subagents, skills, providers, the senses — vision, voice, images — MCP and the scheduler.

12 The tool belt
13 Permissions, allowlist & hooks
14 Subagents & A2A
15 Skills
16 Providers & models
17 Seeing, hearing, speaking, painting
18 MCP — external tool servers
19 Scheduling

CHAPTER 12

The tool belt

Thirty-nine built-in tools in a session of the app without MCP servers and skills (the table “Who gets which tools” below), plus one dynamic adapter per connected MCP tool. Every tool obeys one iron contract: **it never throws** — failures come back as a string starting `ERROR:`, which the model reads and self-corrects from. Every model-supplied path passes the sandbox. Every mutation passes the gate.

The shared rules

THE PATH SANDBOX

Every file tool resolves paths relative to the working directory and refuses anything that leaves it (`ERROR: path is outside the working directory`). The check compares *real* paths, so a symlink sitting inside the working directory cannot be read or written *through* to a file outside it — a link that stays inside still works, because the rule is about leaving and not about links. Tree-walking tools apply the same test to every hit and never enter `.git` , `build` , `node_modules` , `target` . The one exception is the `extended` permission mode (chapter 13): there `view_file` , `read_file` , `list_dir` , `write_file` , `edit_file` , `glob` and `grep` also accept a path outside the working directory, still resolved through its real path. `view_image` stays fenced. In `extended` the launch file refusal is wider than in the other modes: `write_file` and `edit_file` refuse `.spectro/launch.json` and `.claude/launch.json` in any folder, subfolders of the working directory included, where the other modes refuse only the working directory's own.

THE CAPS

Files: a whole read may take 25 % of the run’s context window, estimated at 3 bytes per token; no file over the 2 MiB fuse is read, edited or searched. Output: 10 000 chars shared clamp (surrogate-safe truncation). Glob: 200 results. Shell: 900 s timeout, pipe drained on a background thread — a timeout means “genuinely hung”, never “output too large”. Web: 512 kB streamed body cap before the text clip.

Files & search

`read_file` free — input: { path, offset?, limit? }

Reads a UTF-8 text file relative to the working directory. The file comes back whole when its estimated size (3 bytes per token) fits a quarter of the context window the run has and it is under the 2 MiB fuse. The window is the one compaction is derived from; when the backend states none, the compaction threshold stands in for it. A larger file is refused with its size, its estimated tokens, the window and the share, and can be read in pages with `offset` (1-based line) and `limit` (line count); a page is counted in bytes against the same bound. `read_skill_file` , which reads the files that ship beside a skill, follows the same rule. *Try: “Read build.gradle.kts and tell me the dependencies.”*

`list_dir` free — input: { path }

Lists a directory, sorted, directories marked with a trailing slash; an empty directory answers `(empty)` .

`glob` free — input: { pattern, path? }

Finds files by glob pattern (`**/*.java`) under a directory — pure Java, pruned walk, symlink-guarded, results sorted and capped at 200. Also granted to explore subagents. *Try: “Find every *.test.ts file under spectro-web.”*

`grep` free — input: { pattern, path?, glob? }

Searches file contents by Java regex, returning `path:line:text` hits — no shell involved, so it works even where `run_command` is denied. Skips binary files and files over the 2 MiB fuse, and names each file it skipped for its size at the end of the answer; invalid regex answers `ERROR: invalid regex: …` . *Try: “Search for TODO across the Java sources.”*

`code_graph` free — input: { action, question?, dfs?, budget?, from?, to?, name?, depth?, top? }

Asks the code graph of the session folder, the `graphify-out/graph.json` that Build code graph, `/graph build` or `graphify update` wrote. Five actions run graphify's read-only subcommands: `query` asks a question in words, `path` finds the shortest path between two nodes, `explain` describes one node, `affected` lists what depends on a node and `god_nodes` lists the most connected nodes. The model sees the tool only when graphify is on the PATH and the folder has a graph, and a graph that links out of the folder is refused. A folder has a graph when `graph.json` and its page `graph.html` both lie inside it; the tool, `/graph` and the folder chip read this one rule. The program is the one the `codeGraphBackend` setting names for the folder (chapter 18), today always graphify. While the folder has a graph the session also lists the skill `code-graph` , built into the app, which tells the model when to ask the graph before it reads files, and `/graph ask <question>` in the chat sends a question with a line that asks for this tool (chapter 4). graphify runs without a shell, stops after 30 s, and the answer is cut at 16,000 characters. A name or question that starts with a dash is refused. Explore subagents get it too. *Try: “Use the code graph: what is affected if CodeGraphJobs changes?”*

Mutation & shell

write_file

gated · remembers path

input: { path, content }

Writes a text file, creating parent directories. Success: `Wrote: <path> (<n> bytes)`. An “always allow” remembers the *full path*, never a blanket approval.

edit_file

gated · remembers path

input: { path, old_string, new_string, replace_all? }

Surgical exact-string replacement. `old_string` must be unique (`ERROR: old_string is not unique (n matches)` otherwise) unless `replace_all`; zero matches and empty `old_string` error readably. A denied edit writes nothing. *Try: “In README.md, change the title X to Y.”*

run_command

gated · remembers first token

input: { command }

Runs a shell command in the working directory via `/bin/sh -c`: stderr merged, output drained concurrently and capped, killed on abort. The timeout is the `commandTimeoutSeconds` setting, 900 s as shipped, and the tool tells the model the number it was built with. Non-zero exit returns `ERROR: exit code <n>` *plus the output* — exit-code feedback is what lets the model fix its own mistakes. An “always allow” remembers only the first token (`run_command:git*`).

web_fetch

gated · remembers URL

input: { url }

Fetches an http/https page and returns readable text (scripts/styles dropped, tags stripped, entities decoded, whitespace collapsed, 10 k-char clip). Network egress on model-chosen input is a side effect — hence the gate. Streaming fetch with finite 15 s timeouts and a 512 kB body cap; non-2xx answers `ERROR: web_fetch got HTTP <status>`. Main agent only.

Metadata & delegation

update_plan

free · main only

input: { steps: [{text, status}] }

Publishes the agent's step list to the Plan panel (chapter 6) as the additive `plan` event. Latest wins — each call replaces the whole plan. The status enum (`pending / in_progress / completed`) is *enforced*: a model improvising `"done"` gets an error instead of polluting the wire. Deliberately not given to subagents — a worker's plan would clobber the display.

use_skill

free

input: { name }

Returns the full instructions of a named skill (chapter 15). Registered only when skills are installed; children get it too, so dev-tool workers can load their role skill.

generate_image

gated

input: { prompt }

Generates an image via the configured image provider (chapter 17). Gated because it is a paid cloud call. The image lands content-addressed under `~/.spectro/images/`; the model sees only a short confirmation, and the user sees the picture in the Images panel.

spawn_agent / spawn_agents

free · parent only

input: { type, task } / { agents: [...] }

Starts one subagent (or up to four in parallel) and waits for the result — chapter 14. Free of a gate because the *child's* tools ask for permission individually; delegation is not an escape hatch.

With `background: true` (card 505) `spawn_agents` returns at once with the children's names. They take the chat's slots as a blocking batch does. Their status lines and results reach the parent as messages at its next step. The parent asks one of them how far it is with `send_message` and collects the results with `wait_agents`. The field is offered only where the parent has an inbox and agent messaging is on. At the end of a turn the run waits for its children and hands each result to the model as a message block from that child. At the turn cap, on a stop or on an error it cancels them and waits up to 30 seconds for them to end.

build_plan · write_spec · develop · test

free · parent only

input: { task }

The four development role tools — thin wrappers over a worker spawn, each pointing its child at a skill (chapter 14).

report_status

free · children only

input: { message }

The child's side of the A2A protocol: one short progress sentence per milestone, surfacing live in the parent's UI. Telling the parent what you do is never dangerous.

`send_message` free · main and children — input: { to, message, summary?, notify_when_idle? }

Sends one message to another agent (card 503). `to` is a name from `list_agents`: a session on this machine, `parent`, or a child of the run; the ref in brackets only when two rows share a name. A child agent reaches only its parent. A child that names another child or a session is refused, and the refusal tells it to send to `parent` instead. The parent decides what to pass on, and you see every message in the chat. An agent of the run wins over a session of the same name, and a name that reached one session earlier in the run and now names another one is refused. The result names the summary, the recipient and the `msg_id`; a later `[Delivery notice]` says when the other session held, declined or refused it. `notify_when_idle` asks a session for one notice when it next finishes a turn; only the main agent may ask, at most 32 at a time, and a notice that does not come within 12 hours expires. The message reaches the other model inside an attribution block that says another agent wrote it and that it grants nothing. In `auto` and `extended` a gated call is refused (`peer-relay` in the gate audit) when a message the run took in says something was denied, declined or not authorised and names that call: the command line, a command it chains with `&&`, `;` or `|` or wraps in `sh -c`, its first two words, or a path. The rule reads words. A different command with the same effect, such as `find build -delete` for `rm -rf build`, or a message that names no command, still passes.

`list_agents` free · main and children — input: { }

Lists whom `send_message` reaches, without a model call: the caller's own name first, then the sessions on this machine with their face, status, folder and status line, then the children of the run with their role and last status line. A child sees its parent only. Another session or a child writes the folder, face, status and status line, so each is printed on one line with control characters, brackets, guillemets and quotes turned into spaces. A folder longer than 120 characters keeps its last 119 behind an ellipsis, so the folder's own name stays; a name and the other fields stop at 100. A listing with rows ends with one line saying these fields were written by those sessions and agents and are information, not instructions. Both tools leave the request when `agentMessaging` is off (chapter 30) or the `messaging` group is unchecked.

`wait_agents` free · parent only — input: { ids?, timeoutSeconds? }

The join of a background fan-out (card 505). It waits for the named children, or for every background child of the run, and returns each result once, with the last status line of each child still running. It waits without polling. `timeoutSeconds` runs from 0, which returns at once, to 3,600; left out, it waits until the children have ended. A result the parent already took in as a message is reported as such and not repeated. Its description names the children of the run. No child holds this tool, so a background child cannot join anything and the depth stays one. It belongs to the `messaging` group and leaves the request with the two tools above.

`schedule_wakeup` gated · write · in a loop — input: { delaySeconds, reason, prompt, stop? }

Paces a loop the operator started with `/loop` (card 504). The delay is held between 60 and 3,600 seconds and the wakeup lands on the whole minute at or after it; the result names the time and says when the delay was moved. `delaySeconds`, `reason` and `prompt` are required unless `stop` is true, which ends the loop. The prompt `<<loop>>` fires the loop's own instructions: in full at the first wakeup, after a compaction, after `/clear` and after the session was opened again, as a short reminder otherwise. In a loop with an interval it fires the loop's own task. Any other prompt fires behind a line saying the model wrote it and when, in the chat and for the model, so it never reads as the operator's own words. A reason longer than 500 characters is cut; a prompt the model wrote that is longer than 500 characters is refused. A new wakeup replaces the pending one. The result asks the model to write its update for the tick before the call, and a stop asks for the closing summary before it, because the turn ends when the call returns: the run ends after the round of tool calls that held it, unless the call was refused. A goal's check does not run on that ending. The agent offers the tool only while the session runs a loop, so a session without one sends the tools it sent before. Tier write: in ask mode you see the call, and the auto modes check it against relayed denials as they check every gated call (chapter 10).

`mcp__<server>__<tool>` gated · dynamic — schema from the server

Every tool a connected MCP server advertises, wrapped as a first-class spectroscope tool (chapter 18). Always gated: its inputs are model output and its effects are external.

Who gets which tools

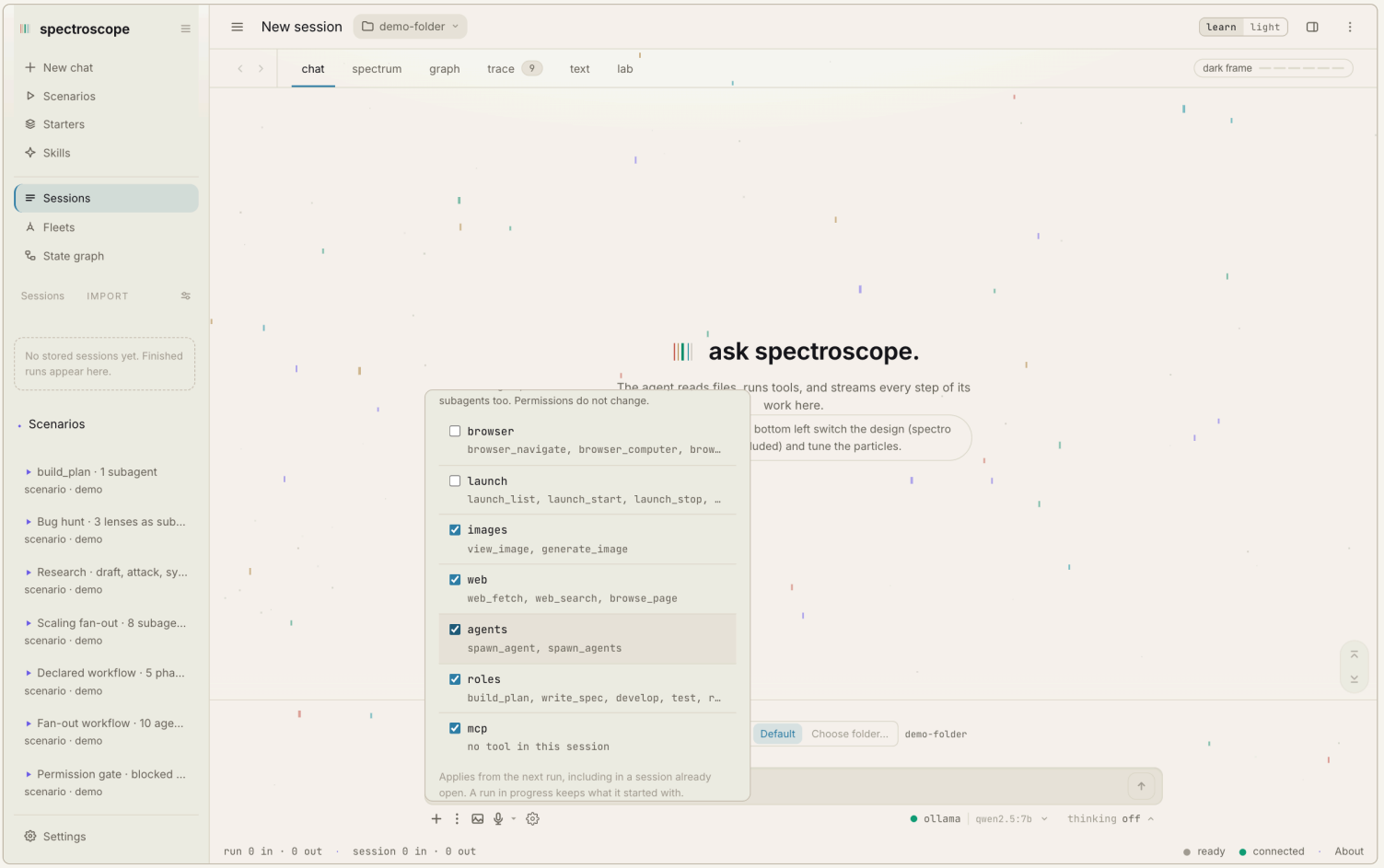
AGENT	REGISTRY
`main`	all of the above except `report_status`, plus the browser, launch, web and image tools: 39 in a session without MCP servers and skills (counted by `SessionToolGroupsTest`); MCP tools on top, `use_skill` when skills exist
`worker-N`	list_dir, read_file, write_file, run_command, edit_file, glob, grep, code_graph, view_image, view_file, use_skill*, **report_status**, send_message, list_agents
`explore-N`	list_dir, read_file, glob, grep, code_graph, **report_status**, send_message, list_agents. Its own registry holds no tool that writes a file or runs a command. Its `send_message` reaches only its parent, so anything an explore child read gets to a worker only through the parent, in the chat

* when skills are installed. Children never receive: spawn/dev tools (nesting depth 1 is structural), generate_image, web_fetch, web_search, browse_page, update_plan, MCP tools.

Tool groups

Every request to the model carries the schema of every tool the session holds, whether the task uses the tool or not. In a session without MCP servers or skills, that block is 18,101 characters, about 4,500 tokens; the context ring's own reader measured it on 2026-10-09. The gear under the message box lists eight tool groups (`browser`, `launch`, `images`, `web`, `agents`, `roles`, `mcp` and, since card 503, `messaging`) with the tools each group holds in this session. The `messaging` group holds `send_message` and `list_agents`, which added 1,073 characters to an Anthropic request when they came (measured on the posted body on 2026-10-11), and `wait_agents` since card 505. When you uncheck a group, the next run leaves its tools out of the request, and so do the subagents that run spawns. With every group unchecked the block dropped to 4,125 characters on 2026-10-09, when the gear had seven groups, and the model sees the standard file tools except `view_image`, which belongs to `images`, plus `run_command`, `update_plan` and `ask_user_question`. `code_graph`, offered where the folder holds a code graph, and `schedule_wakeup`,

offered while a loop runs, belong to no group either. Switching a group off does not change a permission: a tool that is still on asks exactly as before. If the session has a pinned folder, the gear saves your choice in that folder's local settings file under `toolGroupsOff` (chapter 30), and the next session starts with it. Without a pinned folder the gear says that the choice holds for this session only and is not saved. The switch applies to sessions in the app window; the REPL and `spectro run` send every tool.



> The tool groups in the composer gear of a session with a pinned folder, with browser and launch switched off for the next run. Each group lists the tools it holds in this session, and the [local] tag says where the choice is saved. The sentence under the list says that a change applies from the next run.

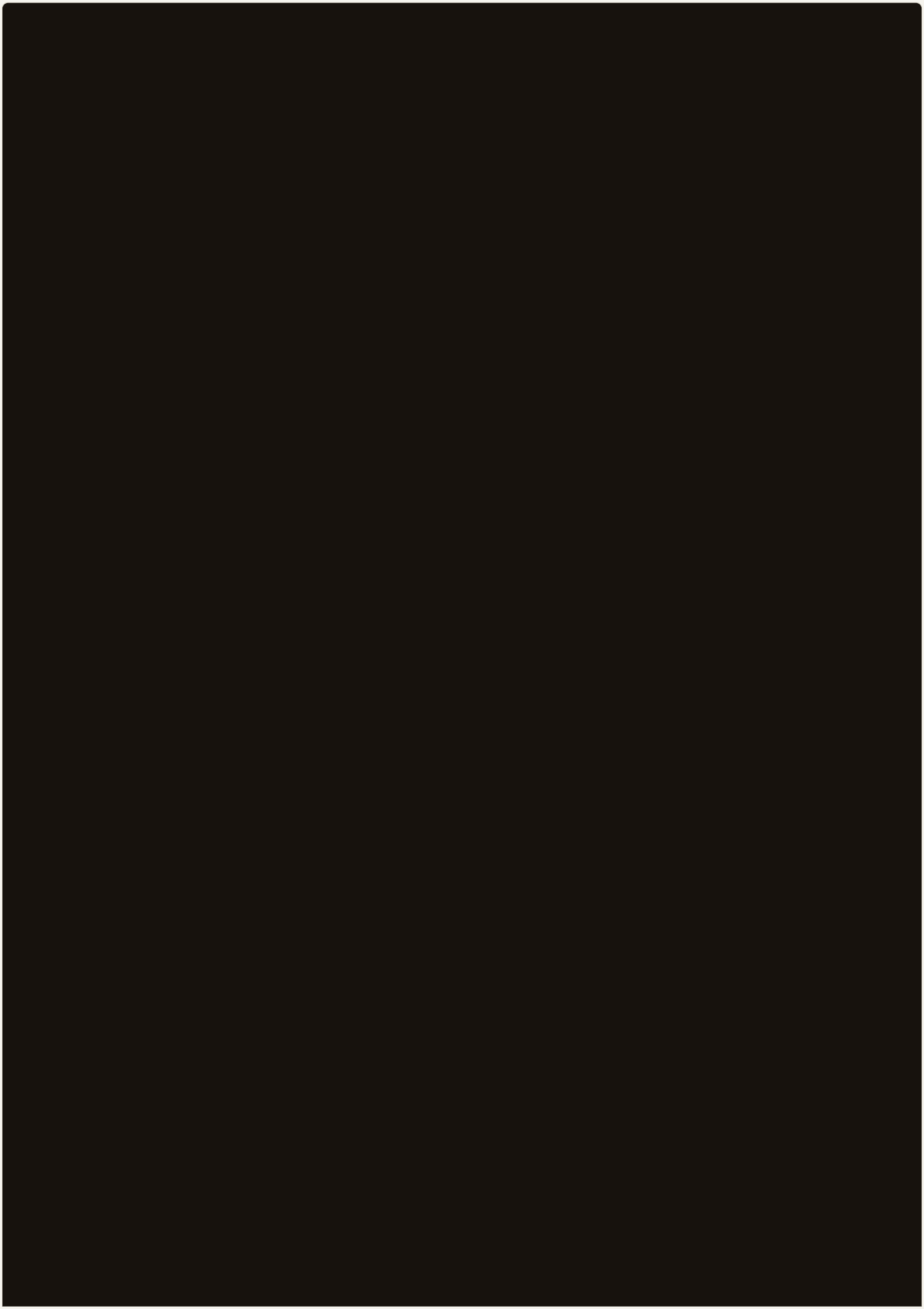
CHAPTER 13

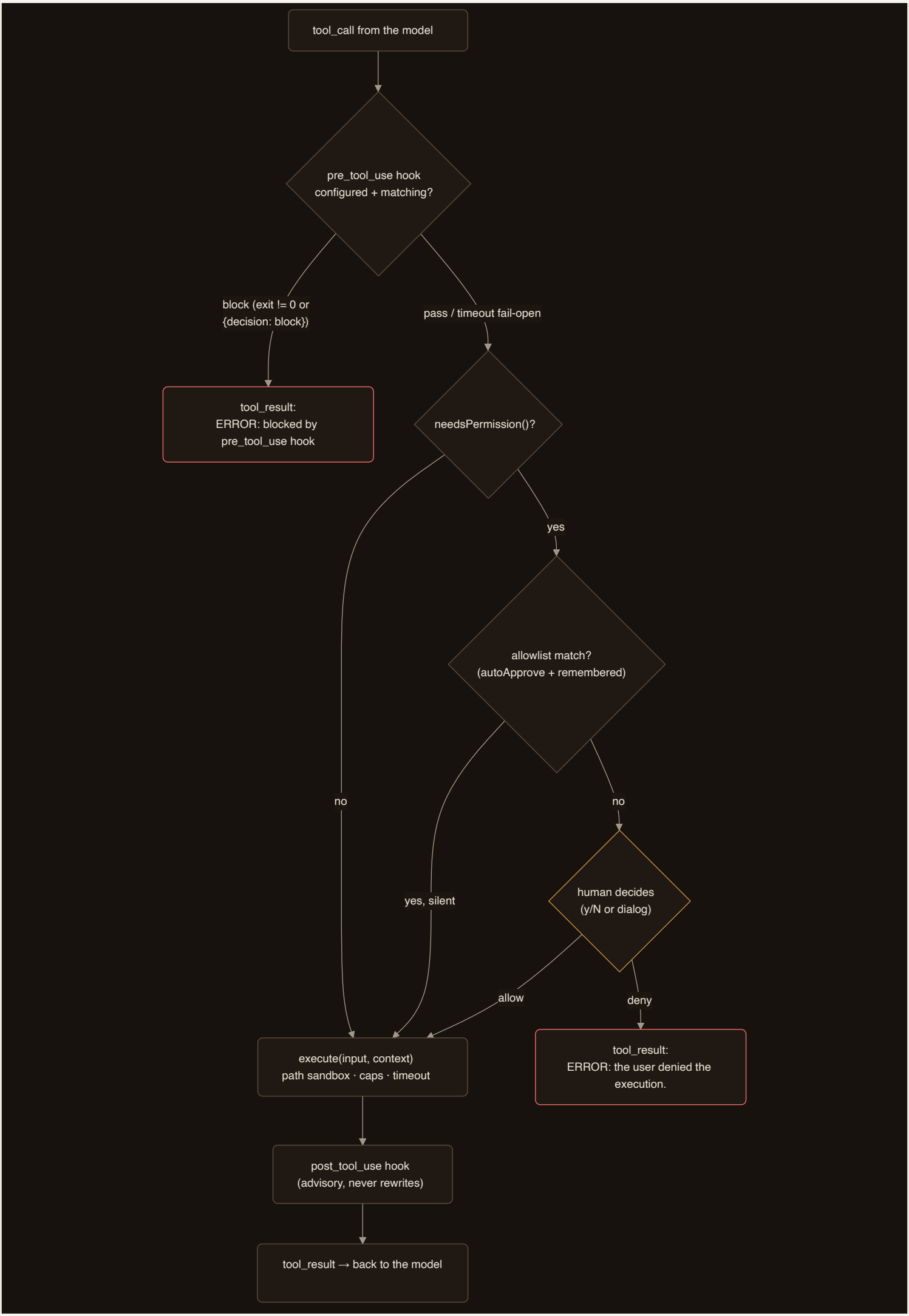
Permissions, allowlist & hooks

You stay in control through three layered mechanisms: the permission gate (a human decides), the allowlist (you decide once, scoped), and hooks (your scripts decide, before the gate even asks). All three leave an audit trail in the session file.

The guard pipeline

Every single tool call passes this pipeline, in this order — the order is load-bearing:




tool_call from the model
pre_tool_use hook configured + matching?
block (exit != 0 or {decision: block})
pass / timeout fail-open
tool_result: ERROR: blocked by pre_tool_use hook
needsPermission()?
yes
allowlist match? (autoApprove + remembered)
no
no
yes, silent
human decides (y/N or dialog)
allow
deny
tool_result: ERROR: the user denied the execution.
execute(input, context) path sandbox · caps · timeout
post_tool_use hook (advisory, never rewrites)
tool_result → back to the model

> Per tool call: pre_tool_use hook (can block before anything else) → permission gate with allowlist short-circuit → sandboxed execution → advisory post_tool_use hook.

Permission modes

MODE	MEANING	WHERE
`ask`	gated tools pause the run until a human decides (terminal y/N, web dialog) — with the allowlist answering first	default; the interactive faces
`readonly`	every gated tool is denied automatically — the agent can look, not touch	headless default (`spectro run` , cron): “capability before convenience”
`auto`	every gated tool is approved automatically	opt-in for unattended runs: `--permissions auto` / per-job `"permissions": "auto"`
`extended`	like `auto` , and the file tools `view_file` , `read_file` , `list_dir` , `write_file` , `edit_file` , `glob` and `grep` accept a path outside the working folder, absolute or with `..` . The agent then reads and writes anywhere your user account can, without asking. Every other mode keeps the fence	opt-in: the composer's mode switch (it holds until the window reconnects and is not written to the project file), the user settings, `--permissions extended` / per-job `"permissions": "extended"` . A folder's settings file may not set it: neither the workspace's own files nor the `.spectro/settings.json` of the folder the server was started in

Even automatic policies emit `permission_request` / `permission_decision` events — the JSONL always shows who allowed what.

The allowlist

Rules live under `autoApprove` in any settings layer and are consulted before any human is asked. Since 0.9 every entry also names a **tier** — what the tool it approves is allowed to DO:

```
{ "autoApprove": [
    "grep#read",                          // approves every grep call
    "run_command#eval-execute:git status*",  // a prefix rule scopes the input as before
    "write_file#write:docs/notes.md*",
    "mcp__playwright__*#read"             // a whole server's READERS, nothing above
] }
```

TIER	WHAT IT COVERS
`read`	looks, never touches — a file read, a page read, a screenshot
`write`	acts on the page, the app or the disk — input, navigation, a file write
`eval-execute`	runs code — in the page, in a Node context, or on this machine

Two rules carry the whole idea. An entry that names *no* tier approves `read` and nothing above it. And a **family wildcard has to name its tier**: `mcp__playwright__*` on its own approves nothing at all, because a wildcard is exactly what somebody writes to stop a prompt storm — and before this, one such line would have approved a Node-context eval beside the screenshots.

Where the tier comes from. Not from the server. Both MCP transports pin protocol revision `2024-11-05` , which carries no tool annotations at all (`readOnlyHint` and its siblings arrived in `2025-03-26`), so no server here can rate its own tools even if it wanted to. The tier is a shipped, versioned data file inside the jar — `resources/permission/tool-tiers.json` , one section per blessed MCP server plus one for the built-in belt. **Anything it does not name is `eval-execute` and prompts**, including every tool of a server the map has never heard of. The honest cost: a server that grows a tool produces prompts on the day it updates, and the fix is an entry in the next release rather than a config edit on your machine.

What happened to the entries you already had. They were migrated once, in place, each rewritten to carry the tier its tool held on the day of the migration — so no entry started or stopped approving anything. The record of that pass, entry by entry, is in `~/.spectro/gate-audit/allowlist-migration.jsonl` . Only entries written after the migration fall under the read-by-default rule. A pre-existing wildcard is left untouched: it matched nothing under the old name-only gate, and stamping it with a tier would have turned dead config into a live family grant.

The list has a screen. Settings → Auto-approvals lists every entry with the tier it carries, per settings layer, says out loud which ones approve running code, and adds or removes entries in your own settings. It renders what the gate decided and never re-decides it: `GET /api/settings/allowlist` reads the entries out through the gate's own parser and the shipped map.

Every gate decision is written down. One line per decision in `~/.spectro/gate-audit/<session-id>.gate.jsonl` — the tool, the tier it resolved to, which section of the map decided that, the map version, allow or deny, who decided, and the raw entry that approved it. A sidecar, not a new event: the RunEvent wire is byte-frozen. The call's *input* never enters the file — a command argument or a URL routinely carries a credential.

Matching reads each tool's **guarded field** — the one input that defines its blast radius. The same table drives what an “always allow” click remembers, so a remembered rule can never silently go dark on the matching side:

TOOL	GUARDED FIELD	AN “ALWAYS ALLOW” CLICK STORES
`run_command`	`command`	the first token: `run_command#eval-execute:git*`
`write_file` / `edit_file`	`path`	the full path: `edit_file#write:src/Main.java*`
`web_fetch`	`url`	the full URL: `web_fetch#read:https://example.com*`
everything else	—	the bare tool name plus its tier

Session-remembered rules live per connection (two tabs never share them). Ticking **Persist** appends the rule to the session's **workspace** project file (`<workspace>/.spectro/settings.json`) through a single serialized writer that preserves every other key and dedupes — falling back to the deprecated launch-dir `.spectro/settings.json` only for a session with no real workspace yet. Because the CLI resolves the same workspace, it honours your web decision there on its next run.

The net fence

`web_fetch` and `browse_page` take their address from model output, and the model reads whatever page it was last shown. So the private world is refused before a request leaves:

REFUSED	WHY
`file://`, and every scheme but http/https	these tools reach the network, not the local disk
10/8, 172.16/12, 192.168/16	the private networks (RFC 1918)
100.64/10	the range a tailnet hands out
169.254/16, fe80::/10, fc00::/7	link-local (where a cloud metadata service lives) and unique-local
multicast, 255.255.255.255	"every host on this segment" is not somewhere a page may send an agent
loopback	unless `allowLocalhost` is set — see below

A host name is judged by where it *points*, not by how it looks: `localtest.me` looks public and resolves to 127.0.0.1, so every name is resolved and every answer is checked. A name that does not resolve at all is left to fail on its own — the fence does not invent an answer.

How far the fence reaches, per tool

`web_fetch` **is fenced at every hop.** It follows redirects itself — up to five — and each address in the chain goes to the fence before a request is made. It has to work that way: the Java HTTP client follows same-protocol redirects on its own, and a first build that asked the fence once let a public page answer `302` and hand back a LAN page and a tailnet page with the fence never consulted for either.

`browse_page` **is fenced at the address you give it, and no further.** Once Chrome is running it follows its own redirects and runs the page's JavaScript, which can navigate anywhere, and none of that comes back past the fence. Two things would close it and both cost something real: pinning Chrome to a single host breaks every page whose script lives on a CDN — which is the reason this tool exists — and running Chrome through a fencing proxy closes it completely but is a component of its own. Until one of them ships, treat `browse_page` as an entry check and prefer `web_fetch` when the page does not need JavaScript.

`allowLocalhost: true` (Settings → Net fence) is the deliberate opt-in for the local verify loop: this product on 8746, ollama on 11434. It opens loopback and nothing else — not the LAN, not the tailnet, never a file URL, and not through a redirect either. It is *process-global*: a workspace scope may not set it, because the workspace is the folder the agent itself writes into, and a switch inside the sandbox is not a switch.

A refusal names the address and the rule and carries nothing else:
`ERROR: browse_page refused 192.168.1.10:8746: it is a private network address, RFC 1918 (rule: rfc1918).` No path, no query string, no userinfo — a URL the model assembled may carry a token in any of the three, and a refusal goes back to the model and into the transcript.

`web_search` is deliberately *not* fenced: it dials the instance *you* configured, not an address the model chose, and a SearXNG on localhost is the normal case.

Hooks

Hooks are config-driven shell commands around every tool call — your policy, in your scripting language, versioned with the project. They come *only* from config, never from tool input: a pre-hook is arbitrary shell running before the gate, so it must never be model-controlled.

```
{ "hooks": [
    { "event": "pre_tool_use",          // or post_tool_use — anything else fails LOUDLY
      "matcher": "run_command",         // tool-name glob: * (default), exact, or prefix*
      "command": "./guard.sh",          // any shell string
      "timeoutSeconds": 5 }             // optional, default 10
] }
```

ENVIRONMENT	the hook process sees `SPECTRO_TOOL_NAME`, `SPECTRO_TOOL_INPUT` (compact JSON) and — post only — `SPECTRO_TOOL_RESULT`; cwd is the agent's working directory.
PRE_TOOL_USE	runs **before the permission gate** and blocks on exit ≠ 0 *or* a stdout verdict `{"decision":"block","reason":"…"}`. A block short-circuits: no dialog, no execution — the model receives `ERROR: blocked by pre_tool_use hook: <reason>`. First matching block wins.
POST_TOOL_USE	runs after execution, **advisory only** — exit code ignored, the result is never rewritten. For logging, notifications, formatters.
FAIL-OPEN TIMEOUT	a hanging hook must not wedge every tool call — on timeout the pipeline continues to the normal gate. Safe because the shared runner drains the pipe concurrently: a chatty hook cannot deadlock itself into the timeout.
SUBAGENTS INCLUDED	children run the *same* hook runner — a hook that blocks a tool also blocks it on a child, or delegation would be a bypass.
WHOLE-BLOCK MERGE	like `mcpServers`: a settings layer that defines `hooks` replaces the entire block below it.

EXAMPLE — VETO EVERY RM

In `.spectro/settings.json`:

```
{ "hooks": [ {
  "event": "pre_tool_use", "matcher": "run_command",
  "command": "case \"$SPECTRO_TOOL_INPUT\" in *'rm '*) echo '{\"decision\":\"block\",\"reason\":\"rm is off limits here\"}' ;; esac"
} ] }
```

Ask the agent to delete a file with `rm` and it instantly receives `ERROR: blocked by pre_tool_use hook: rm is off limits here` — no permission dialog, no execution. `spectro doctor` shows `hooks: 1 configured (pre_tool_use:run_command)`. Verified live.

The audit trail

Every decision is an event: `permission_request` (with the full input) and `permission_decision` (allowed or not) land in the JSONL even for allowlist auto-approvals; hook blocks are visible as their error result. The Trace tab's permission category shows the whole history; the Lab's gate row turns green or red as you step onto it.

CHAPTER 14

Subagents & the A2A protocol

A subagent is not a special construct — it is another `Agent` from the same core, with its own fresh context, its own reduced tool registry, and the same permission gate and hooks as its parent. What makes spectroscope's delegation special is that the protocol between the agents is *visible*: a task/status/result lifecycle you can watch in every view.

Two roles

ROLE	PURPOSE	TOOLS
`explore`	read-only research: map a directory, find usages, summarize	list_dir, read_file, glob, grep + report_status — **read-only by construction**
`worker`	real subtasks that may change things	the full standard set + use_skill + report_status

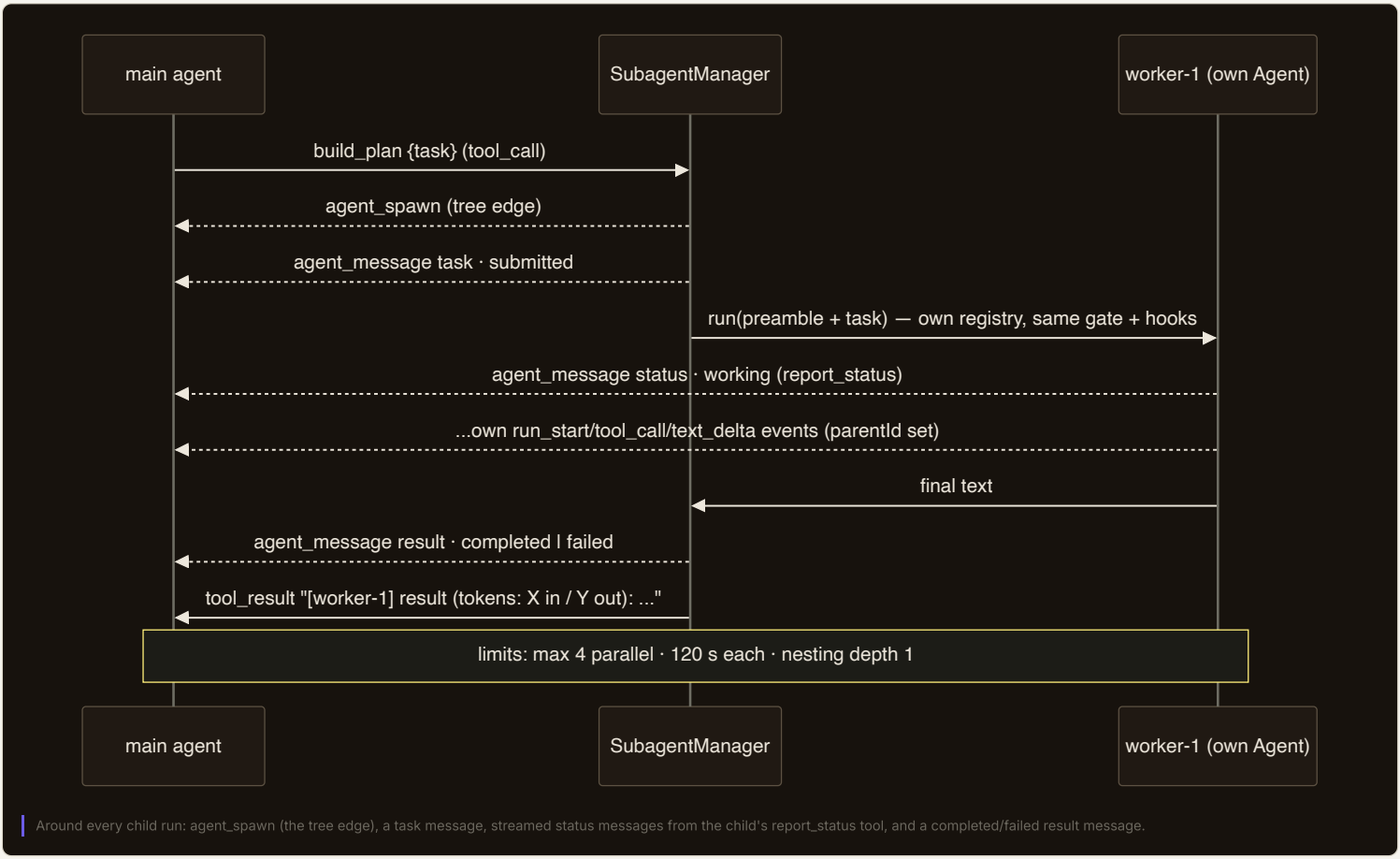
Children see *only their task text* — no conversation history — which is the point: a fresh, focused context. Their final text is all the parent receives, wrapped with the token cost so delegation visibly costs something: `[worker-1] result (tokens: 2.1k in / 0.4k out): …` A child cut off by its budget is the exception: the text of the turn it was cut in comes back as well, below the error and under the marker line `--- what the child had written before the cut, unfinished ---` (card 371).

When the parent delegates

The main agent's system prompt carries one short paragraph on this (card 470). The parent sends a search across many files or folders to an `explore` child and works from the digest it returns. It reads a file it already knows it needs by itself, with `offset` and `limit` when the file is large. It hands an edit that stands on its own as a subtask to a `worker` child. The task text has to be a complete assignment and has to ask for a short digest with path and line, not for file contents; the `spawn_agent` and `spawn_agents` descriptions say the same. A test holds the paragraph to at most five sentences and under 600 characters, measured with the reader behind the context ring's system prompt row. The harness does not enforce the paragraph and does not stop a model that reads everything itself.

LIMITS	at most 4 parallel children (schema-enforced and runtime-checked); one run budget per child, counted from its first token and set by the key `subagentBudgetSeconds`, shipped at two hours, with the current number and the full derivation in that key's row in chapter 30. A child that runs past it is cut, and the parent is handed `ERROR: [worker-1] out of budget: 7200 s of work since its first token (derivation). Raise subagentBudgetSeconds in the settings, or cut the subtask smaller.` In place of `derivation` the budget writes its own account of where the number came from: the floor from the key, and the measured median exchange it was weighed against once this session has measured one. Beside the clock, one token budget per child, set by the key `subagentBudgetTokens` and shipped at ten million: a child whose input plus output has passed it is cut when it starts its next turn, and the parent is handed an `ERROR: [explore-1] out of tokens` sentence that names the spend, the budget and the key. The model request of that turn still goes out and is cancelled; it is not in the spend the sentence names, and a paid backend may still bill its input. Nesting depth 1, because children simply do not have the spawn tools.
CANCELLATION	each child has its own CancelSignal; the parent's cancel cascades — Ctrl+C or Stop ends the whole tree cleanly.
ISOLATION THAT ISN'T	same working-directory sandbox, same permission broker (the dialog names the asking child), same hooks. Delegation never widens what is allowed.

The A2A-lite lifecycle



Around every child run: agent_spawn (the tree edge), a task message, streamed status messages from the child's report_status tool, and a completed/failed result message.

Two additive events carry the protocol — the wire format needed no break. Every spawn emits `agent_spawn` (child id, parent id, task) followed by an `agent_message` with role `task`, state `submitted`. While working, the child's permission-free `report_status` tool publishes role-`status` messages ("working") that surface live in the Agents panel, the chat thread header and the Lab cards. At the end — on *every* path, including timeout and cancellation — a role-`result` message closes the child with state `completed` or `failed`.

Where you see it: chat nests child turns into pastel threads (chapter 5); the Graph tab draws split/join blocks per wave (chapter 6); the Lab gives every child its own loop card with its own packet (chapter 7); the Agents panel keeps the roster with token costs (chapter 6); the CLI renders `[worker-1]`-prefixed lines (chapter 4).

Background fan-out

A parent that passes `background: true` to `spawn_agents` gets the children's names back at once and keeps working (card 505). The children run on the same slots as a blocking batch and under the same clocks: the queue grace, the run budget and the token budget. Each `report_status` line and each `send_message` to `parent` lands in the parent's inbox and is read at its next step, and a child's result arrives there as a message too. The parent can ask a child how far it is with `send_message`, list the children with `list_agents` without a model call, and collect the results with `wait_agents`. Children never message each other. A child's `send_message` reaches only its parent, and every message shows in your chat.

When the parent ends its turn while children still run, the harness waits for them and hands their results in as the next text. Each result is one message block from its child, with the same tag and the same closing note as any message from another agent, so a result cannot pass itself off as the operator. The results count as messages the run took in, however they reach it: through the inbox, through `wait_agents` or handed in at the end of a turn, and the rule that guards the auto modes reads them. A status line is cut at 500 characters. With a goal set the harness waits 30 minutes at most, then 60, then 120 and every 120 after that (card 504), and each time appends a check in that lists each child still running with its role and its last status line, one line per child. Stopping the parent stops every child. A run that reaches its turn cap cancels the children still running and waits up to 30 seconds for them before it ends; a child that ignores the cancel longer than that is logged and left behind.

The development tools

Four role tools wrap a worker spawn with a persona and a skill — no new agent types; specialization is prompt + skill:

TOOL	PERSONA	ASKS FOR SKILL	DELEGATES
`build_plan`	senior PLANNER	`writing-plans`	a feature request → a step-by-step implementation plan (a written document)
`write_spec`	requirements ANALYST	`brainstorming`	a rough idea → a design/spec with decisions and trade-offs
`develop`	IMPLEMENTER	`test-driven-development`	a development task, carried out in small verified steps
`test`	TESTER — "Do NOT fix anything."	`verification`	runs and inspects, reports evidence, changes nothing

The child's prompt is the persona preamble + a standing instruction to report progress via `report_status` + your task text; the A2A task message shows *your* task with the tool name as its label — which is exactly how the UI badges the thread. *Try: "build_plan: draft a plan for adding CSV export to the sessions list."*

Read that third column as a request, not an inventory. The preamble says "if a `use_skill` tool is available, load the skill '`writing-plans`' first", and only `verification` is still seeded into a fresh home (chapter 15). The other three live in the catalogue's `superpowers` pack, where an installed copy answers to `superpowers:writing-plans`, so the bare name comes back as `ERROR: unknown skill 'writing-plans'. Available: …` with the qualified names in the list that follows. The child can take the hop

itself — nothing rewrites the request for it.

CHAPTER 15

Skills

Skills extend the agent with *data, not code*: markdown instruction packages the agent loads when a task matches. The loop is untouched, no new event type exists — a skill use is an ordinary tool call you can see in every view.

The package format

```
~/.spectro/skills/<skill>/SKILL.md           (user scope — a bare name)
~/.spectro/skills/<pack>/<skill>/SKILL.md    (user scope, inside a pack — <pack>:<skill>)
<project>/.spectro/skills/…/SKILL.md         (project scope — wins by name)

---
name: humanizer
description: |
  Remove signs of AI-generated writing from text. Use when editing or reviewing
  text to make it sound more natural and human-written. Based on Wikipedia's
  comprehensive "Signs of AI writing" guide. Detects and fixes patterns including:
  inflated symbolism, promotional language, superficial -ing analyses, vague
  attributions, em dash overuse, rule of three, AI vocabulary words, passive
  voice, negative parallelisms, and filler phrases.
---
# The skill body — full markdown instructions…
```

The frontmatter above is the catalogue's `humanizer`, cut after its description. A directory under a root is a skill when it holds a `SKILL.md`. A directory that holds no `SKILL.md` of its own but does hold skills is a **pack**, and its skills are advertised as `<pack>:<skill>` — the name the system prompt lists and the name `use_skill` answers to, not a label printed beside one. The rule adds exactly one level and stops there: a skill three directories deep is invisible, and everything installed before packs existed sits at the top level and keeps its bare name. That is the whole purpose of the colon — somebody else's `brainstorming` lands beside yours instead of on top of it.

Both roots are scanned in order, user first, project second, so a project skill of the same name wins — the settings hierarchy again. A `.disabled` file inside a skill folder hides that skill from the loader; the same marker on a pack folder skips the pack and everything under it. A broken `SKILL.md` is skipped with one warning line: a bad skill must never take the harness down.

A session of the app reads its skills, commands and agents from three sources instead: the `.claude` and `.spectro` folders of its own working folder (the `.spectro` copy wins inside it), the playbook it picked in developer mode (chapter 15d) and `~/.spectro/skills`. While the working folder has a code graph, the app adds its own skill `code-graph` under the source “built in” (card 587, chapter 9). Between these sources no one wins. A name that exists in more than one stays in each, under the name of its source: `workspace:`, `user:` or the playbook's id, for example `spectro:brainstorming`. A name that exists once keeps its plain form, and a skill or command named like a command of the web chat, such as `clear`, always carries its source. The rail row “Skills and Commands” shows the chat commands and, under “This session”, what the session in view reads at its next turn. The terminal and `spectro run` keep the order of the two roots above.

Frontmatter is hand-parsed — two flat `key: value` lines do not earn a YAML dependency — but flat is not the whole story. A value of `|`, `|-`, `>` or `>-` opens a block scalar, and the indented lines under it are folded into one, because both readers of that string put it on a single line: the prompt's bullet and the settings row. Reading the marker alone was the bug 0.7.0 had to fix before the catalogue could ship — the skill above reached the model as `- humanizer: |`, unfindable, and its continuation line ending in a colon was then read as a further key. One matching pair of surrounding quotes is stripped, since YAML treats them as delimiters. Missing keys fall back: the name to the folder, the description to the first non-blank body line, capped at 120 characters.

Progressive disclosure

Only the *catalog* — one `name: description` bullet per skill — rides in the system prompt under a `## Skills` header, with the standing instruction to call `use_skill` before matching work. The body loads on demand. That keeps the context small however many skills you install, and makes skill usage visible: you can watch the agent fetch its instructions in the chat and trace. It is also the reason the shelf described below is carried but not installed — every skill under a root is a bullet in every run's prompt, and fifty-seven foreign bullets would be a standing tax rather than a gift.

What a fresh home gets

One skill. `verification` — evidence before claims, report what you observed in this run and nothing else — is copied out of the artifact into `~/.spectro/skills` the first time a server starts, and that is the entire seed set. Until 0.7.0 there were four; `brainstorming`, `test-driven-development` and `writing-plans` left it because the catalogue carries versions between three and five times longer, and only a name and a description ever reach the prompt anyway. `verification` stayed because nothing on the shelf replaces it.

Seeding is deliberately timid, and both promises are pinned by tests. An existing folder is never written over, so your edit survives every upgrade. And a `.seeded` ledger in the skills root records every name ever offered, so a skill you deleted stays deleted instead of reappearing on the next boot. Dropping three skills from the seed set therefore removed nothing from any machine: an installation that already had them still has them.

The bundled catalogue

Four collections of foreign skills ride inside every build as classpath resources under `skills-catalogue/`, vendored verbatim at a named commit:

PACK	SKILLS	VENDORED FROM	LICENCE
`humanizer`	1	github.com/blader/humanizer	MIT

`matt-pocock`	35	github.com/mattpocock/skills	MIT
`superpowers`	14	github.com/obra/superpowers	MIT
`ui-ux-pro-max`	7	github.com/nextlevelbuilder/ui-ux-pro-max-skill	MIT

Fifty-seven skills across those four packs — and both totals carry the commands that produced them rather than a memory of a release note:

```
$ find .spectro/skills-catalogue -mindepth 1 -maxdepth 1 -type d | wc -l
       4
$ find .spectro/skills-catalogue -name SKILL.md | wc -l
      57
```

The shelf is enumerated exactly the way that second command works: a skill is *the directory that holds a SKILL.md*, found by walking the resources, so neither of the two nesting depths the catalogue uses — `<pack>/skills/<skill>` and `<pack>/skills/<category>/<skill>` — ever enters the code as a number. Each row the settings page receives carries an opaque id (`<pack>/<name>`), the pack, the description parsed out of the frontmatter exactly as the model would read it, the licence, the upstream repo and commit, a file count and a byte count, plus whether that skill is already on disk.

Every pack keeps its upstream `LICENSE` beside it and a `PROVENANCE.json` naming the repo, the exact commit and the date it was taken — MIT permits the copy, what it requires is that the notice travels with it. One pack goes further because it had to: `ui-ux-pro-max` ships 54 font files, and a typeface's licence is not covered by a repository's MIT statement, so they carry 27 SIL OFL notices of their own, checked rather than assumed.

Installing one

The skills section of the settings page lists the shelf under **Catalogue**, below the skills you already have. Each row that is not installed yet ends in an **install** button, and its tooltip names the pack and the licence *before* you press it: “From superpowers, licensed MIT — LICENSE and PROVENANCE.json travel with it”. A row already on disk says **installed** instead. Nothing is fetched. The whole section works with the network unplugged.

The press posts the row's id to `POST /api/skills/install` as `{"skill": "<pack>/<name>"}`, and that id is matched against the enumerated index by string equality — never resolved as a path. The destination is not something the caller can name: it is the pack and leaf folder the index already carries, which is why a one-line request cannot become a write anywhere else. The copy is assembled in a staging directory that is a *sibling* of the skills root, never inside it, because a half-built folder already holds a `SKILL.md` and the loader would read it as a pack mid-write. Files are written byte for byte out of the jar with no permission carried across — thirteen catalogue files have an execute bit upstream and none of them arrives runnable — and the finished folder is promoted with one atomic move. Beside the skill land the pack's `LICENSE` , its `PROVENANCE.json` , and a `spectro-install.json` recording pack, skill, repo, commit, licence and the date you installed it.

ANSWER	WHEN
200 with the facts	installed — name, pack, files, bytes, licence, commit
409	already installed (“delete it first to install it again” — copying over it would lose your edits and its off switch), or the project already carries it and would win anyway
413	over the ceilings: 400 files or 24 MiB, with the measured numbers in the body
400 / 404	no id in the body; an id that is not on the shelf; *any* request from a non-local origin
500	the copy failed — including a pack whose LICENSE could not be read, which is refused rather than installed unlicensed

One install runs at a time, and a refusal is said rather than swallowed: the panel prints the server's own sentence and deliberately does *not* reload the list, because redrawing rows after a failed copy is how a failure comes to look like a success.

What is installed is managed in the same list. The switch on each row writes or removes the `.disabled` marker. The delete button is armed by the first click and deletes on a second within four seconds, and it appears only for user-scope skills — a project skill answers `a project skill belongs to the repo — remove it there` . Deleting the last skill out of a pack takes the empty pack folder with it. Every route comes in a one-segment and a two-segment form (`/api/skills/{name}/disabled` and `/api/skills/{pack}/{name}/disabled` , the same for `DELETE`), because the pack is a real directory and travels as a real path segment; the colon in `superpowers:brainstorming` is a display name, not a path.

> WHAT THE SHELF IS NOT
>
> Nothing is installed from a live git remote and there is no update check — each pack is pinned to the commit it was vendored at, and a skill that quietly moves under a run nobody re-read is worse than one that does not move. Re-installing is refused rather than treated as a no-op. The catalogue rides in the artifact whether you ever install from it or not. And spectroscope publishes nothing of its own to it: this is a client, not a registry.

`/skills` lists what is installed, under the names the model uses; `spectro doctor` names the same set on one line, which on a fresh home reads `skills: 1 installed (verification)` ; the System-context panel shows the catalog exactly as the model sees it. All three read the library rather than the directory, so a skill you switched off is missing from all three. Changes reach the *next* session rather than the running one — the library is loaded once per connection, and the settings panel says so above the list.

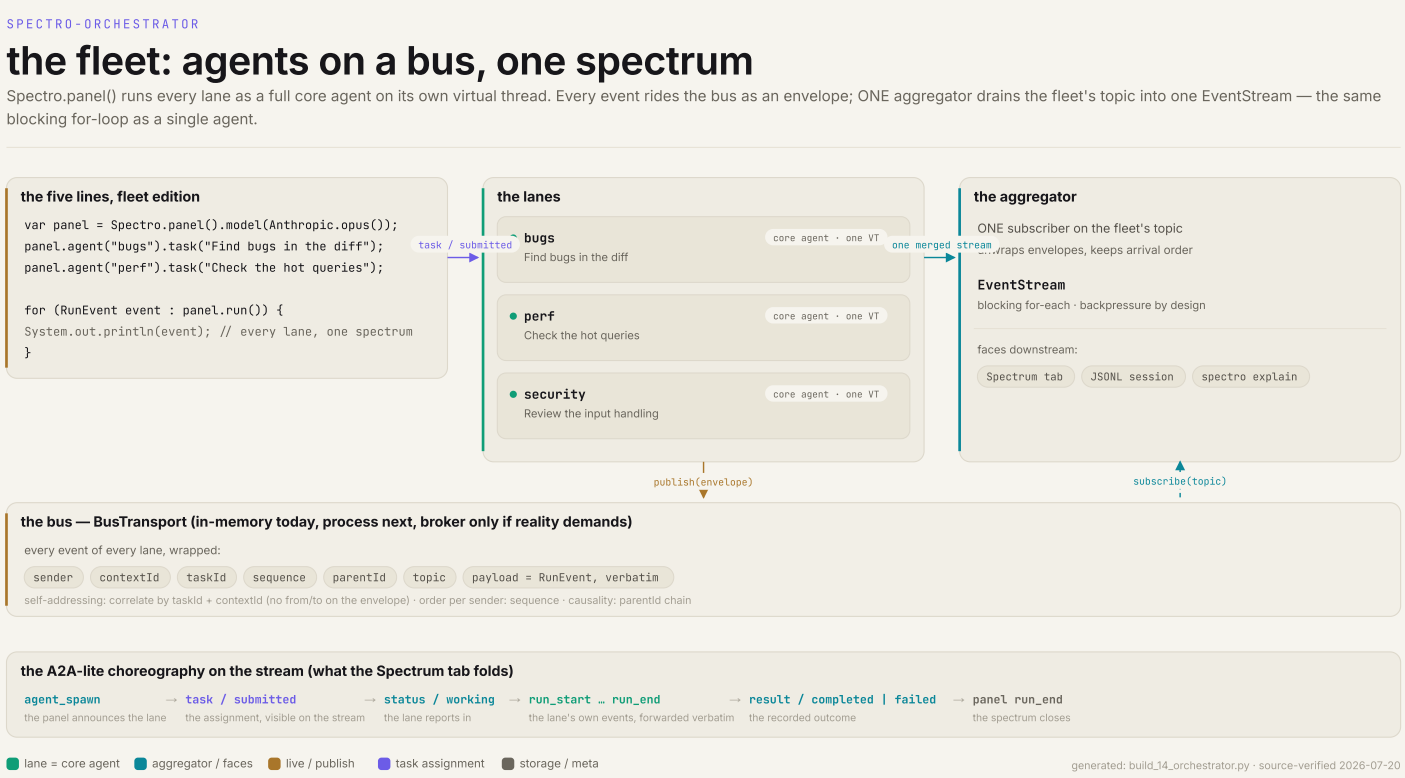
the fleet: many agents, one spectrum

Subagents fan out under one main agent. The fleet is the next level: several full agents, each with its own model, tools and workspace, run in parallel and merge into one spectrum. They can share a single JVM, or run as separate processes on separate machines that report to a hub. Either way you enter a fleet from the sidebar, and what opens is not a session with more lanes in it — a fleet has a bar, a rail and a vocabulary of its own. That is the `spectro-orchestrator` module (migration phase 5), and its facade follows the frozen five-lines style exactly:

```
var panel = Spectro.panel().model(Anthropic.opus());
panel.agent("bugs").task("Find bugs in the diff");
panel.agent("perf").task("Check the hot queries");

for (RunEvent event : panel.run()) {
    System.out.println(event);   // every lane, one spectrum
}
```

Every lane is a complete core agent on its own virtual thread. The panel announces each lane on the stream (`agent_spawn`, then a task message), the lane reports in with a working status, its whole event stream rides along verbatim, and a result message records completed or failed before the panel's own `run_end` closes the spectrum. One broken lane never kills the fleet; it fails loudly and alone.



The code fleet: every lane a full core agent on its own virtual thread, every event wrapped in a bus envelope, one aggregator draining the topic into one EventStream, the same blocking for-loop as a single agent.

what rides the bus

Under the surface, every event of every lane is wrapped in a bus envelope: sender, contextId, taskId, a per-sender sequence, a causal parentId chain, a topic for session isolation, and the RunEvent itself, verbatim. Addressing is self-addressing: consumers correlate by taskId within a contextId instead of from/to routing. The transport behind it is a seam. In-memory serves a fleet inside one JVM; a process transport (`ProcessBus` over a small TCP hub) carries a fleet spread across processes and machines. The envelopes and their consumers are identical either way, so nothing downstream can tell the two apart.

> WHERE IT DOCKS
>
> The bus publishes through the same tracing seam the JSONL sink uses: one registry in the core where the session file and the bus publisher hang side by side, durability first (JSONL before bus). A node's local session file is always the anchor; the bus is what makes it visible to the rest of the fleet.

spectro node: the process fleet

A fleet does not have to live in one program. The launcher runs one prompt as a fleet node whose whole event stream rides the bus to a hub:

```
spectro node -p "Review the input handling" \
    --hub 127.0.0.1:7700 --context pr-42 --role security
```

The node runs headless, exactly like `spectro run`, and publishes as it goes. `--context` names the fleet to join (the bus topic derives from it) and is required; `--hub host:port` (or `$SPECTRO_HUB`) names the hub. Options fill in the rest: `--id` (default `node-<pid>`), `--role` (default worker), `--permissions` (`readonly` by default, `auto`, `ask`, or `extended`), `--max-turns`, and `--linger`. The local session JSONL stays the durability anchor, so a dead or stalled hub never blocks the run: events past the buffers are dropped from the *bus view* only, counted and warned loudly, while at-least-once still holds for everything the outbox accepted. Each process start stamps a fresh wall-clock epoch, so a restarted node reads as a new stream on the bus, delivered rather than misread as a redelivery.

By default a node connects, runs once, and exits. `--linger` keeps it connected and controllable after the run, until a stop arrives over the hub (or, in production, SIGTERM). `--permissions ask` parks each gated tool for an operator to answer over the hub instead of a fixed yes or no (see the control plane below).

A node can also be given a run loop, and that is the difference between a node you can only stop and a node you can talk to. Three flags do it: `--watch <dir>` runs the prompt again on changes in that directory, non-recursively, with a burst of changes debounced into one run; `--listen <port>` runs it on a POST to `/trigger` at that port, where the bind is always `127.0.0.1` and the bearer token is printed at start; and `--every <n>ms|s|m|h` runs it on a timer, at least a second apart. None of them fires at start — the prompt of a triggered node is written against an event, and running it with no event executes half a sentence, so the node boots straight to waiting and says so. Plain `--linger` gives no loop at all: that node parks on its stop latch and has nowhere to put anything you send it.

code fleets on the hub

A `Spectro.panel()` fleet runs in one JVM by default and returns one merged EventStream. Set `$SPECTRO_HUB` (or build the panel with a hub address) and each lane also mirrors its frames to the hub as a node, so a code fleet becomes live-visible in a running cockpit and each lane arrives on the bus rail as its own card. The invariant holds where it matters: the returned EventStream stays byte-identical on the local in-memory spine, the hub is a purely additive mirror, and a dead hub never delays the run's close.

entering a fleet

Fleets are first-class, like sessions. The sidebar's segment row carries three of them — `sessions`, `fleets` and `state graph` — and switching to fleets lists every fleet the hub has seen: a row with its sigil, its context id, and its agent count as online of total. A fleet with a parked gate floats to the top and wears a pulsing `gate open` chip. Click a row to enter it. A browser opened after the fact is not blind to history: entering pulls each node's recent frames from the hub's replay ring, so a gate parked before you arrived is still there waiting. The ring is bounded; the node's own JSONL stays the full record.

With nothing entered, the segment lands on a lobby rather than on the last session left standing, and the lobby's machine-dependent offers say they cannot work *before* they are pressed rather than after. With no hub running, the spawn button is disabled and the place where the ready-made `spectro node …` command would stand carries the sentence that names the variable it needs instead. With a hub up, the lobby adds the part the server cannot answer: spawning also wants `SPECTRO_ALLOW_SPAWN`, and the API does not report whether it is set.

No hub at hand? The lobby's first offer is five recorded fleet scenarios — deterministic, no hub, no key, no provider — that load as a replay fleet and enter it. That is the fastest way to see the whole surface before wiring a real one. A fleet also has an address, `#/fleet/{contextId}`, and pasting one moves the sidebar with it; the address carries the fleet and nothing else, no tab and no reading, so it lands on the bus.

the fleet bar

While a fleet is entered, the app's tab row is replaced by the fleet's own bar: `bus`, `spectrum`, `trace`, a separator, one tab per agent, and a `+` that docks another node. Spectrum and trace survive the swap because both read across the whole fleet rather than one agent of it. Chat, graph, text and lab have no fleet face and are not on the bar. Each agent tab carries a state dot — working, failed, completed or idle, pulsing while a working agent is still connected — and an agent with a decision parked marks its own tab.

Picking an agent tab pins the trace filter with it, so the trace for one agent is one click away and already scoped. There is no way off the bar itself: leaving a fleet is a sidebar move, to a session, to a new chat, or to another fleet. The back and forward chevrons stay in the header, and ⌘← and ⌘→ walk your history as before.

> WHAT A 0.6.1 READER WILL MISS
>
> Two views 0.6.1 offered have no way in any more. The fleet canvas — one spectral node per agent, spawn and task rails between them — was reached through the graph tab of an entered fleet, and the machine room, the whole fleet drawn as one agent-system diagram, through the lab tab. Neither tab is on the fleet bar. The machine room's fold survives where it matters: a bus card is built from exactly the same scene, with the same loop, gate and wiring chips on it. The canvas is out of the app entirely. The surface decision is not closed.

the rail and its cards

The bus is where an entered fleet opens. Every node docks on one rail as a full agent card, and the rail draws with nothing on it too. That is deliberate: the one surface that explains what this application is used to answer an empty fleet with a line of placeholder text, so it only appeared once it no longer needed to explain anything. Now the sentence sits on the rail — “No nodes on the bus yet.”, beside the button that docks one.

A card carries the id, the role and the provider; a restart marker when the node came back with a new epoch, since a restart is a new stream and never a silent continuation; an `offline` label when the node has left; and the task line. Under that sit four chips for what the agent is wired to — `disk`, `shell`, `mcp` and `llm`, each naming the file, command, server or provider in use while it is in use, the `llm` chip wearing a cloud mark when that provider is remote — then a gate row while a decision is parked, the agent's own spectral band, tokens in and out, and a stop control.

Spawn ancestry orders the row and draws no tree. A spawned node docks immediately right of its spawner and carries the parent id above it, but the nodes stay peers on one line, because that is what they are on a bus. Traffic rides the rail underneath them: every `agent_message` is a mark between the two agents' drops, coloured by its role — task, result or status — with its direction written under it, and the most recent one animates from sender to receiver. The rail shows the last twenty-four; the trace keeps every one.

The card head and the card's band are doors. The head opens that agent's own tab; the band opens the trace, already filtered to that agent.

one agent's tab

An agent tab shows that agent's activity as a feed: one row per event, with a timestamp relative to that agent's own first frame, the event type and a one-line preview, dot-coloured across eleven kinds of event. The feed follows the live edge as frames arrive. An agent that has done nothing yet says so instead of showing an empty pane.

talking to a node

Until 0.8.0 both composers were disabled inputs that said why on hover, because the bus control op carried `stop` and `gate` and nothing else. It carries words now.

One component draws the box on both faces — on every bus card, and at the foot of an agent feed — so the two can never disagree about who may be talked to. It is a text input with the placeholder “message this agent …” and a `→` button; Enter sends, Shift+Enter does not. The box is live only for a node that is connected *and* announced a trigger, and the border is the tell: dashed is dead, solid is live. Those two dead states are different problems with different fixes, so the tooltip names which one you have — “the node is not connected”, or “this node cannot take messages — it has no trigger, so it has no run loop. Start it with a trigger to talk to it”.

A send posts to `/api/fleet/{node}/message` behind the same local-origin fence as stop and the gate. The server dispatches a `ctl` frame carrying your words to the node's live connection, and the node's bus reader drops them into the fire slot its trigger loop is already waiting on. The node simply runs again — the operator's words are another thing that fires it, not a second machinery beside the triggers — and the words stay out of stderr, because they are content and that is not where content belongs. Messages that arrive faster than the node fires merge in order rather than being refused, since the endpoint already answered and nobody is left holding a retry.

The answer is one of four short sentences under the box, and none of them claims more than it knows. “sent — best-effort, no ack” is the 202: the words reached the node's connection, which is the most anyone can say, because the control plane has no acknowledgement. “the node cannot take that” is the 409 for a node with no run loop. “the node is gone” is the 404. “not sent” covers everything else, a request that never landed included, and a failed send keeps your draft rather than eating it. Stop is idempotent and its own note tells you to re-issue until the node leaves the roster; a message is not, and says so — sending it twice says it twice.

The bus wire version moved from 3 to 4 for this one verb, which is worth the cost in writing. The gate form could stay on 3, because a pre-gate node dispatches on the call id and simply never enters that branch. A message has no such luck: every shipped version-3 handler reads `if ("stop".equals(action))`, so a v3 node handed `action:"message"` would have dropped the words with no log line at either end — a 202 at the endpoint, silence at the node. The bump makes the line unreadable to v3 rather than quietly misread by it.

the control plane

Watching is one thing; driving is another. The honest state is that most driving still happens from the CLI (`spectro node`) or from code (`Spectro.panel()`). What the cockpit adds is a small set of first-class controls over the hub.

Spawn. Four affordances open the same form: the `+` at the end of the fleet bar, the “dock a node” button at the end of the rail, the sidebar's `+ node` once at least one fleet is listed, and the lobby's own spawn button while none is — that last one is how a first node gets started before there is a fleet to dock into. The web spawn is deliberately read-only: the endpoint is reachable from a browser, so it must never launch a node that writes or runs code, and the server forces a UI-spawned node to `--permissions readonly` rather than trusting the request. It is also opt-in (`SPECTRO_ALLOW_SPAWN`, off by default) and needs a running hub and a reachable node launcher, and because every refusal is a uniform 404 the form names all three instead of pinning one. For a node with full rights it hands you a copy-paste `spectro node …` command, built from the same fields plus the live hub port, with an ask-or-auto toggle. You run that in your own terminal. A replayed scenario fleet offers neither the bar's `+` nor the rail's dock button — there is no hub behind it to dock into.

The hub. The hub is off until you turn it on, with the switch in the settings under Fleet or with `SPECTRO_HUB_ENABLED=true` (`on`, `yes` and `1` work too). It listens on loopback port 7700, and `SPECTRO_HUB_PORT` sets another port from 1024 up. The hub never binds a port under 1024 (it needs admin rights), a port in 8080 to 8100 (the app's own server runs there) or the server's own port; it takes the first free port from 7700 instead. If another program already listens on the port you set, on any address, the hub stays off and the app starts anyway. The fleet settings, the doctor and the server log name the port the hub took, or the reason it is off. A `SPECTRO_HUB_PORT` without the switch also turns the hub on. Both values are read at start, so a change takes effect after a restart.

Stop. A connected agent card carries a `⊗` button that sends a stop to the node over the hub. Best-effort, and it says so: the click confirms the verb was sent, not that the node has stopped (the control plane has no ack). The node leaves the roster when it actually ends; re-click if a stop was lost, and SIGTERM is the ultimate backstop.

Answer a gate. A node started with `--permissions ask` parks each gated tool instead of denying it. The parked gate surfaces on the node's card, on its lane in the fleet's spectrum, and in the same permission dialog you use for your own session, over every tab of the entered fleet, the agent tabs included, because the dialog sits outside the view chain by construction. Approve or deny there and the answer travels to the node over the hub. There is no “remember” here: a remote node keeps its own allowlist, not yours. Like stop it is best-effort; if the node left first, its own close denies the orphaned gate, so the gate clears either way.

> WHO CAN DRIVE
>
> Every fleet write, and the roster read itself, is gated to a local origin: a loopback caller *and* a localhost Host header (the Host check is the DNS-rebinding defense). A remote or rebound page gets a 404 with no hint the endpoint exists. Spawn adds its opt-in flag and forces the node read-only on top. The bus carries prompts, reasoning and tool I/O, so its face is treated as sensitively as the ability to spawn.

A fleet is still not an object anyone creates. No endpoint makes one, and the only path to one is spawning a node into a hub that is already running behind two switches that both default to off. The message verb makes an existing fleet talkable; it does not make a fleet easier to have.

CHAPTER 15C

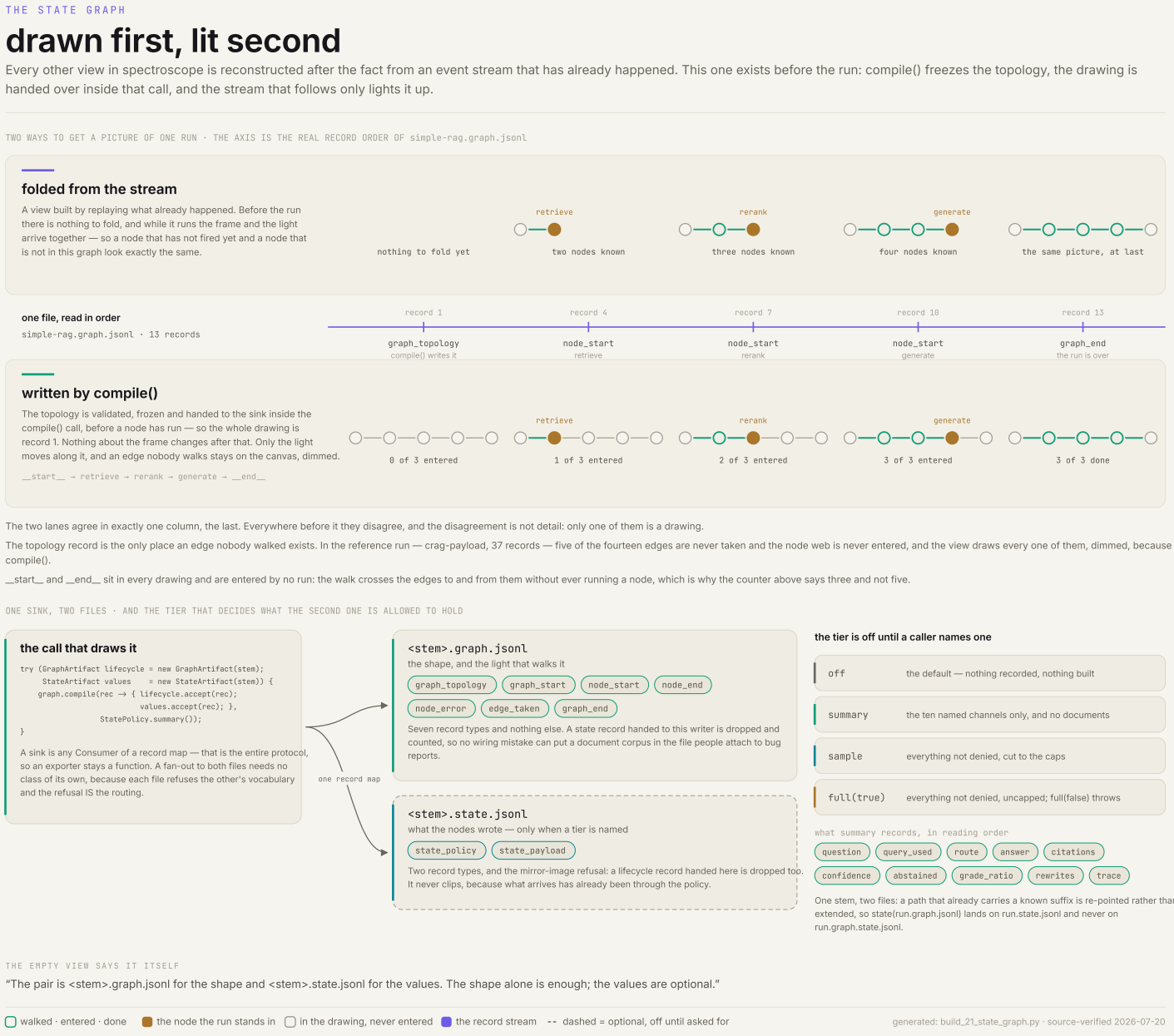
The state graph: drawn before the run

Every other view in this book is reconstruction — a run happened, events arrived, and the picture was folded out of them afterwards. A state graph is the one drawing that already exists before the first token, because its topology is fixed at `compile()`, and the run only lights it up.

Why this one is not a reconstruction

The spectrum, the trace, the flow view, the machine room: each of them reads the same event stream and folds a picture out of it, so each of them can only ever know what already happened. A `StateGraph` is different in kind. You declare its nodes and edges in your own code, `compile()` validates and freezes the topology, and the drawing is knowable at that moment — before a node has run, before a token has moved. The sink receives the topology before the compile call even returns. What the event stream adds afterwards is light, not shape: nodes as they enter and leave, edges as they are taken, values as they are written. The header of a loaded view states the whole idea in one line, on every frame: “Topology fixed at compile(); the event stream lights it up.”

The runtime lives in `spectro-core`, in the published library, not in the app. The app never runs your graph and never asks a server for one; it reads two files your run wrote and draws them. The footer of every loaded run says `offline · no network calls`, which is the point rather than a disclaimer. One warning before you start: the session tab called *graph* is a different thing entirely, and the last section of this chapter separates the two.



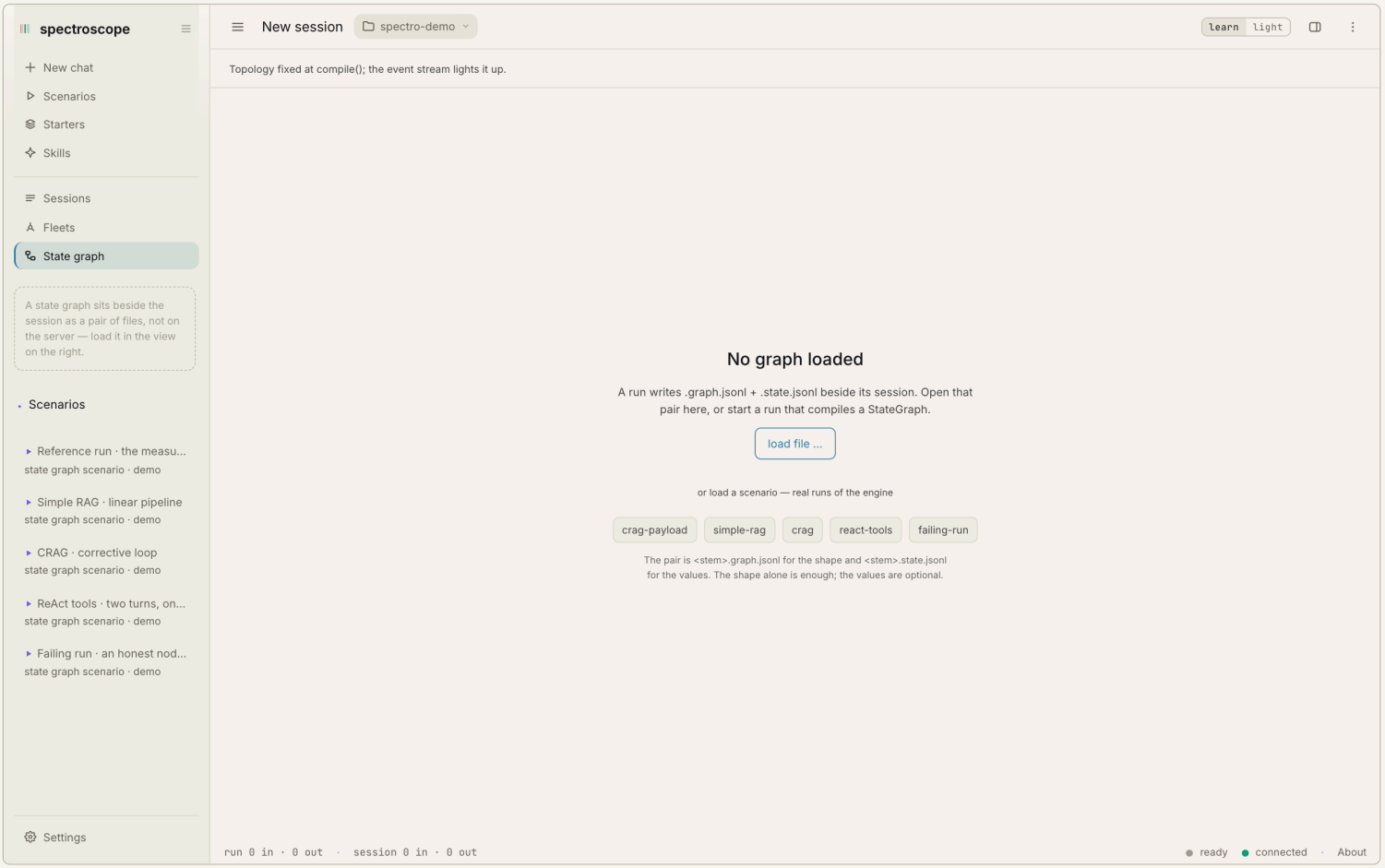
> The whole path in one picture: your code builds a StateGraph, compile() freezes the topology and hands it to the sink before the first node runs, the runtime in spectro-core writes structure and lifecycle into one file and whatever the policy allows into a second, and the browser view reads that pair from disk. Nothing crosses a network on the way.

Reaching it, and the address it does not have

The sidebar carries a segmented control with three buttons: sessions, fleets, and the state graph. In developer mode a fourth button, playbook, joins them (chapter 15d). The third one is the whole surface. Clicking it hands the entire content area to the view — no session tabs above it, because a graph is not a session. It needs no hub, no provider and no key, and unlike the fleets segment beside it, the tutorial in chapter 6c never closes it: this view reads two files off your disk and starts no process, so there is nothing here for a level to protect you from. Under the segment sits a note that says where a graph comes from: “A state graph sits beside the session as a pair of files, not on the server — load it in the view on the right.” Below that, an eyebrow reading `Scenarios` and five rows, one per bundled run.

Now the honest part. The state graph has no address. It is deliberately absent from the router, so there is no `#/stategraph` to bookmark, no deep link into a record, and no way to send a colleague the run you are looking at — the deep links that exist for trace rows and spectrum windows stop at this door. A reload empties the pane, and it has to: the browser cannot re-open a file from your disk without you picking it again. Exactly one piece of view state survives, the horizontal-or-vertical orientation, kept in local storage under `spectroscope:sg-orientation` and defaulting to horizontal. Cursor and picked node reset, because the run they pointed into is gone.

> WHAT A RELOAD COSTS YOU
>
> Loading is a gesture, not a state. Reload the page or open a second tab, and the pane is empty again with its five scenarios on offer. A switch to sessions and back costs nothing — the run, the cursor and the pick are held above the pane, so the segment hands them back. If you want to keep what you are looking at, export it — two files, one click each, described at the end of this chapter.



The pane before anything is loaded: the third sidebar segment is active, the five bundled runs stand in the rail under their human titles, and the shelf in the middle offers the same five runs under their bare file stems. Two pickers, two label sets, one set of runs.

Five bundled runs, under two sets of labels

The empty pane offers a `load file …` button and, under the line “or load a scenario — real runs of the engine”, five chips. The sidebar rail offers the same five runs one column to the left. They do not read the same, and that trips people: the rail prints human titles, the shelf prints bare file stems in mono. Name the surface when you tell someone what to click, because “click `crag`” and “click `CRAG · corrective loop`” are instructions for two different places on one screen.

RAIL TITLE	SHELF CHIP	WHAT RAN	VALUES
Reference run · the measured template	`crag-payload`	the widest of the five — ten nodes, fourteen edges, eleven supersteps	summary
Simple RAG · linear pipeline	`simple-rag`	start, retrieve, rerank, generate, end — one line, no branch	sample
CRAG · corrective loop	`crag`	a router, a grade that branches, and a rewrite that returns to the router	sample
ReAct tools · two turns, one thread	`react-tools`	two runs filed under one thread id, with act returning to plan	summary
Failing run · an honest node_error	`failing-run`	four nodes, three edges, and a generate that ends in an exception	summary

All five were written by the engine itself in real runs, never typed by hand: the demo files are the output of a test that runs the graphs and keeps what they wrote. The two-turn scenario also shows a limitation the view states rather than hides — when one file pair carries several runs, the footer describes the first while the transport walks them all.

The pair of files, and why the shape alone is enough

A run writes two files beside itself. `<stem>.graph.jsonl` carries the structure and the lifecycle — the topology record, graph start and end, node start, node end, node error, and every edge taken. `<stem>.state.jsonl` carries what the nodes wrote. The split is enforced by the writers and not by whoever calls them: the structure file accepts those seven record types and nothing else, the values file accepts only value records, and anything misfiled is dropped and counted rather than smuggled through. The counter surfaces in the footer, next to the count of lines that were not JSON.

The naming is mechanical: the values path is the structure path with its family suffix re-pointed, so `run.graph.jsonl` always pairs with `run.state.jsonl` and never with `run.graph.state.jsonl`. The file picker takes both at once, and it is worth selecting both, because picking only the values half of a pair with nothing drawn yet earns the reply “That was a .state.jsonl on its own. Values need a shape — load the matching .graph.jsonl with it.” The other way round is fine, and this is the sentence the empty pane closes with: the shape alone is enough, the values are optional. A graph with no values file draws completely, counts its supersteps, colours its edges and marks its loops; only the panel goes quiet, saying “No .state.jsonl loaded — the shape is complete, only the values are missing.” while the footer wears a `no .state.jsonl` chip.

Four tiers, and why the default is off

What a run may write into its values file is decided once, at compile time, and carried unchanged to every node. There are four tiers, and the default is the one that records nothing.

TIER	WHAT LANDS IN THE VALUES FILE
`off`	nothing — and nothing is even built. This is the default.
`summary`	only the ten channels on the allow list: `question`, `query_used`, `route`, `answer`, `citations`, `confidence`, `abstained`, `grade_ratio`, `rewrites`, `trace`. Its promise is that no document corpus reached the file.
`sample`	every channel that is not denied, cut to the per-channel caps — `docs` keeps its first three entries and clips the strings under it at 512 B, an unnamed channel falls back to 1024 B, and a whole record is capped at 8192 B.
`full`	every channel that is not denied, uncapped. It will not construct unless you say so: `full(false)` throws.

The reason for that default is worth a sentence, because it is a design decision and not an oversight. A library upgrade that quietly started writing a user's document corpus to disk would be a data incident, not a feature — so there is no ambient switch, no environment variable and no settings toggle that turns recording on behind your back. Recording is switched on by choosing a `compile()` overload, which means it is one visible line in your own source, in a file you own, where you will find it again:

```
var app = graph.compile(sink, StatePolicy.summary());
```

A null policy or `StatePolicy.off()` records nothing; the lifecycle-only overload `compile(sink)` still writes the structure file, so the drawing exists either way. The tier is frozen for the length of the run and announced once, in a `state_policy` record — the one line that later explains why a channel a node wrote is missing from the file. Wherever recording happens at all, one channel is denied by default: `principal`. And no policy field promises redaction: it says `patterns`, never `true`, because patterns catch credential shapes and cannot catch confidential prose.

> OFF IS THE DEFAULT, AND NO DEMO CAN SHOW IT
>
> None of the five bundled scenarios ships at the `off` tier — two record at `sample` and three at `summary`, because a demo of nothing teaches nothing. The tier you will meet on your own first run is the one the app cannot show you.

The transport, and the speed that is not one

Under the header sits the transport: five glyph buttons — `|<`, `<`, play, `>`, `>|` — a speed select, a band of ticks, a counter and a state chip. Only two of the buttons have a name to quote: the rewind announces itself as “first record” and the play button as “play (space)”, flipping to a pause glyph and “pause (space)” while it runs. The other three carry no title and no label, so this book describes them by glyph, as you will have to as well.

The speed select offers `0.5×`, `1×`, `2×`, `4×` and `instant`. The last of those is not a speed. It jumps straight to the last record and pauses — a promise of no waiting, kept literally, because playing a four-hundred record run at a nominal top speed is still half a minute of watching. The four real speeds divide the recorded gap between two records by the speed and clamp the result into a 45–760 ms window. That window is what keeps a run with a two-minute think from stalling the replay, and a burst of ten records in one millisecond from flickering past. A pair carrying no usable timestamps steps at 260 ms divided by the speed.

The band is one clickable tick per record, each tick titled with its record type and node, so you can aim at a node error instead of hunting for it. Every manual seek — any button, any tick — pauses playback first, because a transport that keeps running under your click is a transport that argues with you. Pressing play on a finished replay starts it again from record zero rather than doing nothing. The counter reads `record N/M` and the chip beside it reads `complete` when the cursor stands on the whole run, `mid-run` otherwise. That counter is the one string in this view that never translates; a German session shows `record 13/27` next to `mitten im Lauf`.

The keyboard is shorter than the toolbar: `space` toggles play, `→` and `←` step one record, `Home` returns to the first record, and `End` jumps to the whole run. Keys stay out of the way where they would do damage — they are ignored while a modifier is held, while you are typing in a field, and entirely while your focus sits inside the export dialog; space alone is ignored while focus sits inside the right-hand panel.

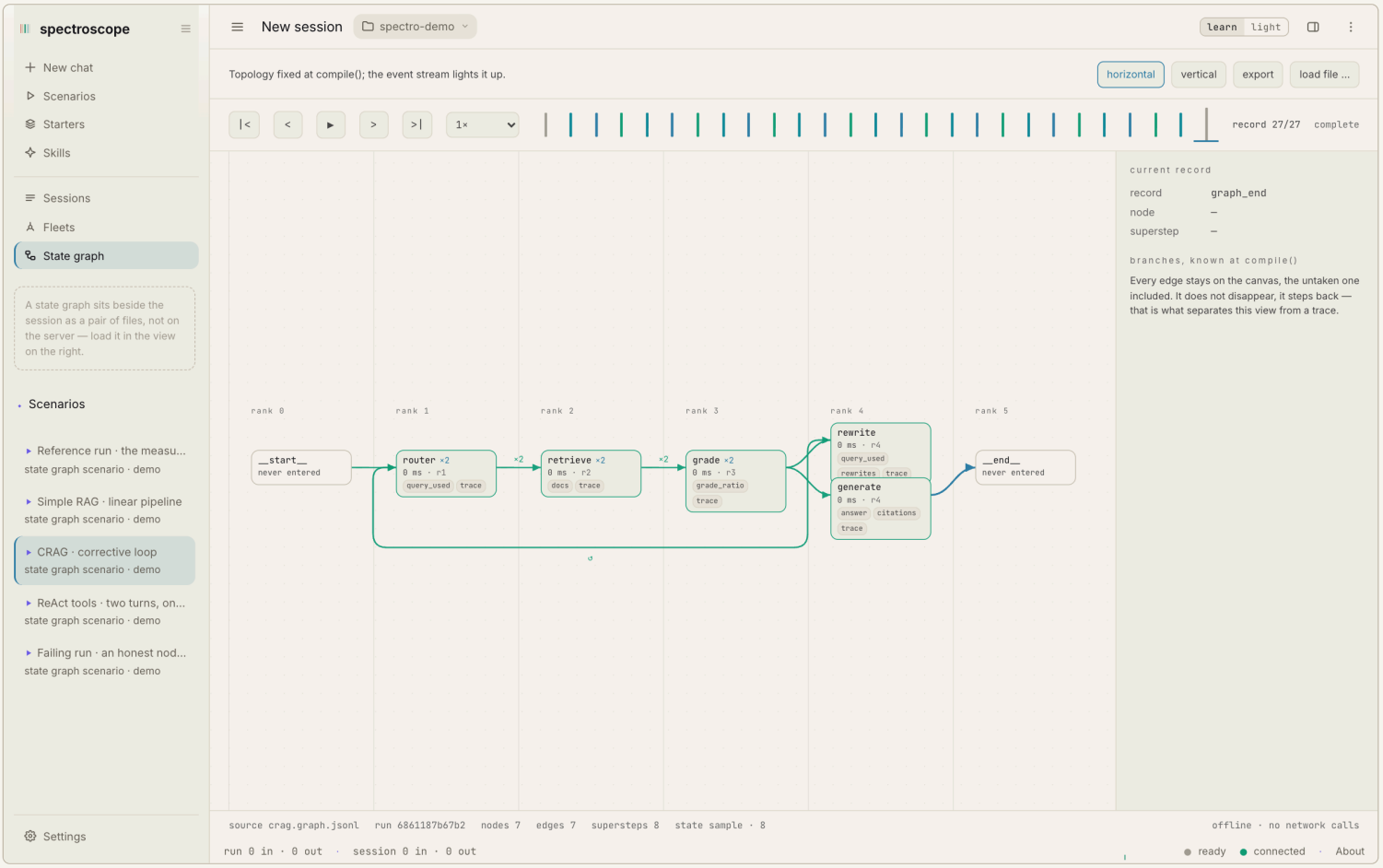
The transport, cropped: five glyph buttons, the speed select, one clickable tick per record with the cursor's tick raised, and the counter. The chip reads mid-run because the cursor is parked at record 13 of 27; at the end of the run it reads complete.

Reading the drawing

The field is organised by rank. Every occupied rank gets a thin ruler drawn in the gutter before its column, running far past the content in both directions, so the stages read as separated columns instead of a cloud of boxes. Each ruler carries a lowercase label: `rank 0`, `rank 1`, and so on. Each node card repeats its own rank in its meta line as `· r2`, with an `r` and not an `s`, because `s` means superstep everywhere else in this view and the two numbers are not the same thing. The meta line also carries the duration in milliseconds once the node has one, and the lifecycle word until then: `never entered`, `running`, `done`, `error`.

Two badges read `×N`. They run on different clocks, which is worth knowing before you catch them disagreeing on screen. On a node card, `×2` means the run entered that node twice — counted over the whole file, never scoped to where the cursor stands, so a card can already say `×2` while the transport is still walking the first visit. On an edge, `×2` means that edge has been walked twice *up to the cursor*, and it climbs as you play. An edge that closes a cycle is marked `↺` instead — but only while it has been walked at most once. From the second walk on, the count replaces the marker, because a loop you have taken three times is better described by the three than by the fact that it loops.

Colour on an edge is three-state and follows the same clock as the edge badge: a quiet arrowhead for an edge not yet taken, the success tint for one that has been walked, and the accent for the edge the cursor is standing on. The untaken edge is the detail to notice. It does not vanish when the run goes the other way; it dims and stays exactly where the compiler put it. The panel explains why, in the view's own words: "Every edge stays on the canvas, the untaken one included. It does not disappear, it steps back — that is what separates this view from a trace." A trace can only show the path taken. A graph shows the branch that was available and declined, which is usually the more interesting half of the question.

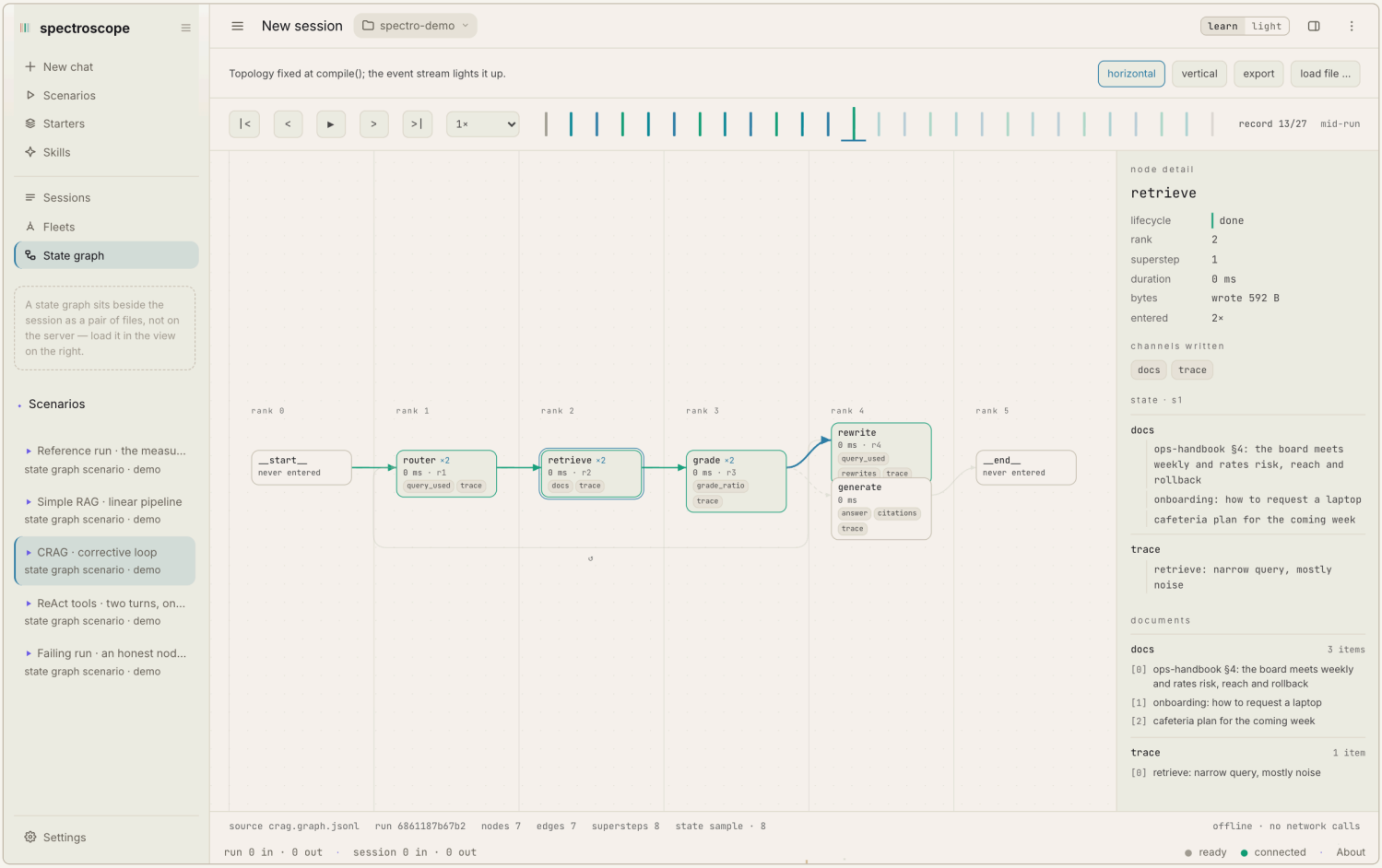


CRAG played to the end. Rank rulers separate six occupied columns, each carrying its lowercase label; router, retrieve and grade each wear a ×2 for their second entry; and the long return lane from rewrite back to router carries ↺ because it has been walked exactly once. Walk it twice and the marker becomes ×2.

The panel: what one node wrote at this visit

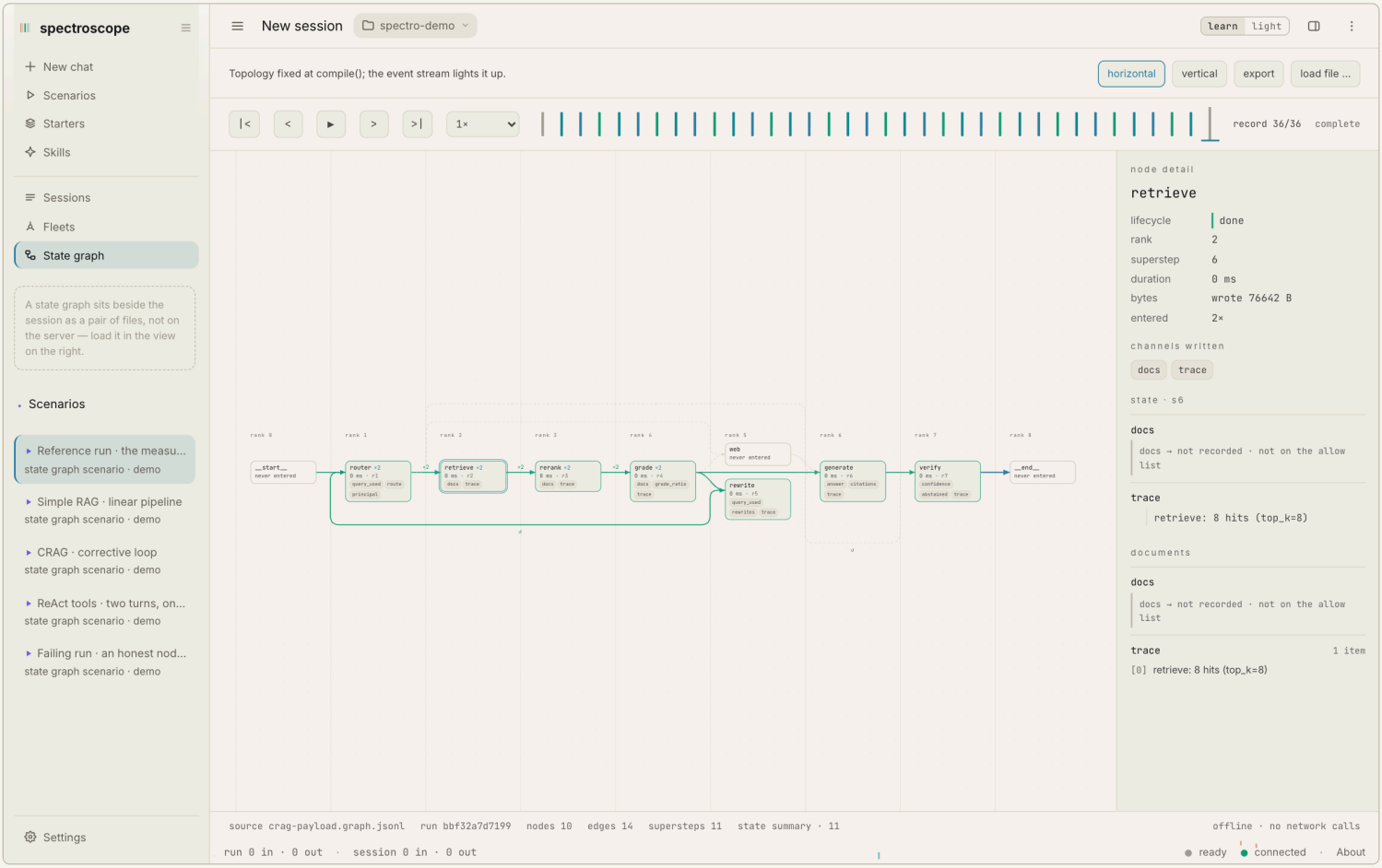
With nothing picked, the right-hand panel describes the cursor: a heading reading `current record` over three rows — record, node, superstep, with an em dash where the file does not carry one — followed by `branches, known at compile()` and the paragraph about the edge that steps back. Click a node card and the panel switches to `node detail`: the node id in mono, then lifecycle, rank, superstep, duration, bytes (`wrote 592 B`) and entry count (`2×`).

The superstep row is the one that repays attention. It is the superstep of the visit the cursor is standing in — not the node's last word. Before the cursor has reached the node the row falls back to its first visit, and it is empty only for a node the run never entered. Everything below it follows the same rule: the `channels written` chips, the heading `state · s1`, and the values under it are what that node wrote at *that* visit, not a merge of everything it ever wrote. A documents strip renders list channels as numbered rows with an item count, and a `clipped` badge marks a value a cap cut short. Where the recorder sampled a list, a badge says how many of how many entries were kept — taken from the recorder's own count of the original, never from the number of rows on screen.



A node picked mid-run. The panel joins what retrieve wrote at the visit the cursor stands in — superstep 1, 592 B, the docs and trace channels — while the card badge reads ×2 and the entered row 2×, because that count is over the whole run and not over the cursor.

The most useful thing the panel does is refuse to be silent about absence. A channel a node wrote but the policy did not record is printed in its own place, followed by `not recorded` and the actual reason: recording is off, the channel was denied, or it is not on the allow list. An empty box would teach you that the node wrote nothing, which is false and expensive to believe.

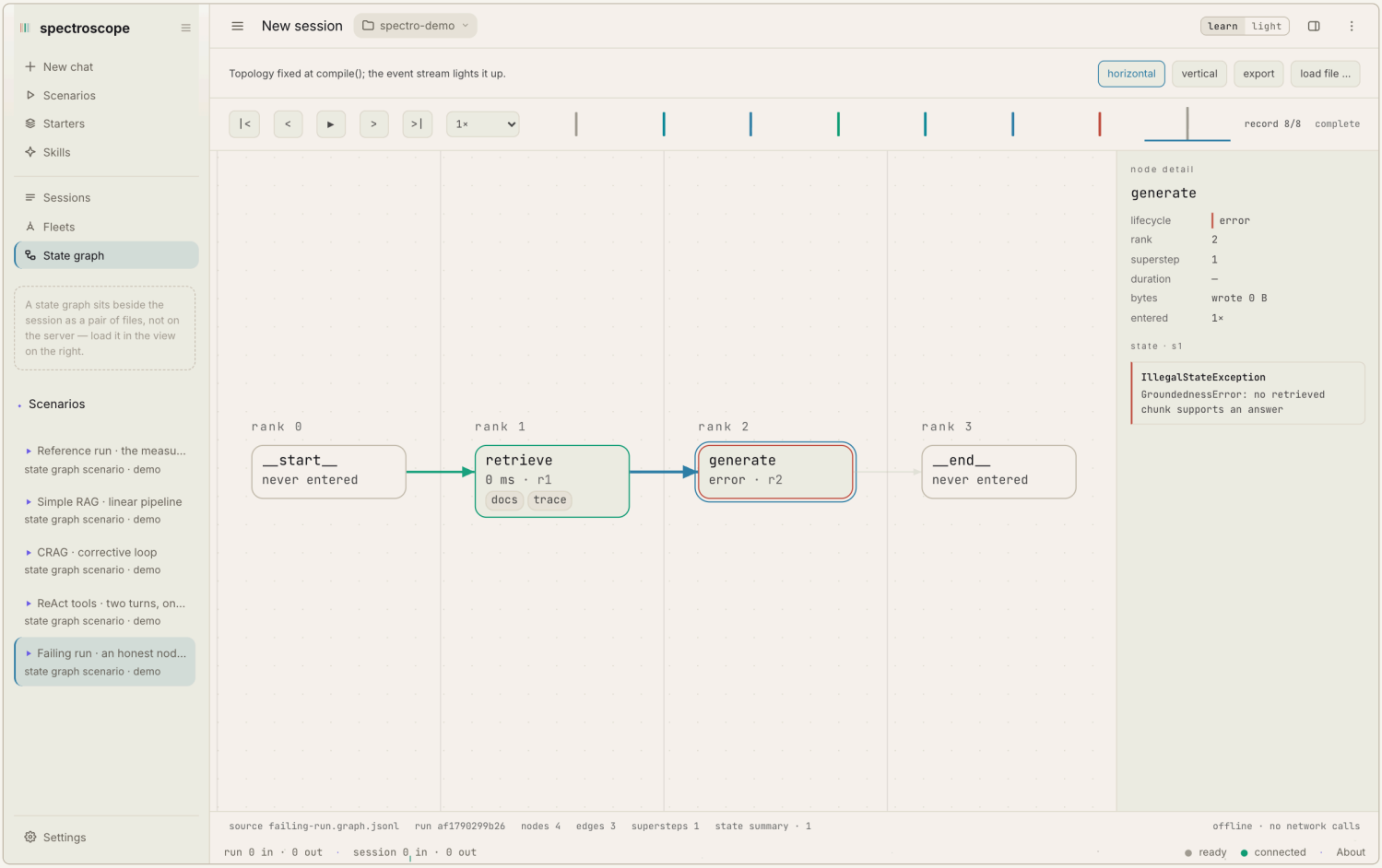


The reference run records at the summary tier, where docs is not on the allow list. The panel prints the channel, the words not recorded, and the reason, beside the channels that did make it into the file. Absence with a reason beats a blank space.

A node that failed

A failure shows up in four places at once, and none of them is a shrug. The card takes an error-tinted border, and its meta line prints `error` where a duration would be, since a node that threw has no end to subtract from. In the panel, the lifecycle chip reads `error` and an error box carries the exception class in bold on its own line, with the message under it — for the bundled failing run, `IllegalStateException` over “GroundednessError: no retrieved chunk supports an answer”. Both halves come straight from the record; an earlier reader looked for a nested object, found none, and drew a failed node with no reason at all.

The box is scoped to the visit the cursor is in: entering the node clears it, the error record sets it, and nothing looks forward. Scrub back to before the failure and the node looks healthy again, because at that record it was.

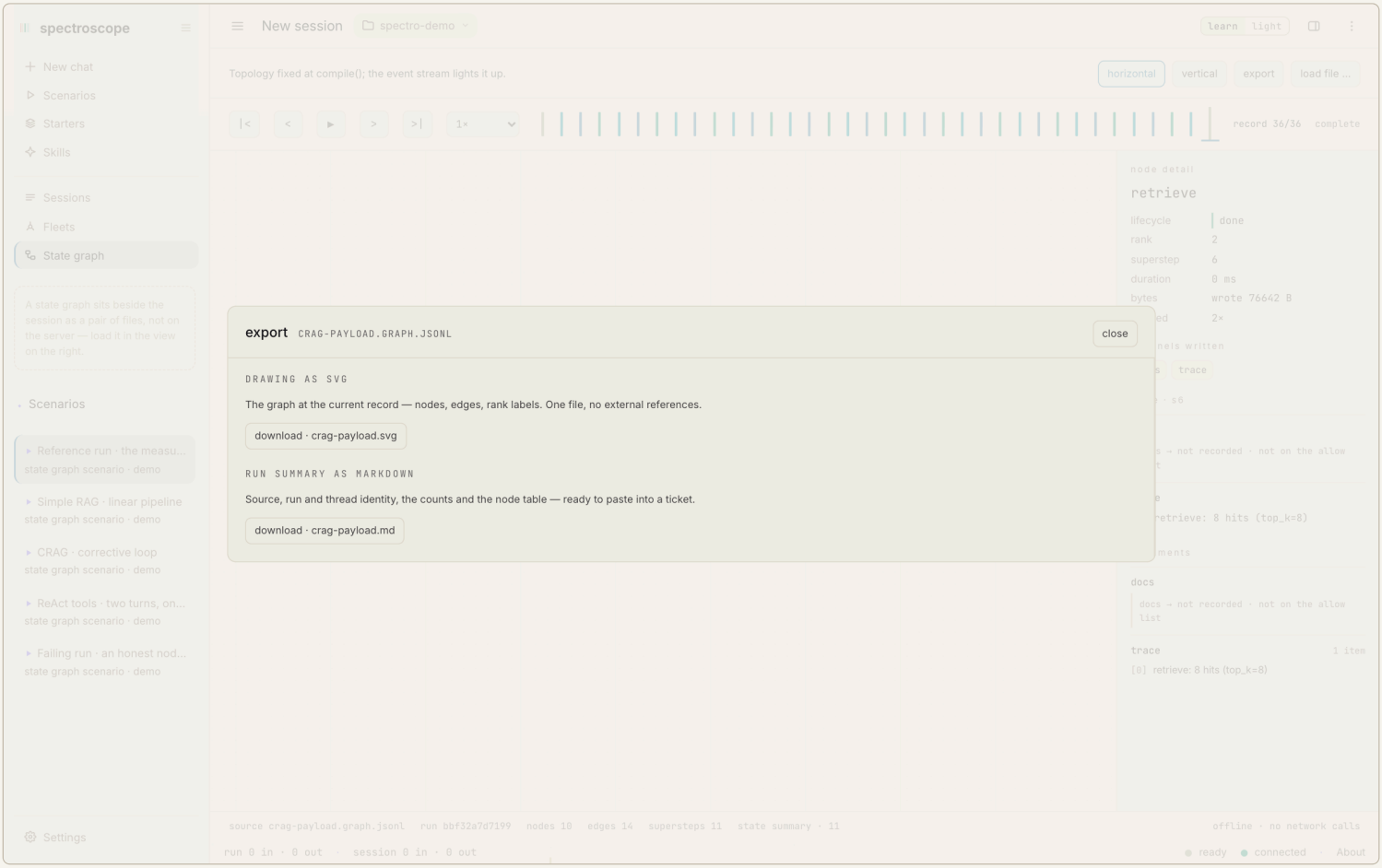


The failing run at its last record: generate carries the error border and prints error where a duration would be, and the panel names the class before the message. Four nodes, three edges, one superstep, and one honest node_error.

Export: two files, no options

The `export` button in the header opens a dialog titled export, with the source file name as its subtitle, a close button, and Escape as the other way out. There is nothing to configure — two groups, one download button each.

The SVG is the drawing at the current record, and it is self-contained: colours resolved to literal values from the export theme, no external references, no fonts to fetch, the source name as its title. It speaks the same visual language as the screen, rank rulers and lowercase rank labels included, and it carries the same `×N` and `↺` grammar, so a picture pasted into a ticket does not need a legend. The Markdown is the run as text: a heading naming the source, a bullet list of source, run and thread identity, node and edge counts, supersteps and the recording policy, then a six-column table of every node — node, rank, lifecycle, superstep, duration, channels written — in the order the drawing reads. Both file names come from the source stem, so `crag-payload.graph.jsonl` leaves as `crag-payload.svg` and `crag-payload.md`.



Export has no options because there is nothing to choose: the drawing at the current record as one self-contained SVG, and the run summary as Markdown, each behind a single download button that names the file it will write.

Why the layout is not dagre

The session graph tab is laid out by dagre; this drawing is laid out by code written for it. That was not a preference. Dagre breaks cycles the way most layered layout engines do — by reversing an offending edge, laying out the resulting acyclic graph, and declaring the job done. Pointed at the corrective loop, it drew `rewrite → router` as `router → rewrite`: the exact opposite of what ran, rendered cleanly and confidently. A graph view that shows the reverse of the truth is worse than no graph view, because it is believed.

What replaced it is a rank layout with gutter routing. A cycle-closing edge is marked as one and routed as a returning lane through the gutters, re-entering its target through the same face every other edge uses, so the arrowhead still points into the box and the direction on screen is the direction that ran. The lane costs vertical space, which is the honest price of drawing a loop as a loop. The session Graph tab keeps dagre and is welcome to it: that graph has no cycles to break.

Two things called graph

The app ships both words, and both of them stay, because they name two different jobs. Chapter 6 documents the session tab: a DAG of one run, one node per prompt, turn, tool call, subagent and answer, reconstructed from the events after they arrived. This chapter documents a segment beside sessions and fleets that draws a topology fixed before the run started. Renaming either one would only have made the other harder to find.

	THE SESSION TAB NAMED GRAPH	THE STATE GRAPH SEGMENT
where it lives	a tab inside one session	the third button of the sidebar's segmented control
what a node is	a prompt, a turn, a tool call, a subagent, an answer	a node you declared in your own code
where it comes from	folded out of the event stream after the fact	the topology record written at `compile()`
when it exists	after events arrive	before the first token
layout	React Flow with dagre	this project's own rank layout with gutter routing
cycles	none to draw	marked, routed as returning lanes, never reversed

CHAPTER 15D

Playbooks: a way of working, drawn

A playbook is a folder that says how a team builds: which steps come in which order, where the work branches, which skill and which model each step uses, and which handover document passes from one step to the next. The developer mode reads such a folder, checks it, draws it and lets you edit it. A session that picks the playbook gets the skills, commands and agents the folder brings, and a session can run by it: each step on the model it names, with the handover checked before the next step starts.

What a playbook is

The central file is `playbook.json`. It describes a graph of three kinds of node. A **step** is work: it names who does it (the chat or a child agent), which skills it uses, which model choice it runs on, whether it stays private or may use a cheaper hosted model, which documents it takes in and which it hands on, and whether it waits for your nod. A **decision** is a check with named outcomes, for example `pass` and `fail`, and every outcome has exactly one arrow leaving it. An **end** closes the path with a result. Arrows connect them, and an arrow that leads back to an earlier node is a loop.

Every loop has a ceiling. A decision that sits on a loop carries `max_rounds`, and when the rounds are used up the decision takes a further outcome called `exhausted`. A playbook whose loop has no such decision is refused when it is loaded, and the finding names the arrow.

The file chooses a model by name, never by address. The names are model choices, a small set that the playbook defines once, and each choice holds a provider and a model with an optional list of fallbacks. A step says `strong` or `fast`, and the choice resolves to whatever your own installation has. This is what lets a team share a playbook without sharing a machine.

The folder

The playbook that ships with the product shows the layout. These are the entries at the top of the folder:

Entry	What it holds
`playbook.json`	The graph, the model choices, the document types and the checks. This is the only file the loader must find.
`skills/`	The skills the steps name. The shipped folder holds the pack `spectropowers`, fourteen skills.
`templates/`	One Markdown template per document type, so a handover document always has the same sections.
`docs/`	A generated `index.html`: the graph as a picture, one section per step and one per document type.
`LICENSE`	The licence text of the material the playbook was built from.
`PROVENANCE.md`	Where the material came from, as which version, and every file that was changed.

The format also has places for `agents/`, `hooks/`, `commands/` and `workflows/`. The shipped playbook leaves the first three empty and ships two workflows (“Running a workflow” below), and version 1 only reads the paths the `contents` list of `playbook.json` names. The loader reads `playbook.json` and those files and nothing else, at most 1 MB per file and 200 files per folder, and only a folder you have registered. What a session does with those entries is described under “Playbook contents”.

The developer mode

developer is the third view mode, next to learn and light (chapter 6c). It opens everything learn opens, without the tutorial, and adds one segment to the sidebar: **Playbook**. The choice is offered in the first question of a new window, and the mode switch at the top right changes it later. In learn and light the segment does not exist and its code is never requested.

The segment has a row of tabs: Steps, Skills, Agents, Hooks, Commands, Workflows, Plugin, Files and New. The section “The bundle tabs” below describes the tabs from Skills to Files, and New is Spectrolyzr. The segment configures and edits playbooks and starts nothing: a run starts from the chat of a session (“Running a playbook”). The Steps tab has four parts.

- **Folders.** The folders you have registered, kept in `~/.spectro/playbooks.json`. A field adds a folder by its absolute path, and the button “Choose” beside it fills the field from the system's folder dialog. The same field is the target of “Copy the spectro playbook into a folder”. A pick changes nothing by itself: “Add” and the copy button act on the field, and a picked folder is checked exactly like a typed one. The playbook folder of Spectrolyzr has the same “Choose” button, and its project folder has one named “Pick”. A button next to each other folder, “Use for this workspace”, pins it to the working folder of the window.
- **The graph.** Steps are cards that show the step name, the model choice and who performs it. Decisions are diamonds, ends are rounded boxes. Each arrow out of a decision carries its outcome as a label, and a loop is drawn as a loop with its round limit, for example “up to 5 rounds”. The colors follow the active design, light or dark.
- **The steps.** One table row per step: who performs it, each skill with “in the playbook” or “missing in the playbook” (read from the skill folders of the playbook itself), the model choice with the provider it names and that provider's state on this machine, the privacy switch, the documents the step takes and hands on, and whether it waits for a nod.
- **Findings.** The problems found in the file, one line each, naming the node or key. The list appears only when there is something to report. A file the reader refuses, for example one with an unknown field or a forbidden key, is not drawn at all. A file that reads but breaks a rule, such as a loop without a ceiling, is drawn with its findings above the graph.

For a session that holds a run the tab has one more line, “Runs of this session: open”. It goes back to the chat and shows the session's runs wide in the Plan panel (“Watching and stopping” below). With no folder pinned the segment shows the folder picker, the copy button and that line, and nothing else.

The spectro playbook

The product ships one playbook, `spectro`. It holds 10 steps, 8 decisions and 2 ends. A first decision classifies the work as a spike, a bounded change or an architectural change, and each class takes its own path.

- The **architectural** path brainstorms with you, checks that the spec has its sections, waits for your nod, writes a plan, checks the plan, waits for your nod again, sets up a worktree and then enters the build loop: implement a task, review it, fix what the review found, for at most 5 rounds per task. A final review follows, and then the branch is finished.
- The **bounded** path is a short design in the chat, an implementation test first, and the finish.
- The **spike** path is a probe that ends in a recommendation.

The model choices are `fast`, `standard`, `strong` and `judge`. The document types are Ticket, Spec, Plan, Task report and Review report, and every step names the ones it takes in and hands on. The skills come from `spectropowers`, a copy of the superpowers skills with house rules worked into the text. `LICENSE` carries the MIT text of the source, and `PROVENANCE.md` names the source version and lists every edited file.

Copying it into a folder

Choose a folder you want to keep the playbook in, ideally one that is its own repository, type its absolute path into the field and press “Copy the spectro playbook into a folder”. The server writes the bundled files there and registers the folder. It never overwrites. If any target file already exists, nothing is written, the answer is a conflict, and the window lists the files that stand in the way. After a copy you can edit `playbook.json` in your own editor or in the segment's editor, which the next section describes. The segment reads the file each time it loads the folder, so the graph shows what the file says now.

Spectrolyzr

Spectrolyzr writes a new project and, if you ask for it, a playbook for it. It can also write a playbook alone. It is one page in the Playbook segment, behind the tab “New”, next to the tab that shows the playbook. Every choice stands on the page at once, so nothing hides behind a step. The project is small, builds, and ships one test that passes. The same choices always produce the same files, byte for byte. The files come from templates inside the app, so nothing is fetched from the network, and no generated project depends on a spectroscope package.

At the top of the page the switch “What to make” offers two options, “Project” and “Playbook”. The page below it changes with the option.

With “Project” the choices stand in one column and the files beside them. On a narrow window the files move under the choices.

- **Archetype, language and name.** Pick an archetype and a language and type a name. The archetypes are **Service**, an HTTP server with one `/health` route; **Library**, one public function; and **Command line tool**, a command that greets a name and returns an exit code. The languages are **TypeScript**, **Python** and **Java**. A name starts with a lower case letter, holds lower case letters and digits, may join parts with single hyphens, and has at most 40 characters. A name that turns into a reserved word of Java or Python is refused, and the field says so.
- **Add-ons.** Any of three, in any combination. **Quality gate** adds one command that runs the type or lint check and the tests, and fails when no test ran. **CI** adds a GitHub Actions workflow that runs that check on every push and pull request. **spectro playbook** writes the playbook of chapter 15d into a folder of its own. With this add-on on, a field “Playbook folder” appears under “Folders”.
- **Folders.** The project folder and, when the playbook add-on is on, the playbook folder. Each field takes an absolute path. The button beside it, “Pick” at the project folder and “Choose” at the playbook folder, opens the system's folder dialog and puts the path into the field. A pick changes nothing on disk, and you can still type or paste into the field. Where the system has no folder dialog, the page says “No folder dialog here. Paste an absolute path into the field.” If a dialog is already open, it says “A folder dialog is already open.”
- **Files.** Under the heading “Files” stands a file tree with the project and, when you chose it, the playbook as two roots. Select a file to read its content and, under “Why this file”, one sentence on why it is there. The summary names the number of files per folder and two commands: the test command, and the check command, which is the gate command when you chose the quality gate. The tree fills in as soon as an archetype, a language and a name are set.

“Generate” stands at the bottom and writes the files. It stays off until the project folder is set, and in the “Playbook” option until the playbook folder is set.

With “Playbook” the page asks for less. It shows the “Playbook folder” with its “Choose” button and the section “Models”, with one choice for each of the four model roles of the shipped playbook, `fast`, `standard`, `strong` and `judge`. A choice offers “Keep the spectro playbook's model” first and then every model the provider registry lists, provider by provider. A role you leave on the first option keeps the model of the shipped playbook. “Generate” writes a copy of the shipped spectro playbook into the folder, sets the model of each role you chose, and registers the folder. No project is written, and the playbook is pinned to no workspace. The folder then appears in the Steps tab. The contents row of the loaded folder counts its skills, commands, hooks, agents and workflows. A playbook made this way carries the two workflows of the shipped playbook, `fanout-review.json` and `fanout-research.json`, for every archetype, byte for byte as the pack ships them.

The playbook folder

The playbook does not go inside the project. The field starts as a sibling of the project folder, named after the project with `-playbook` at the end, and you can change it. It may not be the project folder, inside it, or around it. Generate writes the spectro playbook there, sets its `test` variable to the check command of your project, registers the folder, and pins it to the new project folder. The pin shows in the Playbook segment once the project folder is the working folder of the window. Spectrolyzr does not change the working folder for you.

What Generate refuses

Generate never overwrites. It looks at both folders first. If any target file already exists, nothing is written, and the window lists the files that stand in the way, per folder. A choice the server does not know, or a name it refuses, comes back as a message that marks the field. After a successful run the window names both folders, says the playbook is pinned to the project, and offers a button that copies the path.

A generated Java project has no Gradle wrapper yet, so the command `gradle test` needs Gradle on your machine, next to Java 21. The product's own CI builds every archetype in every language, once bare and once with all three add-ons, and runs its check command, so a template that stops building turns that job red.

Editing a playbook

The segment edits the file with the mouse. The “Edit” button in the header opens the editor for the folder you are looking at. It is offered when the file can be rewritten without losing anything. A file that carries fields the reader does not know stays read only, and the header says why. Nothing on disk changes until you press “Save”.

The editor puts a toolbar above the graph and a panel beside it, or below it on a narrow window. The graph is the picture of the read view, with one difference: the layout engine places every box again after each change, so no box can be dragged to a spot of your choosing. The left mouse button selects and draws arrows. The right and the middle button pan, the wheel zooms, and “Fit” brings the whole playbook back into view.

To add a box, select a step, a decision or an arrow and press “Add step”, “Add decision” or “Add end”. After a step or an arrow the new box is inserted into the arrow that leaves the step, or into the selected arrow, so the path stays whole. After a decision it takes the first outcome that has no arrow yet. A new box gets the next free id, such as `decision_1`, and a plain English name that you can change in the panel.

To connect two boxes, drag from a handle on the right edge of the first and let go anywhere on the second. A step has one handle and a decision has one per outcome, each named when you hover over it. An outcome has exactly one arrow, so dragging from a handle that already has an arrow points that arrow at the new target. Select an arrow and the panel shows its target and, for an arrow that leaves a decision, its outcome, both of which you can change there. “Delete” removes the selected box with its arrows, or the selected arrow. On the canvas, Backspace and Delete do the same.

The panel has three tabs. Selection edits the selected node: a step has an id, a name, a goal, who performs it, a role for a child agent, its skills (the skills of the playbook folder, each marked “in the playbook” or “missing in the playbook” as in the steps table), a model choice that shows the provider's state, privacy, permission, the documents it takes and hands on, and the human nod. A decision has an id, a name, a check, its outcomes and its maximum rounds, and an end has a result. Documents and Checks edit the document types and the checks. Model choices, vars and contents are read only in this version. A text field saves when you leave it or press Enter, Escape brings the old text back, and Backspace in a field never deletes the selected box.

“Undo” and “Redo” step through the last 50 changes. Ctrl or Cmd with Z does the same outside a text field, with Shift for redo. A box that an arrow names but nobody defined is drawn as a dashed box marked “missing”, and its arrow can still be selected, so the mistake stays visible and nothing disappears silently.

The list of findings under the panel is the check of the read view. The server runs it on your draft a moment after each change. “Save” is refused while a finding exists: the status reads “Not saved” with the number of findings, the list names each one, and the file on disk stays as it was. A file you wrote by hand in another layout is rewritten in the editor's fixed form on the first save, which may reformat the whole file. The editor announces that before you save, because the first diff is then larger than your change. After that a save writes structure only, with no coordinates and the keys in one fixed order, so a diff shows your change and nothing else.

If the file changed on disk after you opened it, for example in your own editor, the save is refused with the message that the file changed on disk, and your draft stays on screen. “Revert” drops your changes and reads the file again. “Stop editing” closes the editor. While there are unsaved changes it is disabled, and so is the folder list.

As an example, a review round after *Implement task* in the spectro playbook takes these steps. Select *Implement task* and press “Add decision”. Choose the check `task_review` and type 3 into the maximum rounds. Select the arrow from the new decision to *Review task* and set its outcome to `pass`. Drag the `fail` handle to *Implement task* and the `exhausted` handle to the end `cancelled`. Press “Save”. The file gains one node and three arrows, one existing arrow now points at the new decision, and no other line differs.

The bundle tabs

A playbook folder carries more than `playbook.json`, and the tabs from Skills to Files show and edit the rest. Skills, Agents, Hooks, Commands and Workflows each list the entries of their kind in the `contents` of `playbook.json`. An entry is marked “Listed”, only on disk, or “Listed but missing”. “List it” adds an entry that is only on disk, and “Remove” takes an entry out of `contents` and deletes its files after a confirmation. Beside the list each tab shows a file browser of its folder. Files shows the whole folder. The tabs from Skills to Workflows and the Plugin tab stay disabled until a playbook is loaded; Files needs only a folder. The Workflows tab edits the files, and a workflow runs from the chat of a session with `/playbook workflow` (“Running a workflow”).

Select a file in a browser to edit it on the right. “Save” checks that the file did not change on disk after you opened it and offers “Reload from disk” when it did. Markdown and JSON files show a preview beside the text, and `playbook.json` opens the graph editor described in “Editing a playbook” above. While a file has unsaved changes, the other tabs are blocked. The editor opens UTF-8 text only. `playbook.json`, `LICENSE` and `PROVENANCE.md` belong to every playbook and cannot be deleted. The app reads a folder to six levels and 200 files; the tree says when the folder holds more. The app follows no symbolic link inside a playbook.

The Skills tab imports skills. “Import” lists the skills installed on this machine and the skills of the catalogue, with a checkbox each. An imported skill keeps its licence, and `PROVENANCE.md` gets a line that names its source. Agents and Commands have “New” with a name field: a name holds letters, digits, `-` and `_` and starts with a letter or a digit, and an agent's type is explore, worker or research.

The Hooks tab has “Import hooks”. It copies the hooks of your user settings into `hooks/hooks.json`, together with the scripts in your home folder that a command names, and replaces a `hooks.json` the playbook already has after it says so. A path in your workspace is written as `{project}`, and the contents list shows the working folder there. A hook that names a hidden file or folder in your home folder, such as `~/.ssh`, or a path that leaves the workspace with `..`, is left out, and the list marks it before you import. Other paths in your home folder are written with `~` where they start a word of the command. An import changes nothing in your settings, and nothing copies a playbook's hooks into them.

The Plugin tab lists the Claude Code plugins of your home folder, from the plugin cache and from the marketplaces, with the count of skills, agents, commands and hooks each one carries. “Import” copies the kinds you tick: the skills as one pack, the agents, the commands, and the hooks that run before or after a tool call. Agent fields and hook events the playbook format does not know are left out, and the report after the import lists them with the reason. MCP server definitions are never copied. A plugin without a `LICENSE` is not imported at all.

“Export” saves the folder as `<id>.playbook.zip`, and the same folder always gives the same file. “Import ZIP” reads a bundle in the browser, shows its files and contents, and writes them into an empty folder you choose, which it then registers. The server checks every path and the playbook before anything lands, so a refusal leaves the folder empty. A bundle has exactly one top folder with a `playbook.json` in it. It is refused when it holds more than 200 files or more than 24 MB, an entry deeper than six levels, two entries for the same file, or a binary file outside `docs/` or larger than 2 MB. Export and Import ZIP also refuse a hooks file in which a value matches one of the key patterns the recordings already redact, and name the line and the pattern. The check knows only these patterns: it does not find a password after a command option, for example.

After Generate on the New tab with “Playbook” chosen, and after an import on the Plugin tab, the pane opens the Skills tab of the folder. After Import ZIP it opens the Steps tab of the imported folder.

Playbook contents

A playbook can carry five kinds of content. The Playbook segment shows a Contents row with the count of each kind, and the tabs from Skills to Workflows list the items. Nothing copies a playbook's contents anywhere: the window has no Install or Remove button, and the server has no route for either. A session that picks the playbook reads its skills, commands and agents in place, from the playbook folder.

Kind	In the folder	What a session gets
Skills	`skills/`, one pack folder holding one folder per skill	Each skill under the name of its folder, in every session that picked the playbook. The pack folder is not part of the name; when two skills of the playbook have one name, both take `pack:skill`.

Commands	`commands/`, one Markdown file per command	Each command under the name of its file, invoked in the message box as `/name`, with the rest of the message as arguments.
Hooks	`hooks/`, a `hooks.json` and the scripts beside it	Listed with the command each hook would run and the full text of every script. A playbook's hooks do not run: nothing adds them to your settings.
Agents	`agents/`, one Markdown file per agent with a name, a description, a type and a preamble	Nothing is copied and nothing needs installing: the agent file is read from the playbook folder in place. A child step may name it as the role `agent:name`, and the run starts that child on the type and preamble of the agent file; its bytes are part of the hash the confirmation shows, so a file edited after the confirmation stops the start. The run does not start when the file is missing or is not a plain file in the folder (“agent not found”). In the chat, a session that picked the playbook starts its agents with the tool `spawn_named_agent`. The agent file cannot set a model, a permission or a tool list.
Workflows	`workflows/`, one JSON step list per file	Listed with their hash and never copied. A file that reads as a step list is “runnable” and runs in a session with `/playbook workflow` and its name (“Running a workflow”). Any other file is listed and is not run.

When your workspace or your own folders bring a skill or command with the same name, each of them takes a prefix that names where it comes from, and the playbook's item is invoked as `/<playbook id>:<name>`. A name that a command of the web chat uses, such as `clear`, always takes the prefix.

Every skill and command of the picked playbook adds one line to the system prompt of the session, a number of characters that the contents list adds up. The cost weighs more on a local model with a short context window.

Running a playbook

A run follows the drawing. The playbook becomes one graph on the product's own state graph engine: a step is a node that does the work, a decision is a node that runs its check and returns an outcome, and an arrow is followed only by the outcome it is labelled with. The next step starts after the handover of the previous one has passed its checks.

The confirmation

In developer mode the header carries a playbook chip beside the folder chip: a dropdown of the registered playbook folders and a play button. With nothing picked, the dropdown shows the playbook pinned to the working folder, marked “suggested for this folder”; it is stored for the session only when you pick it or start a run with it. “No playbook” is a pick of its own: the pinned playbook then does not stand in, and a start is refused. The pick belongs to the session and comes back when the session is opened again. The Playbook view only configures and edits: its first tab, “Steps”, shows the playbook and its latest run and starts nothing.

A run starts in the chat of the session. Type `/run-with-playbook` or `/playbook run` and the task in the message box, for example `/run-with-playbook add a CSV export`, and send it. The play button only writes the command into the message box. Without a task the command is refused with its usage line. The task reaches every step, every question the run asks you names it, and it is recorded with the run. Nothing starts yet. The server answers with a sheet, “Start this run?”, that names the task and shows the drawing and these parts:

- One row per step, and one per decision whose check is a `review`: who performs it, the provider and the model its model choice resolves to on this machine, the state of that provider, the privacy switch, the permission and whether the step waits for your nod. Before the sheet opens, the server asks every provider for its model list. When the playbook's model does not serve, the row says why in plain words: the key was rejected (with the HTTP status, and whether the key comes from the environment of the app or is saved in the app, never the key itself), the provider does not answer, it answers but does not list the model, or a private step names a cloud model.
- A model picker in every row. It starts on the playbook's model, labelled as the playbook's default, and offers the models of every provider whose list answered. A private step is offered local providers only, without Ollama's cloud models (a tag `cloud` or one ending in `-cloud`). A pick shows at once that its provider lists the model. It holds for this run only: the playbook file is not written.
- “Skills”, each with its source: “in the playbook” or “missing in the playbook”.
- “Commands this run may execute”, every command of a `command` check word for word, so a command written in a shared folder is read before it can run.
- “What this playbook brings”: the skills, commands, hooks, agents and workflows of the folder, for reading only, each with its source and hash. A skill, command, hook or agent is marked “in the playbook”, and a workflow “runnable” or “not run”. Nothing is installed, so the list names no install state and no item changed since an install. The full contents hash follows the list. The contents are shown here and are not part of the run hash.
- “Hash”: a sha256 over `playbook.json`, every template, every file under the skill folders and every agent file a step names. The sheet shows the first characters.

“Start” stays disabled while the sheet lists reasons under “The run does not start because”. The reasons are a finding in the file, a skill that is missing, a step that asks for the permission extended, a `human` check with more than four outcomes, which is more than a question can offer, and a nod or a `human` check when nobody can be asked. Start also stays disabled while a row has no model this run can use, and a line above the button names those steps and checks. A private step whose model sits on a cloud provider is such a row, and a local pick lifts it. Pressing Start sends the hash and the picks to the server, which loads the folder again and compares. If any file changed since the sheet opened, the run does not start and the answer is “The playbook changed since the confirmation. Open it again.” The server then asks each picked provider for its model list once more and refuses a pick that no longer serves. The sheet stays open until the run starts, and a refusal of this start shows in it as well as in the chat. Stop while the picks are checked again stops the start before the run begins. The run keeps the copy it pinned, so an edit while it runs applies to the next run. No tool of the chat agent can start a run. A playbook that the chat agent wrote reaches a run only through this sheet and your click.

What a step does

A **chat step** runs as a turn of the main agent. If the step's model differs from the chat's current model, the chat is switched to it first and the conversation travels along. The prompt is a short text that the session keeps and a long text that the model receives: the goal, the documents the step reads and writes with their required sections, the full text of each named skill and the template of each document it hands on. When the run ends the chat returns to the model it had before.

A **child step** runs as a child agent in the role the step names (`worker` when it names none), on the provider of its own model choice, whatever the chat uses. A role `agent:name` runs on the type and preamble of that agent file, as pinned at the start. It draws one slot from the chat's pool like any other helper.

After a step the runner looks for the files that changed under the location of each document the step produces. If the step carries a nod, you are asked whether to approve the handover, and anything but approval ends the run.

The checks

A decision runs one check and takes the outcome it returns. Five kinds exist.

Kind	What it does	Outcomes

`sections`	Every listed document exists, every required section has text, and no forbidden string appears. A section is a heading with its name. If there is no such heading, a line that starts with the name as a bold field, such as `**Goal:**` or `**Goal**:`, counts as the section. Its text runs up to the next bold field line or the next heading of level 1 or 2, so `###` sub-headings belong to it. A bold field inside a fenced code block does not count.	pass, fail
`open_items`	No line of the documents is an unchecked task, a list item that starts with `[ ]`.	pass, fail
`command`	Runs a command from `vars` in the working folder. It passes the same host guard and the same permission gate as a command the agent runs, and the gate sees it as `playbook_check`. Exit code 0 is pass.	pass, fail
`review`	A child agent on the model the check names reads the listed documents, cannot write, and answers with one outcome as the last line of its answer. A reviewer runs under the stricter privacy of the steps that produced the documents it reads.	the labels of the check
`human`	Asks you the question of the check and waits. An answer outside the labels counts as no answer.	the labels of the check

A decision that sits on a loop carries `max_rounds`. The number counts how often the decision may send the run back. At the limit it takes the outcome `exhausted`, which the playbook routes to an end. The engine's own step limit stays as a second guard.

Privacy and model choice

Every step has a privacy switch, and the confirmation shows it as private or cheap.

- A `private` step uses only the primary model of its choice, or the model picked for it in the sheet. Its fallbacks are never tried, and a primary on a cloud provider or an Ollama cloud model is refused before the run. If the model does not answer when the step starts, the run stops and asks whether to retry or stop. The question names the provider, the model, the address that was tried and the reason.
- A `cheap` step tries the primary model and then its fallbacks in order and takes the first that answers. The walk is recorded.
- If a chat step would switch to a cloud model after a private step ran earlier in the same run, the run asks first. The history of the chat would reach that provider, so the question names it and offers to continue or stop. It is asked once per run and provider.

Permission

A step may name a permission. The mode that applies is the stricter of the session's current mode and the step's, from strictest to widest: read only, ask, auto, extended. A playbook cannot widen what the session allows, and the step's limit ends with the step.

Watching and stopping

You follow a run beside the chat, in the Plan panel (chapter 6). The panel opens by itself when the run starts. It names the run, its task and how long it has run, draws a strip with one mark per node of the forward path, and has one card per step and decision in the order they ran: who works on it and on which model, the documents it reads and writes, the files its agents changed, how many shell commands ran, and while it runs the last line its agent wrote. A decision shows its round and outcome, and a loop that ran out of rounds reads “limit of 2 rounds reached”. The button “wide” puts the run in the chat's place, with the run's graph on top, drawn from the graph file the runner writes and asked again every second while the run lasts, and one entry in full below it; a press on a node shows that node's latest entry. A press on a card, or “In the chat” in the wide view, goes back to the chat at that step. A press on a document opens it in the Files panel, also one under a dot folder such as `.superpowers/` when this run wrote it (“What a run leaves behind” below). How the run ended comes from its stop reason alone: done, stopped or failed. The model's own plan from `update_plan` shows under the run as “Plan of the model”; the plan the runner writes, one line per node, is not shown there, because the run above draws the same nodes. The footer does not call a chat step unfinished because later steps are still open. Stop ends the run: the running step and the run close as stopped, and the graph file still ends cleanly.

The chat of the session follows the run too. It shows a line when the run starts, with the playbook and the task and the link “open the run”, which brings that run up in the Plan panel, a line when a step starts, with the model it runs on, and a line when the run ends, with the result and the stop reason. The prompt a chat step sends appears as a turn with the header “Playbook” and an accent rule, so it does not read as a message you wrote.

A run ends with one of these words, recorded as the `stopReason` and shown in plain language in the chat's end line.

Word	Meaning
`done`	The path reached one of its ends. The result names which one; in the spectro playbook it is `cancelled` when a loop ran out of rounds.
`aborted`	You pressed Stop.
`step_failed`	A step ended with an error or a child reported one.
`model_unavailable`	A model did not answer and you chose to stop.
`chat_switch_refused`	The chat could not switch to the model of a step.
`privacy_declined`	You declined the switch to a cloud model after a private step.
`unanswered`	A question of a `human` check was not answered.
`review_unlabelled`	A review named none of its outcomes.
`nod_refused`	A handover was not approved.
`recursion_limit`	The engine's step limit stopped the run.
`refused`	The run was not allowed to start, for example because nobody can be asked and the playbook needs an answer.

A window that is closed or a server that is stopped during a run leaves no end record. The Plan panel then says “no end recorded” for the run and for the step that was open.

What a run leaves behind

The session file holds the run as `run_step` lines (chapter 22): one when the run starts, with its forward path, one when each step or decision starts and one when it ends, and one when the run ends. They carry who worked on a step, the model, what it was given and what came out, cut at 4,000 characters, the documents it read and the documents it wrote, each with its path in the working folder. The Plan panel draws the run from these lines, so a stored session shows its runs after a restart. The documents a run wrote open through `GET /api/run-document` (chapter 26), dot folders included: only a path a `step_end` line of that run names, only inside the session's working folder, only while that folder is the one the run worked in, and never a secret such as a `.env` file, a key file or anything under `.git`, `.aws` or `.ssh`. Any other path opens the way the Files panel opens a file.

Two kinds of file live in `~/.spectro/playbook-runs/`, outside the sessions folder so that the session list does not read them. Both are removed when the session is deleted.

- `<session>.<run>.graph.jsonl` is the graph artifact of one run: the topology and one record each time a node starts or ends and each time an arrow is taken. The state graph view reads it.
- `<session>.playbook.jsonl` is the sidecar of the session, one line per fact, several runs one after the other. The session file itself is not changed, so the sidecar is found by the session id. Its record types are `playbook_start` (playbook, folder, hash, the session's permission mode, the chat's model before the run and the models picked in the sheet, if any), `step_start` (step, model choice, whether its model was picked, provider, model, address, the probe with its state and reason, the walk over fallbacks, the effective permission), `step_end` (the child, whether the step succeeded, how long it took, the error), `check` (the check, its kind, the outcome, the loop count, a short detail), `nod` (the answer and how long it took), `privacy` (the provider asked about and the answer) and `playbook_end` (the stop reason, the result and the number of steps). A workflow run writes into the same sidecar: `workflow_start` (the workflow, its file, its hash and how many characters you typed after its name), `workflow_step` (the step, its kind, the round, the children it started, whether it succeeded and how long it took) and `workflow_end` (the stop reason, the result and the number of steps). The sidecar answers which model served which step. Every string passes the same redaction as the other recordings, and the file stops growing at 16 MiB with a line that says so.

Running a workflow

A workflow is a short list of steps in one JSON file under `workflows/`, listed in the `contents` of `playbook.json`. The shipped playbook carries two. In `fanout-review.json` the chat prints the diff of the working folder, three reviewers read it at the same time for correctness, security and readability, and a summariser joins their findings into one review. The note you type after the workflow's name goes to each reviewer and to the summariser. In `fanout-research.json` three explorers look for the answer to your question in the code, the documents and the tests, and a summariser answers from what they found. A file has a `name`, a `description` and `steps`, and a step is one of three kinds:

- `prompt` is one chat turn. Its template is sent as a message of this session, and the answer of the turn is the step's answer.
- `fanout` starts one to four helpers, each with a role and a task, and optionally a `join` with one more task for the summariser. The role is explore or worker.
- `end` ends the run, with an optional result label.

A template may name `{{input}}` (the text you type after the workflow's name), `{{previous}}` (the answer of the step before), `{{folder}}` (the working folder) and, in a join task only, `{{results}}` (what the helpers returned). Any other name is refused when the file is read. A step can repeat until its answer contains the text in `repeat_until`, and then it must carry `max_rounds`, from 1 to 10; a file without it is refused with the line the step starts on. A run whose step used up its rounds without the text stops as exhausted. A whole run has two ceilings, counted when the file is read. It starts at most 20 helper runs, and every round of a fan-out counts its helpers and its summariser. It runs at most 20 step runs, and every round of a prompt or fan-out step counts. A file over either ceiling is refused, and the finding names the step where the count passes it. The format has a JSON schema, `workflow.schema.json`, and it also reads the scenario shape of the Lab, where a step is written as the one key that names its kind.

The Steps tab lists the workflows of the loaded playbook under “Workflows”, each with its name and the start of its hash, and starts none of them. A workflow starts in the chat of a session: `/playbook workflow` or `/run-workflow`, the workflow's name (its file name without `.json`) and the text for `{{input}}`, for example `/playbook workflow fanout-review check the auth module`. It runs at once in that session, from the session's playbook: the one picked in the header chip, else the one pinned to the working folder. An unknown name is refused with the names the playbook has. A workflow that starts a `worker` helper is listed and not run: a worker can write files, and a workflow starts without a confirmation sheet. The list marks such a file “Does not run: not a step list, or the workflow starts a worker helper, which can write files.” A run holds the session like a playbook run, so a message typed meanwhile is refused with the usual note. The run shows in the chat: a line when it starts, “Workflow fanout-review started on: check the auth module”, with the link “open the run”, and a line when it ends, with the stop reason in words. A loop whose rounds ran out before its condition held ends with `exhausted`, “the round limit was reached”. The Plan panel follows it like a playbook run, and its wide view draws the steps as a chain, because a workflow has no graph file. During a `fanout` step no model turn of the session is open: the run starts the helpers with `spawn_agents` in the background, waits for them and collects their results with `wait_agents`, then hands the joined results to one summariser helper. The summary is the last message of the run in the chat. While the helpers work, each status line they report appears in the chat, the session's status line in other sessions' `list_agents` shows the latest of them, and `list_agents` of this session lists the helpers. A helper can write to the run with `send_message` to `parent`, and its message appears in the chat. A helper cannot write to another helper: the summariser gets their results from the run. A message from another session that arrives during the fan-out waits for the next chat turn. A fan-out of three helpers and one summariser starts four helper runs on the provider. The run writes the sidecar records above and no file into the working folder; the helpers write only what their tasks name.

What a playbook file may not carry

A shared folder must not be able to route your material or widen your rights. A key named `apiKey`, `key`, `baseUrl`, `endpoint`, `address`, `hooks`, `mcpServers` or `autoApprove`, anywhere in `playbook.json` except inside the `contents` list, is refused with its path, for example `nodes[1].baseUrl`. The permission of a step can be `inherit`, `readonly`, `ask` or `auto`, and never `extended`. Node kinds for splitting and joining paths, a fallback chain on a step and a nested playbook are recognised and refused as not supported in version 1.

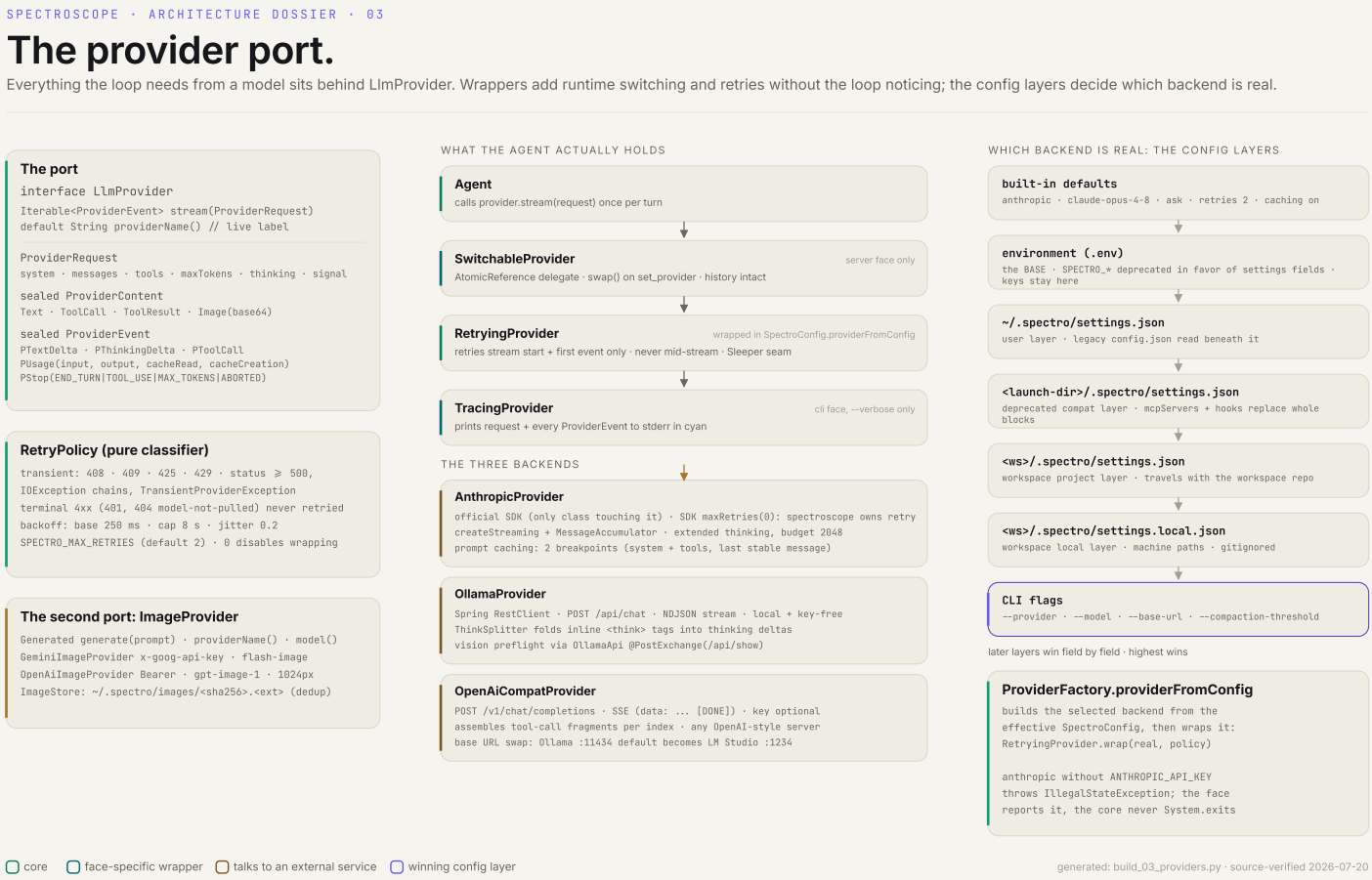
What version 1 does not do

- A run does not resume after a restart. A quit or a server stop ends it, and starting again begins at the first node.
- There is no headless entry. A run starts from the window, and a playbook with a `human` check or a nod is refused when nobody can be asked.
- A shared folder cannot start a run. Only the confirmation sheet does, after you read the commands, the contents and the hash.
- A command, hook or workflow of the folder that changes after the confirmation does not stop a run by itself. Every file under the skill folders, every template and every agent file a step names is part of the hash, and a change to one of them after the confirmation stops the start.
- Between two steps the chat is idle and its composer is enabled. A message typed then is refused with the usual note that a run is active.
- You cannot place a box by hand. The graph carries no coordinates, and the layout engine decides where each box goes.
- A workflow file that is not a step list is listed and not run. A workflow has no split and join of its own beyond the fan-out step, and a run starts only from the chat of a session.
- A playbook's hooks do not run in a session that picks it. The agent's file tools refuse to write under a registered playbook folder, but a shell command run with `run_command` is not fenced from it.
- Paths that split into parallel branches and join again are not part of the format yet.

CHAPTER 16

Providers & models

Everything the loop needs from a model sits behind one narrow port: `Iterable<ProviderEvent> stream(request)`. Swap the implementation and the loop never notices, which is why spectroscope can switch between a cloud API and a local model mid-session.



One port, many backends: the loop only ever sees stream(request). Anthropic, Ollama and the OpenAI-compatible family each implement it; the retry decorator wraps whichever one you picked.

The backends

Nine provider ids resolve to four implementations. Six of them (`openai`, `lmstudio`, `llamacpp`, `openrouter`, `gemini`, `spectro-local`) share one wire protocol and differ in which host they talk to, whether a key is attached, and which reasoning fields that host understands. `copilot` is the fourth: it reaches GitHub Copilot through the Copilot runtime, which spectroscope starts as a separate process (macOS only).

	ANTHROPIC	OLLAMA	OPENAI-COMPATIBLE	COPILOT
provider id	`anthropic`	`ollama`	`openai` · `lmstudio` · `llamacpp` · `openrouter` · `gemini` · `spectro-local`	`copilot`
implementation	`AnthropicProvider`	`OllamaProvider`	`OpenAiCompatProvider`	`CopilotProvider`
transport	official SDK, SSE	NDJSON over `POST /api/chat`	SSE over `POST /v1/chat/completions`	GitHub's Copilot SDK, JSON-RPC to the Copilot runtime
endpoint	`api.anthropic.com` (fixed)	`localhost:11434` (preset)	openai → `api.openai.com`; lmstudio → `localhost:1234`; llamacpp → `localhost:8080`; openrouter → `openrouter.ai/api`; gemini → the Google gateway; spectro-local → a `llama-server` spectroscope starts itself. The three local ids read their own `…BaseUrl` key first (chapter 18); for the rest a set `baseUrl` overrides the preset	none to set: the Copilot CLI found on this Mac (`brew install --cask copilot-cli`), or `COPILOT_CLI_PATH`
key	`ANTHROPIC_API_KEY`	none	`OPENAI_API_KEY` (openai) · `OPENROUTER_API_KEY` (openrouter) · `GEMINI_API_KEY` (gemini) · none (lmstudio, llamacpp, spectro-local)	none: a GitHub sign-in from the model menu
default model	`claude-opus-4-8`	`qwen3`	openai, lmstudio and llamacpp fall back to the placeholder `local-model`, which a local server ignores; spectro-local takes the bundled catalogue's default; gemini and openrouter have none, so name one	`auto`, Copilot's own choice; the menu lists the models the account can use
thinking	extended thinking (2048-token budget)	native `message.thinking` + inline `<think>` tag splitting	`reasoning_content` deltas split out (LM Studio, vLLM). Turning it OFF differs per id: openrouter takes `reasoning.enabled`, spectro-local takes `chat_template_kwargs.enable_thinking`, llamacpp takes both that and `reasoning_effort: none`, LM Studio takes nothing at all	the effort levels the model list names for the chosen model

quirks	usage + parsed tool inputs exist only after the stream ends	mints its own call ids; vision fail-fast for non-vision models	assembles fragmented tool-call deltas per index	the runtime keeps the conversation; built-in tools are off, so the harness runs every tool. Each call reports its cost in GitHub AI credits

One wire, one endpoint per id, no guessing. spectroscope used to redirect a keyless `openai` to a local LM Studio endpoint on your behalf. That guess is gone: the provider id names the endpoint, the key only authenticates. `openai` now always means the cloud. For a local OpenAI-compatible server, pick the id that matches it: `lmstudio` for LM Studio, `llamacpp` for your own `llama-server`, and for anything else (vLLM and friends) `lmstudio` with an explicit `baseUrl`. Same wire either way — but not the same questions: a `llamacpp` endpoint is asked `GET /props` for the window the loaded model really has, where `lmstudio` is asked for a model listing a llama-server does not serve. Pointing `lmstudio` at one works on the wire and loses that.

First run — choosing a backend

Out of the box the provider is `anthropic` with `claude-opus-4-8`, which needs a key. On its first boot every face (`spectro run`, the web chat, `spectro node`, `spectro doctor`, the server) checks whether a `~/.spectro/settings.json` exists yet; when none does, it writes one from your current environment base (whatever you exported, or put in `./.env`), so day one your settings file mirrors exactly what you already had and nothing changes behind your back. With nothing set at all you land on the Anthropic default, and a run without a key stops with a readable “no key” message instead of a mystery failure.

No Anthropic key on hand? Every local backend is keyless — and the first one needs no install either:

- **built-in** (`spectro-local`) — pick a model in the app, it downloads once, done. The first-run sheet leads here, and chapter 16b is all about it.
- **ollama** — install it, pull a model, and it serves `localhost:11434`. Set `provider=ollama`; no key.
- **lmstudio** — run LM Studio in server mode with a model loaded, and it serves `localhost:1234`. Set `provider=lmstudio`; no key.
- **llamacpp** — start your own `llama-server` and it serves `localhost:8080`. Set `provider=llamacpp`; no key. It carries the one model it was started with, and it gets asked what window that model is really loaded at.

The cloud backends that speak the OpenAI wire do need a key: `openai` (`OPENAI_API_KEY`), `openrouter` (`OPENROUTER_API_KEY`) and `gemini` (`GEMINI_API_KEY`). `copilot` takes no key: it signs in with a GitHub account that has a Copilot subscription and needs the Copilot CLI on a Mac (`brew install --cask copilot-cli`). Pick it in the model menu and open its sign-in sheet; what it costs is billed to that account in GitHub AI credits, and the context ring shows the session's total. Set the provider in `~/.spectro/settings.json`, through `SPECTRO_PROVIDER`, with a `--provider` flag, or from the model chip under the message box. The CLI tour is the guided path: `spectro tour` → Settings `[s]` picks a provider, takes the key hidden, and can save it to `./.env` for you.

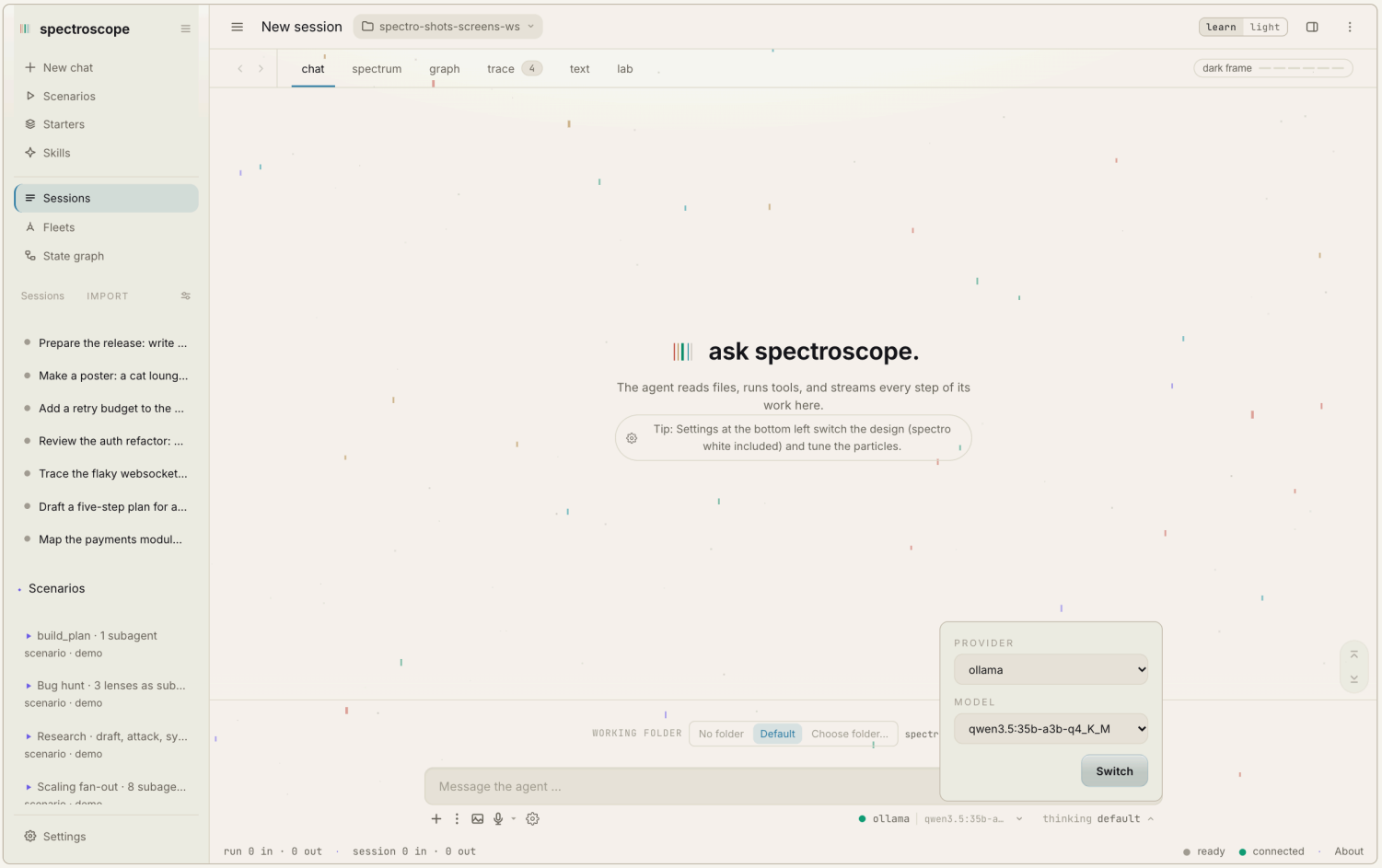
Putting a key in the .env

The simplest home for a secret is a `./.env` file in the repo root: it is gitignored, and spectroscope reads it as the environment base at startup. One `KEY=value` per line, no quotes:

```
# ./.env — gitignored; the environment base layer
ANTHROPIC_API_KEY=sk-ant-...        # anthropic
OPENAI_API_KEY=sk-...               # openai (the cloud); also the image backend's key
OPENROUTER_API_KEY=sk-or-...        # openrouter
GEMINI_API_KEY=...                  # gemini (the LLM and the image backend share it)
# the local ids need no key: ollama, lmstudio, llamacpp, spectro-local
```

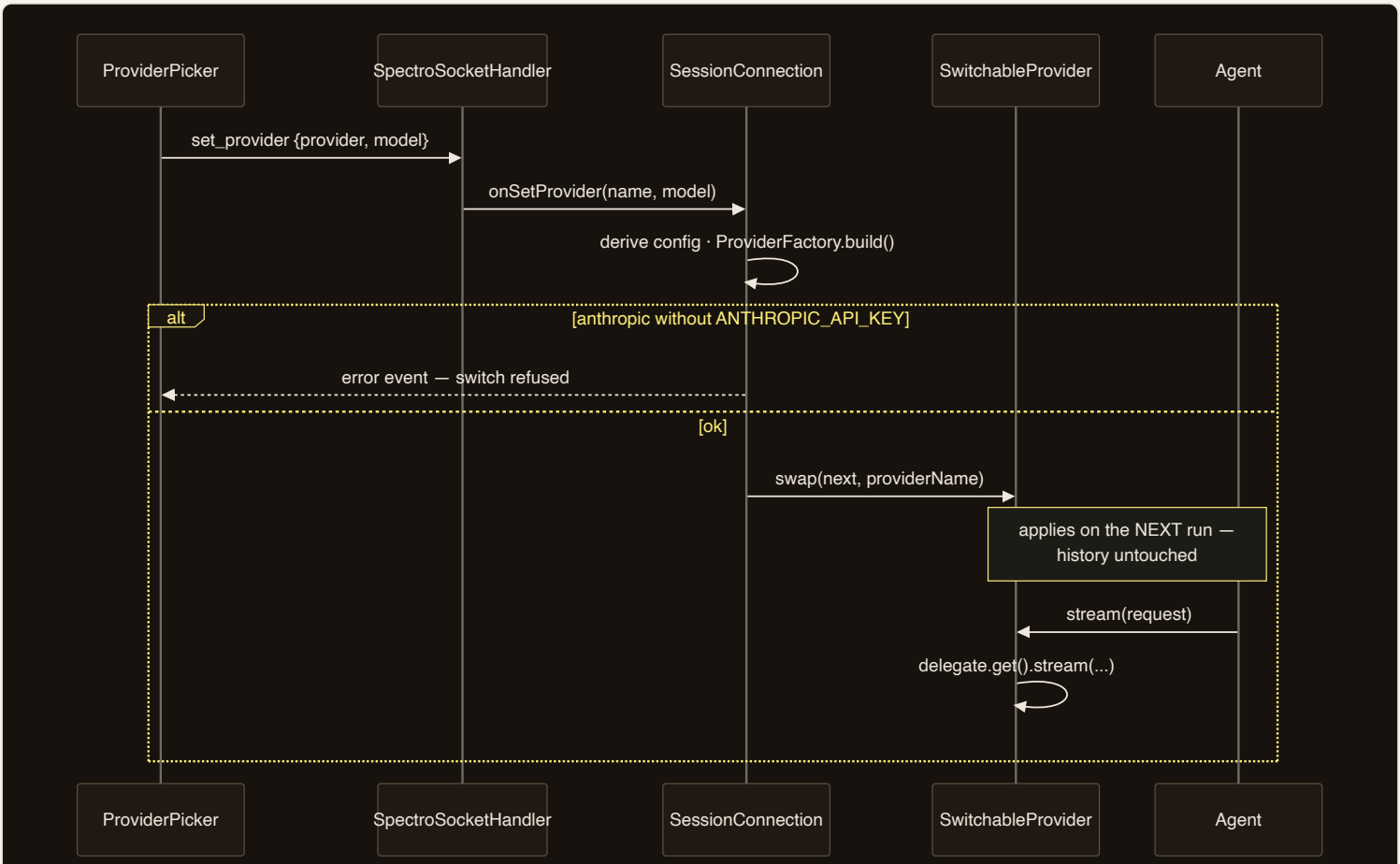
The `.env` is only the *base* layer: a value in `~/.spectro/settings.json` (or a flag) wins over it, and `spectro doctor` names any `SPECTRO_*` variable that is set *and* shadowed, so a value that isn't taking effect never fools you. Keys are secrets: they never enter a session file, the event stream, or the first-run settings seed. In the tour, `[w]` “save key to .env” writes the file for you (gitignored, mode 600). The image-generation keys (`GEMINI_API_KEY`, `OPENAI_API_KEY`) live in the same file — chapter 17.

Switching mid-session



The model picker under the message box, opened upward: provider select plus a real model dropdown. Ollama's list is live from the server, cloud lists are curated, and a custom model is always typeable.

The model chip under the message box opens the picker, and it offers every provider id in the table at the top of this chapter — the cloud backends, the OpenAI-compatible endpoints and the built-in model alike. Nothing has to be chosen in settings to be reachable from there; `SPECTRO_PROVIDER`, `~/.spectro/settings.json` and the `--provider` flag set the same thing before a session starts, which is the one difference. The model dropdown is honest: for Ollama it lists what is actually installed (live from `/api/tags`), for Anthropic what your key can use (its Models API), for an OpenAI-compatible endpoint whatever `/v1/models` returns, each with a curated fallback and “Custom model …” always available; an empty list degrades to free text. Apply sends `set_provider`, and because the agent is built once per connection (it carries your history), the switch swaps a delegate *inside* the provider port instead of rebuilding anything. It takes effect on the next prompt, history intact. Selecting Anthropic without a key is refused with a readable error, and the map in the Lab re-places the LLM node inside or beyond the network boundary to match.



set_provider: validate, build the new backend (key check included), swap the delegate. The next run_start reports the new provider — no new event type needed.

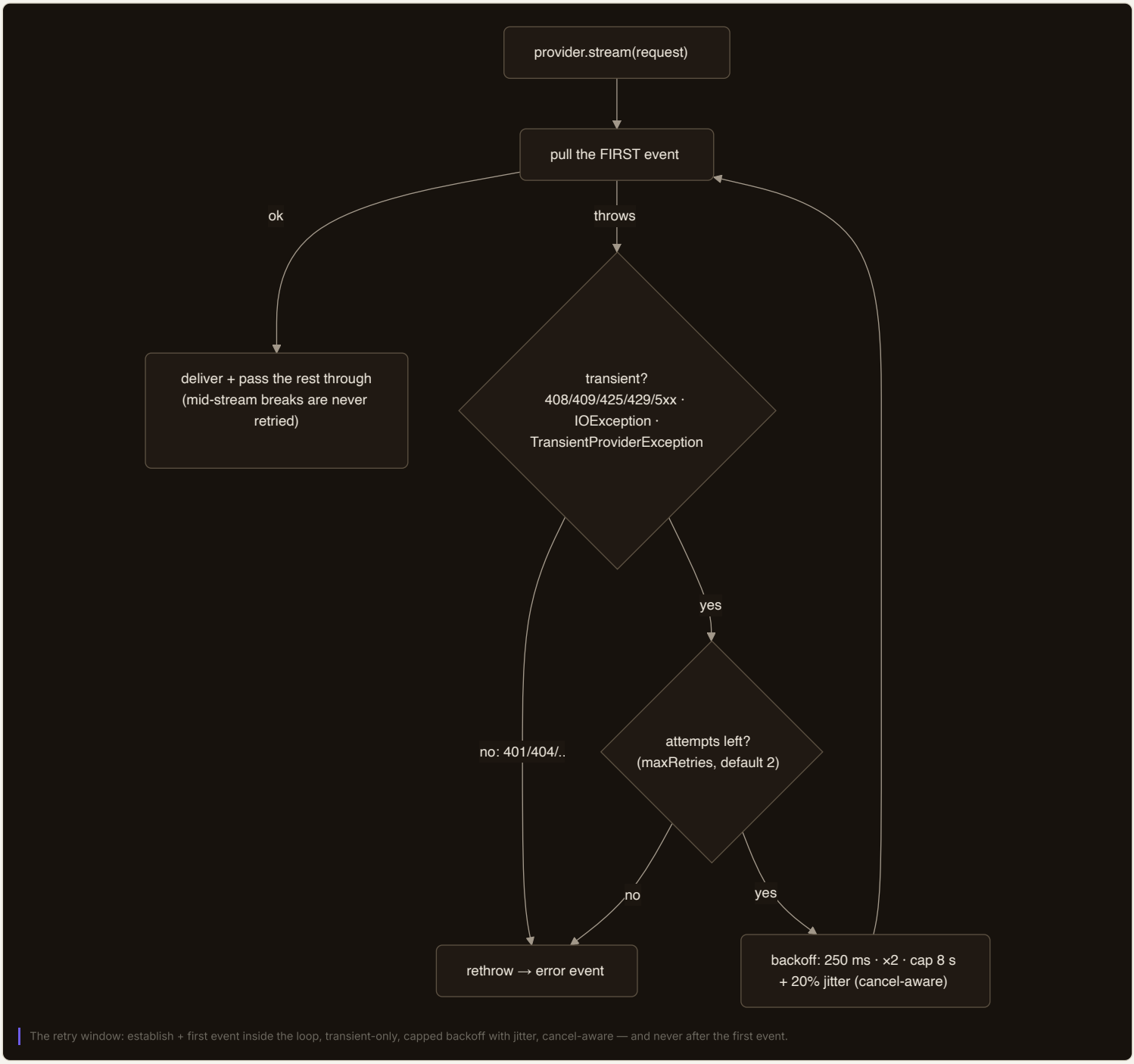
The provider overview

The settings page lists every provider the config knows in one table. A row shows the state in words (needs a key, model not downloaded, configured, answers, not answering), whether a key is present, the address for a local server, the number of models and whether that list came live from the provider or from the curated list, and the age of the last check. *Check* tests one provider and *Check all* tests every one. A check asks the provider for its model list, never for a completion, and it returns within five seconds: a server that accepts the connection and never answers reads “not answering” with the reason `timeout` and its address. Local servers are checked when the page opens, and a provider is checked once after you save its key. Nothing checks on a timer, and the built-in model is never started for a check.

The model picker under the message box reads the same answer. All nine providers stay in its list, but a provider that needs a key or did not answer its last check is greyed out with the reason in the tooltip, and *Switch* stays disabled while a greyed provider is selected. The provider the chat is using right now remains selectable. An OpenAI-compatible server at a private address counts as local without a key, in settings, in the picker and in `spectro doctor`. Scripts get the same rows from `GET /api/providers`, which sends nothing to any provider, and `POST /api/providers/check?provider=<id|all|local>`, which answers local callers only.

Retry — a loop that doesn't tip over

Transient failures (HTTP 408/409/425/429, any 5xx, dropped connections) are retried with exponential backoff and jitter — but **only before the first event** of a response. A stream that breaks mid-answer is never retried (that would duplicate already-emitted text into the transcript); it surfaces as an honest error. Terminal errors — 404 model-not-pulled, 401 bad key — never retry. spectroscope owns retry exclusively: the Anthropic SDK's own retry is disabled so nothing double-retries. Knobs: `maxRetries` (default 2, `SPECTRO_MAX_RETRIES`; 0 disables).



The retry window: establish + first event inside the loop, transient-only, capped backoff with jitter, cancel-aware — and never after the first event.

Prompt caching

On Anthropic, with `promptCaching` on (default), spectroscope places two `cache_control` breakpoints per request: one covering the stable prefix (system prompt + tool list), one on the last *stable* message, the one just before the current turn. Costs and latency drop on every consecutive turn. The `usage` event reports the input tokens no cache served, and the cache-read and cache-creation counts ride beside them. The compaction trigger adds them back, because cached tokens still occupy the context window (chapter 28). The OTLP export sends the uncached input as `gen_ai.usage.input_tokens` and leaves the cache counts out.

A llama.cpp server (`llamacpp`, and the bundled `spectro-local`) gets the request field `cache_prompt`. With it on, the server keeps the part of the chat it has already read and only reads the new turn. Measured on 2026-10-09 with Qwen3-1.7B on llama-server b10090, twelve turns of a chat whose prompt grows from about 1,850 to 2,070 tokens: about 36 ms to the first token on turns 2 to 12 with the cache, about 1,000 ms on every turn without it. A stock llama-server caches even without the field, so the field changes the outcome when a server was started with `--no-cache-prompt`. With `promptCaching` off the `llamacpp` provider leaves the field out and the server's own default decides, so a stock llama-server keeps its cache. The bundled `spectro-local` runtime always sends `true`; the setting does not reach it. When the response reports cached prompt tokens (`usage.prompt_tokens_details.cached_tokens`, or llama.cpp's `timings.cache_n`), they arrive as the cache read on the `usage` event, the same way as Anthropic's: the cached share is taken out of the input count and the context ring adds it back. The OTLP export treats them as it treats Anthropic's. Ollama reuses its cache by itself (measured with 0.32.1) and takes no field; LM Studio documents no such field, so it gets none either.

The wrapper chain

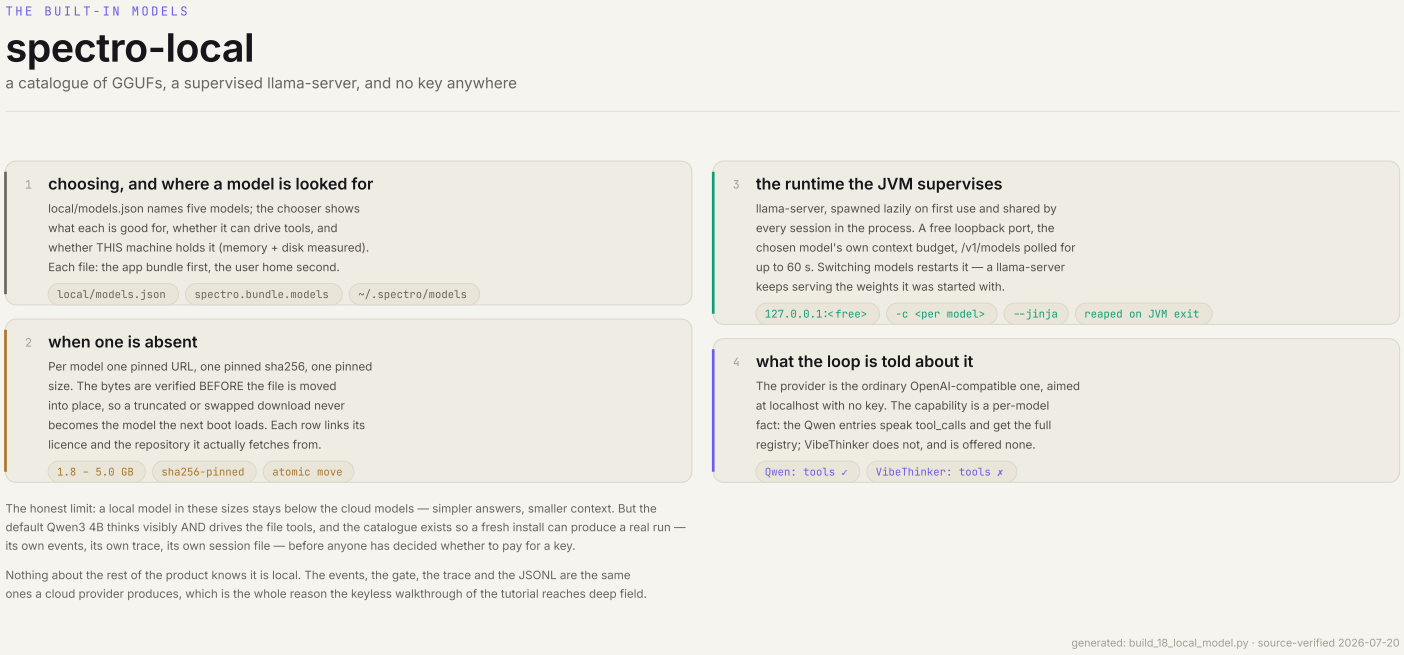
```
Agent loop
  └ TracingProvider      (CLI --verbose only: the wire on stderr)
      └ SwitchableProvider (web face only: the mid-session swap point)
          └ RetryingProvider (all faces, unless maxRetries=0)
              └ AnthropicProvider | OllamaProvider | OpenAiCompatProvider
```

All wrapping happens at one chokepoint (`providerFromConfig()`), so the CLI, the server, headless runs and the scheduler behave identically. The one OpenAiCompatProvider serves all three OpenAI-compatible ids — only its endpoint and key differ.

CHAPTER 16B

The built-in models

Every backend in the previous chapter is something you install or pay for. `spectro-local` is the one that is already here: a small catalogue of open models, driven by a `llama-server` this process starts and supervises. It exists so a fresh install can produce a real run — including real tool calls — before anyone has decided whether to pay for a key.



The keyless path end to end: where the file is found, how it arrives when it is missing, the runtime the JVM supervises, and the capability fact the loop is told.

Choosing a model

The picker entry reads **built-in** and needs no key at all. Applying it opens a chooser rather than switching blindly, because which model runs is a real decision now. Five models ship in the catalogue, smallest first:

The chooser: what each model is good for, whether it can drive tools, how big the download is — and whether this machine can hold it.

model	download	tools	thinking	good for
Qwen3 1.7B	1.8 GB	yes	yes	quick answers on light machines
Qwen3 4B *(default)*	2.5 GB	yes	yes	everyday chat and small agent tasks
Qwen2.5 Coder 7B	4.7 GB	yes	no	coding and tool-driven agent work
Qwen3 8B	5.0 GB	yes	yes	the deepest answers, on 16 GB machines
VibeThinker 3B	1.9 GB	no	yes	watching a model think, without tools

Under each row the dialog answers the question a spec sheet never does: *can this machine hold it?* The server reads total memory and the free space on the models volume, and the row says **fits this machine**, **fits, but it will be tight**, or names what is missing. A model that cannot fit cannot be chosen — the dialog disables the button rather than letting a five-gigabyte download start toward a full disk. (The REST endpoint underneath does not enforce it; the check is advice to the operator, not a lock on the API.) On a machine whose numbers the JVM cannot read, the check says so and steps aside instead of inventing a refusal.

One more thing the dialog checks: whether a `llama-server` exists for this install at all. The desktop run kit bundles one, so there is nothing to install; the bare server jar and the CLI do not, and there the chooser says plainly that `brew install llama.cpp` is still needed — better than a spawn that fails after five gigabytes have already been fetched.

Where a model comes from

Each catalogue entry is looked for in two places, in order: the app bundle (a desktop build that ships a model sets `spectro.bundle.models`) and then `~/.spectro/models/`. Missing means the chooser offers the download; nothing fails.

Every download is pinned: one URL, one sha256, one expected size. The hash is checked *before* the file is moved into place, so a download that was cut short or came back as something else is discarded instead of becoming the model your next boot loads. Four of the five come from Qwen's own repository; VibeThinker's GGUF is a community requantization of WeiboAI's model rather than WeiboAI's own file, which is why its row carries a licence note. Each row links both the licence and the source it downloads from. On the server side two different models can download at the same time without disturbing each other; the dialog itself runs one at a time.

The runtime underneath

Nothing runs until the built-in provider is actually used. On first use the server picks a free loopback port, execs `llama-server` against the chosen GGUF with that model's own context budget, and then polls `/v1/models` until it answers. The budget is a generous sixty seconds, because a cold multi-gigabyte load genuinely takes that long on a laptop.

```
llama-server -m ~/.spectro/models/Qwen3-1.7B-Q8_0.gguf \
             --host 127.0.0.1 --port <free> -c 8192 --jinja
```

One runtime serves every session in the process, and the handle hangs off a JVM shutdown hook, so quitting spectroscope takes the child with it. Switching to a *different* catalogue model shuts the old subprocess down and starts a fresh one — a llama-server keeps serving the weights it was started with, so a live runtime is only ever reused for the same model. From there the ordinary OpenAI-compatible provider does the talking — the same class that serves LM Studio and OpenRouter, aimed at localhost with no key.

The desktop app carries its own signed `llama-server` inside the bundle (llama.cpp's official macOS build, eleven Mach-O files that reference each other only through `@loader_path`, so nothing resolves out of a Homebrew prefix that a user's machine may not have). Running the bare server jar instead, you bring your own — `brew install llama.cpp` puts one on the PATH.

Honest capability flags, per model

This is the part worth knowing before you judge an answer. Whether a model speaks the OpenAI `tool_calls` protocol is a fact about the model, not about the provider, and the catalogue records it per entry. The four Qwen models speak it natively — the agent's file tools, shell and subagents all work. VibeThinker does not: that came out of testing it rather than reading its card — asked to use a tool, it writes the call out as prose, and it can keep going in the think channel for a while afterwards.

So the loop is told the truth per model. A tool-capable entry is offered the full registry; VibeThinker is marked `reasoning` yes, `nativeTools` no, and tools are simply not advertised to it. The alternative — offering tools that come back as text — would have produced a stream of confusing half-failures and blamed them on the user's prompt. A stale selection resolves to the catalogue default the same way the runtime resolves it, so what runs and what is advertised never disagree.

> WHAT TO EXPECT, HONESTLY
>
> A local model in these sizes stays below the cloud models: simpler answers, a smaller context. But the default Qwen3 4B genuinely drives the file tools while showing its thinking, and the first-use sheet tells you the limits of whichever model you picked. When the work gets real, the picker is one click away — and the chapter before this one is about everything else in it.

Everything downstream is unchanged by the choice. The events, the permission gate, the trace, the spectrum and the JSONL on disk are the same ones a cloud provider produces. That is exactly why the tutorial in chapter 6c runs from the dark frame to deep field without a paid key.

CHAPTER 17

Seeing, hearing, speaking, painting

Four bonus-stage senses, all built on the same additive principles: binary data never enters the event stream, external engines sit behind injectable seams, and everything degrades readably when a piece is missing.

Vision — image input

- **Web:** attach images in the composer (picker or drag-and-drop) — they are downscaled client-side, stored content-addressed in the session's blob folder, and restored *before* the prompt text in the provider history (the position matters to the model).
- **CLI:** `spectro run -p "…" --image photo.png` (repeatable).
- **Local models fail fast:** Ollama models without the vision capability are refused with an actionable message ("use qwen3-vl or llava") before any tokens are wasted; cloud models handle images natively.
- Events carry only references (`blobPath` , `sha256`) — session files stay small and `jq` -able; a deleted blob degrades to a placeholder, never a crash.

Voice input — two routes to the same text

A recording becomes text on one of two routes, chosen the way the chat providers are. The **hosted** route posts the clip to OpenAI's transcription endpoint (`gpt-transcribe` at `api.openai.com/v1/audio/transcriptions` , keyed by `OPENAI_API_KEY` — the same key image generation already uses) and needs nothing installed, which is what makes dictation work in a downloaded app. The **local** route runs whisper.cpp as a child process and sends nothing anywhere, which is why it stays. Neither is ranked above the other: `sttProvider` takes `auto` (the default — hosted when a key is there, local otherwise), `local` , or `openai` , and an explicit choice wins even when it cannot run right now, because naming what is missing is more use than a silent reroute — and in the local direction a reroute would mean somebody's voice leaving a machine whose owner asked for the offline path.

The local route's one-time setup is `bash scripts/setup-stt.sh` : whisper.cpp through Homebrew on macOS (`brew install whisper-cpp` ; on Linux the script names the build and stops), then the ggml-small model, ~460 MB, against a **pinned SHA-256** — a mismatch deletes the file and aborts. ffmpeg left this path: the browser converts its own recording now, so the app needs exactly one binary and whisper.cpp is MIT. The CLI's `/voice` still records with ffmpeg, a job the browser cannot do for it, which is why the script reports ffmpeg as absent and never requires it. Recording for whisper is 16 kHz mono WAV, its one input format. `sttLanguage` rides both routes — `auto` by default, so German dictation is not forced through English, or `de` / `en` to pin it.

Settings → Speech to text shows the whole thing without a terminal: the route, the dictation language, the model file with a download button when it is absent, every binary with the path that proves it was found, and one true install line when it is not. There is a button only for the half an app can honestly do — the model is a checksummed download, the binaries belong to a package manager this app must not drive.

- **REPL `/voice` :** record until Enter, transcribe, then an *editable confirmation* — Enter sends, typing replaces, blank discards. An STT error never reaches the agent unreviewed. The turn is audited by a `voice_input` event that never enters the provider history.
- **Web mic button:** a toggle, not a held key — one press arms and records (the tooltip flips to "Stop recording", a level meter moves with your voice, a "Recording 0:04" line counts up), a second press commits. The clip goes to `POST /api/transcribe` and the transcript lands in the composer *draft*, never at the agent. Each route reports its own obstacle with a 503: "Speech-to-text is not installed — run bash scripts/setup-stt.sh." locally, a missing `OPENAI_API_KEY` hosted. Telling somebody to run a shell script when what is missing is a key is the failure this split was built to end.
- **Before any of it:** the first reach for the microphone opens a sheet titled "Speak instead of typing" *instead* of recording, so on the hosted route nobody's audio leaves the machine before a sentence has said that it will. A caret beside the glyph opens "Choose a microphone", which lists "System default" and the named inputs — and before permission has been granted once, a sentence explaining why the browser is withholding the names rather than five blank rows.
- **Seven named failures, seven sentences:** denied, no device, device busy, STT missing, request failed, convert failed, unknown. Only "no microphone found" and "speech to text is not set up on this server" take the button away — a denial is an event, so somebody who then allows it in the site settings can press again without reloading.

Live transcription — text while you are still speaking

On the hosted route the text does not have to wait for the end of the recording. Tick "Write along live" ("Live mitschreiben") in *Settings → Speech to text* and the next press opens a websocket to your own server at `/ws/stt` , which bridges to OpenAI's realtime transcription session (`gpt-live-transcribe`) and holds the key: the browser never talks to the provider directly. Audio goes up in pieces as it is spoken, at 24 000 Hz, because the realtime session refuses 16 000 and whisper reads nothing else — the rate is therefore a function of the route, decided before the audio graph is built rather than discovered by being refused after somebody has already spoken. Anything spoken before the session is configured is held and flushed in order, so the cost is a few hundred milliseconds once instead of the first word of every take.

What arrives is two layers of text in one box. The textarea keeps the draft, crisp, with its caret and its selection; a ghost layer under it repeats that draft in transparent ink and continues it with the provisional words in faint ink, so the faded part begins exactly where the typed text ends. Those words are unselectable, are not in the draft and cannot be sent — only a `final` frame is committed. A guess is never one Enter away from being sent as though somebody had typed it, which is the entire design.

The switch is **off until you turn it on**, and that is a decision rather than caution: a live session is metered by the minute where a clip is billed per clip. It is also never hidden and never quietly honoured. It carries a sentence in every one of its four states, greyed included, because a grey switch with no reason is a vanished button with extra steps: the local route "only writes once the recording is complete — it reads a finished file", and a hosted route with no key says so and points at the providers page. Wanting live text is not consent to send the audio of somebody who chose the offline path off their machine, so no branch anywhere upgrades one route to the other to satisfy a setting. The server refuses the socket by those same two names, plus a third when the provider itself cannot be reached, and a refused session records nothing at all — no model was called and no audio left. Both paths stop at 300 seconds, and on the live one that ceiling is also the only thing that ends a metered session somebody walked away from mid-sentence.

WHAT THE RECORD SAYS, AND WHAT IT DOES NOT

One live socket is one exchange on the llm-wire record: the handshake is the request, every frame the provider sends is a response line kept verbatim with the provider's own type, so a reader can tell a partial from the answer without this code flattening one into the other. The asymmetry is named rather than discovered — the spoken bytes are *not* on the request line, because a websocket has no single request body and the audio flows after it. The record is complete about what the model said and incomplete about what it heard; the batch route, which posts one body, carries the audio itself.

Voice output — spoken answers

Setup: `bash scripts/setup-tts.sh` (piper + the `en_US-lessac-medium` voice). Enable per run with `--speak`, at runtime with `/speak on|off`, or permanently via `"tts": { "enabled": true }` in `~/.spectro/settings.json` (the legacy `config.json` name is still read beneath it for one release — chapter 20). The renderer buffers streamed text to sentence boundaries and speaks *while the rest still streams* (synthesis runs up to three sentences ahead of strictly-ordered playback). Code blocks are never read aloud — you hear “Code block skipped.” Abort stops mid-sentence with no ghost audio. Ollama + piper is a fully offline agent that talks.

Image generation

The `generate_image` tool sits behind its own provider port with two backends — Gemini (`gemini-2.5-flash-image` , `GEMINI_API_KEY`) and OpenAI (`gpt-image-1` , `OPENAI_API_KEY`) — selected by `imageProvider` / `SPECTRO_IMAGE_PROVIDER` or live from the dropdown in the dock's Images panel. Results land content-addressed under `~/.spectro/images/` (same bytes = same file, shared across sessions); the additive `image_generated` event feeds the Images panel; the model itself only sees a short confirmation string. Gated, because it is a paid cloud call. A missing key returns a readable `ERROR:` the model can relay — never a crash.

CHAPTER 18

MCP — external tool servers

spectroscope is an MCP client: plug in any Model-Context-Protocol server and its tools become first-class spectroscope tools — gated, traced, replayable. No new event type was needed: MCP calls are ordinary `tool_call` / `tool_result` lines, so graph, trace and archive render them for free.

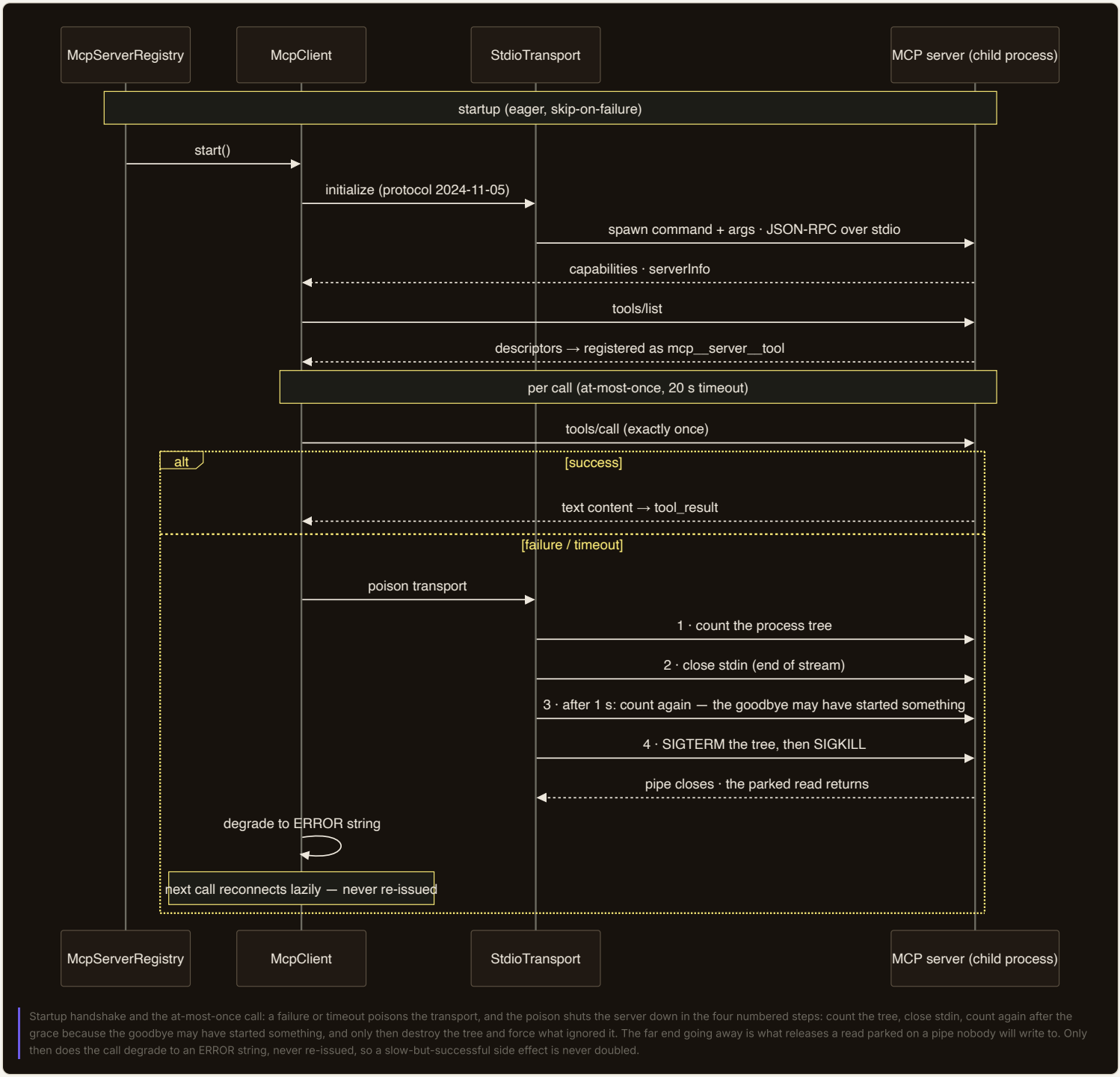
Configuration

The `mcpServers` block has the same shape Claude Desktop and Claude Code use — a config written for those drops straight in:

```
{ "mcpServers": {
    "notes": {                                  // stdio: command (+ args, env)
      "command": "/path/to/spectro-mcp-notes/bin/spectro-mcp-notes",
      "args": ["/path/to/notes-directory"]
    },
    "search": { "url": "http://localhost:9000/mcp", "type": "sse" }   // or HTTP/SSE
} }
```

Servers connect eagerly at startup. One that fails to start is logged and **skipped**, and startup carries on with the rest. How long that costs you depends on how it is broken: a command that does not exist fails in milliseconds, while a command that *does* start and then goes silent waits out the **20-second** handshake budget and then the shutdown that follows it: end of stream, a signal, and force, capped at **5 more seconds**. So **25 seconds** is the most a silent server can cost, and that is the number to hold us to. Measured here, an ordinary silent one is written off in 21.4 s, and one that ignores every step of the shutdown in 23.5 s. Either way the entry is skipped, the process tree it started is destroyed, and the line you get names the server and says what went wrong:
`mcp: notes UNREACHABLE at /usr/bin/notes-server — no answer to 'initialize' within 20000 ms`. Each advertised tool registers as `mcp__<server>__<tool>`, always permission-gated: its inputs are model output and its effects are external. `/mcp` lists servers and tools in the REPL and `spectro doctor` probes reachability; the System-context panel in the web face names the servers *you configured*, which is a different question. It is read from the settings file, so a server that never answered is listed there too.

Call semantics: at-most-once



Startup handshake and the at-most-once call: a failure or timeout poisons the transport, and the poison shuts the server down in the four numbered steps: count the tree, close stdin, count again after the grace because the goodbye may have started something, and only then destroy the tree and force what ignored it. The far end going away is what releases a read parked on a pipe nobody will write to. Only then does the call degrade to an ERROR string, never re-issued, so a slow-but-successful side effect is never doubled.

Every call is issued *exactly once* with a 20-second budget. If the transport died between calls, it is re-established once, lazily. But a call that fails or times out on an established transport is **not retried** — a slow-but-successful `add_note` must not be doubled. Instead the transport is poisoned (the next call reconnects) and the model receives a readable `ERROR: MCP tool '…' on server '…' failed: …`. A dead or slow server degrades; it never crashes the harness.

Every close runs the same steps, and their order is the point. First spectroscope asks the server's process tree who is in it, while there is still someone to ask. Then it **closes the server's stdin**. That end of stream is the ordinary way an MCP server is told to stop, and a server that flushes a cache or says goodbye upstream does it now. Then it waits a second — and asks the tree again, because a server given that second may spend it starting something, and a list drawn up before the goodbye cannot name what the goodbye caused. Only then does it **destroy the server process** and force whatever ignored that.

The last two steps are also what unparks a stuck read, which is why they are not optional on a timeout. A read waiting on a child that has gone silent cannot be cancelled: it ignores an interrupt, and it holds the stream's own lock, so closing the stream to abandon the read only waits on the read. When the far end dies its pipe closes and **the read returns**. Earlier versions closed the stream first, so a server that spawned and then said nothing parked `spectro doctor`, the REPL and any web session for good, and left the child running after the process that started it was gone. Against a server that never speaks, the probe now reports it unreachable inside the budget above, the rest of the output prints, and no orphan survives.

What gets destroyed is the **whole tree**, not only the process spectroscope started. That matters more often than it sounds, because most published servers are configured behind a launcher such as `npx` or a shell wrapper, and then the thing that actually speaks JSON-RPC is a grandchild. Kill the launcher on its own and the operating system hands the server to init, where it keeps running for as long as the machine is up, one leftover per failed connection. This is also why the first count comes before the goodbye: a launcher that exits when its stdin ends would otherwise be gone before anyone asked it, taking the only list of its children with it. The second count is the mirror of that, and it is why there are two: the grace exists so a server can do its cleanup, cleanup work starts processes, and a count taken a second earlier cannot name them. What stays out of reach is what nobody is left to be asked about — anything started after that last count, and anything at all started by a launcher that had *already* exited, because a process the operating system has reaped names no children.

The example server: spectro-mcp-notes

The repository ships a complete, runnable MCP server as its own module — a stdio JSON-RPC 2.0 program whose only dependency is Jackson ("no heavy MCP SDK, no Lucene"). Build it with `./spectro-app mcp-notes`. It serves two tools over a directory of plain text files (default `~/.spectro/notes`, six seed notes included):

- `search_notes` — hand-rolled ranked full-text search (term frequency + substring bonus) returning snippets with their source files;
- `add_note` — appends a note as a new file, with collision-safe naming (two identical texts land in two files, never an overwrite).

Try (with the server configured): "Search my notes for 'permissions' and summarize what you find." — the call passes the permission gate like any other tool, and the Lab's map lights the whole MCP chain: client → network stack → boundary → server.

CHAPTER 19

Scheduling

The scheduler turns spectroscope into an unattended worker: jobs in a JSON file, 5-field cron expressions, honest state tracking and desktop notifications.

```
# ~/.spectro/jobs.json — an array of jobs, validated loudly on load
[ { "id": "morning-report",
    "cron": "0 8 * * *",
    "prompt": "Summarize yesterday's session files under ~/.spectro/sessions.",
    "cwd": "/Users/you/projects/demo",
    "permissions": "readonly" } ]        // readonly (default) | auto | extended
```

COMMAND	BEHAVIOUR
`spectroscope cron`	the foreground daemon — arms every job, reschedules after each firing, Ctrl+C stops
`spectroscope cron list`	each job with its expression, policy and next run time (explicit zone)
`spectroscope cron status`	list + last outcome per job: time, ok/failed/skipped, stop reason, session id, result preview
`spectroscope cron --once <id>`	run one job now; exit 0 only on `ok`

- **Overlap guard:** a job still running when its next slot fires records a `skipped (overlap)` state instead of stacking.
- **Headless policy:** jobs default to `readonly`; every run writes a normal session file (full audit) and its outcome into `~/.spectro/jobs-state.json`.
- **Notifications:** macOS notification per finished job (terminal bell elsewhere); the desktop shell (chapter 10) adds native click-to-open notifications. `"desktopNotifications": "off"` in `~/.spectro/settings.json` turns the first kind off and keeps the line in the job's log (chapter 30).
- **Container-free:** cron-utils computes the times, a single ScheduledExecutorService thread fires them — no Spring scheduling in the core.

IV

PART IV

The file system

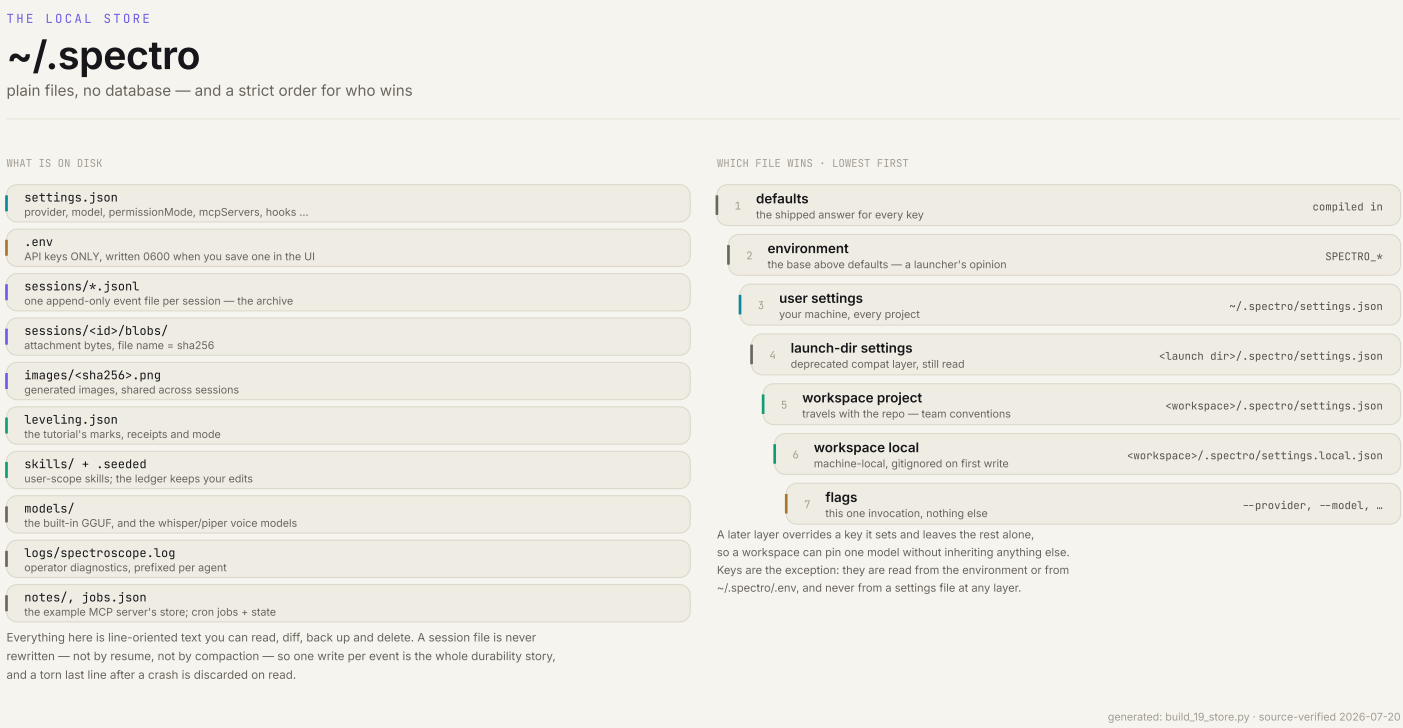
Everything spectroscope writes and reads lives in two places: a per-user home under `~/.spectro` and an optional per-project `.spectro/` folder. This part maps every file — and opens one up.

20 Where everything lives

CHAPTER 20

Where everything lives

spectroscope keeps no database, no hidden caches, no binary state. Everything is plain files you can read, version, back up and delete — most of it line-oriented JSON that `jq` understands directly.



The whole store on one page: what a used home holds, and the seven layers that decide which file wins when more than one sets the same key.

The user home: `~/.spectro`

```
~/.spectro/
├── settings.json            user settings layer (optional) — provider, model, autoApprove,
│                            mcpServers, hooks, tts … every key in chapter 30.
│                            (config.json, the legacy name, is still read beneath it
│                            for one release — `spectro doctor --migrate` renames it)
├── sessions/                the session archive — the heart of it all
│   ├── 20260716-091500-3fa4b2c1.jsonl   one session = one append-only event file
│   └── 20260716-091500-3fa4b2c1/        only when images were attached:
│       └── blobs/
│           └── ca96adb5…f90157          attachment bytes, file name = sha256
├── images/                  generated images, shared across sessions
│   └── 3f7a99c2…81d4.png       content-addressed: <sha256>.<png|jpg|webp>
├── .env                     API KEYS ONLY, mode 0600 — written here when you save a
│                            key in the UI (chapter 33). Never a settings file's job.
├── leveling.json            the tutorial: marks, receipts, the level-up history
│                            and the mode (chapter 6c)
├── skills/                  user-scope skills (chapter 15)
│   ├── .seeded              ledger of what the first boot planted, so your edits
│   │                        survive and your deletions stay deleted
│   └── my-skill/SKILL.md
├── models/                  local model files — everything that runs without a key
│   ├── qwen2.5-coder-7b-instruct-q4_k_m.gguf  a built-in chat model — whichever
│   │                        entries you downloaded in the chooser land here (chapter 16b)
│   ├── ggml-small.bin          whisper.cpp STT model (~460 MB, SHA-pinned)
│   ├── piper/piper           TTS binary
│   └── en_US-lessac-medium.onnx TTS voice
├── logs/spectroscope.log    operator diagnostics, one [agentId] prefix per line;
│                            the level is a settings key, not a code change
├── notes/                   the example MCP server's note store (one .txt per note)
├── jobs.json                cron jobs — an array, hand-edited (chapter 19)
└── jobs-state.json          last outcome per job id — written by the scheduler
```

Directories appear on demand — a fresh install has none of them until the first session, the first generated image, the first setup script. A home that has done nothing but boot and answer one prompt holds five things: `settings.json`, `leveling.json`, `sessions/` with a single `.jsonl`, the seeded `skills/`, and `logs/`.

The launch-dir layer: `<launch dir>/.spectro`

```
<launch dir>/.spectro/
├── settings.json            the launch-dir settings layer — the directory spectroscope was
│                            STARTED in. Still read (deprecated compat layer, one
│                            release); its role as THE project file moved to the
│                            workspace's own .spectro/ pair below. The "Persist"
│                            checkbox appends here only when the session has no
│                            real workspace (throwaway temp desk).
└── skills/                  project-scope skills — win over user scope by name
    ├── brainstorming/SKILL.md
    ├── test-driven-development/SKILL.md
    ├── verification/SKILL.md
    └── writing-plans/SKILL.md
```

Two more files matter at the launch-dir root: the gitignored `./.env` (**secrets only** since the precedence flip — API keys, loaded by Gradle and the launcher, never committed; deprecated `SPECTRO_*` lines still work as the env base, but `.env.example` now documents just the keys) and an optional `SPECTRO.md`, whose content is appended to the system prompt as project context — the agent reads it before your first message.

Two `.env` files, one job between them. The launch-dir `./.env` is the one you write by hand and the launcher loads; `~/.spectro/.env` is the one the UI writes for you when you save a key in the settings, at mode 0600. Both hold secrets and nothing else, both are read the same way when a provider is built, and neither is ever a place for `provider` or `model` — those belong in a settings file, where they can be seen and diffed.

Inside a session file

The session id doubles as the file name: `yyyyMMdd-HHmmss-<uuid8>` — sortable by start time, collision-free. One event per line, compact JSON, UTF-8, append-only: the file is never rewritten, not even by compaction. This is a real session from this machine (a thinking model answering "What is 2+2? Think first." — 43 lines, abbreviated in the middle):


```
{"type":"run_start","runId":"e93e24cf-f415-4873-a21c-cbd73fec673e","agentId":"main",
     "prompt":"What is 2+2? Think first.","provider":"ollama","ts":1783086147207}
{"type":"turn_start","agentId":"main","turn":1,"ts":1783086147208}
{"type":"thinking_delta","agentId":"main","text":"The","ts":1783086147561}
{"type":"thinking_delta","agentId":"main","text":" user","ts":1783086147561}
    … 30 more thinking deltas — the model reasons token by token …
{"type":"text_delta","agentId":"main","text":"2","ts":1783086148244}
{"type":"text_delta","agentId":"main","text":" +","ts":1783086148263}
{"type":"text_delta","agentId":"main","text":" 2","ts":1783086148282}
{"type":"text_delta","agentId":"main","text":" =","ts":1783086148301}
{"type":"text_delta","agentId":"main","text":" 4","ts":1783086148339}
{"type":"text_delta","agentId":"main","text":".","ts":1783086148358}
{"type":"usage","agentId":"main","inputTokens":297,"outputTokens":49,"ts":1783086148379}
{"type":"run_end","runId":"e93e24cf-…","stopReason":"end_turn","ts":1783086148379}
```


Everything you saw in Part II — chat, graph, trace, Lab, plan panel — is a fold over lines like these. A REPL session appends multiple runs into one file; resume appends to the same file again. Chapter 22 documents all 18 line types field by field.

Blobs and generated images

- **Attachments** (what you upload) live next to their session: `sessions/<id>/blobs/<sha256>`. Content-addressed — attaching the same image twice stores one file. Events reference `blobPath` + `sha256` only; a deleted blob degrades to a placeholder on replay, never a crash. Accepted types: jpeg, png, webp, gif.
- **Generated images** (what the agent paints) are shared across sessions in one global store: `~/.spectro/images/<sha256>.<ext>`. The web UI loads them via `GET /api/images/<file>`, where only `64-hex.png|jpg|webp` names are even considered — the content address is the traversal guard.

The workspace: where the agent actually works

The session files above are spectroscope's *records*; the **workspace** is the agent's *desk* — the directory the file tools sandbox against, glob/grep search and `run_command` runs in. Unless a `workspace` is configured (chapter 30), every session gets its own folder under the OS temp dir:

```
<tmpdir>/spectroscope-ws/
├── 20260716-204301-db5f3af7/   one folder per session, keyed by the SESSION id
│   └── add.py                     ← what "write me a script" produces lands HERE,
└── 20260716-204419-39b41a29/      not in the repo you started spectroscope in
```

The key is the session id, so a **continued session finds its files again**; a New chat means a fresh desk. The web server announces the folder over the socket (`workspace_info` — a UI-only frame, never in the JSONL) and the Files tab follows it (chapter 6). The launch dir stays anchored: skills, MCP servers, SPECTRO.md and its own (deprecated) settings layer still load from the directory spectroscope was started in — while the workspace contributes its own settings pair on top (below).

Three ways to place the desk, highest wins: **pick it per session** in the web UI (the Files tab's "Choose folder …" button opens the native macOS dialog on the spectroscope machine; only before the first run, pinned in server memory for the session), **configure it** (`workspace` key / `SPECTRO_WORKSPACE` / `--workspace` — chapter 30), or **let spectroscope mint** the per-session temp folder above.

A *real* workspace (picked or configured — not the throwaway temp desk) carries its own settings pair, the top file layers of the hierarchy (chapter 30) and the files the composer gear and the permission dialog's "Persist" checkbox write:

```
<workspace>/.spectro/
├── settings.json            the workspace PROJECT file — travels with that repo.
│                            Team conventions become checked-in settings:
│                            permissionMode, autoApprove, hooks, mcpServers …
│                            Persisted "always allow" rules land here.
├── settings.local.json      machine-local overrides for THIS workspace — absolute
│                            paths, personal rules. Never committed:
└── .gitignore               spectroscope writes it on the first local save; it lists
                             settings.local.json so the local file stays out of git
```

This pair is not just read, it is **executed**: `mcpServers` spawns processes, `hooks` runs shell commands around every tool call, and a permissive `autoApprove` / `permissionMode` can auto-allow gates that would otherwise ask (chapter 30). Pinning a cloned or otherwise foreign folder as your workspace means reviewing its `.spectro/` first — the same way you would review its build scripts before running them.

Lifecycle: create, continue, delete

- **Created** on the first prompt of a connection (web) or at REPL start; headless and cron runs write the same files.
- **Continued** by `spectroscope --resume <id>`, or in the web UI by opening the session's row and sending a message. The history is reconstructed from the file, and new events append.
- **Deleted** only from the row menu, after a confirmation dialog (chapter 8): `SessionStore.deleteSession` removes the JSONL and the blob folder — and only for ids resolving to a direct child of the sessions directory. Nothing else in spectroscope ever deletes or rewrites a session line.
- **Crash-safe by construction:** one `write` per event, no open handle, no save step. A torn last line after a crash is silently discarded on read; everything that streamed is on disk.

The jq cookbook

Because the format keeps its rules (one object per line, no binary, camelCase), the shell is a first-class session browser:

```
# watch a session live
tail -f ~/.spectro/sessions/<id>.jsonl | jq -c '{type, agentId, name}'

# tool timings
jq -r 'select(.type=="tool_result") | "\(.durationMs)ms\t\(.callId)"' <id>.jsonl

# token totals
jq -s '[.[] | select(.type=="usage")]
       | {in: map(.inputTokens)|add, out: map(.outputTokens)|add}' <id>.jsonl

# the agent tree
jq -r 'select(.type=="agent_spawn") | "\(.parentId) → \(.agentId): \(.task)"' <id>.jsonl
```

PART V

Under the hood

The complete technical reference: the generated architecture dossier, every event on the wire, the provider port, the WebSocket and REST protocols, the loop internals, context management, MCP internals, every configuration key, and the build itself.

21 The architecture dossier
22 The RunEvent protocol
23 Streams & cancellation
24 The provider wire
25 The WebSocket protocol
26 The REST API
27 The agent loop, hop by hop
28 Context: compaction, caching, introspection
29 MCP internals
30 Configuration reference
31 Build & test inventory
A Troubleshooting
B Reproducing this guide

CHAPTER 21

The architecture dossier

Nineteen generated diagrams cover the full build — every class name, wire string, endpoint and count in them was read from this repository, and each comes from a rerunnable Python generator (`docs/diagrams/build_NN_*.py`): change the system, rerun the script, the diagram is current again. The shared visual language: ocean outlines the core, lilac the faces, salmon the model side and external services, moss the disk; sand carries the event wire and stats; coral appears only where a human decides, a run is cancelled or the network boundary is crossed.

The big picture

Chapter 1 already showed diagram 00 (one core, five faces). The build view below adds the honest module structure: five JVM modules, two JS toolchains, and the one deliberate cross-dependency — the server reuses the CLI's Transcriber for `/api/transcribe`.

Diagram 01 — the Gradle build: module dependency edges, version catalog, and the two npm toolchains that live next to the JVM graph on purpose.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 02

The event protocol.

One sealed interface, 18 records, Jackson-typed by a snake_case "type" property. The same line is the socket frame, the JSONL storage row and the graph input.

LIFECYCLE

run_start RunStart
`runId agentId parentId prompt provider attachments`

turn_start TurnStart
`agentId turn`

run_end RunEnd
`runId stopReason`

error ErrorEvent
`agentId message`

PERMISSION GATE

permission_request PermissionRequest
`agentId callId name input`

permission_decision PermissionDecision
`callId allowed`

STREAMING

text_delta TextDelta
`agentId text`

thinking_delta ThinkingDelta — additive
`agentId text`

AGENT TREE

agent_spawn AgentSpawn
`agentId parentId task`

agent_message AgentMessage — additive
`from to role state text label`

CAPABILITIES

voice_input VoiceInput — additive
`agentId durationMs model`

image_generated ImageGenerated — additive
`agentId callId prompt provider model mediaType blobPath sha256`

plan Plan — additive
`agentId steps[text status]`

TOOL LOOP

tool_call ToolCall
`agentId callId name input`

tool_result ToolResult
`agentId callId output isError durationMs`

ACCOUNTING

usage Usage
`agentId inputTokens outputTokens`

context_info ContextInfo — additive
`agentId turn messages estimatedTokens threshold parts`

compaction Compaction — additive
`agentId removedTurns summaryChars`

Wire rules

`@JsonTypeInfo(property = "type")` · `snake_case type values` · `camelCase field names` · `@JsonInclude(NON_NULL)` · `long ts on every event` · `unknown types must be ignored (forward compat)`

The JSONL session file

~/.spectro/sessions/<yyyyMMdd-HHmmss-uuid8>.jsonl · append one line per event, crash-safe, never rewritten (compaction is in-memory only)

{"type":"tool_call","agentId":"main","callId":"call-7","name":"read_file","input":{"path":"src/App.tsx"},"ts":1783000012400}

resume: readSessionEvents drops a torn last line · loadSession finds the main agent structurally (first run_start without parentId) · orphan tool_calls dropped · adjacent roles merged

interop invariant: byte-identical with the TypeScript edition · proven by CrossEditionReplayTest + JsonlFormatInteropTest against interop/ts-edition-session.jsonl

additive event (safe extension) · core protocol · gate · agent tree · accounting · capabilities

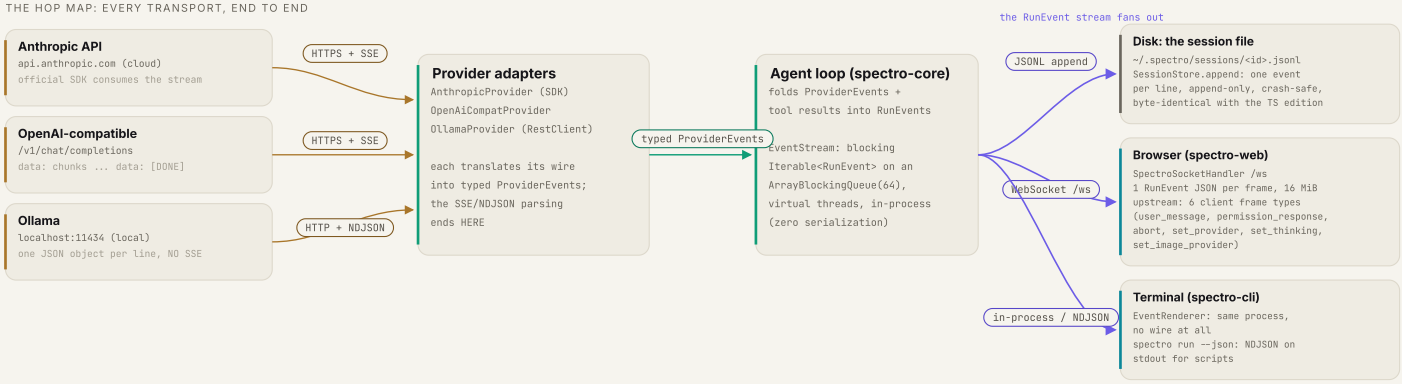
generated: build_02_events.py · source-verified 2026-07-20

Diagram 02 — the RunEvent protocol: all 18 record types with exact wire strings, additive events marked, the wire rules, and the anatomy of a session file including the cross-edition invariant.

The protocols.

Five wire shapes carry one event model. SSE lives only between the cloud LLM APIs and the provider adapters; inside the harness everything is an in-process stream, the browser gets WebSocket frames, the disk gets JSONL, and agents talk to each other through plain events.

THE HOP MAP: EVERY TRANSPORT, END TO END



MCP side channel (tools, not events)

McpClient ⟷ server: JSON-RPC 2.0 over stdio, one message per line (protocol 2024-11-05)
optional HttpSseTransport: JSON-RPC over HTTP POST, the RESPONSE arrives as an SSE body
results surface as ordinary tool_call / tool_result events: nothing new on the wire

Where SSE actually lives

1 cloud LLM APIs stream SSE: Anthropic (inside the official SDK) and OpenAI-style endpoints
2 optional: an MCP server over HTTP answers SSE
x Ollama does NOT: it streams NDJSON
x the agent NEVER sees SSE: adapters translate to typed events before anything leaves the provider
x the browser does NOT get SSE: it gets WebSocket

Where JSON lines live (same framing, three documents)

1 the session file: one RunEvent per line (the storage format, shared with the TS edition)
2 MCP stdio framing: one JSON-RPC message per line
3 spectro run --json: NDJSON RunEvents on stdout

do not confuse the framing (JSON per line) with the document type: three different vocabularies

How agents talk to each other (A2A-lite): no network, just events

a child is its own Agent loop in the SAME JVM (virtual thread); MergedEventStream = one queue, many producers, so the whole conversation lands in the JSONL and every face for free

permission gates ride along: a child's permission_request goes to the SAME human broker; hooks apply to children too · additive events, wire stays byte-identical

external wire (SSE / NDJSON / JSON-RPC) · in-process (no serialization) · the RunEvent stream · disk · faces · -- dashed = side channel

generated: build_13_protocols.py · source-verified 2026-07-20

Diagram 13 — protocol breakdown, hop by hop: SSE lives only between the cloud APIs and their adapters (and the optional MCP-HTTP transport); Ollama streams NDJSON; in-process it is the blocking EventStream; WebSocket to the browser; JSONL on disk; JSON-RPC/stdio to MCP servers; A2A between agents is events on the merged stream — no network.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 03

The provider port.

Everything the loop needs from a model sits behind LlmProvider. Wrappers add runtime switching and retries without the loop noticing; the config layers decide which backend is real.

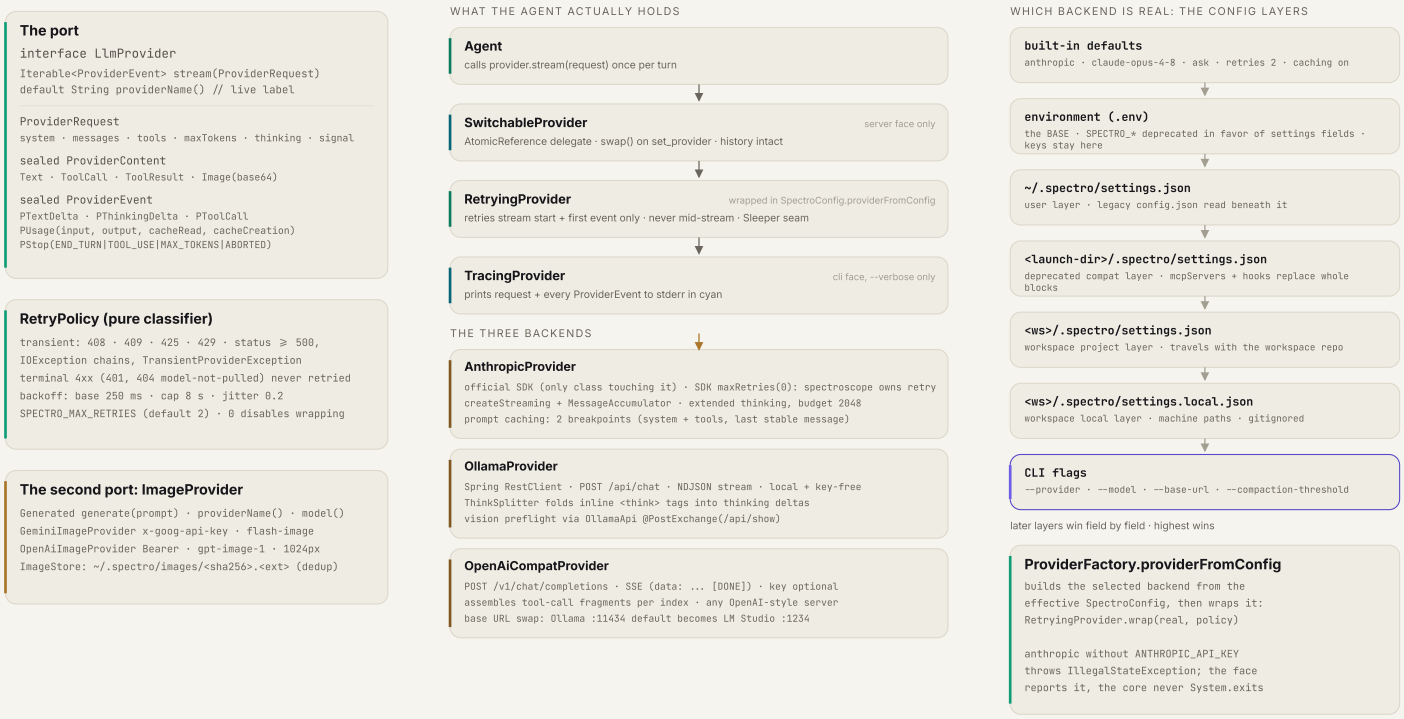


Diagram 03 — the LlmProvider port: the sealed request/event vocabulary, the wrapper chain (Switchable → Retrying → backend), the three backends with their transports, the retry policy table, the ImageProvider port and the config precedence.

THE BUILT-IN MODELS

spectro-local

a catalogue of GGUFs, a supervised llama-server, and no key anywhere

1 **choosing, and where a model is looked for**

local/models.json names five models; the chooser shows what each is good for, whether it can drive tools, and whether THIS machine holds it (memory + disk measured). Each file: the app bundle first, the user home second.

`local/models.json` `spectro.bundle.models` `~/.spectro/models`

2 **when one is absent**

Per model one pinned URL, one pinned sha256, one pinned size. The bytes are verified BEFORE the file is moved into place, so a truncated or swapped download never becomes the model the next boot loads. Each row links its licence and the repository it actually fetches from.

`1.8 – 5.0 GB` `sha256-pinned` `atomic move`

3 **the runtime the JVM supervises**

llama-server, spawned lazily on first use and shared by every session in the process. A free loopback port, the chosen model's own context budget, /v1/models polled for up to 60 s. Switching models restarts it — a llama-server keeps serving the weights it was started with.

`127.0.0.1:<free>` `-c <per model>` `--jinja` `reaped on JVM exit`

4 **what the loop is told about it**

The provider is the ordinary OpenAI-compatible one, aimed at localhost with no key. The capability is a per-model fact: the Qwen entries speak tool_calls and get the full registry; VibeThinker does not, and is offered none.

`Qwen: tools ✓` `VibeThinker: tools ✗`

The honest limit: a local model in these sizes stays below the cloud models — simpler answers, smaller context. But the default Qwen3 4B thinks visibly AND drives the file tools, and the catalogue exists so a fresh install can produce a real run — its own events, its own trace, its own session file — before anyone has decided whether to pay for a key.

Nothing about the rest of the product knows it is local. The events, the gate, the trace and the JSONL are the same ones a cloud provider produces, which is the whole reason the keyless walkthrough of the tutorial reaches deep field.

generated: build_18_local_model.py · source-verified 2026-07-20

Diagram 18 — the built-in models: where a GGUF is looked for, the pinned download and the hash checked before the move, the llama-server the JVM supervises, and the capability flags the loop is told per model.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 04

One turn through the loop.

The agent streams a model turn, guards every tool call through hooks, allowlist and the human gate, feeds the result back, and repeats until the model stops calling tools.

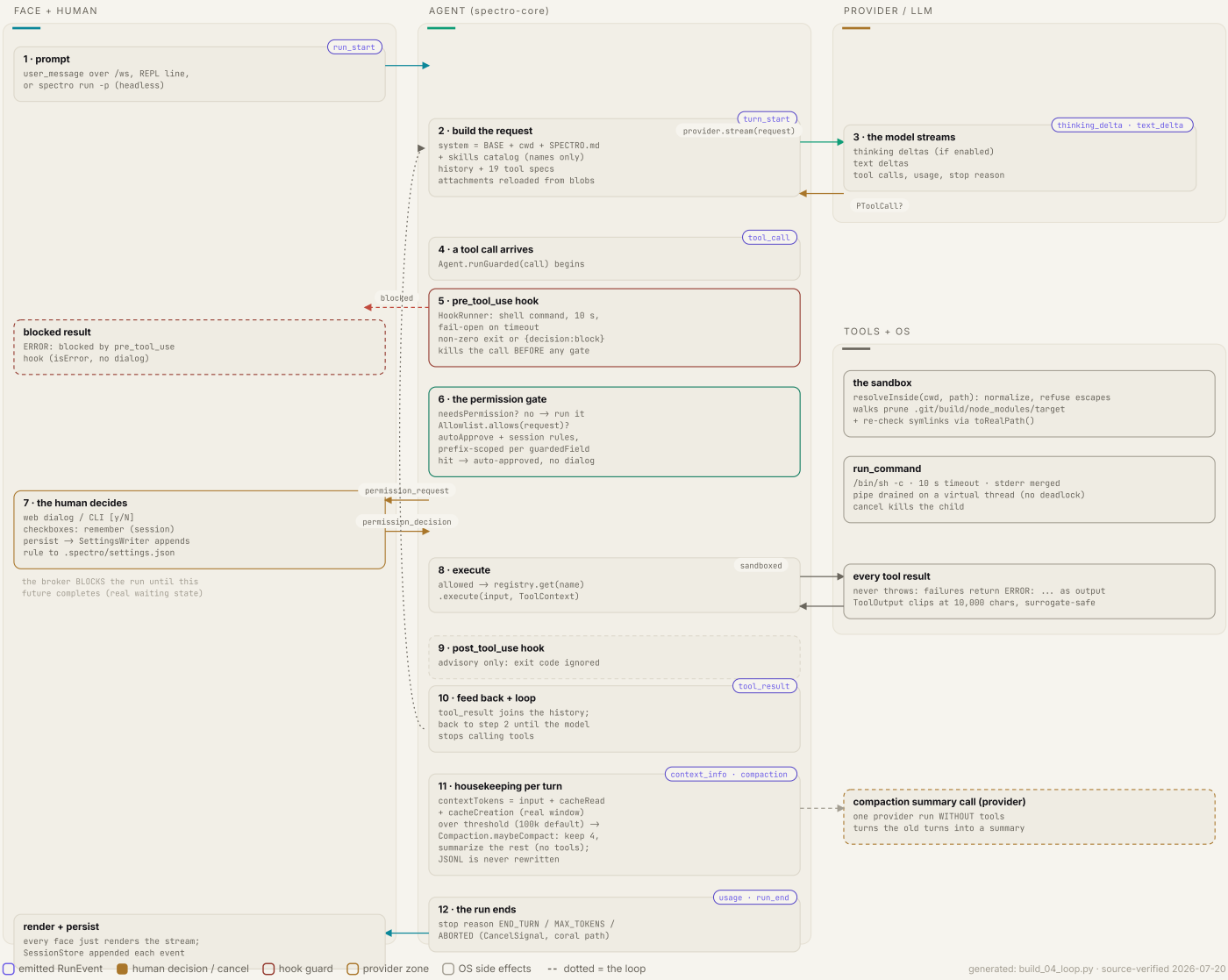


Diagram 04 — one turn as a swimlane: request build, streaming, the pre_tool_use hook, allowlist and human gate, sandboxed execution, the feedback loop into the next turn, compaction and run end.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 05

The tool belt.

19 tools behind one registry and one gate. Read-only tools run free; anything that writes, executes, spends money or leaves the machine asks first. Course maxim: tool inputs are model output, and therefore untrusted.

Files + search

`list_dir` — list a directory (sandboxed)	free
`read_file` — read text, whole if 1/4 of the window fits	free
`glob` — pattern find, pruned walk, cap 200	free
`grep` — regex search: path:line:text	free
`write_file` — write + mkdirs · rule scoped by path	gate + prefix rule
`edit_file` — exact-string replace, unique match · by path	gate + prefix rule

Execution + network

`run_command` — /bin/sh -c, 10 s · rule scoped by first token	gate + prefix rule
`web_fetch` — http(s) via HttpFetcher, HTML to text · by url	gate + prefix rule

Media + knowledge

`generate_image` — paid cloud call → ImageStore + image_generated	gate
`use_skill` — return a SKILL.md body on demand	free
`update_plan` — emit the plan event · main agent only	free

Delegation (parent only)

`spawn_agent` — one child: explore or worker · waits	free
`spawn_agents` — up to 4 children in parallel	free
`build_plan` — worker + writing-plans skill	free
`write_spec` — worker + brainstorming skill	free
`develop` — worker + test-driven-development skill	free
`test` — worker + verification skill	free

Child side + MCP

`report_status` — child → parent agent_message (working)	free
`mcp__<server>__<tool>` — proxy to a remote MCP tool · dynamic set	gate

The sandbox

```
resolveInside(cwd, path): normalize,
refuse anything outside the working dir
tree walks prune .git · build · node_modules
· target, then re-check symlink escapes
via toRealPath()

ShellCommand: stderr merged, output pipe
drained on a virtual thread, cancel kills
```

Tool contract

```
String execute(JsonNode input, ToolContext)
never throws: failures return ERROR: ...
ToolOutput: one shared 10,000-char clip,
surrogate-safe
ToolContext(cwd, signal, agentId, callId,
emit) · tools can emit additive events
```

Allowlist: what a remembered "always allow" actually covers

one guardedField map feeds remembering AND matching, so a click never becomes a blanket approval:

`run_command : command (first token) e.g. run_command:git*` · `write_file / edit_file : path` · `web_fetch : url` · `other tools : bare name`

sources: project autoApprove rules + session-remembered rules · persist appends via SettingsWriter (read-modify-write, no clobber)

free (read-only or harmless) · gated by the permission broker · delegation · external side effects

generated: build_05_tools.py · source-verified 2026-07-20

Diagram 05 — the tool belt: every tool grouped by family with its gate class (free / gated / gated + prefix rule), the sandbox rules, the tool contract, and the allowlist's guardedField mapping.

Delegation

SPECTROSCOPE · ARCHITECTURE DOSSIER · 06

Delegation: agents spawning agents.

The parent agent spawns typed children through ordinary tools. Every child is its own loop with its own toolset and CancelSignal; an additive agent_message lifecycle makes the protocol between them visible.

Parent agent (main)

gets from SubagentManager:

`spawn_agent` `spawn_agents` `build_plan` `write_spec` `develop` `test`

dev tools are thin role wrappers over a worker
spawn: no new agent type, prompt + skill only

SubagentManager

```
MAX_PARALLEL_CHILDREN = 4
CHILD_TIMEOUT_MS = 120,000
child ids: explore-1, worker-2, ...
children NEVER get spawn/dev tools
(nesting depth 1 by construction)
every child also gets report_status
parent cancel cascades per-child signals
children run the parent's HookRunner
```

MergedEventStream

```
one LinkedBlockingQueue, many producers
spectroscope-parent-pump virtual thread drains parent,
each child adds itself as a producer
every child event carries its own agentId
```

ROLE CATALOG

explore (read-only scout)

EXPLORE_TOOL_NAMES:

`list_dir` `read_file` `glob` `grep`

own system prompt from RoleCatalog.SYSTEM_PROMPTS

worker (does the work)

```
the full child base toolset (files, shell, web, ...)
+ use_skill when skills are loaded
dev tools point it at a skill:
```

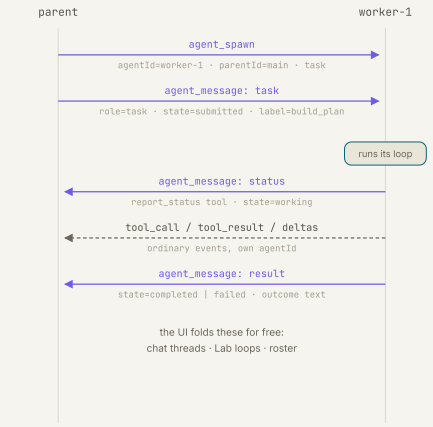
DevSpec: tool → skill

`build_plan`	→ `writing-plans`
`write_spec`	→ `brainstorming`
`develop`	→ `test-driven-development`
`test`	→ `verification`

Introspection (feeds /api/context)

```
RoleCatalog.roleProfiles(childBaseToolNames)
→ RoleProfile(type, kind, systemPrompt, tools,
readOnly, skill) · parentTools() summaries
```

THE A2A-LITE LIFECYCLE (additive agent_message)



Gates + hooks apply to children too

```
a child's permission_request rides the SAME
broker to the human; permission_decision
carries no agentId and is routed back by
callId (gateOwners in the web Lab)
children inherit the parent HookRunner
```

additive A2A events · read-only role · worker role · gate

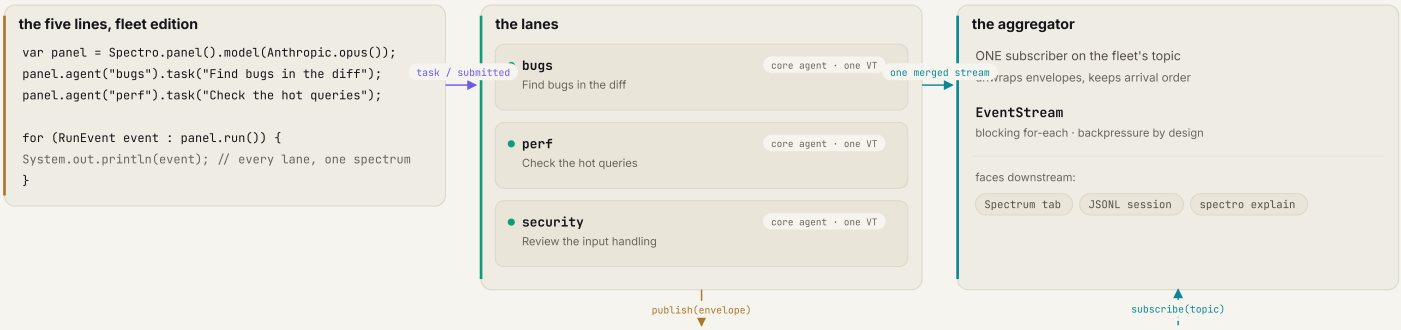
generated: build_06_subagents.py · source-verified 2026-07-20

Diagram 06 — subagents: the manager's limits, the explore/worker role profiles, the dev-tool-to-skill table, and the A2A-lite lifecycle as a parent/child sequence.

SPECTRO-ORCHESTRATOR

the fleet: agents on a bus, one spectrum

Spectro.panel() runs every lane as a full core agent on its own virtual thread. Every event rides the bus as an envelope; ONE aggregator drains the fleet's topic into one EventStream — the same blocking for-loop as a single agent.



the bus — BusTransport (in-memory today, process next, broker only if reality demands)

every event of every lane, wrapped:

`sender` `contextId` `taskId` `sequence` `parentId` `topic` `payload = RunEvent, verbatim`

self-addressing: correlate by taskId + contextId (no from/to on the envelope) · order per sender: sequence · causality: parentId chain

the A2A-lite choreography on the stream (what the Spectrum tab folds)

agent_spawn	→ task / submitted	→ status / working	→ run_start … run_end	→ result / completed | failed	→ panel run_end
the panel announces the lane	the assignment, visible on the stream	the lane reports in	the lane's own events, forwarded verbatim	the recorded outcome	the spectrum closes

lane = core agent · aggregator / faces · live / publish · task assignment · storage / meta

generated: build_14_orchestrator.py · source-verified 2026-07-20

Diagram 14 — the orchestrator: the panel facade, one full agent per lane, the bus transports, the aggregator, and the A2A-lite choreography that turns a fleet into one merged stream.

The web face

SPECTROSCOPE · ARCHITECTURE DOSSIER · 07

The web backend.

Spring Boot on 127.0.0.1:8080. One TextWebSocketHandler speaks the RunEvent wire; a handful of read-only REST endpoints serve replay, context, models and the workspace. One jar serves the built UI.

transport · core wiring · GET (read-only) · POST / decision

generated: build_07_server.py · source-verified 2026-07-20

Diagram 07 — spectro-server: the WebSocket frame vocabulary in both directions, all REST endpoints with their guards, and SessionConnection's responsibilities.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 08

The browser face.

Events arrive on one socket, are batched per animation frame and folded by one pure reducer. Every tab is a different reader of the same UiState, so live runs, archives, imports and scenarios all render identically.

Diagram 08 — spectro-web: the rAF-batched pure-reducer pipeline, the tabs as lenses over one UiState, the header and sidebar anatomy, the persistence stores and the design system.

Runtime and integration

SPECTROSCOPE · ARCHITECTURE DOSSIER · 09

Start it: launcher, desktop, doctor.

One executable per checkout fixes the two host problems (JDK version, .env) before any command runs. The desktop face is a supervisor: Electron manages the Spring Boot JVM as a child process.

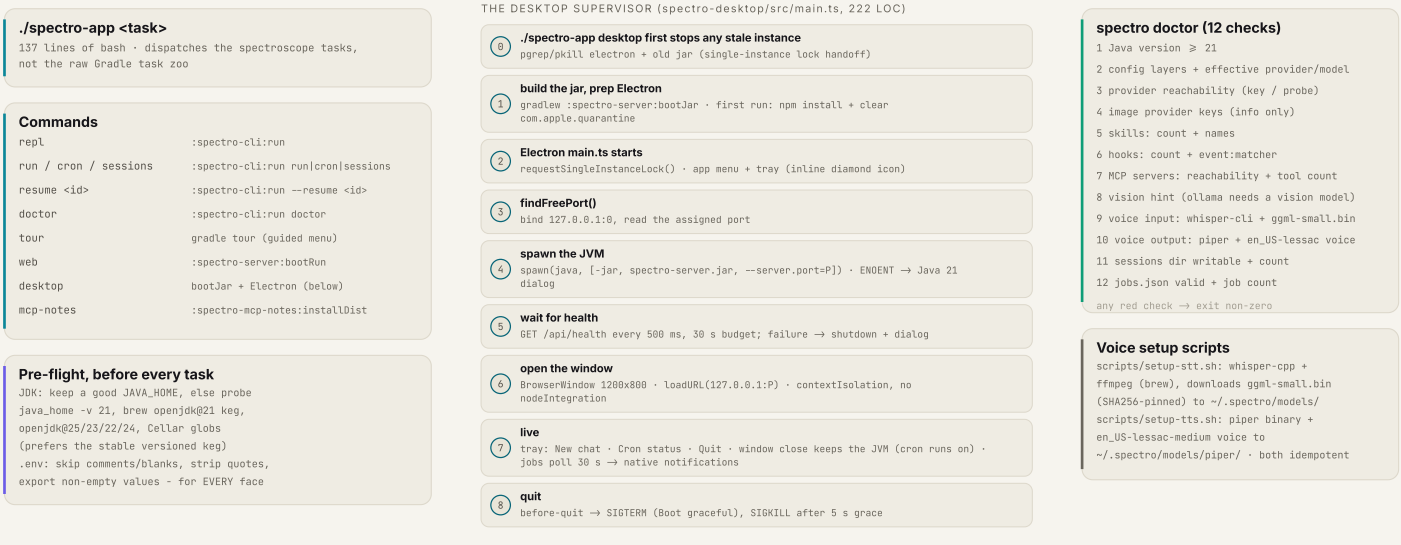
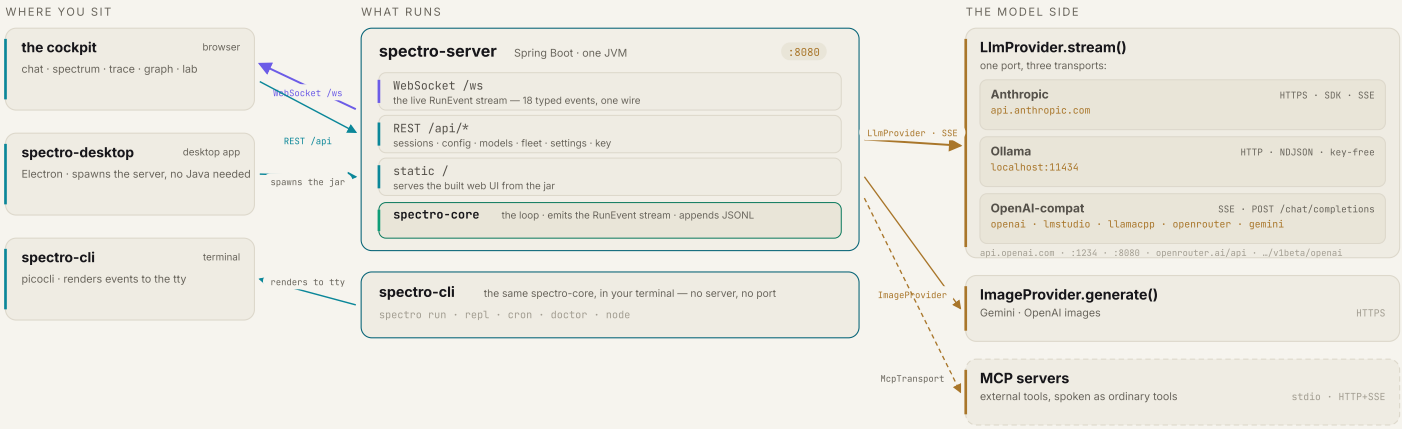


Diagram 09 — runtime: the ./spectro-app command table and its pre-flight (JDK resolution, .env), the nine-step Electron supervision sequence, the twelve doctor checks and the voice setup scripts.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 15

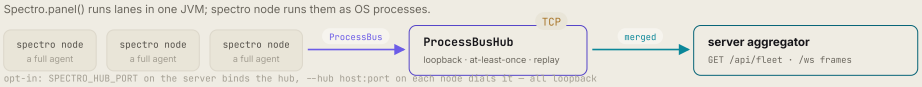
Every process, every wire.

One core, several faces, one model side. This is the deployment: the processes that run, the ports they bind, and the exact protocol on every connection between them.



Fleet: many agents, watched as one

Spectro.panel() runs lanes in one JVM; spectro node runs them as OS processes.



opt-in: SPECTRO_HUB_PORT on the server binds the hub, --hub host:port on each node dials it — all loopback

Disk: local-first, everything the run persists

`~/.spectro/.env` · `~/.spectro/config.json` · `~/.spectro/sessions/<id>.jsonl` · `sessions/<id>/blobs/<sha256>` · `~/.spectro/images/<sha256>` · `~/.spectro/jobs.json` · `<project>/.spectro/settings.json` · `SPECTRO.md`

Everything binds 127.0.0.1 — local-first, no TLS by design. The model side is the only network the agent reaches out to.

faces / surfaces · spectro-core · model side / external · the event wire · port / live endpoint · -- dashed = external / out-of-process

generated: build_15_deployment.py · source-verified 2026-07-20

Diagram 15 — deployment: every process, every port and the protocol on each wire, from the browser socket to the TCP hub and the JSONL on disk.

THE LOCAL STORE

~/.spectro

plain files, no database — and a strict order for who wins

WHAT IS ON DISK

- `settings.json` — provider, model, permissionMode, mcpServers, hooks …
- `.env` — API keys ONLY, written 0600 when you save one in the UI
- `sessions/*.jsonl` — one append-only event file per session — the archive
- `sessions/<id>/blobs/` — attachment bytes, file name = sha256
- `images/<sha256>.png` — generated images, shared across sessions
- `leveling.json` — the tutorial's marks, receipts and mode
- `skills/ + .seeded` — user-scope skills; the ledger keeps your edits
- `models/` — the built-in GGUF, and the whisper/piper voice models
- `logs/spectroscope.log` — operator diagnostics, prefixed per agent
- `notes/, jobs.json` — the example MCP server's store; cron jobs + state

Everything here is line-oriented text you can read, diff, back up and delete. A session file is never rewritten — not by resume, not by compaction — so one write per event is the whole durability story, and a torn last line after a crash is discarded on read.

WHICH FILE WINS · LOWEST FIRST

1. **defaults** — the shipped answer for every key — compiled in
2. **environment** — the base above defaults — a launcher's opinion — SPECTRO_*
3. **user settings** — your machine, every project — ~/.spectro/settings.json
4. **launch-dir settings** — deprecated compat layer, still read — <launch dir>/.spectro/settings.json
5. **workspace project** — travels with the repo — team conventions — <workspace>/.spectro/settings.json
6. **workspace local** — machine-local, gitignored on first write — <workspace>/.spectro/settings.local.json
7. **flags** — this one invocation, nothing else — --provider, --model, …

A later layer overrides a key it sets and leaves the rest alone, so a workspace can pin one model without inheriting anything else. Keys are the exception: they are read from the environment or from ~/.spectro/.env, and never from a settings file at any layer.

generated: build_19_store.py · source-verified 2026-07-20

Diagram 19 — the local store: what a used home holds, next to the seven settings layers and the rule for which of them wins.

MCP: external tools, no new events.

A container-free MCP client wraps every remote tool as an ordinary spectroscope Tool. Calls surface as plain tool_call / tool_result, so graph, trace and JSONL show them for free and the wire stays byte-identical.

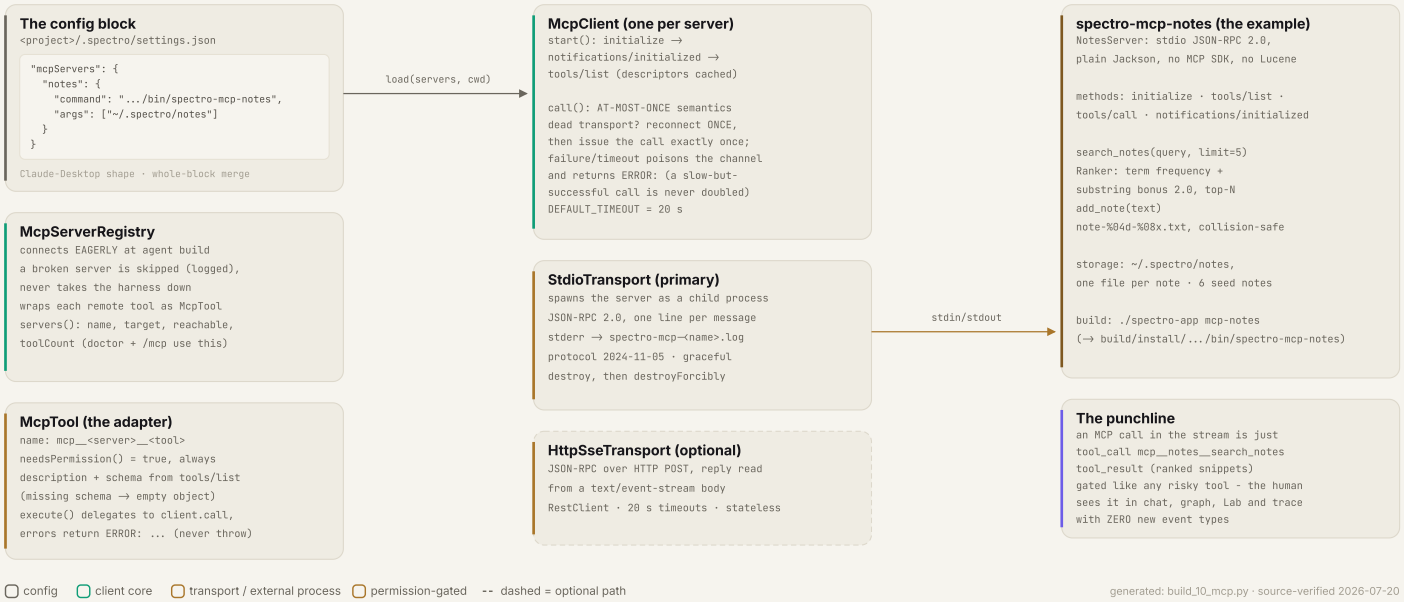
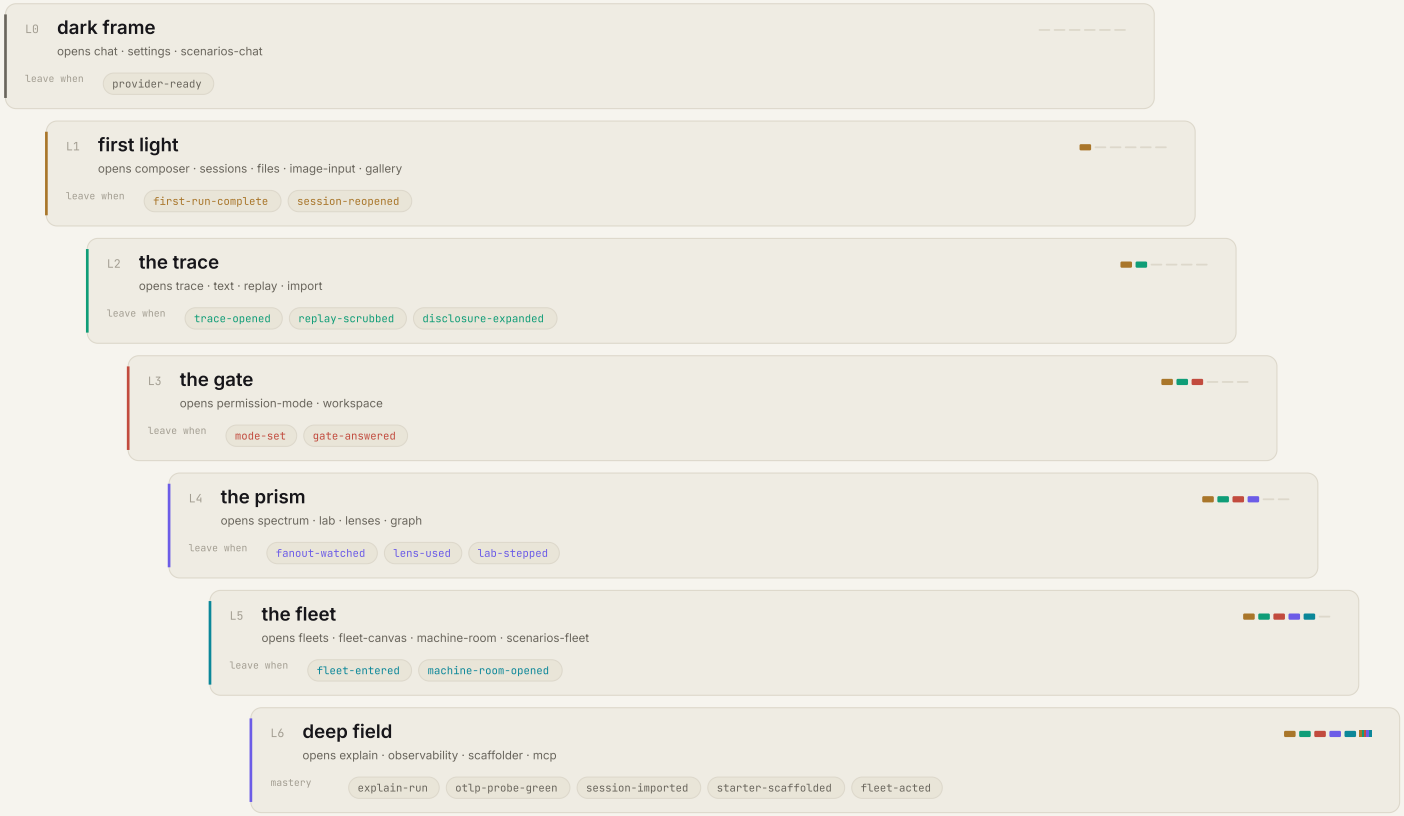


Diagram 10 — MCP: the config block, the registry's eager-connect/skip-on-failure policy, the at-most-once client, both transports, the McpTool adapter and the spectro-mcp-notes example server.

LEVELING

the tutorial

seven states, and the surfaces each one opens



dark frame · first light · deep field are the telescope's own words: the calibration image taken with the shutter closed, the first real picture, and what appears when you stare at a patch of sky that looked empty.

Every criterion is reachable without a paid key: a local model counts as light, and the bundled scenarios count as fan-outs and fleets.

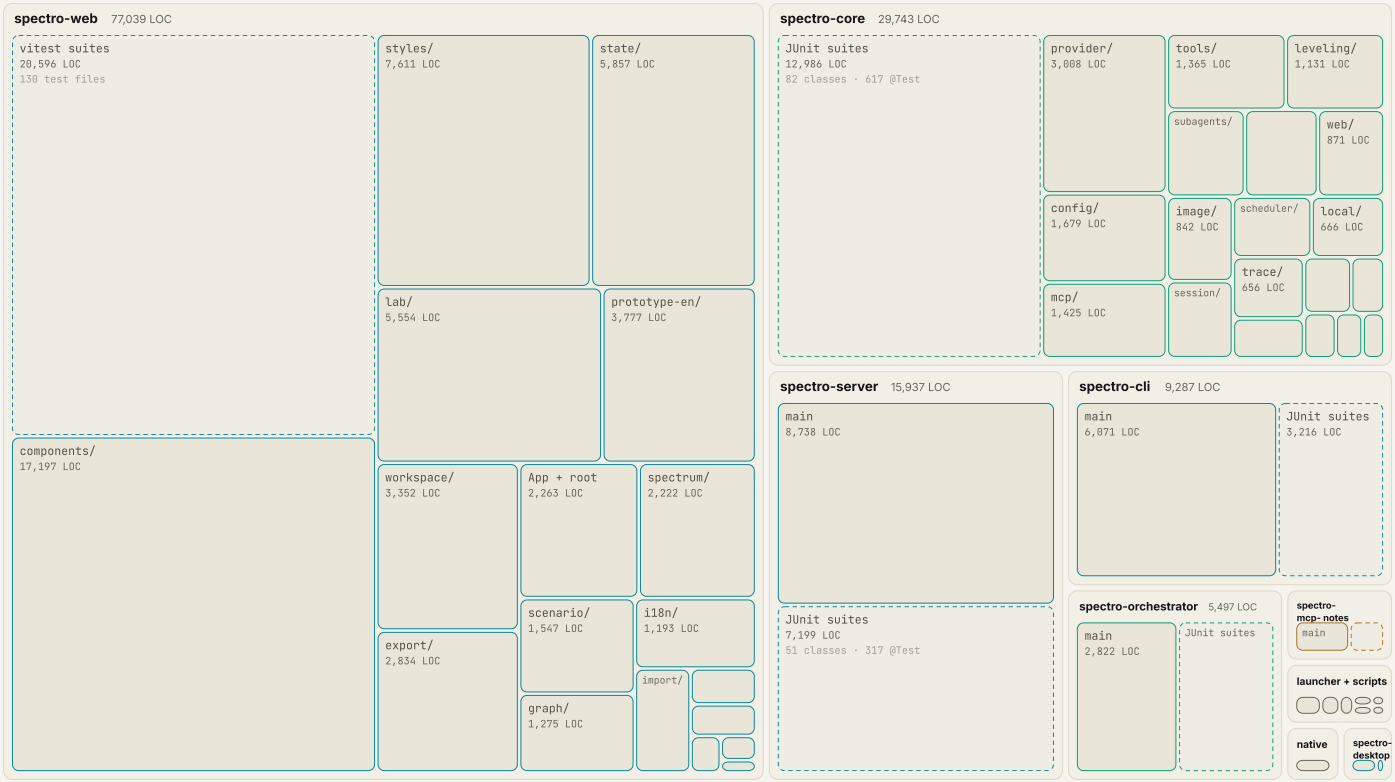
generated: build_17_leveling.py · source-verified 2026-07-20

Diagram 17 — the tutorial as one picture, generated from the same levels.json the engine, the web face and spectro level all read, so the poster cannot drift from the product.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 11

Code inventory.

Area proportional to lines of source (main + tests), counted from the tree at build time (this build: 2026-07-31).



libraries (core, orchestrator) · faces (cli, server, web, desktop) · example MCP server · launcher · scripts · native -- dashed tile = test code

Total: 140,257 lines across 7 modules + launcher scripts + the native PTY helper, counted from this tree at build time.

generated: build_11_treemap.py · source-verified 2026-07-20

Rule: newline count per file under <module>/src — .java | .ts | .tsx | .css (native: .c, scripts: .sh); generated bundles and non-source files fall out via the extension rule.

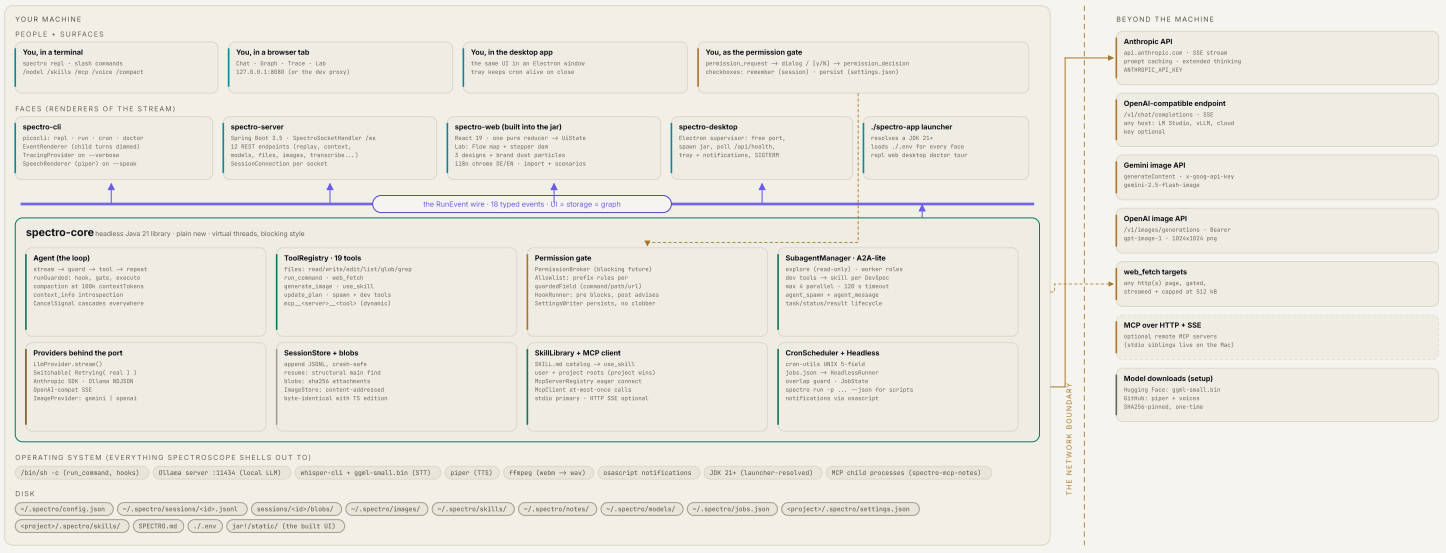
Too small to label: spectro-web markdown/ (329) · spectro-web translate/ (289) · spectro-web transport/ (154) · spectro-web reactflow/ (120) · spectro-web effects/ (60) · spectro-core dev.spectroscope.core (root) (956) · spectro-core dev.spectroscope (facade) (426) · spectro-core events/ (398) · spectro-core skills/ (279) · spectro-core hooks/ (216) · spectro-core permission/ (182) · spectro-core log/ (148) · spectro-mcp-notes JUnit suites (412) · launcher + scripts ./spectro-app (260) · launcher + scripts build-desktop-runkit.sh (184) · launcher + scripts fetch-llama-server.sh (139) · launcher + scripts setup-stt.sh (96) · launcher + scripts setup-tts.sh (89) · launcher + scripts build-spectro-pty.sh (63) · launcher + scripts build-release-assets.sh (60) · native spectro-pty.c (414) · spectro-desktop main.ts (301) · spectro-desktop menu.ts (95)

Diagram 11 — the code inventory as a treemap: area proportional to lines of source per module and package, test code as dashed tiles, measured counts in the footer.

SPECTROSCOPE · ARCHITECTURE DOSSIER · 12 · THE WALL

spectroscope, the whole machine.

Agent harness, five faces, one event wire. Everything on this wall exists in the repo; the dashed line is the real network boundary.



4 + 2	41,071	348 + 244	18	19	12	6 + 3	7	3 + 2 + 2	16
modules + JS toolchains	lines of source	JUnit + vitest, green	RunEvent types	tools	REST endpoints	WS frame types + designs	teaching scenarios	LLM + image + voice engines	workshop stages (0-9 + b1-b6)

faces + surfaces · core · external service · disk · the event wire · human decision / network boundary

generated: build_12_poster.py · source-verified 2026-07-20

Diagram 12 — the wall poster: people, faces, core, operating system, disk, and everything beyond the network boundary in one picture.

CHAPTER 22

The RunEvent protocol

Eighteen event types, one sealed Java interface, one JSON line each. This chapter is the complete wire reference — every field, every optionality, every rule. The format is binding and shared byte-for-byte with the TypeScript edition (`docs/concept/JSONL-FORMAT.md` is the contract document).

The wire rules

1. **snake_case types, camelCase fields.** `"type":"tool_call"` carries `callId`, never `call_id`.
2. **One line = one event**, compact JSON, UTF-8, `\n`-terminated. No arrays, no comments, no blank lines.
3. **Append-only.** A written line is never modified, reordered or deleted — compaction included.
4. **Additive evolution only.** New types and new optional fields may appear; renaming, removing or reinterpreting is forbidden. Old files replay forever.
5. **Tolerant consumers.** Unknown types and fields are skipped, never errors; an unparsable (torn) trailing line is silently discarded.
6. **Null optionals are absent.** Optional fields are omitted from the JSON entirely (`@JsonInclude(NON_NULL)`), matching the TS edition.
7. **Structured tool input.** `input` travels as a JSON object, never as a serialized string — consumers parse trees, never string-match JSON.
8. **Only data.** No binary, no object references, no cycles; every line round-trips losslessly.

IDS — `sessionId` = file name (`yyyyMMdd-HHmmss-<uuid8>`) · `runId` = UUID per run · `agentId` = `"main"` or `explore-1` / `worker-2` … · `callId` links tool_call ↔ permission_request/decision ↔ tool_result.

TIMESTAMPS — `ts` = epoch milliseconds, monotone within a file (write order).

The catalog

`run_start` — since stage 3

FIELD	TYPE	
`runId`	string	UUID per run
`agentId`	string	`"main"` or child id
`parentId`	string?	only on child runs — the structural marker of a subagent
`prompt`	string	the user/task text
`provider`	string? *additive*	kept live-accurate across mid-session switches
`attachments`	Attachment[]? *additive*	image references: `{kind, mediaType, blobPath, sha256}` — bytes never enter the event
`origin`	string? *additive · card 554*	`"playbook"` on the turn a playbook step sends into the chat; absent on every prompt a person typed

`turn_start` — since stage 3

`agentId`, `turn` (1-based; the loop caps at 15). On resume a turn boundary flushes the reconstruction buffer.

`text_delta` — since stage 3

`agentId`, `text` — one streamed chunk of the answer. Consecutive deltas concatenate into the assistant text on resume.

`thinking_delta` — additive · stage 7+

`agentId`, `text` — one chunk of the model's reasoning. A sibling of `text_delta` but a separate stream: rendered apart, and it **never re-enters the provider history** (the resume fold ignores it).

`tool_call` — since stage 3

`agentId`, `callId`, `name`, `input` (JSON object). MCP calls (`mcp__server__tool`) and skill uses are ordinary tool calls — no dedicated types exist.

`permission_request` — since stage 3

`agentId`, `callId`, `name`, `input` — emitted when a gated tool is about to run in ask mode; same `callId` as the pending call.

`permission_decision` — since stage 3

`callId`, `allowed` (boolean). One of only two types **without an `agentId`** — consumers route it back via `callId`. Emitted for every decision, allowlist auto-approvals included (the audit trail).

tool_result

since stage 3

`agentId`, `callId`, `output` (always a string, clipped surrogate-safely), `isError` (true iff the output starts `ERROR:`), `durationMs`.

agent_spawn

additive · stage 5

`agentId` (the *child*), `parentId`, `task`. The tree edge — the whole agent hierarchy lives in these two id fields.

compaction

additive · stage 4

`agentId`, `removedTurns`, `summaryChars`. An audit line only: compaction replaces the *in-memory* history — the file is never rewritten.

context_cleared

additive · card 471

`agentId`, `removedMessages`. Written when the operator types `/clear` in the web chat. Nothing before it is removed from the file; a resume rebuilds the provider history only from the events after the main agent's last marker.

playbook_progress

additive · card 554

`agentId` (`"playbook"`), `runId`, `phase`. A playbook run tells the chat of its session three things. Phase `start` carries `playbook` (the name) and `task` (what the run was asked, absent without one); phase `step` carries `node`, `name`, `provider` and `model`; phase `end` carries `stopReason`, `result` and `detail`. The full record of the run stays in its sidecar.

playbook_picked

additive · card 554

`dir`, `name`, `none`. The playbook the session picked, in the header dropdown, with `/playbook use` or by starting a run. A wake and a resume read the last one back; a line without `dir` withdraws the pick, and a line with `none: true` is the explicit "No playbook", which no pinned folder stands in for.

run_step

additive · card 586

`agentId` (`"playbook"` for both kinds), `runId`, `runKind` (`playbook` or `workflow`), `phase`. One line per moment of a playbook or workflow run, for the run panel beside the chat; `playbook_progress` and `plan` are written as before. Phase `run_start` carries `name`, `task` (absent without one), `nodes` (the forward path, `[{id, name, kind}]`) and `workspaceFingerprint` (the first 16 hex digits of the SHA-256 of the run's real working folder, never the path). Phase `step_start` carries `node`, `name`, `kind` (`step`, `decision`, `prompt` or `fanout`), `entry` (1, then 2 and on for a loop), `performer` (`chat` or `child`), `role`, `provider`, `model`, `picked`, `input` with `inputChars`, `reads` (`[{doc, path}]`, the path only for a file inside the working folder) and, on a decision or a repeating workflow step, `round` and `maxRounds`. Phase `step_end` carries the same `node`, `name`, `kind` and `entry`, `status` (`done`, `failed` or `stopped`), `durationMs`, `childId` or `children`, `output` with `outputChars`, `documents` (`[{doc, path, change}]`, `change` `created` or `changed`), `label` (a decision's outcome, `exhausted` included), `error`, and `round` and `maxRounds` as at the start. Phase `run_end` carries `stopReason`, `result`, `detail` and `durationMs`. Every `step_start` has one `step_end` with the same node and entry, also when the step fails, is stopped or the host throws. A refused run writes `run_start` and `run_end` with `stopReason` `refused` and no steps. `input`, `output`, `error` and `detail` are cut at 4,000 characters; the full texts stay in the chat turn, the child's thread and the llm wire. Fields a phase does not set are left off the line. The documents of `step_end` lines are what `GET /api/run-document` opens.

voice_input

additive · bonus 2

`agentId`, `durationMs`, `model` (e.g. `"ggml-small"`). Written *before* the `run_start` of a spoken turn; pure provenance — a resumed voice turn is byte-identical to a typed one.

usage

since stage 3

`agentId`, `inputTokens`, `outputTokens`, per agent. `inputTokens` is the part of the prompt no cache served. When the provider reported a prompt cache, `cacheReadTokens` and `cacheCreationTokens` ride beside it and are omitted otherwise. The event never adds them up: the compaction trigger and the context ring do that. A provider that reports what a call cost in GitHub AI credits (`copilot`) adds `aiCredits`, the cost of that one call; it is omitted for every other provider, and the context ring shows the session's sum.

run_end

since stage 3

`runId`, `stopReason` — no `agentId`. Stop reasons: `end_turn` (regular), `unfinished` (the run answered while its `plan` still had open steps — the loop reads the ledger at the exit), `max_tokens`, `tool_use` (edge case), `aborted` (cancel/Ctrl+C), `max_turns` (the turn brake, 1000 by default), `error` (preceded by an `error` event). A run that never wrote a plan keeps `end_turn`: the absence of the `plan` event is the fact, and nobody can grade a run that never said what it was doing.

error

since stage 3

`agentId?`, `message` — the failure channel, followed by `run_end {stopReason:"error"}`. Also the web server's own error path (a refused provider switch, a too-large attachment) — always a first-class event, never a separate frame type.

`image_generated`

additive · bonus 4

`agentId`, `callId`, `prompt`, `provider`, `model`, `mediaType`, `blobPath` (relative to `~/.spectro`, e.g. `images/3f7a….png`), `sha256`. The event carries the reference; the bytes live in the content-addressed store.

`context_info`

additive · full build, opt-in

`agentId`, `turn`, `messages`, `estimatedTokens`, `threshold`, `thresholdSource` (`override` | `window` | `model` | `fallback` | `window_override`: which fact produced the threshold; absent in pre-0.9 sessions), `contextWindow` (the window that threshold was measured against; absent when none is known), `parts[]` (`{label, chars, estTokens}`: system prompt / tool schemas / conversation). Emitted once per turn when introspection is on; sizes are chars/4 *estimates* — `usage` stays the truth. Drives the context ring.

`window_override`

additive · card 390

`tokens` (absent after a clear, never 0), `threshold`, `thresholdSource`, `contextWindow` (absent when none is known), no `agentId`. The operator set or cleared the context window for this session from the context ring. The server writes the line and then sends it to the page, so the ring moves without a run. The threshold, source and window are what the harness derives from the choice at that moment: `window_override` and 70 % of the window while one is set, the automatic source after a clear. A resumed session takes its window from the **last** such line; the line adds nothing to the rebuilt conversation.

`agent_message`

additive · full build (A2A-lite)

`from`, `to`, `role`, `state`, `text`, `label?` — note the addressing is from/to, not agentId. Roles: `task` (parent→child, state `submitted`), `status` (child→parent, `working`, fed by `report_status`), `result` (child→parent, `completed` / `failed`). `label` names the dev tool (`build_plan`…) and is absent on plain spawns. Role `peer` is a message between agents: state `sent` or `received`, the message's `msg_id` in `label`, and state `returned` when a helper ended with the message still unread. A `returned` line runs from the helper to the sender (the parent's id, or `operator` for a reply typed on the page), and its `text` is the line the sender reads.

`plan`

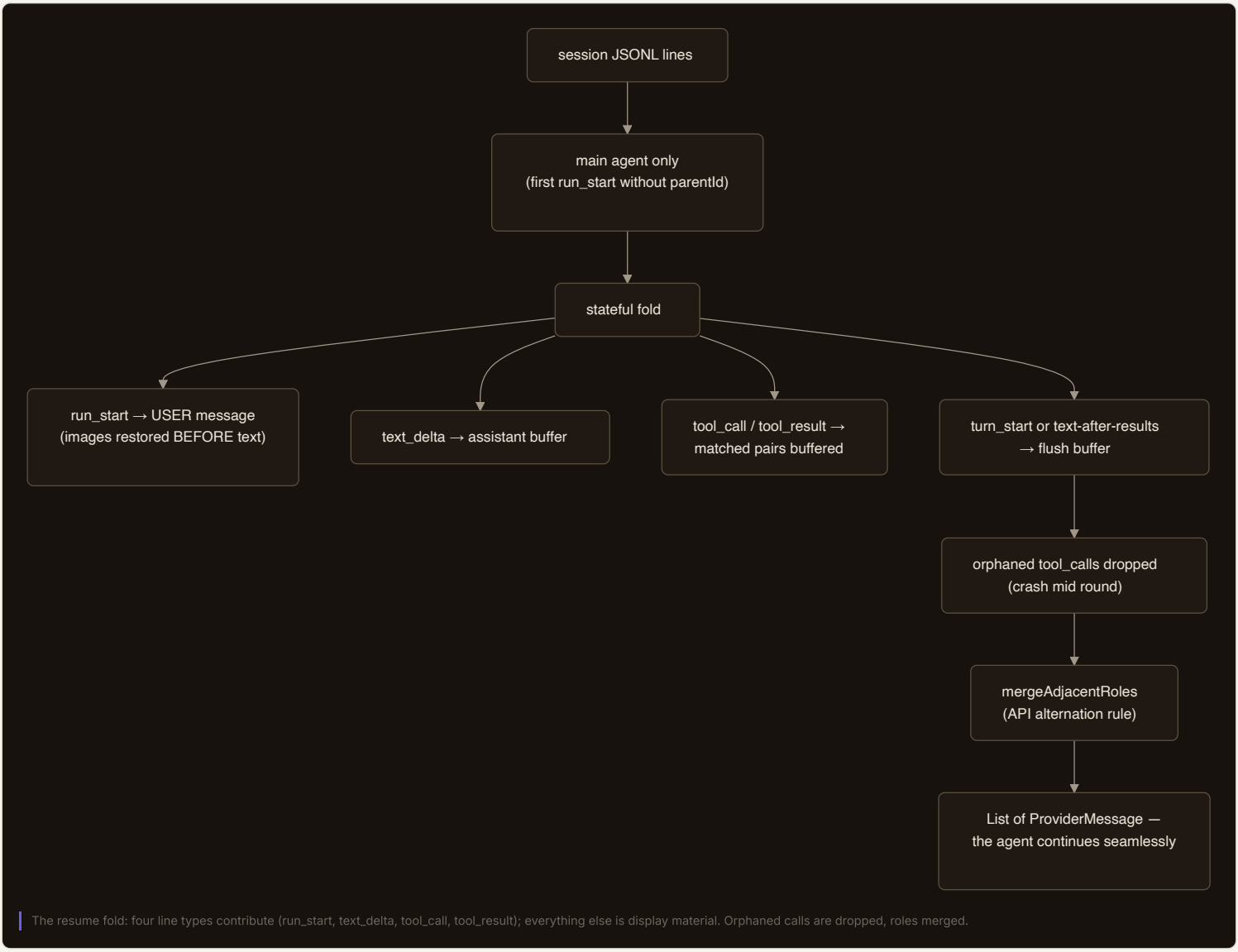
additive · full build

`agentId`, `steps[]` (`{text, status}`). Latest-wins full replacement from `update_plan`; statuses are the canonical English enum `pending / in_progress / completed`, enforced at the write boundary.

The cross-edition proof

The invariant is tested, not asserted: `CrossEditionReplayTest` replays the TypeScript edition's canonical example session verbatim — every line parses into a typed event, **re-serialization reproduces the other edition's bytes line for line**, and the resume fold reconstructs a usable history even though that file uses compact ids (`a0`) instead of `"main"`. Two interop hardenings came out of that work and are now permanent behaviour: the main agent is found *structurally* (first `run_start` without `parentId`), and text arriving after buffered tool results flushes the reconstruction even when a foreign file omits `turn_start`.

Resume: lines become a conversation



The resume fold: four line types contribute (run_start, text_delta, tool_call, tool_result); everything else is display material. Orphaned calls are dropped, roles merged.

Only four line types rebuild the provider history — everything else (`usage`, permissions, `thinking_delta`, `compaction`, `voice_input`…) is display and audit material. Child events never leak into the history: their results are already inside the parent's `tool_result`. Orphaned tool calls (a crash mid-round) are dropped because the provider API would reject the history; attachment images are restored *before* their prompt text, at their original position.

CHAPTER 23

Streams & cancellation

The Java-21 answer to the TypeScript edition's async generators: a blocking `Iterable` over a bounded queue, produced by a virtual thread, plus a cooperative cancel signal. No reactive framework anywhere.

EventStream

```
public interface EventStream extends Iterable<RunEvent>, AutoCloseable {
    void cancel();
    @Override void close();   // idempotent; also cancels
    static EventStream start(CancelSignal signal, Consumer<Consumer<RunEvent>> body) { … }
}
```

- **Backpressure for free:** a bounded queue (64) — a slow renderer blocks the producer instead of ballooning a buffer.
- **The producer** runs on a virtual thread named `spectroscope-agent`; a `finally` always enqueues the end sentinel, so a consumer's for-each can never hang, even when the body throws.
- **The sentinel** is a private instance recognized by reference identity — never handed out, never serialized.
- `close()` also interrupts a producer parked on a full queue — try-with-resources cleans up even when a consumer bails early.

MergedEventStream

Subagents write into a shared merge: one unbounded queue, many producers (the parent pump plus one forwarder per child), exactly one consumer. Unbounded on purpose — child producers must never block behind a slow renderer. This is why A2A needs no network: the “protocol between agents” is just events interleaving on the one stream that every renderer and the session store already consume.

CancelSignal

```
public final class CancelSignal {
    public synchronized void cancel();                  // idempotent, runs listeners
    public boolean isCancelled();
    public synchronized Runnable onCancel(Runnable l);  // returns a deregistration handle
}
```

The loop checks it at safe points; providers close their HTTP streams on it; running shell commands are force-killed by a per-call listener that deregisters itself afterwards (the handle return — a run's listener list must not grow per tool call). `onCancel` fires immediately if already cancelled, closing the classic registration race at spawn time. Children get their own signal, cascaded from the parent's — one Ctrl+C ends the whole tree with `run_end {stopReason:"aborted"}`, not a stack trace.

CHAPTER 24

The provider wire

Between the agent loop and any model sits one sealed vocabulary. The loop consumes five event kinds and produces one request record — everything provider-specific stays inside the adapters.

The port

```
public interface LlmProvider {
    Iterable<ProviderEvent> stream(ProviderRequest request);   // blocking, lazy
    default String providerName() { return null; }             // live label for run_start
}

record ProviderRequest(String system, List<ProviderMessage> messages,
                       List<ToolSpec> tools, int maxTokens, boolean thinking,
                       CancelSignal signal)

record ProviderMessage(Role role /* USER | ASSISTANT */, List<ProviderContent> content)

sealed interface ProviderContent
        permits TextContent, ToolCallContent, ToolResultContent, ImageContent

sealed interface ProviderEvent permits PTextDelta, PThinkingDelta, PToolCall, PUsage, PStop
record PUsage(int inputTokens, int outputTokens,
              int cacheReadTokens, int cacheCreationTokens,
              Double aiCredits /* null unless the provider reports a cost */)
record PStop(StopReason reason /* END_TURN | TOOL_USE | MAX_TOKENS | ABORTED */)
```

Contract points: all four backends return *lazy* iterables (tokens reach the UI while the HTTP stream is still open); `PUsage.inputTokens` is the part of the prompt no cache served (it feeds the wire `usage` event) while the two cache fields ride separately and are added back only by the compaction trigger and the UIs; a cancel mid-stream surfaces as `PStop(ABORTED)`, never as an exception.

Backend mappings

Anthropic (SSE via the official SDK)

- SDK client built with `maxRetries(0)` — spectroscope owns retry.
- Deltas translate one-to-one; **usage and fully-parsed tool inputs exist only in the accumulated final message** — the classic SDK trap — so `PToolCall`s, `PUsage` and `PStop` emit after the stream is exhausted.
- Extended thinking with a 2048-token budget (clamped below `maxTokens`); thinking requests carry no sampling parameters.
- Vision: image blocks are reordered before text within a user message.
- Cancel closes the SDK stream; the resulting error maps to `ABORTED` iff the signal fired.

Ollama (NDJSON)

- `POST /api/chat` read line by line; typed wire records (`ChatRequest(model, stream, messages, tools, options, think)`); tool results become `role:"tool"` messages; images ride as raw base64 arrays.
- No call ids from the server — the adapter mints `ollama-call-<nanos>`.
- Token counts arrive only in the final `done:true` chunk.
- **Thinking, two ways:** the native `message.thinking` field, plus a streaming state machine that splits inline `<think>` tags even when a tag is torn across chunk boundaries.
- **Vision fail-fast:** before sending images, `POST /api/show` checks the model's capabilities — a text-only model is refused with an actionable message instead of a confusing 400.
- Error classification at the source: retryable HTTP → `TransientProviderException`; terminal (404 model-not-pulled, 401) → `IllegalStateException` — never retried.

OpenAI-compatible (SSE)

- `data:` lines until `data: [DONE]`; keep-alive comments skipped; `stream_options.include_usage` puts usage in the final chunk.
- **Fragmented tool calls** accumulate per index until the turn closes; arguments arrive as a JSON *string* and are re-parsed (unparseable → `{"raw": …}`).
- Bearer auth only when a key is present — LM Studio and friends run keyless.
- Prompt cache (card 468): the `llamacpp` and `spectro-local` dialects send `cache_prompt: true` while `promptCaching` is on; with it off the field is left out. No other dialect gets the field. A cached share is read from `usage.prompt_tokens_details.cached_tokens`, or from llama.cpp's `timings.cache_n` and `timings.prompt_n` when the usage has no details; it leaves `inputTokens` and arrives as `cacheReadTokens`, the split Anthropic reports.
- Endpoint by provider name, no key-based swap: `openai` → `https://api.openai.com`, `lmstudio` → `http://localhost:1234`, `llamacpp` → `http://localhost:8080`, `openrouter` → `https://openrouter.ai/api`, `gemini` → the Google gateway; an explicit `baseUrl` always wins, and the local ids read their own key first (chapter 18). One wire protocol, one host per id.

Copilot (JSON-RPC to a local runtime, through the Copilot SDK)

- The SDK starts the Copilot CLI as a child process; what a call sends goes from there to GitHub's Copilot service, so the host the app names is `api.githubcopilot.com`, or the host the runtime reports for a data-residency account. The runtime keeps one session per conversation and is sent only the new message.
- The runtime's own tools are off and every harness tool replaces the runtime tool of the same name, so the harness runs every tool.
- `assistant.usage` carries the tokens and `copilotUsage.totalNanoAiu`. Divided by 1e9 that is `PUsage.aiCredits`, the cost of that one call in GitHub AI credits, set only when the runtime calls the cost `complete`. The premium request multiplier (`cost`) is never read. The session's credit total adds these per-call amounts, the title call included; the runtime's `session.usage_checkpoint` carries the running total, which the app does not read.

CHAPTER 25

The WebSocket protocol

One endpoint (`/ws`), one connection per browser tab, one agent per connection. Server→client traffic is *nothing but serialized RunEvents* — there is no separate frame vocabulary in that direction. Client→server is six small frames.

Client → server

FRAME	SHAPE	SEMANTICS
`user_message`	`{type, text, attachments?: [{mediaType, dataBase64}]}`	starts a run (refused with an error event while one is active). Attachments are decoded and stored *before* the run starts; 5 MB cap per attachment; supported: jpeg/png/webp/gif
`permission_response`	`{type, callId, allowed, remember?, persist?}`	answers a parked permission future. `remember` adds a session rule, `persist` appends it to `.spectro/settings.json`
`abort`	`{type}`	fires the current run's CancelSignal
`set_provider`	`{type, provider, model?}`	mid-session backend switch; validated; refused without a key; applies on the next run
`set_image_provider`	`{type, provider}`	`gemini` | `openai` ; effective on the next generation
`set_thinking`	`{type, enabled}`	reasoning visibility; applies on the next run
`set_playbook`	`{type, dir|null, none?}`	card 554: the session's pick from the header dropdown or `/playbook use` ; only a registered folder; `null` withdraws it; `null` with `none: true` picks "No playbook", and the folder pinned for the workspace no longer stands in; written as `playbook_picked` and answered with `playbook_info`
`confirm_playbook`	`{type, dir?, prompt}`	card 554: `/run-with-playbook <prompt>` asks for the confirmation sheet; the folder is the frame's `dir` , else the session's pick, else the folder pinned for the workspace; answered with `playbook_confirm` ; starts nothing. Since card 584 the server first asks every known provider for its model list, side by side and off the socket thread, so the answer can take up to one check budget (6 s) while every other frame of the session is still answered
`start_playbook`	`{type, dir, hash, prompt, picks?, ask?}`	the sheet's Start; `hash` must match the bytes; since card 554 `prompt` is required and becomes the run's task in every step. Card 584: `picks` is `{<node id>: {provider, model}}` , the model picked in the sheet for a step or a review check, for this run only. The server reads it as untrusted input after the hash: an object of at most 256 picks of plain text, a node of the pinned playbook that runs on a model, a provider the app knows, and a model that serves on a fresh check of its provider; a private step never takes a cloud provider or an Ollama cloud model (a tag `cloud` or one ending in `-cloud`). A pick equal to the step's own model is no pick. A refusal is an `error` event and starts nothing; for a start that carried `ask` , the page's id for it, the refusal also comes as `playbook_start_refused` . An `abort` while the picks are checked again stops the start before any run begins. The picks are recorded in the run's sidecar and never written to the playbook folder
`start_workflow`	`{type, name, input, lang, dir?}`	card 554: a workflow of the session's playbook by name (the file below `workflows/` without its extension), started at once; an unknown name is refused with the names the playbook has
`get_session_items`	`{type}`	card 558: asks for the session's skills, commands and agents; answered with `session_items` ; reads only

Nuances the names hide: the client frame is `permission_response` — `permission_decision` is the *RunEvent* that flows back and into the file. Cancel is spelled `abort` on the wire. Resume is *not* a frame: it is the `?resume=<id>` query parameter at connect time (kept across auto-reconnects). The web UI connects that way when you send the first message into a stored session. Opening its row before that is pure REST.

Server → client

Every event of a run goes to the socket *and* the session file — the same object. Server-side errors (invalid frame, refused switch, oversized attachment) arrive as first-class `error` RunEvents. One synchronized writer per connection; a dead socket swallows sends — "a dead socket is not a run failure; the JSONL already has it."

A few frames are for the page only and never reach the file. Card 554 adds two: `playbook_info {dir, name, found, registered, none}` names the session's picked playbook (`found` only for a registered folder that still holds a playbook, `none` for the explicit "No playbook"), and `playbook_confirm {dir, prompt, preview}` carries what the confirmation sheet draws. Since card 584 the socket's preview comes from a fresh model list check of every known provider and carries two more lists: `models` , one row per step and per review check `{node, name, kind, choice, privacy, provider, model, serves, viaProvider, viaModel, reason, httpStatus, keySource}` , where `reason` is one word for why the playbook's model does not serve (`rejected-key` , `refused` , `timeout` , `http-<status>` , `bad-answer` , `not-listed` , `private-on-cloud` , `needs-key` , `needs-signin` , `needs-download` , `check-timeout` , `check-error` , `configured` , `unknown`) and `keySource` says for a rejected key whether it came from the environment of the app or was saved in it, never the key; and `offers` , one row `{provider, kind, models}` per provider whose list answered. A private step on a cloud model is that row's reason, which a local pick lifts, and no longer one of the `refusals` . `GET /api/playbooks/start-preview` has the same shape but reads the stored answers without a request, so its rows say `configured` for every provider nobody checked, and its `refusals` leave the private step rule to the rows as well. Card 584 also adds `playbook_start_refused {ask, message}` , the refusal of one start for the sheet that sent it, so the sheet shows only the refusal of its own start; the chat has the same message as an `error` .

Card 558 adds `session_items {playbook, items}`, sent after every `playbook_info` and on `get_session_items`: the skills, commands and agents the session's next turn reads. `playbook` is `{dir, id, name}` of the picked playbook whose items are in the list, or `null`; each item is `{kind, name, plain, source, namespace, description}`, with `kind` `skill`, `command` or `agent` and `source` `workspace`, `playbook` or `user`. A name that exists once is plain; a name that exists in more than one source is `<namespace>:<plain>` in every one of them, the playbook's namespace being its id. The playbook's items are read from its folder in place; nothing is copied.

Threading model

- The Tomcat thread only parses frames; every run executes on a virtual thread (`spectroscope-run`).
- Permission waits *park* the agent's virtual thread on a `CompletableFuture` keyed by `callId` — cheap, and the browser dialog answers with the same id. On disconnect every pending future completes with *deny*, so no thread ever hangs.
- Two tabs = two connections = two agents, two session files, two independent remembered-rule lists. Only the persisted settings file is shared — its writer is serialized.
- Text frames up to 16 MB (base64 images need the headroom; the Spring default of 8 kB would close the socket with status 1009).

CHAPTER 26

The REST API

The session stream is read-only over REST — the socket carries run mutations — but a handful of control writes exist: the transcription POST, the guarded session DELETE, the three settings PUTs, the workspace picker and the image copy-to-workspace POST (plus the fleet control POSTs). Bound to 127.0.0.1; auth and TLS are a deliberate course boundary. Every route below except `GET /api/health` passes a single filter first, which wants a loopback peer *and* a localhost `Host` header: a page rebound from some other name to 127.0.0.1 reaches the port but cannot forge that header, so it reads a blank 404. The per-row notes name the extra guards each endpoint adds on top.

ENDPOINT	RETURNS	NOTES & GUARDS
`GET /api/health`	`{"status":"ok"}`	the desktop shell's startup probe
`GET /api/sessions`	`[{id, startedAt, firstPrompt, tokens, provider}]`	fold over the sessions dir; broken files skipped
`GET /api/sessions/{id}/events`	the session's RunEvents as a JSON array	replay + resume seeding; torn last line dropped
`DELETE /api/sessions/{id}`	204 · 400 · 404	**the one destructive endpoint**: id-shape guard (`[A-Za-z0-9][A-Za-z0-9-]*`) → 400; unknown → 404; the store additionally refuses ids that do not resolve to a direct child of the sessions dir. Removes JSONL + blob folder
`GET /api/config`	`{provider, model}`	boot truth; live switches are client-side overlay
`GET /api/commands`	`[{id, name, sub?, aliases, takesArgs, group, surfaces, developerOnly, wired, help: {en, de}, usage?: {en, de}}]`	card 554: the one list of chat commands; the web builds its `/help` , its slash picker and the Skills and Commands page from it, the REPL prints the same list. Card 588: `help` in English and German, and `usage` , the template of a command that takes words such as `/graph ask <question>` , in both languages; `null` for a command that takes none
`GET /api/context`	system prompt, tools, skills, MCP servers, thinking, provider/model, subagent role profiles	stateless — assembled from the same constants the live agent uses; no agent built, MCP not connected
`GET /api/models?provider=`	`[string]`	ollama: live from `/api/tags` with finite 1.5 s/2.5 s timeouts (a stalled Ollama must not pin a worker); cloud: curated; errors → `[]`
`GET /api/images/{file}`	image bytes	name must match `[0-9a-f]{64}\.(png|jpg|webp)` — 400 *before* any filesystem access; the content address is the traversal guard
`GET /api/jobs/state`	`{jobId: JobState}`	cron outcomes; corrupt file → `{}` ; polled by the desktop shell
`GET /api/files`	the sandboxed workspace tree	that session's resolved workspace only (no session, no tree); hidden + ignored dirs skipped; depth ≤ 8; ≤ 2000 entries with an honest `truncated` flag; local-origin fenced
`GET /api/file?path=`	one file for preview	`resolveInside` compares REAL paths (no traversal, no symlink escape) + per-segment hidden/ignored re-check (the `.env` answers 404 even by direct URL); every document served with `Content-Security-Policy: sandbox allow-scripts` ; 413 oversized, 415 binary; local-origin fenced
`GET /api/run-document?session=&run=&path=`	one document a run wrote, for preview	card 586: only a path a `run_step` `step_end` line of that run in the session's own file names as a written document, dot folders such as `.superpowers/` included; when the run's `run_start` carries `workspaceFingerprint` , the session's current real folder must match it; never, in any letter case, on the asked and the real path: `.env` , `.env.*` , `.env-*` , `.envrc` , `.netrc` , `.npmrc` , `*.pem` , `*.key` , `*.p12` , `*.pfx` , or a `.git` , `.aws` or `.ssh` segment; then the sandbox and the answer of `/api/file` . 400 for a malformed session or run id; 404 for everything refused, so a refusal and a missing file read the same; the web asks only on a press, never polls; local-origin fenced
`POST /api/transcribe`	`{text}`	browser audio → ffmpeg 16 kHz mono → whisper-cli; 503 with a setup hint when STT is not installed; the transcript goes to the composer, never to the agent
`GET /api/claude/transcripts`	`{limitBytes, truncated, transcripts: [{path, project, file, size, modifiedAt, loadable}]}`	lists `~/.claude/projects/**/*.jsonl` (newest first, ≤ 300 rows, `truncated` says when that cap fired); `limitBytes` is the ceiling the content endpoint enforces and `loadable` is its per-row verdict; local-origin fenced
`GET /api/claude/transcripts/content?path=`	raw JSONL	must end `.jsonl` ; canonical real-path must stay inside the real base (no traversal, no symlink escape); 128 MiB cap, streamed; local-origin fenced

CHAPTER 26B

The llm-wire

The session file records what the app said about a run; this file records what the backend actually posted to a model and what came back, verbatim. One NDJSON sidecar per session, written at send time — a crash mid-generation still leaves the request on record.

THE LLM WIRE

the one exchange that costs money

The session file is spectroscope's own protocol, already summarised. What the model was actually asked, and what it actually answered, is the expensive exchange — and it now lands beside the session as a third file, verbatim.

ONE EXCHANGE · THE RECORDER LISTENS, AND THE WIRE IS NEVER INTERRUPTED

there is no switch

The server, the CLI and the headless runner each construct a recorder unconditionally. There is no flag, no environment variable and no settings key that turns it off.

A library caller is the one exception: AgentOptions.llmWire is nullable, and null records nothing.

LlmWireRecorder — one file per session, appended, flushed as written

`begin(WireRequest)` — **appends llm_request AT SEND TIME**
a crash mid-stream still leaves the request on record

`end(WireOutcome)` — **appends llm_response WHEN IT CLOSES**
lines for a stream, body for a single payload

The two halves pair by xid, never by position — parallel subagents share one recorder, and every line is one atomic append with no handle held between lines. A write failure prints one stderr line and never kills the run.

256 MiB, and it latches

The first payload that would cross the ceiling writes ONE llm_wire_truncated marker. From then on every body is dropped with omitted:"ceiling" — while the ledger row keeps its measured bodyBytes, so the count of calls stays true even when the content stops.

THREE LINE TYPES · AND THE ONLY THING THE WRITER STOPS

llm_request
written at send time
fields, in declaration order
`type · xid · agentId · turn · kind · provider · model · transport · method · url · headers · fidelity · body · bodyBytes · omitted · ts`

llm_response
written when the exchange closes
fields, in declaration order
`type · xid · agentId · turn · status · fidelity · lines · body · lineCount · bodyBytes · aborted · error · durationMs · omitted · ts`

llm_wire_truncated
written once, at the ceiling
fields, in declaration order
`type · reachedBytes · ceilingBytes · ts`

what the writer stops
credential header VALUES, matched case-insensitively by NAME

authorization · proxy-authorization · x-api-key · api-key · x-goog-api-key · cookie · set-cookie

`authorization: <the value the provider set>` — as the provider set it
`authorization: REDACTED(<n> chars)` — as it lands on disk
`content-type: application/json · passes through, untouched`

Names stay, values go, and the map is key-sorted so the file is deterministic. n counts UTF-16 code units, which is a small oracle and a recorded open finding: the graph layer already reports a coarse byte bucket instead, and the wire does not yet.

Serialization is NON_NULL, so an absent field is simply missing from the line rather than present as null: a status the backend never sent leaves no key behind. And transport is the recorded label, not a closed set — sdk, http, websocket and process all appear on disk.

fidelity — the label is a promise somebody measured, not a hope

bytes	the recorded string IS what went over the socket
sdk-json	the SDK owned the socket; the request body is byte-equal, loopback-proven
sdk-events	rebuilt from the SDK's typed events — content-equal per event, keep-alives absent
encoded	the recording's own base64 of real input bytes that never rode a socket as a string

Bodies are never redacted, and the twelve pattern rules are not this file's.

The full system prompt, the entire replayed history, the tool schemas and base64 images ride verbatim — that is the file's purpose, and it makes the sidecar exactly as sensitive as the conversation itself. The pattern rules (anthropic-key, jwt, iban and nine more) belong to graph/Redaction.java and are applied to <stem>.state.jsonl; nothing pattern-matches inside a wire body.

THE FAMILY · WHAT LANDS BESIDE ONE RUN, AND WHAT SWITCHES IT ON

`~/.spectro/sessions/<id>.jsonl` — the app's own protocol: one appended line per RunEvent — the archive you reopen	always: the session is this file
`~/.spectro/llm-wire/<id>.llm.jsonl` — the backend exchange, verbatim: one line out, one line back, paired by xid	always on in the server, the CLI and the headless runner
`<stem>.graph.jsonl` — the topology at compile(), then the seven lifecycle records of the walk	when a caller hands compile() a sink
`<stem>.state.jsonl` — what the nodes wrote, cut to the tier's caps	off until a caller names a tier

The wire file sits NEXT TO sessions/, not inside it: the session list reads every sessions/*.jsonl in full, and a wire file can be orders of magnitude larger than the session it records.

Voice has no session, so both speech routes share a day file instead — stt-<yyyy-MM-dd>.llm.jsonl, in the same directory, belonging to no session at all.

A finished exchange also becomes a line of the session file, as the llm_exchange RunEvent, appended to the file first and mirrored to the socket second — so a reopened archive still knows every model call it made, and the bodies stay in the sidecar.

The graph pair has no fixed home: the caller hands one stem to the two writers and they re-point it, which is exactly how the bundled demo scenarios were produced — by the engine's own test, not by hand.

HOW THE FILE IS READ, AND HOW IT GOES AWAY · THE THREE READS ARE FENCED TO LOOPBACK

GET	`/api/sessions/{id}/llm-wire`	the whole file, one application/x-ndjson attachment
GET	`/api/sessions/{id}/llm-wire/index`	the bodiless ledger — bodies are read past and never returned
GET	`/api/sessions/{id}/llm-wire/exchange/{xid}`	one exchange, both lines, parsed — fetched on the gesture that opens it
DELETE	`/api/sessions/{id}`	cascades to the sidecar unconditionally, so an orphan is deletable too

In the browser there is no wire tab: the surface is the session's trace tab, where one exchange becomes three rows and an expanded row offers three faces. The whole file downloads from the archive bar of a stored session, beside export, and only when the index answered non-empty — so the link never names an empty file.

the network boundary · the exchange itself ▢ the recorder and the session's own stream ▢ what is guarded, and what is not -- dotted = the tap: a copy, not a hop

generated: build_22_llm_wire.py · source-verified 2026-07-20

One exchange end to end: the request line written the moment the call leaves, the stream lines buffered as they arrive, the response line appended at close, the two halves joined by xid, and the three places a reader meets that pair again, the trace rows, the second spectrum baseline and the download link of a stored session.

The sidecar file

One file per session at `~/.spectro/llm-wire/<session-id>.llm.jsonl` — beside the sessions directory and deliberately not inside it, because the session list folds every `sessions/*.jsonl` in full and a wire file can be orders of magnitude larger than the session it records. The id is never trusted as a path: the one path rule the recorder and all three read endpoints share refuses anything that is not a plain basename (`[A-Za-z0-9][A-Za-z0-9-]*`), so a `?resume=` parameter cannot steer the sidecar into another directory.

Voice happens before any session exists, so both speech routes share a day file named `stt-<yyyy-MM-dd>.llm.jsonl` in the same folder — the batch transcription POST and the live socket append to the same one. The sidecar is per session everywhere else, per day for speech, and a reader hunting their dictation under a session id will not find it.

Every line is one atomic write. The recorder opens the file with `O_APPEND`, writes the whole serialized line in a single channel write and closes again, holding no handle in between, so two recorders on the same file — the speech day file, a doubly resumed session — cannot interleave fragments mid-line. A write failure prints one line to stderr and is silent afterwards: the record is an additive mirror of a run and must never be the thing that kills it.

The ceiling is 256 MiB per file and it latches. The first payload that would cross it writes a single `llm_wire_truncated` line carrying `reachedBytes` and `ceilingBytes`; from then on every body is dropped and its line says `omitted:"ceiling"` while the ledger keeps the measured `bodyBytes` — the history of the calls survives even when their contents do not. The byte count is seeded from a file that already exists, so a resumed session gets no fresh allowance, and a file already past the ceiling latches without a second marker because the line that crossed it wrote one.

The three line types

An exchange is two lines. `llm_request` is written the moment the call leaves, `llm_response` when it closes, and the two are joined by `xid` — a UUID, never a position, because parallel subagents share one recorder and their lines interleave. Serialization is `NON_NULL` throughout: an absent field is missing from the line, never present as `null`.

`llm_request`

written at send time

FIELD	TYPE	
`xid`	string	UUID — the only join between the two halves of one exchange
`agentId`	string	the calling agent; the provider itself knows neither agent nor turn
`turn`	int?	1-based; absent where no turn exists (speech)
`kind`	string	`chat` | `compaction` | `image` | `stt` — the call site's knowledge, not the provider's
`provider` · `model`	string	the backend label as the session knows it, and the model id the request names
`transport`	string	who owned the socket
`method`	string?	`POST` on every HTTP and SDK route; absent where no verb exists (the speech socket, the local process)
`url`	string	the full request URL; a route with no host names itself there instead
`headers`	object?	the headers as the provider set them, credential values redacted, key-sorted; absent when none were recorded
`fidelity`	string	what the recorded body is worth
`body`	string?	the request body, verbatim; absent past the ceiling
`bodyBytes`	number	UTF-8 size of that body — measured whether or not the body survives
`omitted`	string?	`"ceiling"`, and nothing else today
`ts`	number	epoch millis at send time

`llm_response`

written at close

FIELD	TYPE	
`xid` · `agentId` · `turn`		as on the request; the `xid` is what pairs them
`status`	int?	the HTTP status — absent when the connection never answered, and absent on a natural Anthropic finish, where the SDK exposes none
`fidelity`	string	this half's own label, which may differ from the request's
`lines`	string[]?	the received stream lines, verbatim, without their trailing newline
`body`	string?	a single-payload response (image generation, an error detail) instead of lines — the two are mutually exclusive
`lineCount`	int?	appears only when a streamed body was dropped at the ceiling; while the lines themselves are on the line, counting them is the reader's job
`bodyBytes`	number	UTF-8 size of the body, or of the summed lines
`aborted`	bool?	`true` when a cancel tore the stream down; absent otherwise, never `false`
`error`	string?	the failure in one line; absent on a clean close
`durationMs`	number	close minus send, computed from the request's own stamp
`omitted` · `ts`		as on the request; `ts` is the close

`llm_wire_truncated`

at most once per file

`reachedBytes`, `ceilingBytes`, `ts` — no `xid`, because the marker belongs to the file and not to an exchange. It means “bodies stop here”, and the ledger rows after it are still complete.

An aborted exchange records the lines that were consumed before the teardown. Bytes the server had sent but nothing had read are dropped on purpose — draining them to make the record look tidier would resurrect the slow-cancel bug that the abort path exists to avoid.

A finished exchange is written a third time, into the session file itself, as the additive RunEvent `llm_exchange` — file first, socket second — so a session reopened next month still knows every model call it made even when the sidecar has been deleted. It carries the summary and no bodies. The `llm_request` frame is the counterexample and the one exception to chapter 25's rule that server-to-client traffic is nothing but serialized RunEvents: it is a hand-built frame, sent and never stored, because the only facts that exist at send time are the ones that say nothing about how the call went.

Fidelity and transport

Fidelity is part of the record and never an implication. Each value is a separately measured promise about what the recorded string is worth, and the two halves of one exchange can carry different labels.

FIDELITY	THE PROMISE
`bytes`	the recorded string *is* what went over the socket — the harness's own HTTP providers post the exact string they hand the tap
`sdk-json`	the SDK owned the socket and the recorded request body is byte-equal to what it posts, proven over loopback
`sdk-events`	the stream reconstructed from the SDK's typed events: content-equal per event, field order not guaranteed, keep-alives absent
`encoded`	the recording's own base64 of real input bytes that never rode a socket as a string — the speech audio

Anthropic therefore writes `sdk-json` on the request and `sdk-events` on the response, while Ollama and the OpenAI-compatible provider write `bytes` on both halves: they own their connection and record the string they post. The detail pane prints one honest sentence per fidelity, per side, rather than one verdict for the exchange.

`transport` is the second recorded label and names who owned the socket: `sdk` a vendor SDK, `http` the harness's own client, `websocket` the live speech route, `process` the local `whisper-cli` — whose URL is spelled `process://whisper-cli` and whose trace row prints the executable instead of an invented HTTP verb. No comment in the source lists all four; the field is the word the call site chose, and those are the words on disk.

Always on, with one exception

There is no flag, no environment variable and no settings key. The web server opens a recorder wherever it mints a session store, fresh and resumed alike; the command line opens one beside its own store; the headless and cron runner does the same. Recording the backend conversation is what running spectroscope means, not a mode you enter — and the state-graph file that ships in the same release is the opposite case, where value recording stays off until you turn it on.

The exception is the library. `AgentOptions.llmWire` is nullable, and a provider that gets no tap records nothing and behaves byte-identically to a build without the feature; the pre-wire constructor arity still exists for exactly that. An embedder who wants the record hands the builder an `LlmWireRecorder`, and a session id is all the recorder needs to place the file.

What is redacted

Exactly one thing: the value of a credential header. Seven names are matched case-insensitively — `authorization`, `proxy-authorization`, `x-api-key`, `api-key`, `x-goog-api-key`, `cookie`, `set-cookie` — and each match's value becomes the literal `REDACTED(n chars)`. Every other header passes through untouched, and the map is sorted by key so the same request writes the same line twice.

The rule lives in the recorder and not in the adapters: a provider hands over the headers it really set, real values included, so no adapter can forget to scrub. Two ordinary local setups therefore record no credential at all — a keyless LM Studio sets nothing but `Content-Type`, and the Anthropic path records no headers whatsoever because the SDK owns the socket, which the detail pane says in those words: “no headers recorded — the SDK owns the socket here.”

> NOTHING IN THE BODY IS SCRUBBED
>
> Bodies are recorded as they are: the full system prompt, the entire replayed history, every tool schema, base64 images and documents, the compaction summary. A secret pasted into a prompt is a secret in this file. The twelve pattern rules that *do* catch credentials by shape — private keys, provider keys, bearer strings, JWTs and the rest — belong to a different file, the state graph's `.state.jsonl`, where the clipper applies them to recorded values; they never run over the llm-wire. Treat `~/.spectro/llm-wire/` as being exactly as sensitive as the conversations it holds.

Reading a recorded exchange

There is no llm-wire tab. The surface is the session's `trace` tab, where a finished exchange arrives as three consecutive rows: `llm_request` with an ↑ at the instant it was sent, `llm_response` with a ↓ at the instant it closed, and `llm_exchange` as the closing summary. Only the first and the third are lines anywhere — the response row is manufactured for display, half a step in front of the summary, and exists in no file.

Two filters find them. `llm` is a filter chip of its own rather than a fold into `other`, and beside it sits an LLM-direction filter with three settings: to the LLM (↑ request), from the LLM (↓ response), harness-internal (·). A vertical glyph means the row actually left this machine; the dot means it never did.

An expanded llm row offers three faces where other rows offer four. **Structured** breaks the request into its parts — the system prompt with its character count, the messages with their per-message block counts, the tool schemas — and a body in a shape it does not recognize says so and hands over to the tree, because an empty pane would read as “the request was empty”. **Insight** is a tree over the parsed body. **Wire** prints the request line as HTTP writes it, then the recorded headers, then the body, then the response one numbered line at a time. **Source** is not offered for these three types: the endpoint re-serializes parsed nodes, so the line this frame was read from, byte for byte, is not a thing that exists here.

Two caps keep that pane readable and both announce themselves. The response stops at 200 lines with “showing X of Y response lines.” and a button for the rest. Base64 is never printed anywhere: each run of it is cut out and replaced by a mark carrying the length measured on the recorded line. The answer is not reassembled either — the reassembled text is the chat, and a second reassembly in the browser would be a second truth. Bodies never travel on the socket at all; the live frames carry metadata only and the pane fetches one exchange's bodies on the gesture that opens it.

Spectrum draws a second baseline under every agent lane, carrying that agent's exchanges on the same time axis as the app-protocol ticks above it, so a call sits under the moment that caused it. The two tracks stay apart on purpose: the app protocol and the conversation with the model are two stories about one moment. A lane whose agent never called a model is empty rather than absent, and a mark is coloured by what was measured and nothing else — it came back, or it never answered.

The whole file downloads as a plain link labelled `llm wire` next to `download .jsonl`, saving as `<session-id>.llm.jsonl`. It is offered only when the index answered non-empty, so the link never names an empty file. Both links sit on the bar of a stored session that opens read-only, which happens while another window is working in it. A session you can type in shows the composer instead and has no download links. For such a session, read the route directly (`GET /api/sessions/{id}/llm-wire`, below).

The three reads

All three sit behind the same front door as the session reads of chapter 26: a loopback peer, a localhost `Host` header and an `Origin` that is loopback or absent. Every refusal answers 404 and explains nothing — a foreign caller, a malformed id and a session that never called a model are indistinguishable from outside.

ENDPOINT	RETURNS	NOTES & GUARDS
`GET /api/sessions/{id}/llm-wire`	the sidecar verbatim	the wire-record twin of the session export: `application/x-ndjson`, `X-Content-Type-Options: nosniff`, served as an attachment named `<id>.llm.jsonl` — recorded model traffic is caller-shaped text and must never reach an HTML parsing context
`GET /api/sessions/{id}/llm-wire/index`	the bodiless ledger: one object per exchange, in file order	the two halves pair by `xid` *while* the file streams past; bodies and stream lines are read, counted and never returned, because one sidecar can dwarf the session it records. A torn tail line is skipped; an exchange still open reports `status` null
`GET /api/sessions/{id}/llm-wire/exchange/{xid}`	`{request, response}`, both lines parsed	the drill-in behind one ledger row: `xid` must be UUID-shaped, and the scan stops as soon as both lines are found, so an early exchange in a huge file never costs a full read. `response` is null while the exchange is open
`DELETE /api/sessions/{id}`	204 · 400 · 404	the session delete of chapter 26 cascades here **unconditionally**, not gated on the session file existing — which is what makes the speech day files and an orphaned sidecar deletable at all. 404 only when neither the session nor a sidecar was there

What it does not record

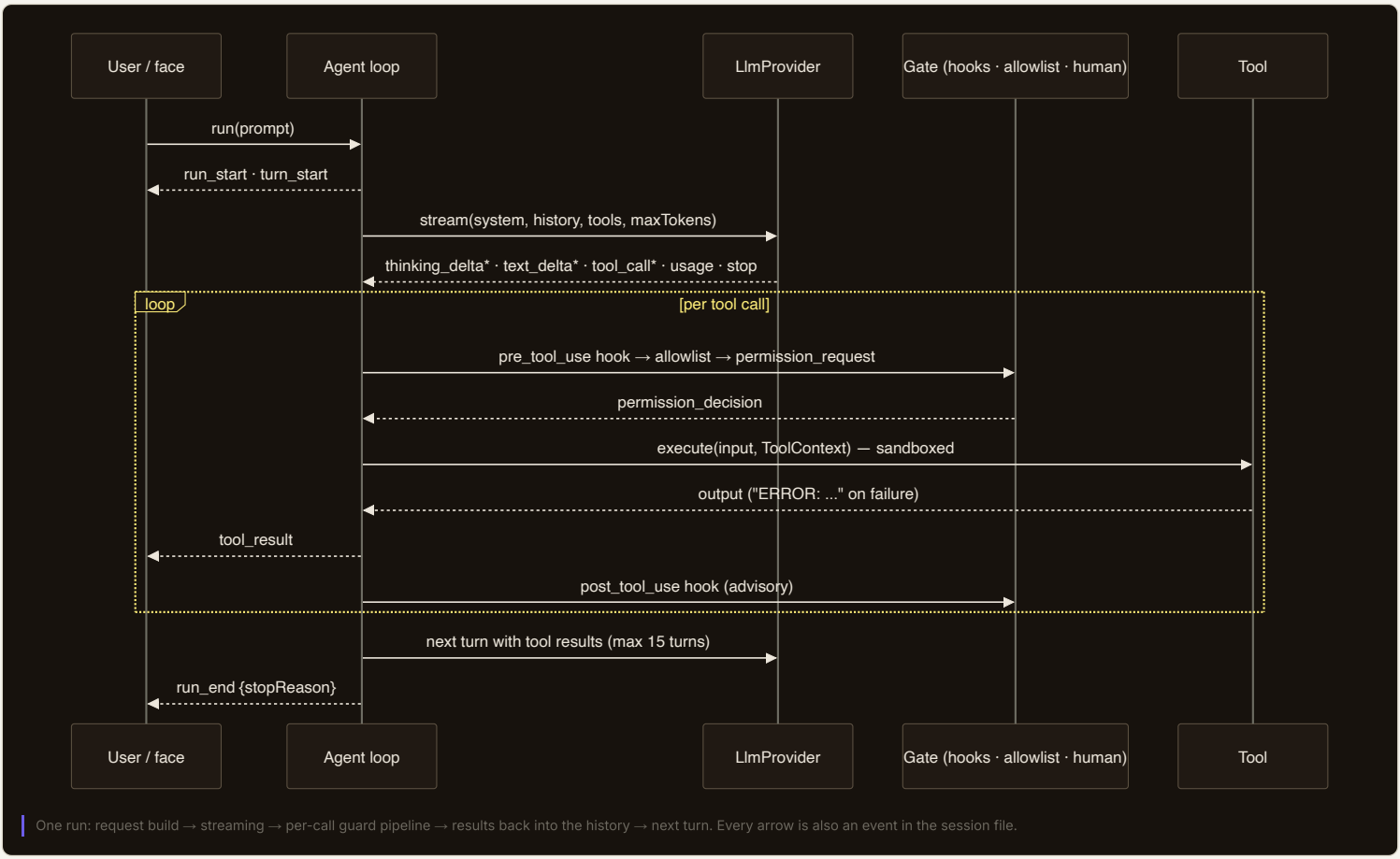
The session-less endpoints — explain and translate — and the orchestrator's panel lanes are declared, not captured, and the interface says so rather than implying coverage it does not have. A live speech session is complete about what the model said and deliberately incomplete about what it heard: the spoken bytes ride the socket in pieces and are not on the request line, which the record names instead of hiding. What the file holds is the model traffic of a session — not every model call the application makes.

One finding is open, recorded rather than quietly carried: the header redaction prints the credential's exact length. The number in `REDACTED(n chars)` narrows the search for anyone who obtains the file, and it is taken with Java's `String.length()`, so it counts UTF-16 code units and not UTF-8 bytes — equal for an ASCII key, unequal in general, and not a byte count whatever it looks like. The graph layer, reviewed in the same week, already answers with a coarse band instead — `1-8`, `9-16`, `17-32`, `33-64`, `65-128`, `129+`, measured in UTF-8 bytes — and its own source names the exact length here as the reason. The wire is to follow; until it does, that number is real.

CHAPTER 27

The agent loop, hop by hop

The whole harness reduces to one loop in one class: build a request, stream the response, execute the tool calls behind the guards, feed the results back, repeat — at most fifteen times. Everything else in this book hangs off one of these hops.



One run: request build → streaming → per-call guard pipeline → results back into the history → next turn. Every arrow is also an event in the session file.

The turn, precisely

1. **Run start.** Fresh UUID, `run_start` with the *live* provider label (a switched backend reports itself truthfully). Attachments convert to image content *before* the prompt text in the first user message.
2. **Per turn** (1…15):
   a. `turn_start`; with introspection on, a `context_info` estimate of what the next call will carry;
   b. the compaction check (chapter 28) — a no-op below the threshold;
   c. the request: system prompt, full history copy, tool specs, maxTokens (default 32 000, lowered to fit the context window when the window is known, card 488), thinking flag, the cancel signal;
   d. streaming: text deltas emit immediately; thinking deltas emit but never join the history; tool calls collect; usage updates the context measure; the stop reason records;
   e. abort check — a cancelled run ends here with `aborted`;
   f. history append: the assistant message (text + tool-call blocks) goes in *before* any results — the API's ordering rule;
   g. no tool calls → the exit reads the run's own `plan` ledger and `run_end` carries the mapped stop reason — or `unfinished`, when the last `update_plan` still had open steps and the mapped reason would have been `end_turn`. Only that one value is displaced: a brake, a token cap or an abort already says a limit intervened, and the verdict must not eat the reason;
   h. the tool round: every call passes the guard pipeline below, in order; all results of the round return to the model in *one* user message — denials and errors included, as data the model can read and react to.
3. **The plan is still open and the leash has budget** → the harness writes its own message naming the steps that are still open and starts *another turn in the same loop*, recording `continuation {decision:"continued"}`. Two things it will not do: continue a run that has nothing to show since the last continuation (`decision:"no_progress"`), and continue past its budget (`decision:"budget_exhausted"`, and the run then ends as `unfinished_after_continuations`). A continuation never resets the turn counter, so the cap below is the whole ceiling.
4. **The turn cap exceeded** → `run_end {stopReason:"max_turns"}` — the runaway brake. `15` as shipped, settable per agent.
5. **Any exception** → `error` + `run_end {stopReason:"error"}` (or `aborted` if the signal fired). The stream terminates on every path.

The guard pipeline (per tool call)

Chapter 13 showed it from the user's side; this is the implementation order inside `runGuarded`, and it is load-bearing:

1. `pre_tool_use` **hooks** — before anything else, so a policy veto costs no dialog. First matching block wins; timeout fails open.
2. **The gate** — only for `needsPermission()` tools: `permission_request` → the broker blocks the virtual thread (terminal read, or a parked future in the web) → `permission_decision` — always emitted, allowlist answers included. Denial returns the literal `ERROR: the user denied the execution.`
3. **Execution** — `tool.execute(input, context)`; the context carries the sandbox root, the cancel signal, the caller's agent id, the call id and an `emit` sink through which artifact tools publish their additive events (`plan`, `image_generated`, `agent_message`).
4. `post_tool_use` **hooks** — after the fact, advisory, never rewriting.

The headless variant

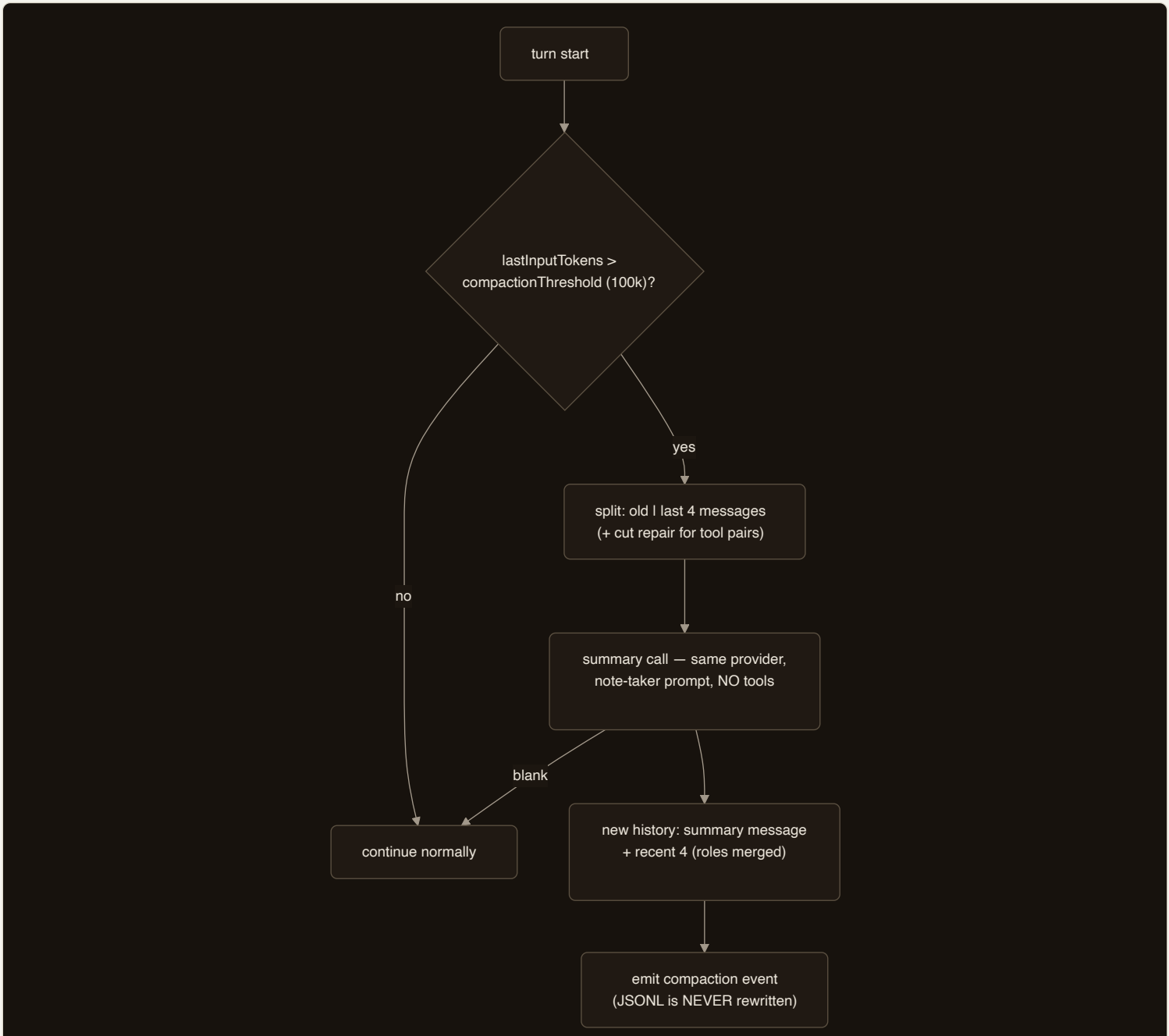
`spectro run` and cron jobs share a runner with a fixed system prompt for unattended operation (“there is no human at the terminal: do not ask questions … if a tool is denied, do not retry it”, followed by the same paragraph on reading and delegating the attended faces carry; every sentence in it that mentions a child names `spawn_agent`, so none of them applies here), a constant-policy broker (readonly denies, auto allows), standard tools (never the spawn tools; the configured MCP servers mount only when the `headlessMcp` setting or `spectro run --mcp` asks for them — chapter 30 has the switch and its auto-approval warning), an external turn brake, and a result record (final text, stop reason, session id, exit-ok). Every headless run writes a normal session file — automation is fully auditable after the fact.

CHAPTER 28

Context: compaction, caching, introspection

Three mechanisms manage the finite context window: compaction shrinks the history when it grows past the threshold, prompt caching makes the stable prefix cheap, and introspection shows you both at work.

Compaction



Above the threshold: split the history, summarize the old part with the same model (no tools), swap in "summary + last 4 messages". In memory only — the file grows, never shrinks.

- **Trigger:** the last turn's context size vs. `compactionThreshold` — checked at the start of each turn. The threshold is **derived** when nobody sets one: seven tenths of the window the backend says it loaded (LM Studio answers `loaded_instances[].config.context_length` on its own `/api/v1/models`, ollama answers `context_length` on `/api/ps`, and a llama.cpp server — `llamacpp` or the built-in runtime, which is one — answers `n_ctx` on `GET /props`), asked once per run. The last three tenths are kept back because the very next call after the trip is the summarizer, which re-sends the history *and* asks for a completion on top. A backend that states no loaded window may still be running a model whose vendor *publishes* one — anthropic and every hosted id an OpenAI-compatible gateway is really serving — and then the threshold is seven tenths of that published figure. An ollama *cloud* model (its entry on `/api/tags` carries `remote_host`) has no loaded instance on your machine, and ollama publishes its window on `/api/show` as the `model_info` key ending in `.context_length`; that figure is read before the vendor table, and ollama's answer is kept for as long as the session stays on that model, so `glm-5.3:cloud` at 1 048 576 compacts at 734 003. For a local model the same key is the trained length, not the loaded one, and is not read. Only a backend that states no window *and* publishes none keeps the old 100 000, and a server that answers "no

such endpoint" is not asked again for the rest of the session. An explicit setting always wins — and costs no round trip at all, because there is nothing the backend could say that would change it. Children inherit that setting, so one number governs the whole tree. The measure is honest under caching: `inputTokens + cacheRead + cacheCreation`, because cached tokens still occupy the window even though the provider bills them separately.

- **Mechanism:** everything except the last four messages is summarized by the same provider (note-taker system prompt, summary-only instruction, no tools, and a completion budget sized to the room the threshold actually left — on a model loaded with 8 192 tokens it asks for 2 458, not 32 000); the new history is one summary message plus the recent four, with a cut repair so no tool result loses its call. A failed summary is an `error` event and an unchanged history — never a broken run.
- **The file never shrinks.** A `compaction` audit line is appended; a later resume reconstructs the *full* history from the file and compacts again if needed — the file is always the unabridged truth.
- **Force it:** `/compact` in the REPL calls the same mechanism regardless of the threshold.

Prompt caching

Chapter 16 covered the two breakpoints (system + tools; the last stable message). The reference detail: only `text`, `tool_use` and `tool_result` blocks can carry a breakpoint (when the stable message ends in an `image` or `document` block, that request has no message breakpoint); the index is recomputed fresh every call, so a history reshaped by compaction simply gets a new breakpoint; and the wire `usage` event never includes the fold — a cache hit must not change the bytes an old consumer sees.

Introspection

With introspection on (both interactive faces enable it), every turn emits `context_info`: message count, an estimated token total against the threshold, and labeled parts — system prompt, tool schemas, conversation — each measured in chars and estimated at chars/4. The **context ring** under the web composer renders it live (green < 70 %, amber ≤ 90 %, red above); its popover shows the parts table with the honest caveat that the estimate is an estimate — the `usage` line is the truth. The “Context window” scenario (chapter 7) demonstrates the whole arc: fill → warn → compact → continue.

CHAPTER 29

MCP internals

Chapter 18 explained MCP for users; this is the wire level. The `dev.spectroscope.core.mcp` package is container-free: hand-rolled JSON-RPC 2.0 records, a transport seam, a client with strict delivery semantics, and a registry that treats broken servers as skippable.

JSON-RPC framing

- Protocol version pinned: `2024-11-05`; client info `spectroscope / 1.0`.
- **stdio transport** (primary): one JSON object per line; stdout is protocol, stderr goes to a per-server log file under the system temp dir. Handshake: `initialize` → `notifications/initialized` → `tools/list`. Reads are bounded (20 s) by a watcher thread; **a timed-out channel is poisoned** — the uninterruptible reader is unblocked by closing the streams, and the channel refuses further requests until reconnect.
- **HTTP/SSE transport** (optional): each JSON-RPC frame POSTs to the configured URL; the reply's `data:` lines are concatenated per the SSE spec (a plain JSON body is accepted too); 20 s connect/read timeouts; stateless.
- Tool results: the `text` of all `content[]` blocks joined by newlines; unexpected shapes return the raw JSON rather than failing.

Client semantics

- **At-most-once**: `tools/call` is issued exactly once per tool call. Dead transport *before* the call → one lazy reconnect. Failure or timeout *during* the call → poison + readable `ERROR:`, never a re-issue (a slow-but-successful `add_note` must not run twice).
- **Registry policy:** eager connect at startup, log-and-skip on failure, descriptors cached, config order preserved; handles (name, target, reachable, tool count) feed `spectro doctor` and `/mcp`. Closed on shutdown.
- **Adapter hygiene:** a server-supplied schema that is missing or not an object is replaced by `{"type":"object"}` — untrusted server output must not crash the provider advertisement.
- **Scope:** MCP tools bind to the session at connect time, independent of LLM-provider switches; children never see them.

The reference server

`spectro-mcp-notes` doubles as executable documentation of the server side: a single-threaded stdio loop answering `initialize`, `tools/list`, `tools/call`; JSON-RPC errors `-32601` / `-32603` where they belong; tool-level failures as MCP-style `isError` results (still JSON-RPC successes). Its store is a directory of text files with collision-safe creation; its ranker a few dozen lines of term frequency + substring bonus — search as a small program, not a search engine. Eighteen tests include one that spawns the real child JVM and speaks the protocol end to end.

CHAPTER 30

Configuration reference

Seven layers, 53 keys (three of them whole-block), plus a small handful of true environment-only secrets. Missing files are fine at every layer; malformed JSON fails loudly — a broken config is a programming error, not a fallback case.

The layers

```
built-in defaults
  <  environment  SPECTRO_*                      (usually injected from ./.env —
                                                  now the BASE, directly above defaults)
  <  ~/.spectro/settings.json                    (user; config.json read beneath it,
                                                  for one release)
  <  <launch-dir>/.spectro/settings.json         (deprecated compat layer)
  <  <workspace>/.spectro/settings.json          (project — travels with the workspace)
  <  <workspace>/.spectro/settings.local.json    (machine-local, gitignored)
  <  CLI flags    --provider --model --base-url --compaction-threshold --workspace
```

The flip (owner decision 2026-07-18, "settings-productization"): env used to sit just below the flags, outranking every settings file — editing `./.env` was the loudest way to configure spectroscope, and a stale line nobody remembered could silently shadow a deliberate choice made in the Settings page. Now env is the BASE, one step above the built-in defaults: it still seeds the starting configuration (a good fit for bootstrap, CI, or a one-shot experiment), but ANY settings file — user, project or local — outranks it. *"Die env ist die Basis, die Settings geben den Ton an."* Later layers win field by field — except `autoApprove`, `mcpServers` and `hooks`, which merge as **whole blocks**: a layer that defines the block replaces it entirely (no per-entry merge). Unknown JSON keys are ignored everywhere, so additive fields never break older builds.

Two resolution moments. Config resolves at two different times, and not every field is meaningful at both. The *process* moment (server boot, `spectro doctor`, CLI startup, a stateless REST call) has no session yet, so only `defaults < env < user < launch-dir < flags` applies — this is where `workspace` itself resolves, and the one-per-process `logLevel`. The *session* moment (the agent build, the composer gear, a per-connection re-apply) joins the session's own resolved workspace pair — project, then local — directly below launch-dir: two concurrent sessions with different workspaces can legitimately run different providers or permission modes. A workspace scope may not set `workspace` itself (a folder must not repoint the agent elsewhere) or `logLevel` (one log file per process) — both fail loudly if you try.

Workspace-supplied config is executed, not just read. A workspace's own `.spectro/settings.json` can set `mcpServers`, `hooks`, `autoApprove` and `permissionMode` — and every one of those takes effect the moment the workspace resolves: an `mcpServers` entry spawns its process, a `hooks` entry runs a shell command around every tool call, and a permissive `autoApprove` / `permissionMode` can auto-allow gates that would otherwise ask. Pinning a folder as your workspace therefore runs whatever its checked-in settings declare — the course maxim *"tool inputs are model output and therefore untrusted"* extends one layer further to *folder-supplied config is untrusted too*. Review a cloned or otherwise foreign repository's `.spectro/` before pointing spectroscope at it, the same way you would before running its build scripts.

The settings API. `GET /api/settings[?session=]` returns the resolved configuration alongside per-field **provenance** (which layer won, which lower layers were shadowed) and the raw, non-empty layers — without `?session=` it is the process-moment view, with one the session-relative view. `PUT /api/settings/{user|project|local}` writes a schema-validated partial patch to exactly one scope; a `null` value removes that key. Secret-shaped keys (`*_API_KEY`, `*_TOKEN`) are always rejected — a settings file is never a place to paste a key.

Two gears, two scopes. The gear at the foot of the sidebar (the Settings page) edits the GLOBAL, user-scope fields only: session defaults (provider/model/thinking/image backend), the default workspace (origin badge + a "reset" that falls back to the layer below), `logLevel`, and the machine-tool paths (`chromeBinary` / `imageModel` / `sttModel`). A second, compact gear sits in the composer row and edits THIS session's workspace: permission mode as a keyboard-navigable text list (live immediately via `set_permission_mode`, persisted to the workspace project file once one is pinned), always-allow rules (add and delete), machine-local overrides, and raw-JSON editors for `mcpServers` / `hooks`. An unpinned session — still in its throwaway temp folder — shows the composer gear's project sections disabled: there is no durable folder to write to yet.

When a saved setting lands. Every tool that reads a setting reads it again on the call, so saving one changes what the session you already have open does next: the `web_search` tier (`searxngUrl` and the two search keys), the image backend and `imageModel`, `chromeBinary`, the `allowLocalhost` net fence that `web_fetch`, `browse_page` and the browser and launch families share, and the dictation settings (`sttProvider`, `sttLanguage`, `sttModel`), which the transcription route reads per request. A call already in flight finishes on what it started with; the next one asks again.

The one exception, and it is a control rather than a rule. The image backend also has a dropdown in the *composer* row. Picking a backend there speaks to the running session directly, and that choice outranks a saved `imageProvider` until the session ends — so if you used it, the settings page will not move this session's image backend, and the sentence under that dropdown on the settings page says so. Nothing else on the page has a second live control like this. (Where the configured backend has no API key and another one does, the backend with a key is used. That is not a remembered choice: it is re-derived from the settings and the keys on every call, so giving the configured backend a key ends it.)

Four things are settled when the agent is built and stay settled for that session, because changing them mid-session would mean killing processes or rewriting a conversation that already happened: the **workspace**, the **MCP servers**, the **shell hooks**, and the base of the **system prompt** with its skills and `SPECTRO.md` / `AGENTS.md`. A setting with the reach `next-run` may add a paragraph to that base when a run starts, so the system prompt a run sends is fixed for that run, and the next run of the same session may send a different one. The **allowlist** (`autoApprove`) is session-scoped too, and that one is a decision rather than a cost: the belt reads settings live so the operator reaches the agent, and the allowlist is what protects the operator from the agent — a run that may write files must not be able to widen its own permissions between two tool calls. The **OTLP exporter** is settled the same way: it is built where the session's store is minted.

Provider, model, the provider address and `thinking` are the third case, and the one that reads as a contradiction until it is spelt out. Saving them on this page applies from the next session — the agent is built with the provider it was built with, and a run does not change backends underneath itself. The *model picker under the message box* is the live path: it swaps the running provider through the same session, which is a different door from the settings file. So "live via the picker" is true of the picker and not of the page, and the page now says the page's half rather than the picker's.

When each key lands

Each key in the "Every key" table below has one reach: the moment a change made with the key's own control starts to act in a session that is already open. For most keys that control is the settings page; for `toolGroupsOff` it is the composer gear. The settings page prints the answer under each of its controls, and the composer gear prints it under the tool groups. A test holds this table, the page's own table (`settingsReach.tsx`) and the list of keys in `SpectroConfig` to one another, so a new key without a reach fails the build.

REACH	A SAVED CHANGE ACTS	KEYS
`live`	at once, in a session already open: the next tool call or request reads the settings again, and the window redraws the two dock widths where they stand. A call already in flight finishes on what it started with. `sessionsPerChat` acts here for the slots of the chat: the next helper that asks for one is admitted by the new count. The descriptions of the spawn tools keep the count the run started with and follow at the next run	`allowLocalhost`, `searxngUrl`, `rtkFilter`, `chatReserveWidth`, `dockMaxWidth`, `logLevel`, `imageModel`, `sttModel`, `sttProvider`, `sttLanguage`, `sttRunner`, `chromeBinary`, `sessionsPerChat`, `codeGraphBackend`
`live-unless-picked`	like `live`, unless the image backend was picked in the composer during this session; that pick stays on top until the session ends	`imageProvider`
`next-run`	at the start of the next run of a session already open, the next time you send a message. A run in progress keeps the value it started with. `toolGroupsOff` acts here when you switch it in the composer gear. A list written into a settings file while a session is open is not read again by that session and applies from the next session: the agent can write that file, and a run must not change its own tool list	`continuationBudget`, `toolGroupsOff`, `careParagraph`, `readSharePercent`, `agentMessaging`
`next-session`	when the next session builds its agent. A session already open keeps the value it was built with	`provider`, `model`, `baseUrl`, `ollamaBaseUrl`, `lmstudioBaseUrl`, `llamacppBaseUrl`, `compactionThreshold`, `permissionMode`, `workspace`, `autoApprove`, `thinking`, `mcpServers`, `progressGuardWrites`, `progressGuardFailures`, `progressGuardPlanTurns`, `maxTurns`, `maxTokens`, `subagentBudgetSeconds`, `subagentBudgetTokens`, `questionsPerRun`, `maxQuestionOptions`, `maxQuestionChars`, `maxRetries`, `commandTimeoutSeconds`, `toolResultElision`, `promptCaching`, `hooks`, `otlpEndpoint`, `otlpBasicAuth`, `localModeKeys`, `tts`
`headless-run`	only where an unattended run starts: the cron daemon and a fleet node at their own start, and `spectro run` on each call for the key it reads. No session in the app window reads them	`headlessMcp`, `desktopNotifications`

Doctor never lets a shadowed variable go unnoticed. Every run, `spectro doctor` walks the same provenance and prints one line for every `SPECTRO_*` variable that IS set but is no longer the effective source, naming both the variable and the settings layer that now wins:

```
env SPECTRO_MODEL is set but shadowed by user settings (effective model comes from user)
```

On first boot after the flip, if no user settings file exists yet, spectroscope materializes one FROM the current env base (secrets excluded, only the fields the env actually sets) — so day one changes nothing functionally, and from then on that file is the truth the doctor line, the Settings page and every face agree on.
`spectro doctor --migrate` renames the old `~/.spectro/config.json` to its new name, `settings.json`, whenever the new name is not yet present.

Deprecation table

Every `SPECTRO_*` variable below still works exactly as before — as the env BASE — but is deprecated in favor of the settings field it feeds:

ENV VARIABLE (DEPRECATED)	SETTINGS FIELD
`SPECTRO_PROVIDER`	`provider`
`SPECTRO_MODEL`	`model`
`SPECTRO_BASE_URL`	`baseUrl`
`SPECTRO_WORKSPACE`	`workspace`
`SPECTRO_THINKING`	`thinking`
`SPECTRO_IMAGE_PROVIDER`	`imageProvider`
`SPECTRO_IMAGE_MODEL`	`imageModel`
`SPECTRO_STT_MODEL`	`sttModel`
`SPECTRO_STT_PROVIDER`	`sttProvider`
`SPECTRO_STT_LANGUAGE`	`sttLanguage`
`SPECTRO_CHROME`	`chromeBinary`
`SPECTRO_MAX_RETRIES`	`maxRetries`
`SPECTRO_PROMPT_CACHING`	`promptCaching`
`SPECTRO_LOG_LEVEL`	`logLevel`

Not deprecated — these never had a settings-field counterpart: `compactionThreshold` and `permissionMode` (aside from its live socket message) are flag/UI-only, no env form ever existed for them; the API keys in "Environment-only variables" below are true secrets and never graduate into a settings file.

Also not deprecated, because they arrived together with the fields they feed: `SPECTRO_OLLAMA_BASE_URL` → `ollamaBaseUrl` and `SPECTRO_LMSTUDIO_BASE_URL` → `lmstudioBaseUrl`, and `SPECTRO_LLAMACPP_BASE_URL` → `llamacppBaseUrl`. They are the environment form of the three per-provider addresses in the table below, and they sit in the same env layer as everything above — a settings file still outranks them.

Every key

Some whole-number keys have a floor, the lowest value they take, and their rows below name it. A save below the floor is refused with a message that names the key, the value and the floor, and nothing from that save is written. A value below its floor in a settings file is skipped when the file is read: that key falls back to the layer below while the rest of the file applies. The settings page names the skipped value beside the field's origin. A number field on that page saves nothing while it is empty or holds anything other than a whole number at or above its floor, and it shows the saved value again when you leave it. Where the floor is 0, zero is a legal value and the row says what it does.

KEY	TYPE · DEFAULT	ENV BASE (DEPRECATED)	MEANING
`provider`	`anthropic` | `ollama` | `openai` | `lmstudio` | `llamacpp` | `openrouter` | `gemini` | `spectro-local` | `copilot` · `anthropic`	`SPECTRO_PROVIDER`	the LLM backend; unknown values fail loudly, and the message lists these same nine
`model`	string · `claude-opus-4-8`	`SPECTRO_MODEL`	when no layer set a model, the default follows the provider: `ollama` → `qwen3` ; `openai` , `lmstudio` and `llamacpp` → `local-model` , a placeholder each of them ignores in favour of what is loaded; `spectro-local` → whichever model the bundled catalogue leads with; `copilot` → `auto` , which lets Copilot pick the model; `gemini` and `openrouter` have no honest default at all, so one of the layers has to name a model
`baseUrl`	URL · `http://localhost:11434`	`SPECTRO_BASE_URL`	the shared FALLBACK address, and no longer the last word: ollama reads `ollamaBaseUrl` first, lmstudio reads `lmstudioBaseUrl` first and llamacpp reads `llamacppBaseUrl` first, whichever layer any of those values came from, so for those three this field only applies when their own is unset. Ignored by anthropic. For the remaining openai-compatible providers the old rule stands: a preset endpoint by name (openai → `api.openai.com` , openrouter → `openrouter.ai/api` , gemini → the Google gateway), no key-based swap, and an explicit URL wins. The default doubles as the "unset" sentinel — typing ollama's own `http://localhost:11434` here reads as "nothing configured"; the three fields below carry no sentinel and are honored verbatim
`ollamaBaseUrl`	URL · *unset* (preset `http://localhost:11434`)	`SPECTRO_OLLAMA_BASE_URL`	where ollama lives — the address the run, the live model list, the reachability probe and the doctor line all dial. Set it to put ollama on another machine (`http://gpu-box:11434`) without moving any other provider. No sentinel: only unset or blank means "not configured"
`lmstudioBaseUrl`	URL · *unset* (preset `http://localhost:1234`)	`SPECTRO_LMSTUDIO_BASE_URL`	the same for LM Studio, kept apart from ollama's — one address per provider was the point: before this, pointing the shared field at LM Studio's port moved ollama there too, wrong port and all
`llamacppBaseUrl`	URL · *unset* (preset `http://localhost:8080`)	`SPECTRO_LLAMACPP_BASE_URL`	where your own `llama-server` lives. The preset is llama.cpp's documented default port. A llama-server is not LM Studio wearing a different hat: it serves exactly the one model it was started with and ignores the model field in a request, and it answers `GET /props` with the context size that model is actually loaded at — so the harness reads the window instead of guessing it from a table. Pointing `lmstudio` at a llama-server works on the wire and loses both of those
`compactionThreshold`	int · *unset* (derived; `100000` when nothing is known)	— (flag only)	compaction trigger in input tokens. Unset is not 100 000 any more: the harness asks the backend what window the loaded instance serves and compacts at seven tenths of it, so a model loaded with 204 288 tokens is no longer summarized away at 100 000 — and where no instance answers but the model's vendor publishes a window, it is seven tenths of that. Set it and your number wins outright. Reaches every way a run starts — the browser session, the REPL, `spectro run` , a cron fire, a fleet node and every child agent. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()` , used by a JVM program that took spectroscope as a dependency) and a lane of a **fleet panel** declared by that same program (`Spectro.panel()`). Both take their numbers from the code that declares them, never from a settings file on whatever machine they happen to run on. The same window also bounds `read_file` : one whole-file read may take a quarter of it, and where no window is known the threshold stands in for it
`permissionMode`	`ask` | `auto` | `readonly` | `extended` · `ask`	—	chapter 13; headless `run` has its own `--permissions` ; live via the composer gear's `set_permission_mode` . `extended` lets the file tools reach outside the working folder, so a workspace scope may not set that one value, and neither may the launch dir's `.spectro/settings.json` : it is dropped with a notice and your user setting stays in force. `ask` , `auto` and `readonly` stay workspace-settable
`workspace`	path · *unset*	`SPECTRO_WORKSPACE` / `--workspace`	the agent's working directory (file tools, glob/grep, run_command); unset = a per-session folder `<tmpdir>/spectroscope-ws/<session-id>` — a resume finds its files again, and the project you started spectroscope in stays clean; process-global — a workspace scope may not set this (a folder must not repoint the agent)
`autoApprove`	string[] · `[]`	—	allowlist rules: `tool#tier` , `tool#tier:prefix*` or `family*#tier` (chapter 13) — whole-block; lives in the workspace project file once one is pinned
`allowLocalhost`	bool · `false`	`SPECTRO_ALLOW_LOCALHOST`	the net fence's local-verify-loop opt-in: `web_fetch` / `browse_page` may reach loopback. Never widens to RFC 1918, the 100.64/10 tailnet or `file://` — `web_fetch` fences every redirect hop, while `browse_page` is fenced only on the address you hand it (Chrome's own redirects and script navigation are outside it — chapter 13). Process-global: refused in a workspace scope, user scope only
`searxngUrl`	URL · *unset*	`SPECTRO_SEARXNG_URL`	the SearXNG instance `web_search` prefers over every other tier (chapter 13). Process-global: refused in a workspace scope, user scope only — the searcher carries no net fence, so this field alone decides an address the tool dials
`imageProvider`	`gemini` | `openai` · `gemini`	`SPECTRO_IMAGE_PROVIDER`	image-generation backend
`thinking`	bool · `true`	`SPECTRO_THINKING`	reasoning-stream visibility (REPL `/think` ; in the web UI under Settings, Models). Reaches every way a run starts — the browser session, the REPL, `spectro run` , a cron fire, a fleet node and every child agent. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()` , used by a JVM program that took spectroscope as a dependency) and a lane of a **fleet panel** declared by that same program (`Spectro.panel()`). Both take their numbers from the code that declares them, never from a settings file on whatever machine they happen to run on. A child agent is built with the value its session's tool belt was built with, so the REPL's `/think` moves the running parent and not the children spawned around it; the next session reads the key again, for the whole tree
`mcpServers`	object · `{}`	—	Claude-Desktop-shaped server map — whole-block (chapter 18). The REPL and a web session mount every entry; a headless run mounts them only with the opt-in below or its `--mcp` flag
`headlessMcp`	bool · `false`	`SPECTRO_HEADLESS_MCP`	the headless faces' MCP opt-in (card 220): when true, `spectro run` , every cron fire and every triggered fleet node mount the configured `mcpServers` the way the REPL does — read at each headless start, never by an interactive session. A manual `spectro run` overrides it per invocation with `--mcp` / `--no-mcp` . Under `--permissions auto` the opt-in approves every tool every configured server offers, unwatched. Process-global: refused in a workspace scope, user scope only — the switch that widens an unattended run must not live in the folder the agent writes into
`desktopNotifications`	str · `on`	none	whether a finished `spectro cron` job shows a desktop notification. `on` , the shipped value: the end of every job, and a job that cannot start because its folder is missing, shows a banner with the job id, its status and the start of its result or its stop reason. On macOS the banner comes from `osascript` ; on other systems it is a terminal bell and a log line. `off` writes the same line to the job's log and shows nothing. Only the cron daemon and `spectro cron --once` send these; `spectro run` , the web session and a fleet node never do, and the desktop app's own notifications (chapter 10) do not read this key. Read when `spectro cron` starts, so restart a running daemon after a change. `spectro doctor` prints the state. Process-global: refused in a workspace scope, user scope only, because the agent writes into that folder and could otherwise turn off the report of its own failed job
`progressGuardWrites`	int · `3`	—	the progress guard's first detector (card 262): how many DISTINCT earlier paths must already carry the exact bytes a write is about to repeat before the run says what it saw and asks the operator — carry on, change course, or end it. `0` turns the detector off. Cut from a measured hour in which a local model wrote the same 283 bytes to 31 paths while the harness said nothing. Content shorter than 64 characters is never compared, so a scaffold's empty `__init__.py` files cannot trip it. Process-global: refused in a workspace scope, user scope only — a folder the agent writes into must not be able to switch off the detector watching it. Floor `0`

`progressGuardFailures`	int · `3`	—	the second detector: how many times in a row ONE call with byte-identical input must fail before the same question is asked. The counter resets on any success of that same call, so a flaky test that fails twice and then passes stays silent. `0` turns it off. Process-global: refused in a workspace scope, user scope only — a folder the agent writes into must not be able to switch off the detector watching it. Floor `0`
`progressGuardPlanTurns`	int · `0` (off)	—	the third net: how many consecutive turns a plan may sit unchanged, with a step still open, before the same question is asked. **Off as shipped** — it needs a plan that exists and is maintained, and a model whose profile says it was not trained for tool use is handed no tool belt and writes none. A plan whose steps are all completed never stalls. Process-global: refused in a workspace scope, user scope only — a folder the agent writes into must not be able to switch off the detector watching it. Floor `0`
`continuationBudget`	int · `3`	none	how many times ONE run may be restarted by the harness after it stopped with its own plan still open (card 266). The harness writes the continuation itself and names the steps that are open, so the model does not have to cooperate. The models this was built for cannot call a tool to ask for another turn. It refuses to continue a run that has nothing to show since the last continuation, and it never resets the turn cap, so the real ceiling stays one number. `0` turns the leash off. Attended faces only: the browser session and the interactive REPL wire it, `spectro run` and cron do not. The browser session reads it from the settings files at the start of every prompt, so a save reaches the next run of a session already open. Floor `0`
`maxTurns`	int · `1000`	—	the runaway-loop brake: how many turns ONE run may take before the harness ends it with `stopReason: max_turns`. It is a ceiling, not a detector — nothing was noticed, the run simply ran out of room, which is why a run that hits it stops without a closing word from the model. Counted per run, and a continuation (see `continuationBudget` above) never resets it, so this number is the real ceiling on one prompt. The shipped value was 15 until 2026-09-01 and had never been measured against anything: a census of 7,139 real agent sessions put the median run at 14 turns and p95 at 129, with **48 % of them going past 15** — so the old ceiling ended half of all real work in the middle. Card 373 moved it to 1000 on 2026-09-17, measured on the owner's own runs rather than on that census: the longest took 118 turns in a little over two hours, so an unattended night is about 440, and the census p99 is 288. The 150 that ran from 2026-09-01 until then sat above the census p95 and below an unattended night, which is why the number moved a second time. Raise or lower it freely; a long run costs tokens, not safety, because every tool call still passes the permission gate. Reaches every way a run starts — the browser session, the REPL, `spectro run`, a cron fire, a fleet node and every child agent. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()`, used by a JVM program that took spectroscope as a dependency) and a lane of a **fleet panel** declared by that same program (`Spectro.panel()`). Both take their numbers from the code that declares them, never from a settings file on whatever machine they happen to run on. `spectro run --max-turns N` is a second, outer brake belonging to that one invocation: it can end a run EARLIER than this number, never later. Floor `1`
`maxTokens`	int · `32000`	—	the completion budget ONE provider call may spend, in output tokens. It is the ceiling the harness ASKS for, not a promise about what a model grants: a backend may hold a lower one of its own, and the OpenAI-compatible provider clamps every request to a hard 16 000 whatever this says. When the run knows the model's context window (the window the backend says it loaded, the one published for the model, or the session window set from the context ring), each request asks for at most the window minus the compaction threshold, and at most what the window has left after that request's estimated input and a reserve of 256 tokens, with a floor of 512 tokens (card 488). The estimate can be low on a first request, so a request can still ask for more than the window has left. On an 8 192 window that is 2 458. An image counts as 4 784 tokens of that input and a PDF page as 7 784, whatever their size in bytes; the pages are counted from the page objects of the file, and a PDF that keeps them in compressed object streams counts as one page. Under an explicit `compactionThreshold` the backend is not asked for its window: a model with a published window keeps that window, and a model with none is held against the window the threshold is 70 % of, so a threshold of 5 000 on such a model asks for at most 2 143. A threshold at or above a known window leaves 512. Only a run with neither a known window nor an explicit threshold sends this value unchanged. The older Anthropic models take their reasoning budget as a number that has to stay strictly below this one and be at least 1 024, so on those models a request that asks for 1 024 or less goes out without thinking. The models on adaptive thinking, the shipped default among them, and every non-Anthropic backend are untouched by this at any value. The default has been 32 000 since the first commit. What changed with this key is that it can be typed at all: the builder seam existed, was public and documented, and had **zero callers**, so an audit asking "is it parameterised?" scored it as reachable while no operator could change it. Reaches every way a run starts — the browser session, the REPL, `spectro run`, a cron fire, a fleet node and every child agent. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()`, used by a JVM program that took spectroscope as a dependency) and a lane of a **fleet panel** declared by that same program (`Spectro.panel()`). Both take their numbers from the code that declares them, never from a settings file on whatever machine they happen to run on. Floor `1`
`subagentBudgetSeconds`	int · `7200`	—	the smallest run budget ONE child agent gets, in seconds, counted from its first token. Two hours as shipped, the owner's number of 2026-09-17. The budget is derived: the floor you type here, or three times the median exchange this session has measured, whichever is larger; the measured term is capped at 30 minutes, the floor never is. While the session has measured nothing, the budget is exactly the floor. Once it has measured something, a slow backend can raise the budget above a floor under 30 minutes. It cannot raise the shipped two hours, because the cap on the measured term sits below them. A child that runs out is cut, and what it had written in the turn it was cut in comes back to the parent under a marker (card 371). Raising it raises what a stuck child costs: one that wedges after its first token holds its parent for this long, plus up to 45 minutes of queue grace before that token. **No ceiling is enforced**, for the reason `commandTimeoutSeconds` gives. Bound when the session's subagent manager is built: a save reaches the next session. Reaches the browser session and the REPL, and every child they spawn. The headless faces build no subagent manager at all, so `spectro run`, a cron fire and a fleet node have no children for it to reach. Floor `1`
`subagentBudgetTokens`	int · `10000000`	—	the token spend past which ONE child agent is cut, counted as input plus output the way each of its exchanges reports usage. Ten million as shipped (card 394). Cache reads and cache writes are reported apart and are not counted, so the budget binds hardest on a backend without a prompt cache, which reports the whole resent conversation as input on every exchange. The check runs when the child starts another exchange: a child whose spend has passed the budget is cut there. The model request of that exchange still goes out and is cancelled as it starts; a cancelled request reports no usage, so it is not in the spend, and a paid backend may still bill its input. The run ends with the stop reason `child_token_budget_exhausted`, and the parent is handed `ERROR: [explore-1] out of tokens: n spent (in in / out out), past its budget of budget (subagentBudgetTokens). Raise subagentBudgetTokens in the settings, or cut the subtask smaller.` followed by what the child had written in its last turn, under the same marker as a child that ran out of time. A child whose final answer is the exchange that passed the budget keeps that answer. Each child has a budget of its own, so the four children of one `spawn_agents` call may together spend four times this number. **No ceiling is enforced**, for the reason `commandTimeoutSeconds` gives. Bound when the session's subagent manager is built: a save reaches the next session. Reaches the browser session and the REPL, and every child they spawn. The headless faces build no subagent manager at all, so `spectro run`, a cron fire and a fleet node have no children for it to reach. Floor `1`
`questionsPerRun`	int · `9`	—	how many times ONE run may stop and put a decision to you through `ask_user_question`. Spent only by a well-formed call, so a malformed one costs nothing and the person is still owed a question; a fresh run gets a fresh budget. `0` switches the asking off entirely — the tool stays registered and always declines, which is the reachable form of "the model never learns it exists". Card 265 shipped this as a constant and called the number a guess awaiting a word; card 356 made it settable and card 365 raised it from three to nine, after a census found that one asking session in six wanted more than three. Floor `0`
`maxQuestionOptions`	int · `4`	—	how many choices one question may offer. The answer bar draws them in a row, so a large number is narrow rather than broken. The value is not only enforced — it is formatted into the tool's own schema and description, so the model is told the same number it will be held to. Floor `1`
`maxQuestionChars`	int · `500`	—	how long one question may be. A question is read under time pressure, which is the whole argument for a limit; like the option cap it is announced to the model rather than only enforced against it. Floor `1`
`maxRetries`	int · `2`	`SPECTRO_MAX_RETRIES`	transient retry budget; 0 disables; garbage values fail readably
`commandTimeoutSeconds`	int · `900`	—	the wall-clock budget ONE `run_command` call gets before the child process is killed and the model is told `ERROR: command timed out after N s`. Fifteen minutes as shipped since card 370, the owner's number; ten seconds before it, which is less than an `npm install` or a cold test run takes, so those calls could only ever come back as a timeout. **No ceiling is enforced**, deliberately: every shell call passes the permission gate anyway, so the person setting a large number here is the person approving the call it applies to. The tool tells the model the number it was actually built with, so a shrunk budget is announced rather than hidden. Bound when a face builds its tool belt: a save reaches the next session, in the browser session, the REPL, `spectro run`, a cron fire, a fleet node and every child agent. One place builds a belt and does **not** read it, because it is not a face of the app: the **embedded library** facade (`Tools.all()`, used by a JVM program that took spectroscope as a dependency), which takes its numbers from the code that declares them. Floor `1`
`rtkFilter`	str · `off`	—	whether the agent's shell lines go through rtk, a CLI proxy that runs a developer command and prints less of its output. `on` asks `rtk rewrite` what each `run_command` line could become and runs the answer; `off`, the shipped value, runs the line the model wrote. Measured on one machine with rtk 0.45.0 on 2026-09-21: 77.7 % fewer bytes on `ls -la` and 51.0 % on `git status`, and nothing at all on `git log`, `git diff --stat` and `grep -rn`. **It ships off because rtk swallows failures.** On the same binary, `rtk test sh -c 'exit 7'` returned 0 in silence and `rtk err sh -c 'exit 3'` returned 0 while printing `[ok] Command completed successfully (no errors)`, a success line the child never wrote. The verbs measured to do that are refused whatever the switch says: `test`, `err`, `find`, and `gradlew`, whose filter is written for Android and returns 43 bytes where `./gradlew --version` returns 568. Spectroscope keeps its own verdict: the permission gate is asked about the line that will actually run, and the

			event carries the line the model wrote beside it, so a rewrite is never invisible. Goal checks and the hooks you wrote are untouched. When rtk is not on the PATH the line runs raw and nothing fails. Read on every tool call, so a save reaches the session you already have open
`toolResultElision`	str · `off`	none	whether an old, large tool result is sent again on every later turn. `off`, the shipped value since 0.15.4 (card 600), sends every result in full on every turn. `on` applies to every provider, local ones included, and to every child agent: a result that more than four assistant turns have followed and that is longer than 2,000 characters is replaced in the outgoing request by one line. The line names the tool, the path or the first word of the command (at most 40 characters), and the size, and says what a second call does. For a tool that only reads, it asks the model to call the tool again to read the current state. Any other tool, MCP tools included, would run again with its effects. It never carries file content. The history, the session file, the trace and the Lab keep the whole result. The llm wire shows the line, because it records what was sent. Results are replaced in batches that save at least 8,000 characters, so a provider's cached prefix survives between batches. A result that calling the tool again cannot give back (a child agent's report, your answer to a question, a generated image) always stays whole. The context ring reads the size that is sent. Compaction fires at the same threshold, which the smaller requests reach later. The summarizer reads the whole history where it fits within that threshold. Where it does not, it reads the oldest replaced results as their one-line replacements. Measured on 2026-10-09 by replaying 1,387 main-agent requests from 368 local session files, with each recorded compaction applied where it happened, and counting conversation characters only (no system prompt, tool schemas or images): pooled over all requests the saving is 25.7 %, but one session of 525 requests carries it; without that session it is 4.9 %, and the average over sessions, each counted once, is 0.37 %, because in most sessions no result grows old enough. On a local model the same batches cost time instead: each one changes the start of the request, so the backend processes the prompt again from there, and on 2026-10-13 two helpers of a session on a local llama.cpp model spent 83 to 87 % of their re-processed prompt tokens on the requests right after these batches. The composer gear switches the key under Local mode, with the checkbox **Optimize tool results cache (use with cloud models only)**, which saves it in your user settings. User scope only: a workspace scope (a folder's `.spectro/settings.json` or `settings.local.json`) may not set it: the key is dropped from that file and the notice names it. The checkbox writes the user settings, so its choice holds for every folder. Read when the session's agent is built, so a save reaches the next session. Reaches the browser session, the REPL, `spectro run`, a cron fire, a fleet node and every child agent. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()`) and a lane of a **fleet panel** (`Spectro.panel()`). Both keep the shipped value unless the program that declares them sets `AgentOptions.toolResultElision`
`toolGroupsOff`	string[] · `[]`	none	the tool groups a session leaves out of every model request: `browser` (the seven `browser_*` tools), `launch` (the five `launch_*` tools), `images` (`generate_image`, `view_image`), `web` (`web_fetch`, `web_search`, `browse_page`), `agents` (`spawn_agent`, `spawn_agents`), `roles` (the five role tools), `mcp` (every `mcp__*` tool) and `messaging` (`send_message`, `list_agents`, `wait_agents`, card 503). A tool in a switched-off group is not sent to the model at all, so its schema stops riding along on every request and the context ring's *tool schemas* part shrinks by it. A child agent has the same groups off, and its role can take more tools away but never add one back. The standard file tools except `view_image` (it belongs to `images`), `run_command`, `update_plan`, `ask_user_question` and the skill tools belong to no group and are always sent, and so are `code_graph` and `schedule_wakeup` where a run offers them. Switching a group off changes no permission: a tool that is still on asks exactly as before. Empty as shipped, which sends every tool. The composer gear switches groups for the open session from its next run and, when a folder is pinned, saves the list in that folder's local file; without a pinned folder it saves nothing. A list in a settings file is read when a session starts and again when a folder is pinned, so a pinned folder's list applies before the first prompt. A list written into the file while a session is open is not read again by that session; it applies from the next session. **Only a session in the app window reads it**, together with every child it spawns. The Local mode switch in the same gear writes `browser`, `launch`, `images` and `roles` for one chat; a list you set yourself replaces that preset and the gear says it changed (card 493). Five places build an agent and do **not** read it and send every tool, so a list at user scope has no effect there: the **REPL** (its agent and its children, two places), the **unattended runs** of `spectro run`, a cron fire and a fleet node, the **embedded library** facade (`Spectro.agent()`) and a lane of a **fleet panel** (`Spectro.panel()`). Measured on 2026-10-09 in a session without MCP servers or skills, with the ring's own reader: the schema block is 18,101 characters with every tool and 4,125 with all seven groups of that day off. An unknown group name is refused on save with a message that lists the eight, and a file that holds one is refused on load
`sessionsPerChat`	int · *unset*	none	how many model sessions one chat may run at once, the main agent and its helpers together. The main agent holds one for as long as the chat runs. A helper takes a slot when it starts and gives it back when it finishes. A helper that finds no free slot waits for one of the same chat, the chat shows it as waiting, and the helpers asked for first start first. Nothing is refused and nothing outside the chat is counted, so two chats at `3` can put six requests on one model server at the same time. The default count is `3`, the main agent and two helpers: the Local mode switch in the composer gear writes it for one chat and its row there edits the count (card 493), and the care paragraph names those two helpers while the key is unset; otherwise you set the key in a settings file. The key itself ships unset and the default does not act while the switch is off, so a chat with the switch off sends the same request as before: no limit per chat, and one `spawn_agents` call starts up to four helpers at once. With a count set, the descriptions of `spawn_agent` and `spawn_agents` tell the model how many helpers run at once, which is the count minus one, and one `spawn_agents` call may ask for four helpers or for as many as run at once, whichever is larger. A helper's grace before its first token starts when it holds a slot, so a wait for a slot does not use it up. In the app window the count is read again at the top of every prompt, so a save reaches the next prompt of a session you already have open. The descriptions keep the count the run started with, so the cached start of the prompt stays valid for the whole run. A helper that already runs keeps its slot when the count drops. The REPL reads it when it starts. `spectro run`, a cron fire and a fleet node carry it and register no spawn tools, so it limits nothing there. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()`) and a lane of a **fleet panel** (`Spectro.panel()`). A count of `1` is refused; to run a chat without helpers, put `agents` in `toolGroupsOff`. Floor `2`
`careParagraph`	str · `off`	none	whether every run ends its system prompt with a short paragraph that asks the model to go carefully: work in small steps, read only the file it needs next, read large files in parts with `offset` and `limit`, not list or search the whole workspace in one call, check each result, keep answers short. The paragraph names no backend, so it reads the same on a local and on a hosted model. While the run offers `spawn_agent`, one more sentence asks it to start at most as many subagents at once as `sessionsPerChat` allows: the count minus one. While the key is unset it takes the default count of `3`, the main agent and two helpers, so it names two. With the `agents` tool group off, and in a child agent or a headless run, which offer no spawn tool, that sentence is left out. The model can ignore the paragraph; nothing in it is enforced. The Local mode switch in the composer gear turns it on for one chat (card 493). `off`, the shipped value, sends no paragraph and the system prompt is the one sent before the key existed. A run reads the key when it starts and keeps the text for every request of that run. The browser session reads the settings again before every message, so a saved change applies from its next run. `spectro run` reads it on each call; the REPL, the cron daemon and a fleet node read it when they start, so a change reaches them at their next start. The context ring and the System context panel show it. Reaches the browser session, the REPL, `spectro run`, a cron fire, a fleet node and every child agent. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()`) and a lane of a **fleet panel** (`Spectro.panel()`). Both keep the shipped value unless the program that declares them sets `AgentOptions.careParagraph`
`readSharePercent`	int · `25`	none	the share of the context window, in per cent, that one whole-file `read_file` may take (card 493). A file whose estimated size, at 3 bytes per token, is larger than this share of the window is refused with a message that says the numbers and asks the model to page with `offset` and `limit`; a page is held to the same share. A fixed fuse of 2 MiB per read stays on top. The `read_file` description names the share, so the model is told the number it is held to. `25`, the shipped value, is the share the release cut on 2026-10-09 had as a constant, and the request is then the one that release sent. The Local mode switch in the composer gear writes `10` for one chat; that value is a proposal nobody has measured. A run reads the key when it starts and keeps it for every request and every read of that run, so its description and its check name the same number and the cached start of the prompt stays valid. The browser session reads the settings again before every message, so a saved change applies from its next run. `spectro run` reads it on each call; the REPL, the cron daemon and a fleet node read it when they start. A child agent takes the share of the run that spawned it. Reaches the browser session, the REPL, `spectro run`, a cron fire, a fleet node and every child agent. Two places build an agent and do **not** read it, because neither is a face of the app: the **embedded library** facade (`Spectro.agent()`) and a lane of a **fleet panel** (`Spectro.panel()`). Both keep the shipped value unless the program that declares them sets `AgentOptions.readSharePercent`. Floor `1`
`localModeKeys`	string[] · `[]`	none	the keys the Local mode switch of the composer gear wrote into a folder's local file (card 493). You do not set it by hand. When a session in the app window switches Local mode on with a folder pinned, it writes `sessionsPerChat`, `toolGroupsOff`, `readSharePercent` and `careParagraph` into that folder's `.spectro/settings.local.json`, leaves out every one of them you had already set there yourself, and lists the ones it wrote here. The next session in that folder starts with the switch on. Switching it off removes the listed keys and this one, and keeps the keys you set yourself. A key the switch does not write is refused on save and on load. The session reads it when it starts and when a folder is pinned. No other face reads it
`agentMessaging`	bool · `true`	none	whether agents on this machine message each other (card 503). On, the browser session and the REPL offer the main agent `send_message` and `list_agents`, every child agent carries both, and they reach its parent and no one else, and the process takes messages from other sessions. Off, both tools leave the `tools` array of the next run, a call to one of them is refused as unknown, the process still registers so others can list it but says it is not accepting messages, and every inbound message is answered `refused`. Messages already recorded stay in the chat. A run reads the key when it starts; the browser session reads the settings again before every message, so a saved change applies from its next run, and the inbound refusal follows a save at once. The `messaging` tool group of the composer gear switches the two tools and `wait_agents` off for one session without touching the key. The headless faces register no messaging tool and refuse every inbound message whatever the key says. Two places build an agent and do **not** read it: the **embedded library** facade (`Spectro.agent()`) and a lane of a **fleet panel** (`Spectro.panel()`); neither registers a messaging tool
`codeGraphBackend`	str · `graphify`	none	the program that builds and answers the code graph of a folder (card 587). The `/graph` commands, the code graph rows of the folder chip and the model's `code_graph` tool read this one key, so another graph program can take graphify's place later. `graphify`, the shipped value, is the only backend the app knows today; any other value is refused on load and on save. A folder may name it in its own settings. Every reader asks for the folder at hand on each read, from your settings, the start folder's and that folder's own: the session's `code_graph` tool, the offer of the bundled `code-graph` skill, `/graph`, the code graph rows of the folder chip and the System context view. A folder whose settings file is refused costs that file, as at the first run: the start folder and your settings decide. The REPL, `spectro run`, a cron fire and a fleet node offer no code graph tool and do not read it
`chatReser`	int · `360`	—	how many pixels of the chat row the right dock may never take, whatever you drag. Introduced as a fixed number because the proportional cap before it left the chat at 356 px on a 1150 px window; a setting since card 361, because on a wide screen it is the wrong limit. Read by the app's layout, not by the agent. `0` keeps no width back for the chat. Floor `0`

`veWidth`			
`dockMaxWidth`	int · `1200`	—	the widest the right dock may be dragged, in pixels. It is a separate key from the reserve above because the two bind on *different screens*: below a chat row of roughly 1560 px the reserve decides the dock's limit and this ceiling never comes into it, at or above that width this ceiling decides and the reserve never does. Raising only one of them is why "the dock has a hard edge" survived one fix. Read by the app's layout, not by the agent. The floor is the dock's own minimum width. Floor `260`
`promptCaching`	bool · `true`	`SPECTRO_PROMPT_CACHING`	Anthropic `cache_control` breakpoints, and the `cache_prompt` field of the `llamacpp` provider (card 468): on sends `cache_prompt: true`, off leaves the field out and the server's own default decides; a stock llama-server reuses its cache either way. The bundled `spectro-local` runtime always sends `true`. No field goes to Ollama, LM Studio or the other OpenAI-compatible endpoints, because their documentation names none; Ollama was measured to reuse its cache without being asked
`logLevel`	`error` | `warn` | `info` | `debug` | `trace` · `info`	`SPECTRO_LOG_LEVEL`	operator-log detail (chapter 31); process-global — one log file per JVM, refused in a workspace scope, user scope only
`imageModel`	string · *unset*	`SPECTRO_IMAGE_MODEL`	overrides the image backend's own default model; machine-tool path — belongs in user or workspace-local scope
`sttModel`	path · *unset*	`SPECTRO_STT_MODEL`	absolute path to the whisper.cpp model (default `~/.spectro/models/ggml-small.bin`); machine-tool path
`sttProvider`	`auto` | `local` | `openai` · `auto`	`SPECTRO_STT_PROVIDER`	which way speech to text goes (chapter 17): `auto` is hosted when a key is there and local otherwise; an explicit choice is never rerouted, even when it cannot run
`sttLanguage`	`auto` | `de` | `en` · `auto`	`SPECTRO_STT_LANGUAGE`	the language dictation is transcribed in, on both routes; `auto` lets the model detect it
`sttRunner`	`auto` | `cli` | `server` · `auto`	none	what runs whisper for live text on the local route (card 528). `cli` starts one `whisper-cli` process per chunk, and every process loads the speech model again, so the loading card shows on every press. `server` starts one `whisper-server` from the same Homebrew package on the first press, on `127.0.0.1` and a free port the app picks, keeps the model loaded and sends each chunk to it; it stops when the app stops or after 30 minutes with nothing to do, and the next press starts it again. `auto`, the shipped value, runs the server when `whisper-server` is installed and `whisper-cli` otherwise. `server` without `whisper-server` on the machine also runs `whisper-cli`, and `/api/stt/status` says so. Read when a recording starts, so a saved change applies from the next press. The batch route (live text off) always runs `whisper-cli`
`chromeBinary`	path · *unset*	`SPECTRO_CHROME`	override for the system-Chrome binary `browse_page` discovers automatically; machine-tool path. Process-global: refused in a workspace scope, user scope only — it names an executable `browse_page` launches, and the workspace is the folder the agent itself writes into
`hooks`	array · `[]`	—	pre/post tool hooks — whole-block (chapter 13)
`otlpEndpoint`	URL · *unset*	`SPECTRO_OTLP_ENDPOINT`	the OTLP/HTTP traces endpoint; unset = the exporter is off (see Observability below)
`otlpBasicAuth`	`pk:sk` · *unset*	`SPECTRO_OTLP_BASIC_AUTH`	optional Basic-auth pair for the OTLP endpoint (Langfuse's public:secret); never echoed by any read
`tts`	`{enabled, voice}` · off, `en_US-lessac-medium`	—	CLI-side voice output block in the same user settings file (not a `SpectroConfig` field)

Every `SPECTRO_*` name in the "env base" column still works exactly as before, but only as the BASE layer — see the deprecation table above for the field each one now defers to, and run `spectro doctor` to see which of yours are currently shadowed. The three "machine-tool path" fields are session-scoped like any other field, but discouraged from a committed project file: the Settings page and the composer gear's local-overrides section are where they belong.

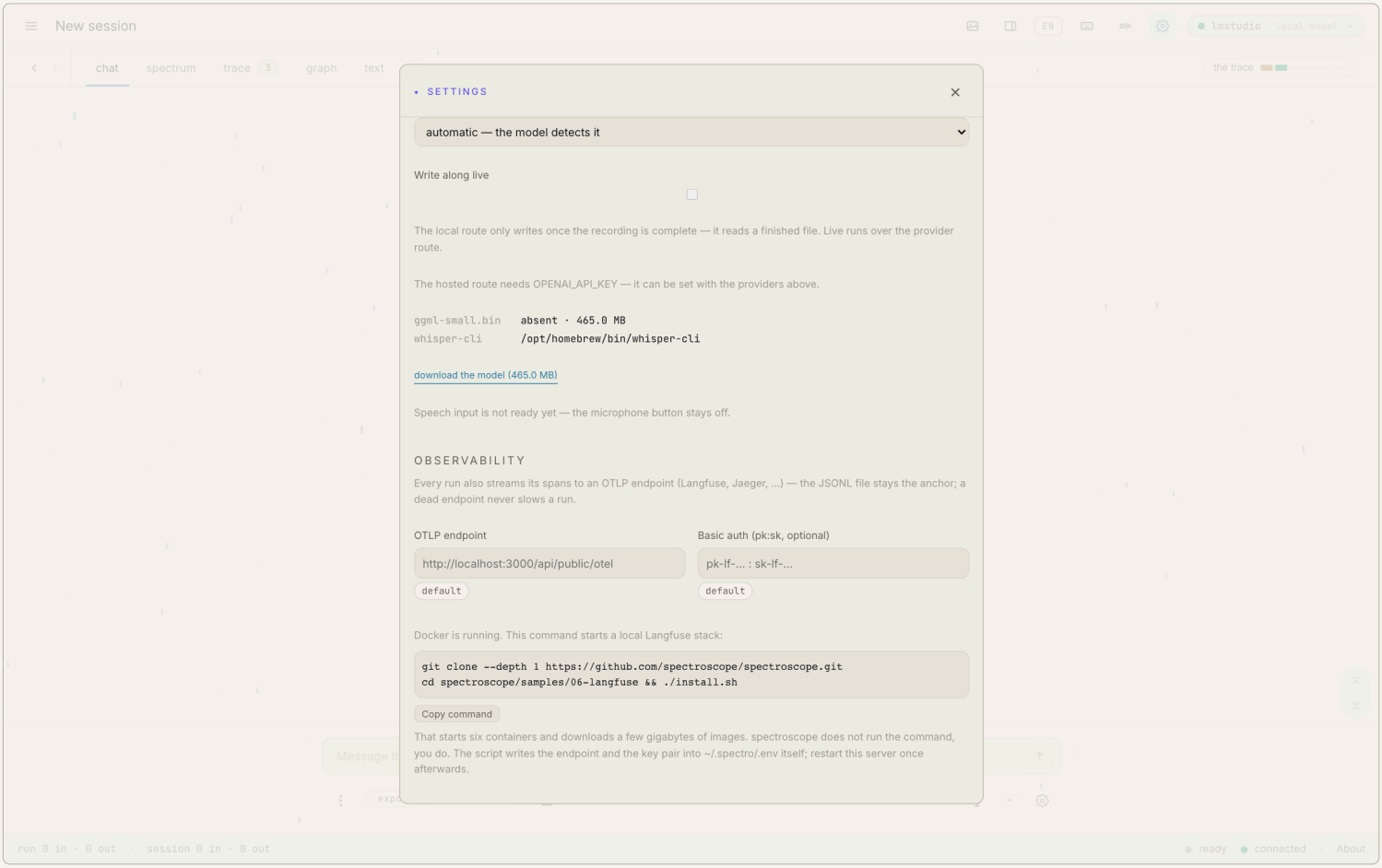
Observability: streaming to OTLP

Every run already writes its `session.jsonl`; that file is the durable record and never needs a network. On top of it, a run can ALSO stream its spans to any OTLP/HTTP backend — Langfuse, Jaeger, Phoenix, anything that speaks the OpenTelemetry protocol over HTTP. The export is additive: the JSONL is the anchor, and a dead backend warns once and never slows a run.

Two settings turn it on, both off by default — the endpoint and an optional Basic-auth pair, either through *Settings → Observability* or the two `SPECTRO_OTLP_*` env bases in the table above:

```
SPECTRO_OTLP_ENDPOINT   = http://localhost:3000/api/public/otel/v1/traces   (Langfuse)
SPECTRO_OTLP_BASIC_AUTH = pk-lf-...:sk-lf-...                               (optional)
```

The **doctor** carries an OTLP line: it posts an empty — but valid — OTLP batch to the configured endpoint and shows green when the endpoint accepts it, so you learn a real run's spans will land before you spend a run finding out. The auth value is never echoed. Each session becomes one trace whose span tree mirrors the run (agent → turns with token usage → tool calls → gates), carrying both Langfuse's `langfuse.observation.type` and OpenTelemetry's `gen_ai.*` conventions so either kind of consumer reads it. The full attribute mapping, the endpoint variants (Langfuse / Jaeger / Phoenix) and the backfill path for older sessions live in `docs/OBSERVABILITY.md`.



Settings → Observability: the OTLP endpoint and the optional pk:sk pair. The doctor's OTLP line probes the endpoint for real — an empty valid batch — so a green light means a run's spans will land.

Environment-only variables

Everything that used to live only in the environment has graduated to a settings field (deprecation table above). What is left is **secrets** — spectroscope never writes an API key to a settings file, and the `PUT` endpoints refuse to accept one outright (any `*_API_KEY` / `*_TOKEN`-shaped key is rejected):

VARIABLE	CONSUMED BY
`ANTHROPIC_API_KEY`	the Anthropic SDK; checked up front — a missing key refuses provider construction with a readable message
`OPENAI_API_KEY`	optional Bearer for the chat provider; required by the OpenAI image backend
`OPENROUTER_API_KEY`	Bearer for the `openrouter` openai-compatible provider
`GEMINI_API_KEY`	the Gemini image backend
`TAVILY_API_KEY`	the Tavily tier of `web_search`; absent falls back to the keyless DuckDuckGo tier

Three `.env` injection paths, one parse rule. Gradle's run tasks (CLI and server) and the `./spectro-app` launcher all read the project root's gitignored `.env` identically: comments and blank lines skipped, first `=` splits, one quote layer stripped, **empty values excluded** — a `KEY=` line can never blank a variable you exported in your shell. The launcher's copy exists because the desktop face bypasses Gradle entirely. All JavaExec tasks also run from the project root, so the project settings layer is always found.

The single writer. Only `SettingsWriter` ever writes a settings file — two entry points, the same serialized read-modify-write discipline: `appendAutoApprove` (the permission dialog's "Persist"/"dauerhaft" checkbox) appends one allowlist rule to the workspace project file; `patch` (behind every `PUT /api/settings/*` call, so behind both gears) applies a schema-validated partial patch to any of the three files, rejecting secrets and process-globals in the wrong scope before anything touches disk. Both preserve every other key already in the file, and a `local` write also ensures the workspace's `.spectro/.gitignore` lists `settings.local.json`. Everything else only reads.

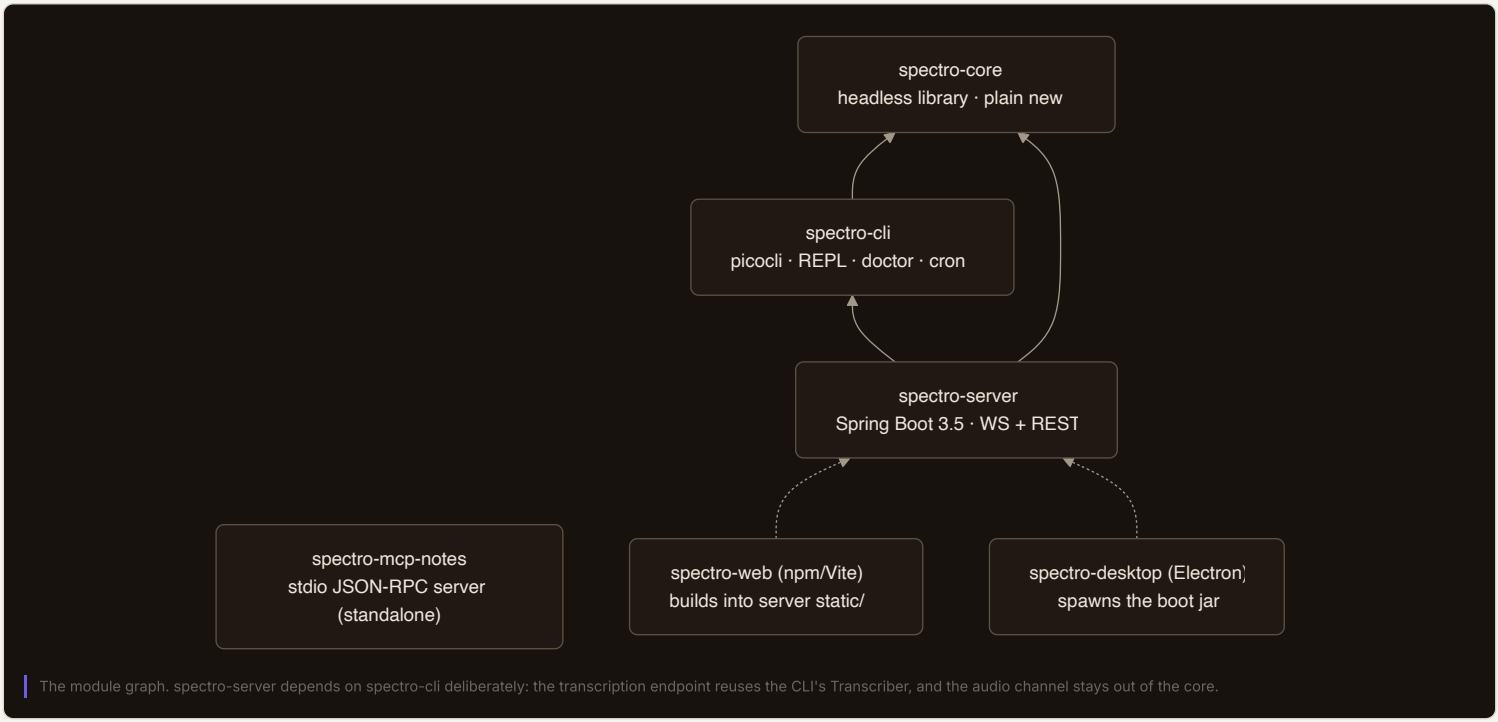
SPECTRO.md

A `SPECTRO.md` in the working directory is appended to the system prompt as a "Project context" section — provider-neutral, visible in the System-context panel. The counterpart of a CLAUDE.md, for spectroscope.

CHAPTER 31

Build & test inventory

Five JVM modules, two npm toolchains, one version catalog — and a test suite that never needs an API key.



The module graph. spectro-server depends on spectro-cli deliberately: the transcription endpoint reuses the CLI's Transcriber, and the audio channel stays out of the core.

Modules

MODULE	ROLE	KEY FACTS
`spectro-core`	the headless library	java-library; Jackson is `api`, through its BOM (JsonNode appears in public contracts); the Anthropic SDK is `implementation`, and no other class may import it; the Copilot SDK is `implementation` as well, used only by the Copilot provider, with its own Jackson family excluded; Spring *Framework* as a library (RestClient + HTTP interfaces); cron-utils
`spectro-cli`	terminal face	application; picocli; Logback keeps the console WARN-quiet so the ANSI face stays clean and the diagnostics go to the log file; registers the `tour` task; stdin wired for the REPL
`spectro-server`	web backend	Spring Boot 3.5.3 (only here); starters web + websocket; depends on core *and* cli; serves the built UI from `static/`
`spectro-mcp-notes`	example MCP server	standalone; Jackson only; its test spawns the real child JVM
`spectro-orchestrator`	the fleet facade	java-library; Spectro.panel() lanes as core agents on a bus; POM pins spectro-core at the same version; Maven Central library
`spectro-web`	browser UI	npm/Vite; React 19, React Flow 12, dagre; `npm run build` writes straight into the server's resources
`spectro-desktop`	desktop shell	npm/Electron 43; TypeScript strict; zero runtime dependencies (health polling via built-in fetch)

Version pins

LIBRARY	VERSION	
`com.anthropic:anthropic-java`	2.34.0	owns SSE; client built with maxRetries(0)
`com.github:copilot-sdk-java`	1.0.20-preview.0	a preview release, MIT licence; the Copilot provider (macOS only) starts the Copilot runtime through it; its Jackson 2.22.2 family is excluded, so it runs on the BOM's 2.18.2. The published `spectro-core` therefore depends on a preview artifact: this is the version the provider was measured against with a live Copilot runtime, and the stable 1.0.19 was not tested
`com.fasterxml.jackson:jackson-bom`	2.18.2	a platform, not a number: `jackson-databind` is declared under it with no version of its own and is the one `api` dependency
`info.picocli:picocli`	4.7.6	CLI framework
`org.springframework:spring-web`	6.2.8	Framework as library — no Boot in the core
`com.cronutils:cron-utils`	9.2.1	computes slots; a ScheduledExecutorService fires them
`org.junit.jupiter`	5.10.2	
Gradle wrapper	9.6.1	checked in
Spring Boot	3.5.3	spectro-server only

react / @xyflow/react / dagre	19.1 / 12.11 / 0.8.5	spectro-web
electron / electron-builder	43.3 / 26.0	spectro-desktop
`org.slf4j:slf4j-api` / `ch.qos.logback:logback-classic`	2.0.18 / 1.5.38	the two library modules speak the API only; the faces run Logback, lifted off Boot 3.5.3's managed 1.5.18

The Jackson row is a platform import, and that is the interesting one. The published POM used to declare `jackson-databind` at 2.17.2 while `anthropic-java` 2.34.0 brings the 2.18.2 family in behind it, and every build in this repository passed anyway — Gradle resolves highest-wins and quietly landed the whole tree on 2.18.2. A resolver that chooses differently does not: Maven's nearest-wins, or the classpath an editor extension assembles, can pair a 2.18 databind with a 2.17 core, and a databind newer than its own core throws `NoSuchMethodError` on `ParserMinimalBase.<init>(StreamReadConstraints)` the first time anything is deserialized. Importing the BOM pins the family together instead, and the import travels into the generated POM, so the guarantee holds for every consumer and not only inside this build. For a published library the POM is the contract — and the contract was the part that was wrong.

The test suite

MODULE	TESTS	WHAT THEY PIN
spectro-core	270	event round-trips and wire shapes, the loop against fake providers, sessions/resume/delete, compaction, subagents (parallelism, timeout), scheduler, all three provider mappings against scripted local servers, MCP client/transports, hooks, allowlist, retry
spectro-cli	33	renderer, overrides plumbing, allowlist, voice seams
spectro-server	33	REST guards (incl. the delete endpoint), a full WebSocket round-trip against an Ollama mock, workspace sandbox, transcripts controller
spectro-mcp-notes	18	store, ranker, stdio protocol in-JVM and as a real child process
JUnit total	**354**	plus **252 vitest** for the web UI (reducer, threads, stepper, scene models, scenario compiler, importer, markdown parser, layout stores)
spectro-web (vitest)	252	

Every test runs key-free and network-model-free: fake providers, scripted HTTP servers, injectable process seams, and a redirected `user.home` so no test can ever touch your real `~/.spectro`. Gate: `./gradlew build` + `cd spectro-web && npm test`.

CHAPTER 32

Releases & working with the source

Two libraries go to Maven Central, one download per app module goes to the GitHub release, and the whole tree is MIT and reproducible from a script. Rule of thumb: every module ships *something*. This chapter is the map of what that something is, and how to work with the source that builds it.

What a release ships

The seven modules split cleanly into reusable Java (published to Maven Central so you can depend on it) and runnable faces (attached to the GitHub release, one download each). One module is the exception, and it is called out below.

MODULE	BUILD	SHIPS AS
`spectro-core`	Gradle · java-library	Maven Central library
`spectro-orchestrator`	Gradle · java-library	Maven Central library (POM pins core at the same version)
`spectro-cli`	Gradle · application	GitHub asset `spectro-<v>.zip` (the terminal face, `bin/spectro`)
`spectro-server`	Gradle · Spring Boot	GitHub asset `spectro-server-<v>.jar` (executable; embeds the built web UI)
`spectro-mcp-notes`	Gradle · application	GitHub asset `spectro-mcp-notes-<v>.zip` (the sample MCP server)
`spectro-desktop`	npm · Electron	GitHub asset `spectroscope-<v>-<arch>.dmg` (the desktop run kit; bundles server + JRE)
`spectro-web`	npm · Vite	**not a standalone download** — built into the server jar (and the desktop kit)

Diagram 01 — the Gradle build: five JVM modules with their dependency edges and the version catalog, plus the two npm toolchains (spectro-web, spectro-desktop) that live next to the JVM graph on purpose. What ships from each is the subject of this chapter.

Why the web has no download. `spectro-web` is a single-page app; it needs the server's REST and WebSocket API on the same origin, so a bundle handed out on its own could not run. The release build compiles the UI *first*, writing its output straight into `spectro-server`'s static resources, so the server jar below already carries the current UI. The desktop kit carries that same server jar, so the web face is present in three assets and standalone in none.

`scripts/build-release-assets.sh` produces every GitHub asset in one pass into `build/release-assets/`; the two libraries are the exception and go to Maven Central instead. On a machine without a full JDK/Node for the Electron step, `SKIP_DESKTOP=1` builds everything except the `.dmg`.

The libraries on Maven Central

The two headless modules are published under the **verified namespace** `dev.spectroscope`. Depend on the orchestrator and you get the core transitively:

```
dependencies {
  implementation("dev.spectroscope:spectro-core:0.15.4")
  implementation("dev.spectroscope:spectro-orchestrator:0.15.4")  // POM pins spectro-core at 0.15.4
}
```

`v0.1.0` is the first published version. **Maven Central is append-only**: a published version cannot be pulled or overwritten, only superseded, so the next release is always a bump (`0.1.0 → 0.1.1 → …`), never a re-publish. That is why the release ritual gates hard before the single irreversible step.

Only these two ship to Maven. `spectro-server` is a Spring Boot application: its plain `jar` is a thin, non-runnable shell, so it ships as an executable `bootJar` on the GitHub release, never to Maven. `spectro-web` and `spectro-desktop` are npm/frontend modules and cannot go to Maven Central at all.

The desktop run kit

The `.dmg` is a **self-contained** app. The Electron shell (`spectro-desktop`) spawns and supervises the server, and the build bundles both the server jar *and* a `jlink`'d JRE into the app, so the target machine needs no Java installed: double-click, the server starts with it, the cockpit opens. When the window closes, the shell reaps its child JVM. The script that builds it is `scripts/build-desktop-runkit.sh`.

Two honest limits, and the release notes state them:

- **Per-platform.** Electron and the bundled JRE are OS/arch specific, and the script builds the *host* target only. Windows, Linux and Intel each need building on and for themselves.
- **Ad-hoc signed, not notarized.** On first download macOS reports an *"unidentified developer"*; open it once with right-click → Open (or clear the quarantine flag with `xattr -cr`) and it launches normally from then on. A local build has no quarantine flag and starts directly.

For a zero-warning, plain double-click app you need a paid **Apple Developer Program** membership, a Developer ID Application certificate, and notarization. The one spectroscope-specific twist is signing the bundled JRE under the hardened runtime; the full end-to-end procedure is `docs/DESKTOP-SIGNING.md`. Sign only with the ad-hoc identity (`-`) otherwise, never a corporate certificate the machine may happen to list.

Working with the source

The layout is the module table above, read as folders. `settings.gradle.kts` includes the five JVM modules (`spectro-core`, `-cli`, `-server`, `-mcp-notes`, `-orchestrator`); the two npm modules (`spectro-web`, `spectro-desktop`) sit next to the Gradle build with their own toolchains, on purpose — a Java backend and JS frontends are separate worlds and the build does not pretend otherwise. The gate is two commands:

```
./gradlew build                          # the whole JVM suite + all tests
( cd spectro-web && npm test )           # the web UI suite (vitest)
```

Neither needs an API key: the tests run against fake providers and scripted local servers, and the one live contract check self-skips unless `ANTHROPIC_API_KEY` is set (chapter 31 has the full inventory). The whole tree is **MIT licensed** (`LICENSE`, © 2026 Christopher Ezell) — use it, fork it, ship it, keep the notice.

The release ritual

Cutting a release is a written runbook, `docs/RELEASE-PLAYBOOK.md` (`RELEASING.md` is the narrower "just Maven Central" companion; the two agree). The order exists to put every irreversible step behind a green gate:

1. **Preflight** — clean tree, on `main`, and a version strictly higher than the last published (Central is append-only).
2. **Bump versions together** — the two Maven libs (`spectro-core`, `spectro-orchestrator`) and the app versions that name the assets (`spectro-cli`, `-server`, `-mcp-notes`, and `spectro-desktop/package.json`).
3. **Full green gate** — `./gradlew test` plus the two `javadoc` tasks, then vitest. At `v0.1.0` the baseline was **678 JUnit** (one skipped: the live-Opus check) and **383 vitest**.
4. **Dry-run the publish** — `publishToMavenLocal` for both libs confirms GPG signing and POM generation (every artifact gets a `.asc`; the orchestrator POM depends on `spectro-core` at the same version) without touching the portal.
5. **Commit + tag** `v<v>`, push both.
6. **Publish the libraries** — `./gradlew publishAndReleaseToMavenCentral`. This is the one irreversible step; wait for the coordinates to resolve on `repo1.maven.org` before trusting them.
7. **Build the assets** — `scripts/build-release-assets.sh`, then `gh release create v<v>` with everything in `build/release-assets/`.
8. **Flip the install snippets** — the landing page and the dev portal go from "planned" to the real coordinates; push is deploy, so verify the live copy last.

One-time owner prerequisites (a Central Portal token and GPG key in `~/.gradle/gradle.properties`, a full JDK for `jlink`, Node for the web bundle and Electron, and `gh` authenticated) are listed at the top of the playbook. The gitignored build artifacts (`jre/`, `build/`) never get committed.

The brand mark

The mark is a spectrum. Five vertical lines of different weights and colours, read left to right — the same figure the event stream makes when it fans into its lines. It sits on the cover of this guide. Colour lives only on the lines (a coral edge, amber, a broad teal, cyan, a brown edge); the wordmark is always **lowercase**, one word, set flat with no shadow and no glow. That restraint is deliberate and matches the interface, where colour is reserved for the spectral lines and nothing else carries it.

CHAPTER 33

Saving a key, safely

You can paste an API key into the cockpit and have spectroscope save it for you. That is a web page writing a secret to disk, so it only ships fenced. This is the honest account of the fences: what the feature does, why it takes effect without a restart, and exactly which requests it refuses.

Where the key goes

The key lands in one place: `~/.spectro/.env`, written at `0600` (owner read and write only). It is **never** put in `settings.json` — those files are meant to be shared and checked into a project, so a secret there is a leak waiting to happen. It never enters a session file, a log line, or any GET response: the API reports a key's *presence* (`true` / `false`), never its value. There is one write path, the POST below; the value leaves your browser once, as that request body, and is never sent back.

Why it works without a restart

A running JVM cannot change its own environment: `System.getenv` is frozen for the life of the process. So a key merely written to disk would be inert until the next launch. spectroscope closes that with a read seam. Every provider resolves its key as *environment first, then* `~/.spectro/.env` (`SpectroConfig.resolveApiKey`). Providers are built per chat, so the moment you save a key the next new chat picks it up: no restart, and it works the same from the jar and the desktop app, not only the launcher's `./.env`. A real environment variable or a launcher-loaded `./.env` still wins, so nothing you set on the command line is ever overridden.

The two fences on the write

The endpoint is `POST /api/onboarding/key`. It is bound to `127.0.0.1` like the rest of the server, and it passes only when **both** of these hold:

FENCE	WHAT IT CHECKS	WHAT IT STOPS
local origin	the TCP peer is loopback *and* the `Host` header is `localhost` / `127.0.0.1`	a remote caller, and a DNS-rebinding attack that aims a public name at your loopback
same origin	the `Origin` header is absent (a non-browser client) *or* points at loopback	CSRF: a page on `evil.com` firing a `fetch` at your local server. Its request is still loopback, but its Origin is its own domain, so it is refused

The second fence matters because the read side of this controller deliberately allows cross-origin requests, so the Vite dev server on another port can read `/api/config`. Reads are harmless; a *write* is not, so the write checks the Origin itself instead of trusting the CORS policy. A refused request answers `404`, not `403`: an attacker gets no signal that the endpoint is even there.

What an attacker can and cannot do

Plainly, for the sceptics:

- A website you visit **cannot** write a key here. The Origin fence answers it 404 (verified: an `evil.com` Origin is rejected, a loopback Origin is accepted).
- A remote machine **cannot** reach the endpoint. The server binds loopback, and the Host fence 404s a rebinding attempt.
- Nobody can **read** a saved key back through the API. Every surface is presence-only; the value sits on disk at `0600` and in the process, nowhere else.
- The honest residual: a malicious program *already running on your machine* could POST a key (its Origin is loopback) or just read the file directly. But a hostile local process can do far worse than this anyway, so it is outside the line this feature draws. That line is external websites and remote hosts, and both are closed.

You never have to use it

The dialog is a convenience, not the only door. The same key is read from a real environment variable, from a `./.env` next to the launcher (`./spectro-serve --env-file`), or typed hidden on the command line with `./spectro-serve set-key`. All four land in the same resolver, so use whichever you trust. The cockpit field is there for the person who would rather not open a terminal.

APPENDIX A

Troubleshooting

SYMPTOM	CAUSE & FIX
`zsh: operation timed out: ./spectro-app`	The folder arrived as a download — every file carries the macOS quarantine flag, and Gatekeeper hangs in an online notarization check on exec. Fix: `xattr -cr <path>/spectroscope` (read-only `.git/objects` complaints are harmless).
`ollama unreachable` in doctor	The Ollama server is not running — open the app or `ollama serve`. Probe: `curl -s http://localhost:11434/api/tags`.
Gradle: "release version 21 not supported"	Your default JDK is older than 21. The launcher normally solves this; without it, install `brew install openjdk@21` or set `JAVA_HOME`.
git hangs / `Operation timed out` on `git diff`	On a sync filesystem (OneDrive/iCloud) the fsmonitor daemon can wedge. Fix: `git config core.fsmonitor false`.
Model without vision error	You attached an image while a text-only local model is selected. `ollama pull qwen3-vl` (or llava) and switch, or use a cloud provider.
Mic button disabled with a tooltip	STT is not installed — `bash scripts/setup-stt.sh`; or the browser denied microphone access.
`Voice output disabled: piper, voice or audio player missing`	`bash scripts/setup-tts.sh` (note: its checksums are pin-on-first-run — the script prints the sums it computed and asks you to verify and paste them).
Anthropic switch refused in the picker	`ANTHROPIC_API_KEY` is not set for the *server* process — put it in `spectroscope/.env` and restart `./spectro-app web`.
Desktop window blank after a rebuild	An old instance held Electron's single-instance lock. `./spectro-app desktop` stops stale instances first — use it instead of a bare `npm start`.
MCP server UNREACHABLE in doctor	The configured `command` path does not exist on this machine (the settings file carries machine-specific absolute paths) or the dist was never built: `./spectro-app mcp-notes`.
`SPECTRO_MAX_RETRIES must be an integer`	Exactly what it says — the env var refuses garbage loudly instead of silently defaulting.

APPENDIX B

Reproducing this guide

This document is generated — never hand-edited — and every asset in it is reproducible from the repository.

SCREENSHOTS	`docs/guide-assets/capture_screens.sh` runs `capture_screens.mjs` for both themes: Playwright on the system Chrome against a server jar on a copy of the curated `demo-home`. EN chrome and the learn mode are seeded before load, scenario replays cover the feature states, and live runs against a local ollama model cover the plan tab, thinking, the permission dialog, the Files tab, explain, a continued session and two sessions at once. The delete shot only opens the dialog and cancels it. The tutorial plates come from `capture_leveling.sh`, the chat plates from `capture_chat.sh` and the state graph plates from `capture_stategraph_shots.mjs`. Output: `docs/guide-assets/shots/*.png` and `shots-light/*.png`.
MERMAID DIAGRAMS	`docs/guide-assets/mermaid-src/*.mmd` rendered to SVG by `render_mermaid.mjs` (mermaid 11, dark theme on the spectroscope tokens) — the guide inlines the SVGs so the PDF needs no JavaScript.
ARCHITECTURE SVGS	generated by `docs/diagrams/build_NN_*.py`; rerun with `for g in build_*.py; do python3 $g; done`.
TERMINAL CAPTURES	real output of `./spectro-app` and `./spectro-app doctor` on 2026-07-16, ANSI stripped at build time.
ASSEMBLY	`python3 docs/guide-assets/build_user_guide.py` — inlines the fonts (from the server's static assets), all images as data URIs, and the content parts from `docs/guide-assets/parts/` into the self-contained `docs/USER-GUIDE.html`.
PDF	headless Chrome: `--headless --no-pdf-header-footer --print-to-pdf=docs/USER-GUIDE.pdf docs/USER-GUIDE.html`. Dark pages incl. margins come from the thead/tfoot page-spacer technique with `@page margin 0`.
SOURCE OF TRUTH	the prose and the figures were verified against the **v0.1.0** source tree; gate at that time: 678 JUnit + 383 vitest, 0 failures. The cover carries the edition date, not a re-verification: this edition ships with **v0.15.4**. The run view in the Plan panel with its wide view, the opening of the documents a run wrote, the agent picker of the context ring and of the Plan and System context panels, and the Plan of the model were checked against that tree on 2026-10-13. The playbook start from the chat with the playbook chip, the confirmation and the run lines in the chat, the playbook's skills, commands and agents in a session, the contents list without install states, the shared command list with `/help`, the terminal's `/clear`, the minimal density, the model picker of a stored session and Stop on a backend that does not answer were checked against the previous release's tree or the branches merged into it on 2026-10-12 and 2026-10-13. Messages between agents and background helpers, the goal and `/loop`, running a workflow, the bundle tabs of the playbook pane, the code graph rows and the `code_graph` tool, the dock's tabs default, the tool counts of chapter 12 and the config reference rows for agentMessaging, sttRunner and toolGroupsOff were checked against the tree of the release before it on 2026-10-11 and 2026-10-12; the folder chooser in the Playbook segment, the row of tabs in developer, back and forward in the header, and Spectrolyzr as one page with a new playbook option against the tree of the release before that on 2026-10-10; the rest has not been re-checked. Where the guide and the product disagree, the product is right.

Credits and colophon

spectroscope is designed and built by **Christopher Ezell**, with Claude (Anthropic) doing much of the heavy lifting across code, design and copy. It began as the reference harness of a build-an-agent-harness workshop and grew into its own product; the brand, the orchestrator and this guide came after.

The guide is generated, never hand-written. The type is Inter and JetBrains Mono, both embedded so the pages read the same on any machine. The architecture plates are drawn by Python generators; the sequence and flow figures are Mermaid; the screenshots are real captures of spectro web. Everything assembles into one self-contained HTML file and a PDF, in an espresso-dark and a paper-light edition. The wire format is shared byte-for-byte with the TypeScript edition, so a session recorded by one is replayable by the other.

Fonts: Inter and JetBrains Mono, both SIL Open Font License 1.1. Diagrams: Mermaid (MIT). Rendering and capture: headless Chrome. spectroscope itself is MIT licensed and pre-release. Contact: chris (at) spectroscope.ai